



Automata Formal Languages & Logic

Introduction to the Course

Prakash C O

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Introduction

Dictionary Definition of an Automaton

noun (plural automata)

1. A moving mechanical device made in imitation of a human being.
2. A machine that performs a function according to a predetermined set of coded instructions.



Automata Formal Languages & Logic

Introduction

- The term “Automata” comes from the Greek word “αὐτόματα” which implies “self-acting”. An automaton (Automata in plural) is an abstract self-propelled machine which follows a predetermined sequence of operations automatically.

- What is Automata Theory?

Study of abstract computing devices, or “machines”

Automaton = an abstract computing device

Note: A “device” need not even be a physical hardware!

- A fundamental question in computer science:

Find out what different models of machines can do and cannot do

Automata Formal Languages & Logic

Introduction

What is Finite State Machine (FSM)?

- An automaton with a finite number of states is named a Finite Automaton (FA) or Finite State Machine (FSM).
- Finite-state machine (FSM) or finite-state automaton (FSA) is **an (abstract) mathematical model of computation**. FSM can be **in exactly one of a finite number of states at any given time**.



Automata Formal Languages & Logic

Introduction

What is Finite State Machine (FSM)? Cont..

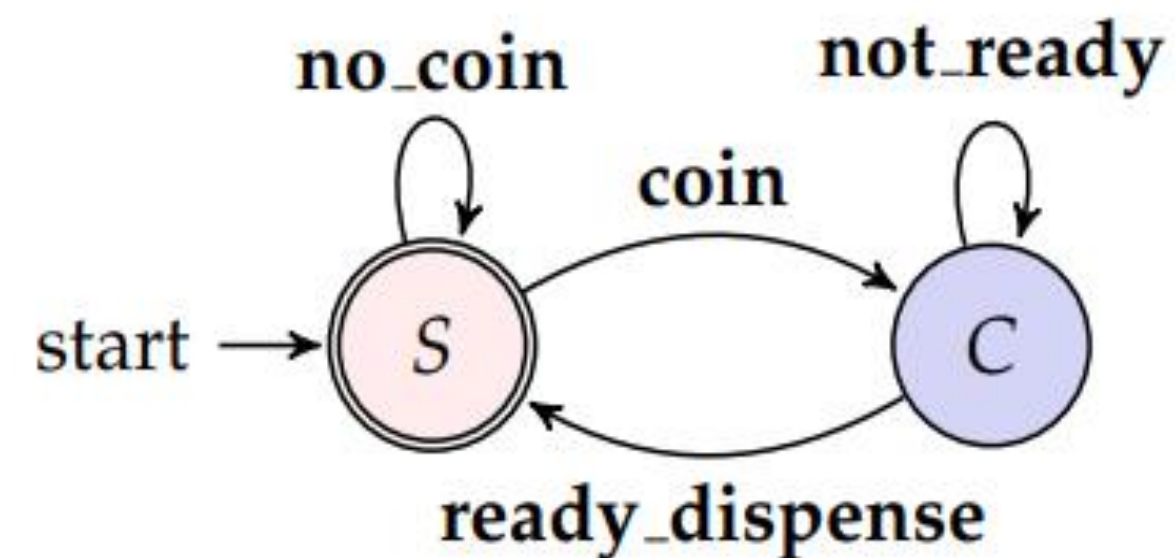
- The FSM can change from one state to another in response to sequence of inputs; the change from one state to another is called a **transition**.
- An FSM is defined by a list of its states, its initial state, its final state(s) and the inputs that trigger each transition.



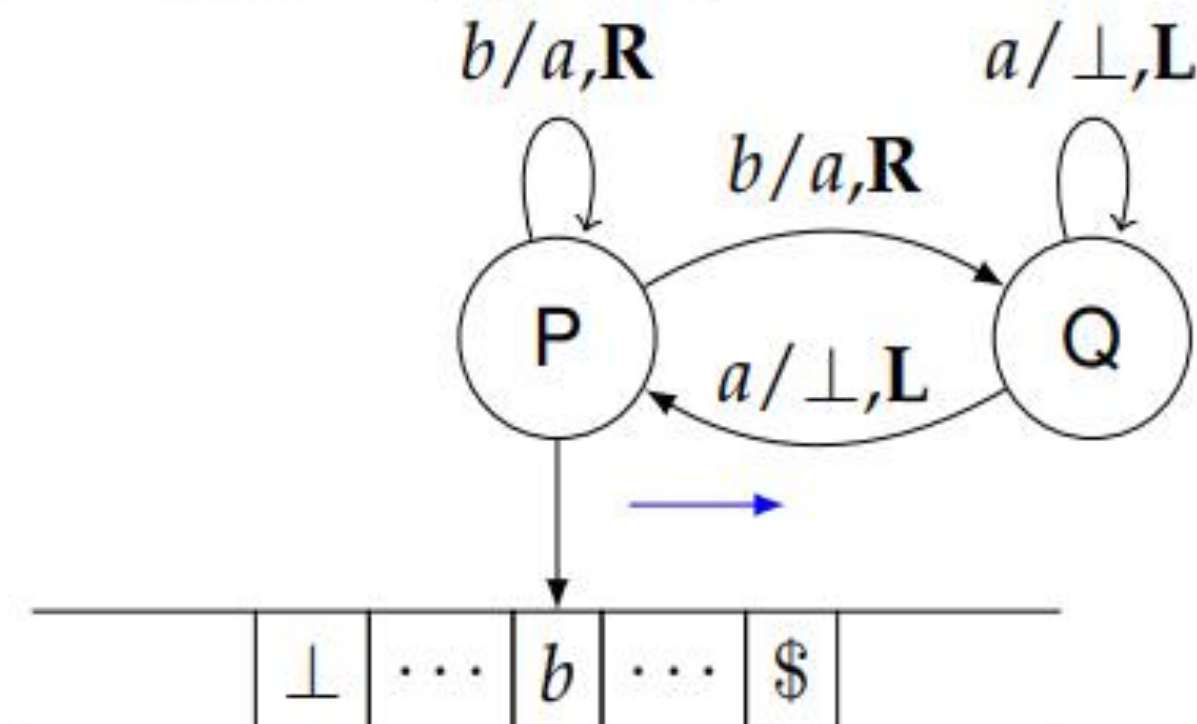
Automata Formal Languages & Logic

Introduction

Finite instruction machine with finite memory (**Finite State Automata**)



Finite instruction machine with unbounded memory (**Turing machine**)



The Turing Machine is a theoretical model of a simple computing device that can perform calculations and manipulate symbols on an infinite tape.

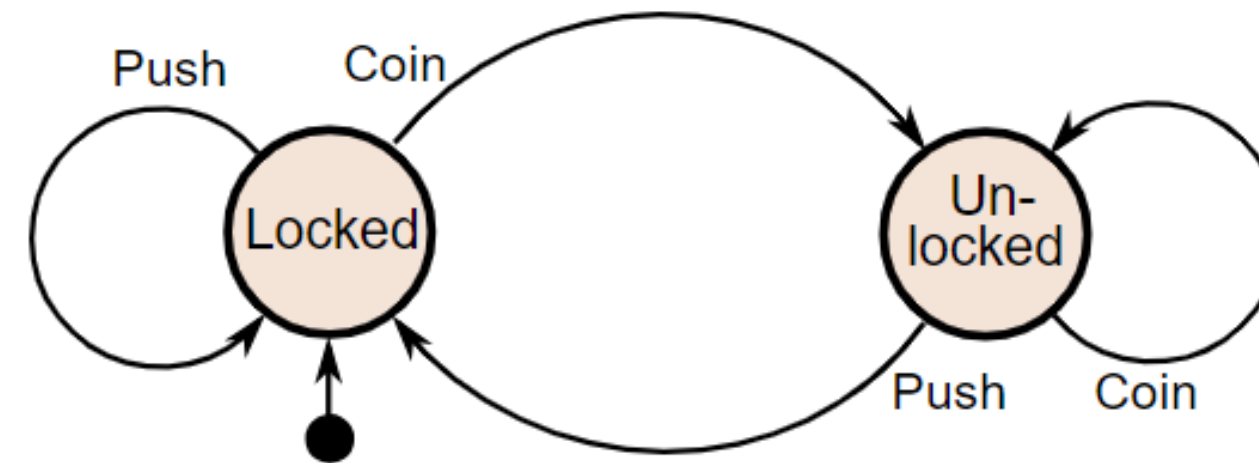
A Turing machine is an improved version of the Finite state Machine.

Automata Formal Languages & Logic

Introduction

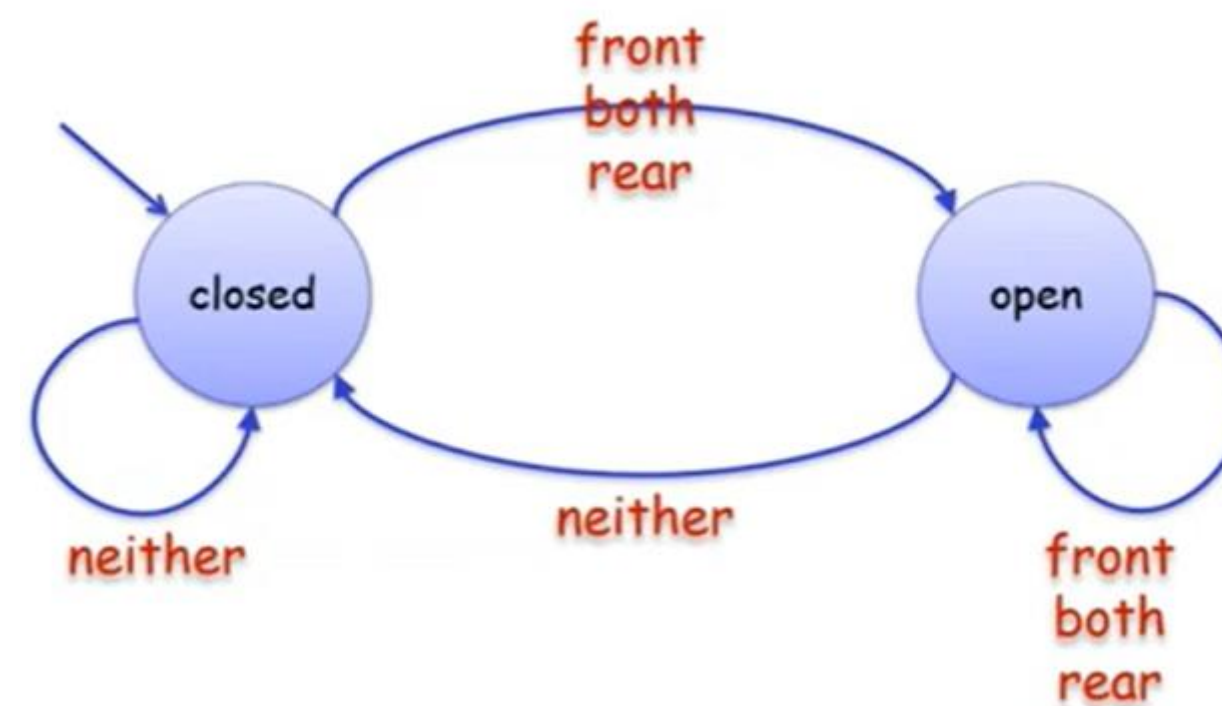
Finite State Machine (Finite State Automata)

➤ Example 3: coin-operated turnstile



State diagram for a turnstile

➤ Example 4: Controller used for an automatic sliding door (simple example of a FSM)



Automata Formal Languages & Logic

Introduction



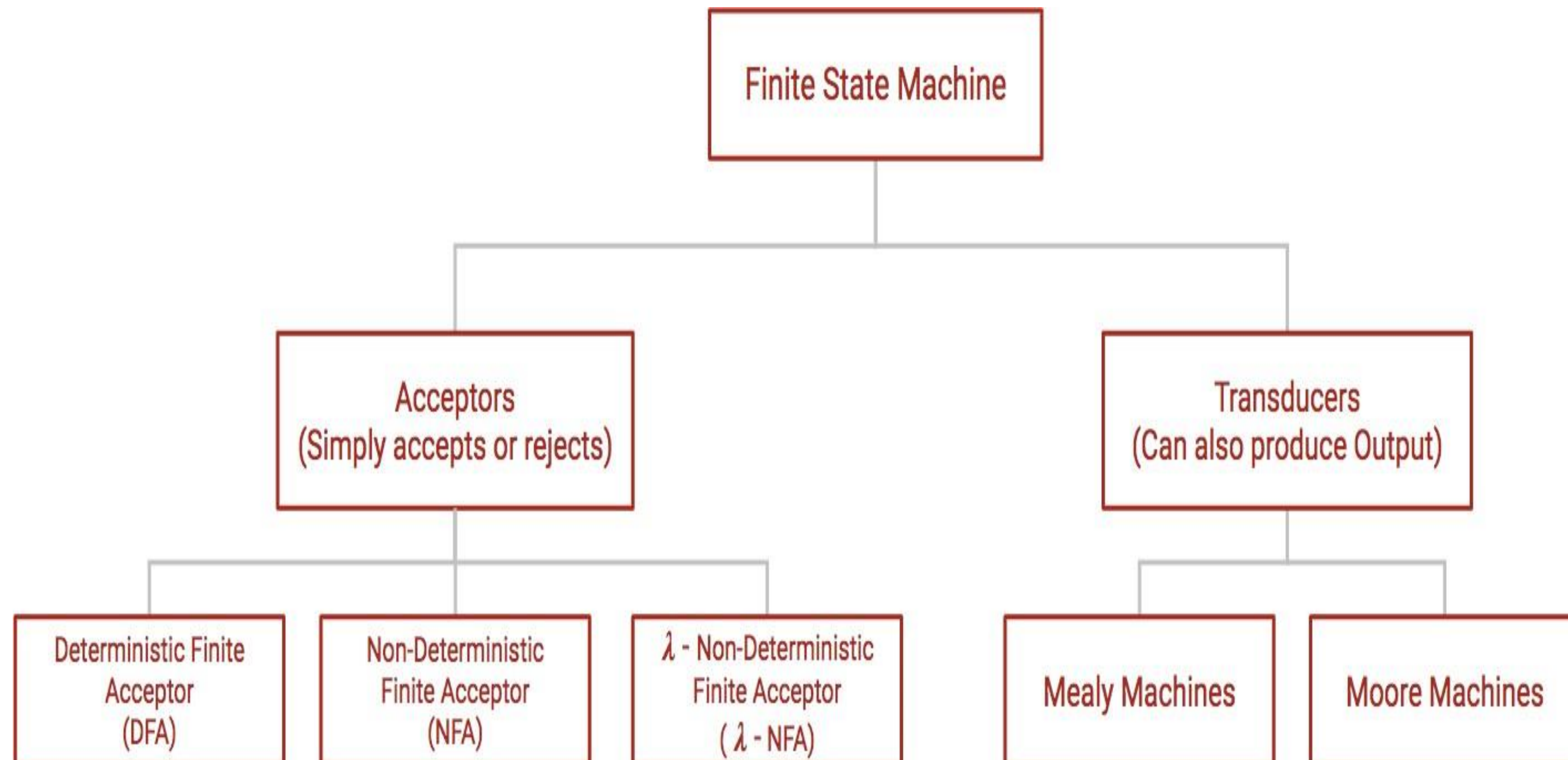
Finite State Automata (FSA) / Finite State Machine (FSM)

- Through automata, computer scientists are able to understand how machines compute functions and solve problems and more importantly, what it means for a function to be defined as *computable* or for a question to be described as *decidable*.
- There are four major families of automaton:
 1. Finite-state machines (Acceptors/Recognizers and Transducers)
 1. Pushdown automata
 2. Linear-bounded automata
 3. Turing machine

Automata Formal Languages & Logic

Introduction

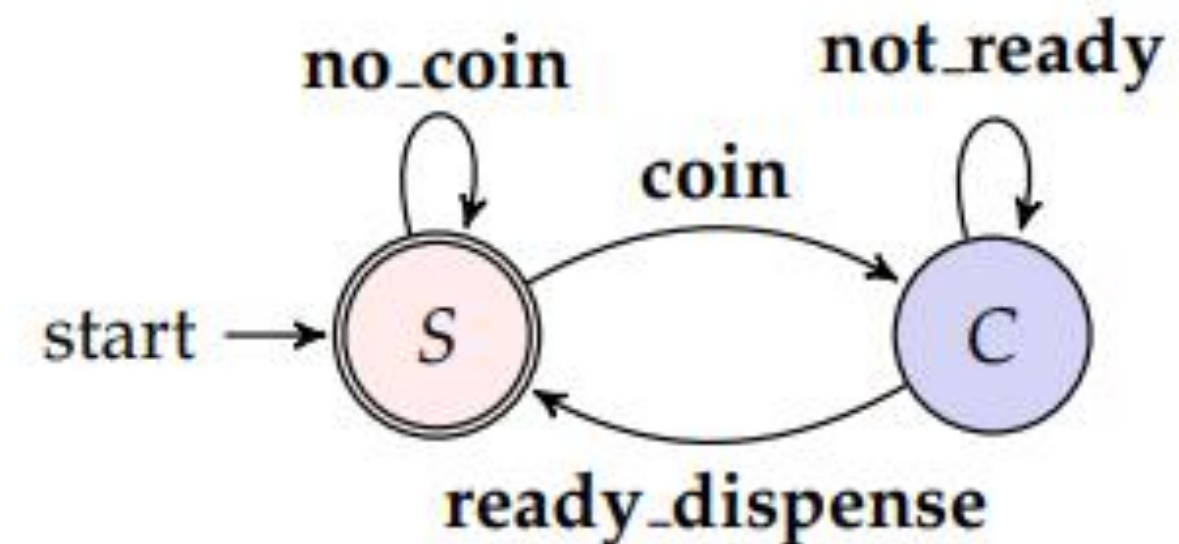
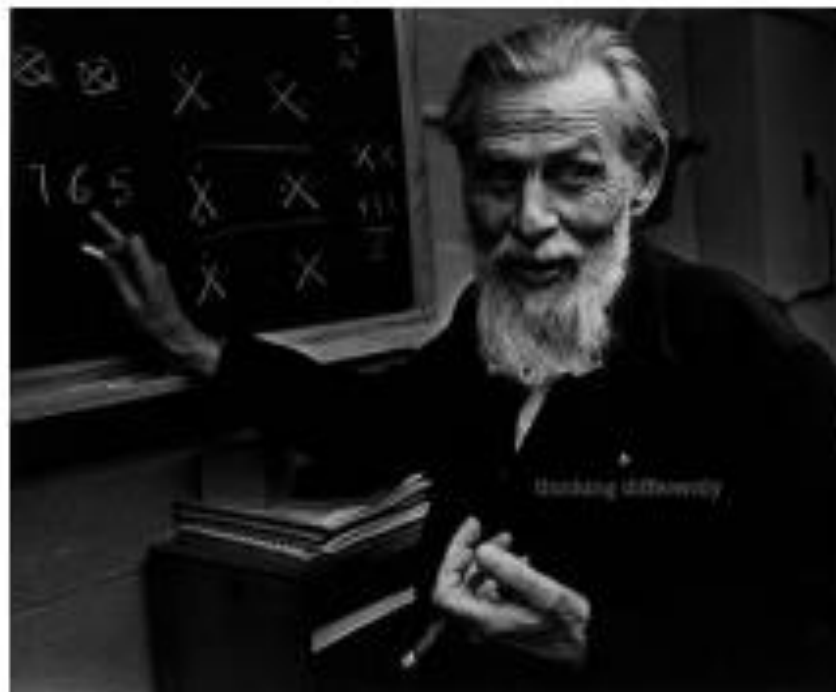
Finite State Automata (FSA) / Finite State Machine (FSM)



Automata Formal Languages & Logic

Introduction

Finite State Automata/ Finite State Machine



Introduced first by two neuro-psychologist Warren S. McCulloch and Walter Pitts in 1943 as a model for human brain!

Finite automata can naturally model microprocessors and even software programs working on variables with bounded domain - capture so-called regular sets of sequences that occur in many different fields (logic, algebra, regex).

Applications in digital circuit/protocol verification, compilers, pattern recognition/matching, etc.

Automata Formal Languages & Logic

Introduction - Formal Languages

Formal Languages:

- In logic, mathematics, computer science, and linguistics, **a formal language consists of words (or strings) whose letters are taken from an finite non-empty alphabet and are well-formed according to a specific set of rules called a formal grammar.**
- Words that belong to a particular formal language are sometimes called *well-formed words* or *well-formed formulas*. **A formal language is often defined by means of a formal grammar such as a regular grammar or context-free grammar, which consists of its formation rules.**

Automata Formal Languages & Logic

Introduction - Formal Languages

Formal Languages:

- Automata theory is closely related to formal language theory. In this context, **automata are used as finite representations of formal languages** that may be infinite.
- Automata(or FSMs) are often classified by the class of formal languages they can recognize, as in the Chomsky hierarchy, which describes a nesting relationship between major classes of automata.
- A formal language is designed for use in situations in which natural language is unsuitable, as for example in mathematics, logic, or computer programming.
The symbols and formulas of such languages stand in precisely specified syntactic and semantic relations to one another.

Automata Formal Languages & Logic

Introduction - Formal Languages

- Formal languages are languages which only consider about the well-formedness nothing else. That means the sufficient and necessary condition is to adhere to its rules.
- **Here are some examples of formal languages:**
 - $L = \Sigma^*$, the set of *all* words over Σ ;
 - $L = \{a\}^* = \{a^n\}$, where n ranges over the natural numbers and " a^n " means " a " repeated n times (this is the set of words consisting only of the symbol " a ");
 - The set of syntactically correct programs in a given programming language (the syntax of which is usually defined by a **context-free grammar**);
 - The set of inputs upon which a certain **Turing machine** halts; or
 - The set of maximal strings of **alphanumeric ASCII** characters on this line, i.e.,
the set {the, set, of, maximal, strings, alphanumeric, ASCII, characters, on, this, line, i, e}.

Automata Formal Languages & Logic

Introduction - Formal Languages

➤ **Formal languages are normally defined in one of three ways**, all of which can be described by automata theory:

1) **Regular expressions** (only for regular languages)

- Regular Expressions are an algebraic way to describe regular languages.

2) **Standard Automata** - that accepts (or recognizes) the language.

3) **A formal grammar system** - that generates the language

- **Regular (Type-3) grammars** - generate the regular languages - recognized by FSM.
- **Context-free (Type-2) grammars** - generate the context-free languages - recognized by PDA.
- **Context-sensitive (Type-1) grammars** - generate context-sensitive languages - recognized by LBA.
- **Recursively enumerable (Type-0 or Unrestricted grammar) grammars** - generate recursively enumerable or Turing-recognizable languages - recognized by TM.

Automata Formal Languages & Logic

Introduction - Formal Languages

- Formal languages are normally defined in one of three ways, all of which can be described by automata theory:

Formal Language $L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

- 1) Regular expressions (only for regular languages)

Regex $\Rightarrow a^*$

- 2) Standard Automata - that accepts (or recognizes) the language



- 3) A formal grammar system

Grammar $\Rightarrow$ $S \rightarrow aS$
 $S \rightarrow \epsilon$

Automata Formal Languages & Logic

Applications

1. Robotics and Control Systems:

FSMs are used to model the behaviour of Robots and Control systems.

2. Compiler Design:

Finite automata are extensively used in Compiler Design for Lexical analysis, which involves tokenizing and scanning the source code of a programming language.

Lexical analyzers employ finite automata to recognize and classify different lexical units such as keywords, identifiers, operators, literals and punctuation symbols.

- Lexical analysis (Finite state automata),
- Syntactical analysis (Pushdown automata),
- Code selection (Tree automata)

Automata Formal Languages & Logic

Applications



3. Pattern Recognition: Finite automata are employed in pattern-matching and recognition tasks. Applications include text processing, string matching, and searching algorithms.

4. Implementing artificially intelligent agents in games, but many games include characters with simple, state-based behaviors that are easily and effectively modeled using state machines.

5. Natural Language Processing: Finite automata are used in natural language processing for tasks such as text tokenization, morphological analysis, and part-of-speech tagging. They help in parsing and understanding the structure of natural language sentences.

Automata Formal Languages & Logic

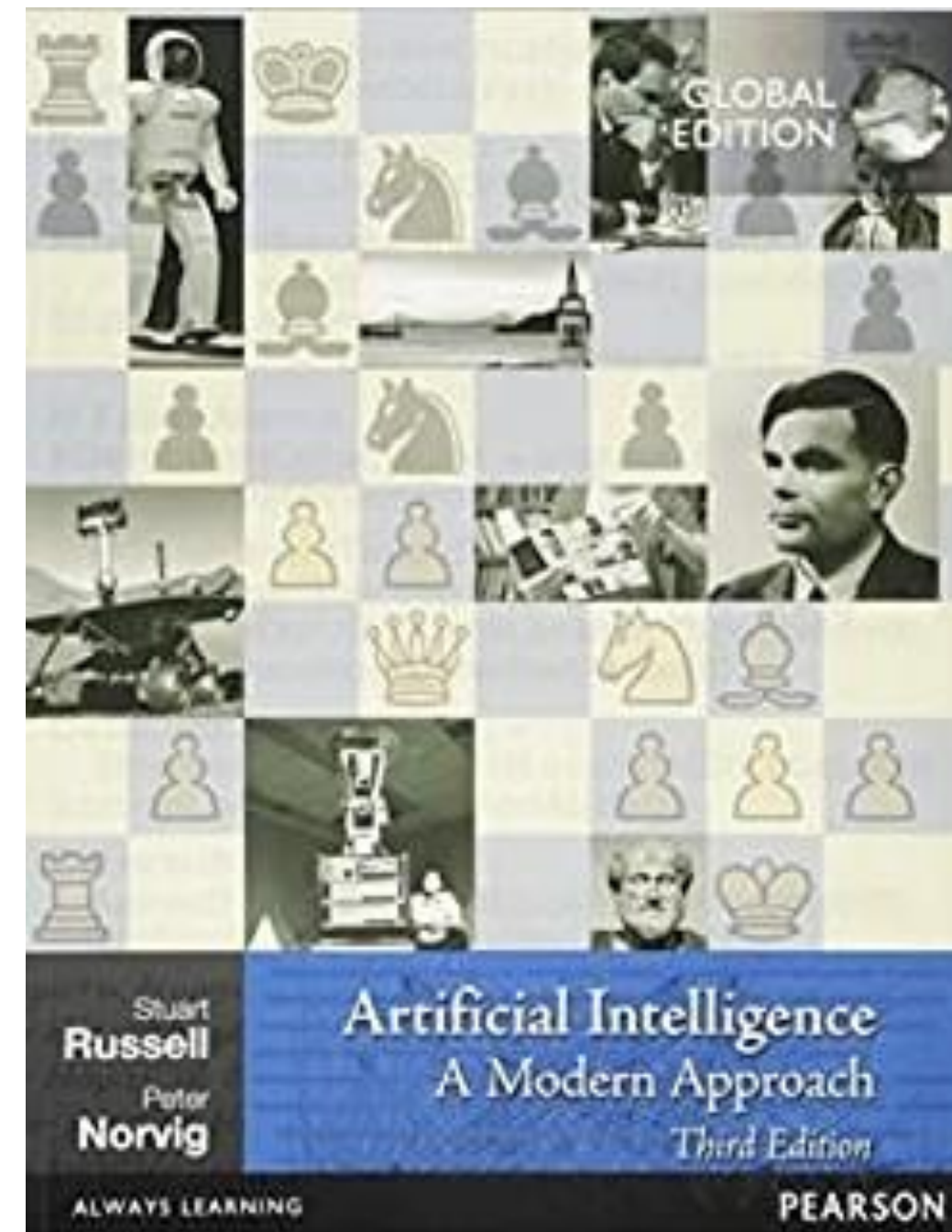
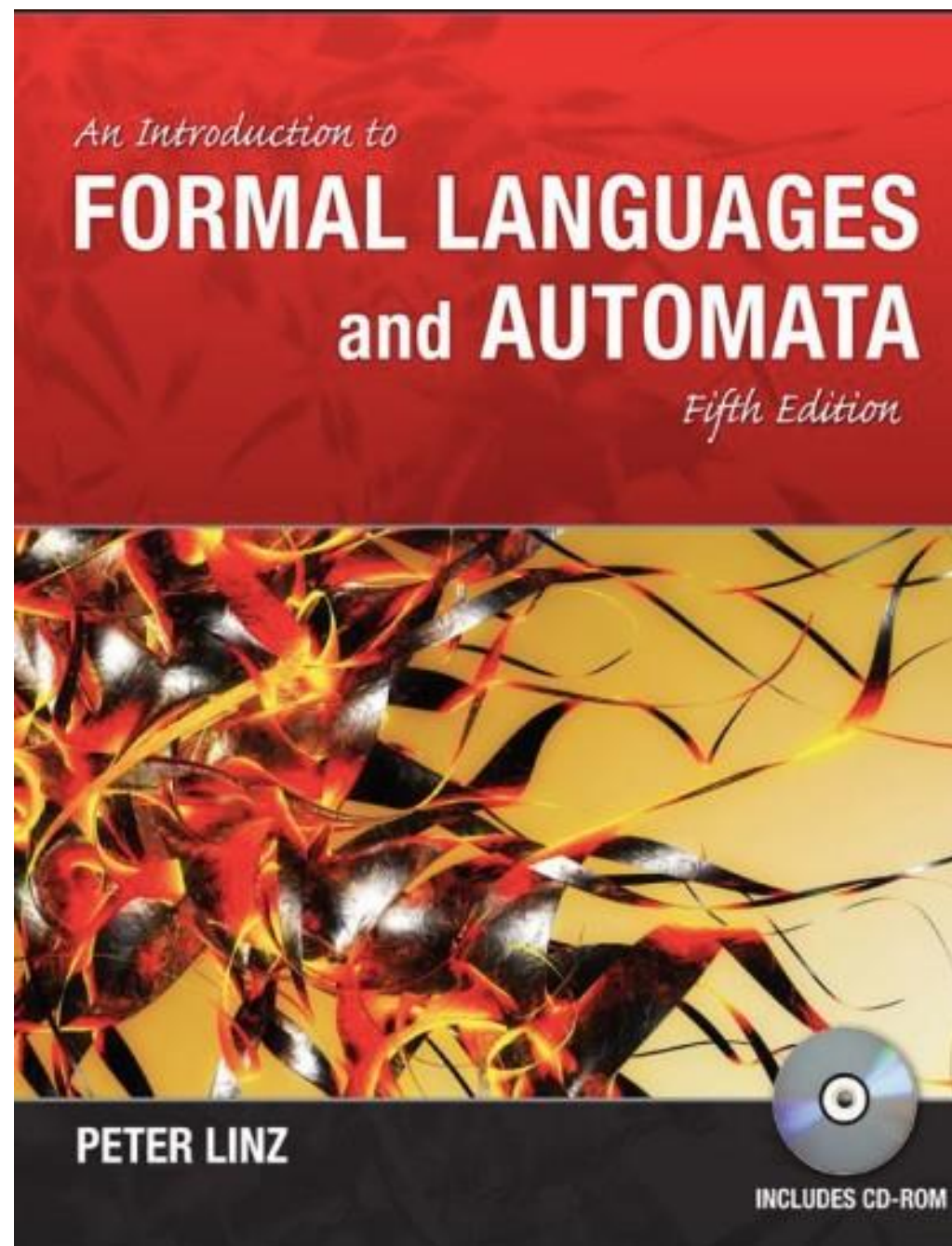
Applications



- . **6. Digital Circuit Design:** Finite automata can model the behavior of digital circuits and aid in their design and testing. Sequential circuits, such as counters and state machines, can be represented and analyzed using finite automata.
- . **7. Network Protocol Design and Analysis:** Network Protocol Design and Analysis: FSMs are employed to model and analyze communication protocols (e.g., [TCP/IP](#), HTTP, ...), They help to ensure correct message sequencing and behaviour.
- . **8. Software Engineering:** State machines are extensively used in software development to model the behaviour of complex systems, such as user interfaces, protocol implementations, and control systems.
- . **9.**

Automata Formal Languages & Logic

Introduction - Course Textbook



Automata Formal Languages & Logic

Introduction - Course Evaluation Policy

Evaluation Policy

Evaluation component	Test Mode	Conducted for Marks	Scaled to Marks
ISA-I	CBT	40	20
ISA-II	CBT	40	20
Experiential Learning	Syntax validation - Coding Assignment in Class using PLY tool: Building a foundation for further semester courses.	6 Marks	10
	Unit-wise Problem-solving Assignment in Class: 5 questions/unit, student should solve in a sheet of paper and submit.	4 Marks (1 Mark/Unit)	
Total (ISA)			50
ESA	Pen and Paper	100	50
Total (ISA+ESA)			100



THANK YOU

Prakash C O

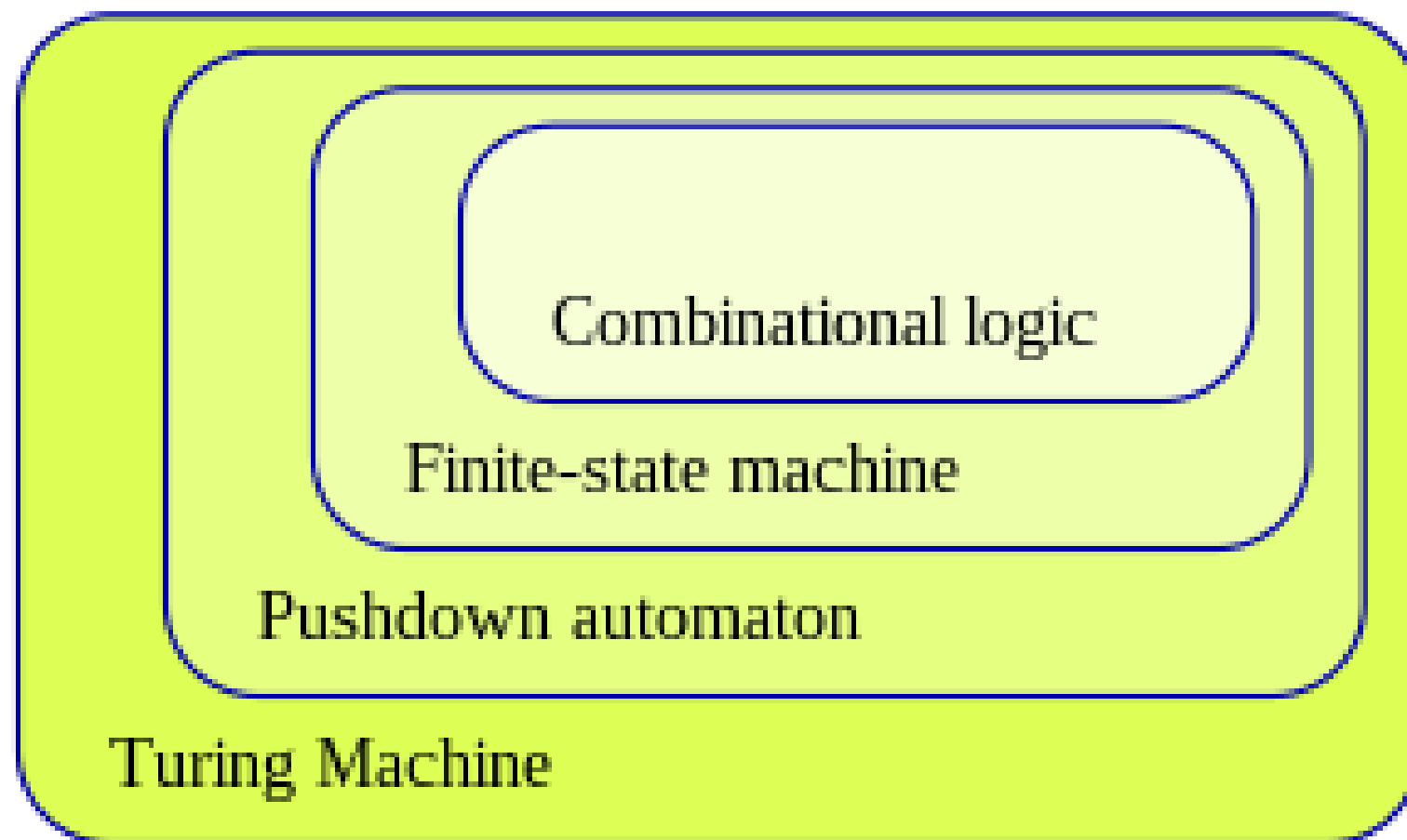
Department of Computer Science & Engineering

coprakasha@pes.edu

Automata Formal Languages & Logic

Introduction - Additional Info

Automata theory



In automata theory, **combinational logic** is a type of digital logic that is implemented by **Boolean circuits**, where the output is a pure function of the present input only. This is in contrast to sequential logic, in which the output depends not only on the present input but also on the history of the input. In other words, sequential logic has *memory* while combinational logic does not.

Automata Formal Languages & Logic

Introduction - Additional Info

ω -automaton (or stream automaton)

- In automata theory, an ω -automaton (or stream automaton) is a variation of a finite automaton that runs on infinite, rather than finite, strings as input.
- An ω -automaton either accepts or rejects infinite inputs. Since ω -automata do not stop, they have a variety of acceptance conditions rather than simply a set of accepting states.
- **Classes of ω -automata** include the Büchi automata, Rabin automata, Streett automata, parity automata and Muller automata, each deterministic or non-deterministic. These classes of ω -automata differ only in terms of acceptance condition.

Automata Formal Languages & Logic

Introduction - Additional Info



Tree Automata:

- Most machine models like Turing machines and finite automata, work on inputs that are strings over some finite alphabet. However, often problem descriptions have inputs that are more structured than strings. For example, the input could be a graph, or a tree, or a partial order, etc.

Automata Formal Languages & Logic

Introduction - Additional Info

Automata and Logic:

- **The connection between automata and logic** goes back to work of Biichi [Bii60] and Elgot [El61], who showed that finite automata and monadic second-order logic (interpreted over finite words) have the same expressive power, and that **the transformations from logic formulas to automata and vice versa are effective.**
- **Mathematical logic and automata theory are two scientific disciplines with a fundamentally close relationship.**

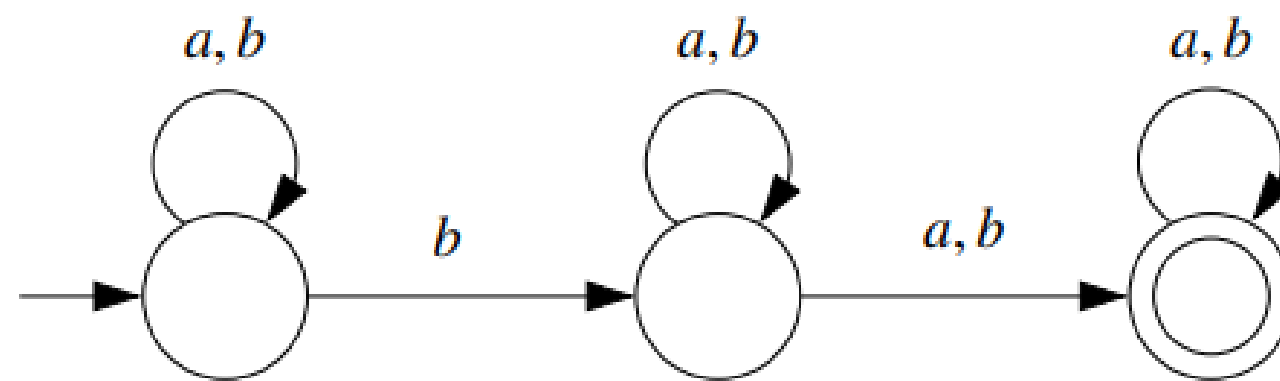
Automata Formal Languages & Logic

Introduction - Additional Info

Automata and Logic:

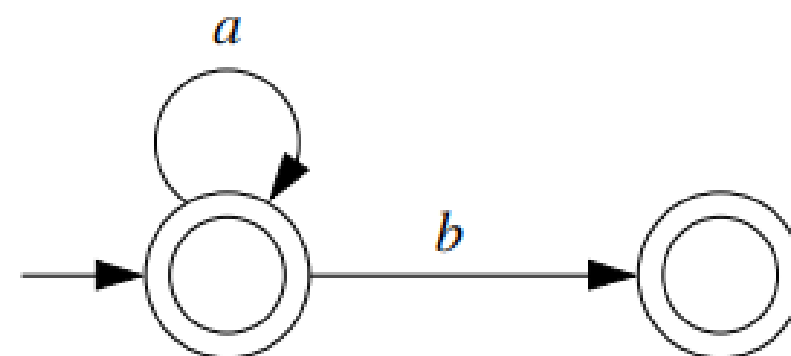
➤ Example:

Intersecting the automata for $\neg last(x)$ and $\neg Q_a(x)$, and subsequently projecting onto the first component, we get an automaton for $\exists x (\neg last(x) \wedge \neg Q_a(x))$



$[\exists x (\neg last(x) \wedge \neg Q_a(x))]$

Determinizing, complementing, and removing a useless state yields the following NFA for $\neg \exists x (\neg last(x) \wedge \neg Q_a(x))$:



$[\neg \exists x (\neg last(x) \wedge \neg Q_a(x))]$

Automata Formal Languages & Logic

Introduction - Additional Info

Applications of FSMs/Automata Theory:

- <https://code.tutsplus.com/finite-state-machines-theory-and-implementation--gamedev-11867t>
- <https://robotika.sk/go/FSM/finite-state-automata.pdf>
- <https://people.computing.clemson.edu/~goddard/texts/theoryOfComputation/5.pdf>
- <https://www.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-applications.pdf>
- <https://yashindiasoni.medium.com/real-world-applications-of-automata-88c7ba254e80>



Automata Formal Languages & Logic

Unit 1 - Basic Notations

Prakash C O

Department of Science & Engineering

Automata Formal Languages & Logic

Unit 1 - Basic Notations

- Alphabet (Σ)
- String or word (w)
- Empty String [ϵ (“epsilon”) or λ (“lambda”)]
- Length of a String ($|w|$)
- Powers of an Alphabet (Σ^k)
- Kleene Closure or Kleene Star or Kleene operator($*$)
- Kleene Plus ($+$)
- Zero or one occurrence ($?$)
- Language (L)

Automata Formal Languages & Logic

Unit 1 - Basic Notations



Alphabet (Σ):

- An alphabet is a finite, non-empty set of symbols.
- We use the symbol Σ (sigma) to denote an alphabet.

➤ Examples:

- Binary: $\Sigma = \{0,1\}$
- All lower case letters: $\Sigma = \{a,b,c,...z\}$
- Alphanumeric: $\Sigma = \{a-z, A-Z, 0-9\}$
- DNA molecule letters: $\Sigma = \{a,c,g,t\}$

...

String or word (w):

- A string or word is a finite sequence of symbols chosen from Σ
- Empty string is ε (“epsilon”) or λ (“lambda”)
- Length of a string w , denoted by “ $|w|$ ”, is equal to the number of (non- ε) characters in the string

E.g., if $w = 010100$ then $|w| = 6$

if $w = 01\varepsilon 0\varepsilon 1\varepsilon 00\varepsilon$ then $|w| = ?$

if $w = \varepsilon$ then $|w| = ?$

- xy = concatenation of two strings x and y

Automata Formal Languages & Logic

Unit 1 - Basic Notations



Powers of an alphabet:

- Let Σ be an alphabet.
- Σ^k = the set of all strings of length k

Example: if $\Sigma = \{a, b\}$ then,

$\Sigma^0 = \{\lambda\}$ or $\{\epsilon\}$ i.e., the set of all strings of length 0

$\Sigma^1 = \{a, b\}$ i.e., the set of all strings of length 1

$\Sigma^2 = \{aa, bb, ab, ba\}$ i.e., the set of all strings of length 2

....

Automata Formal Languages & Logic

Unit 1 - Basic Notations



Kleene star (or Kleene closure): Kleene star is a unary operation, either on sets of strings or on sets of symbols or characters.

The application of the Kleene star to an alphabet Σ is written as Σ^*

If Σ is an alphabet, then Σ^* is the set of all strings over symbols in Σ , including the empty string ϵ .

The definition of Kleene star on Σ is

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

Kleene plus: The Kleene plus omits the Σ^0 term in the above union. In other words, the Kleene plus on Σ is

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots \quad \text{i.e., the set of strings of length } > 0$$

Automata Formal Languages & Logic

Unit 1 - Basic Notations



Language(L): L is said to be a language over alphabet Σ , only if $L \subseteq \Sigma^*$
 $\Rightarrow$ this is because Σ^* is the set of all strings (of all possible length including ϵ) over the given alphabet Σ

Examples:

Let L be *the* language of all strings consisting of n 0's followed by n 1's (i.e., $0^n 1^n$):

$L = \{\epsilon, 01, 0011, 000111, \dots\}$

Let L be *the* language of all strings of with equal number of 0's and 1's:

$L = \{\epsilon, 01, 10, 0011, 1100, 0101, 1010, 1001, \dots\}$

Definition: $\emptyset$ denotes the Empty language, Let $L = \{\epsilon\}$; Is $L = \emptyset$?

Automata Formal Languages & Logic

Unit 1 - Basic Notations



Finite or Infinite Languages:

Finite Language has a finite set of words/strings.

$L = \text{set of string over } \Sigma = \{0, 1\} \text{ such that } |w| = 2$

$L = \{00, 01, 10, 11\}$

Infinite Language has a infinite set of words/strings.

$L = \text{set of string over } \Sigma = \{0, 1\} \text{ such that } |w| > 2$

$L = \{000, 101, 010, 011, 100, 110, 001, 111, 1100, \dots\}$

Automata Formal Languages & Logic

Unit 1 - Basic Notations



The Membership Problem

Given a string $w \in \Sigma^*$ and a language L over Σ , decide whether $w \in L$ or not.

Example:

Let $w = 100011$ and $\Sigma = \{0,1\}$

Q) Is $w \in$ the language of strings with equal number of 0s and 1s?



THANK YOU

Prakash C O

Department of Computer Science and Engineering

coprakasha@pes.edu

+91 98 8059 1946



Automata Formal Languages & Logic

Unit 1 – DFA

Preet Kanwal and Prakash C O

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1: Deterministic Finite Automata (DFA)

Preet Kanwal and Prakash C O

Department of Computer Science & Engineering

Formal Definition of a DFA

A DFA can be represented by a 5-tuple $M=(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states.
- Σ is a finite non-empty set of input symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states ($F \subseteq Q$).

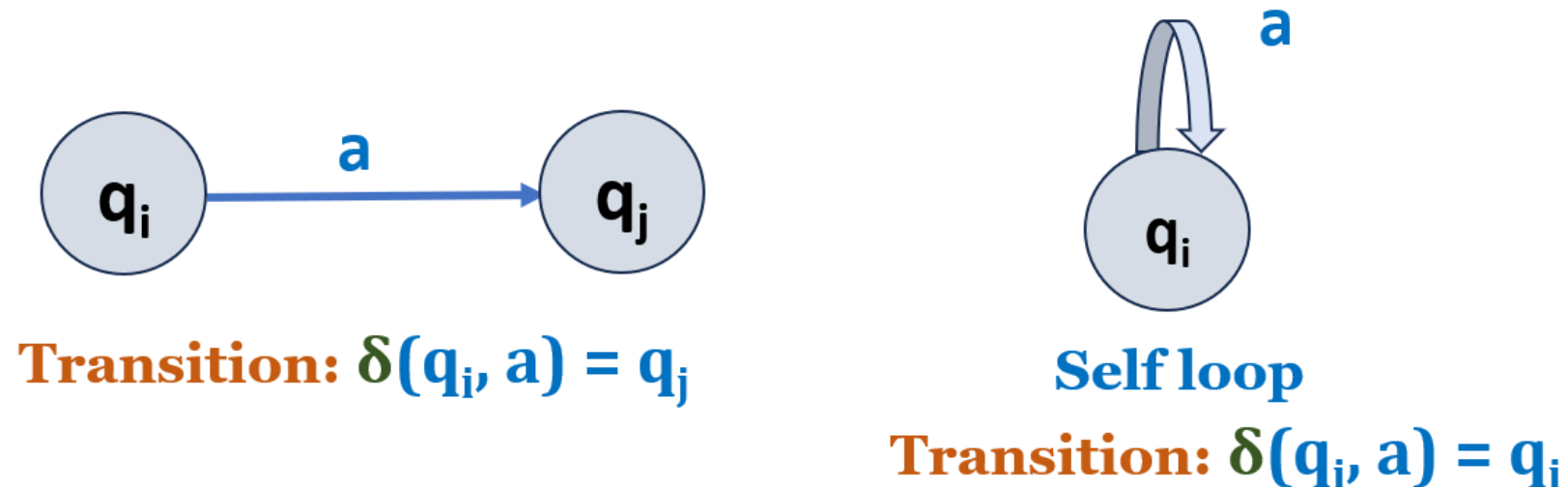
Machine M can be defined in 3 different ways:

1. Transition Diagram or State Transition Diagram or State Diagram
2. Transition Table
3. Transition Function

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

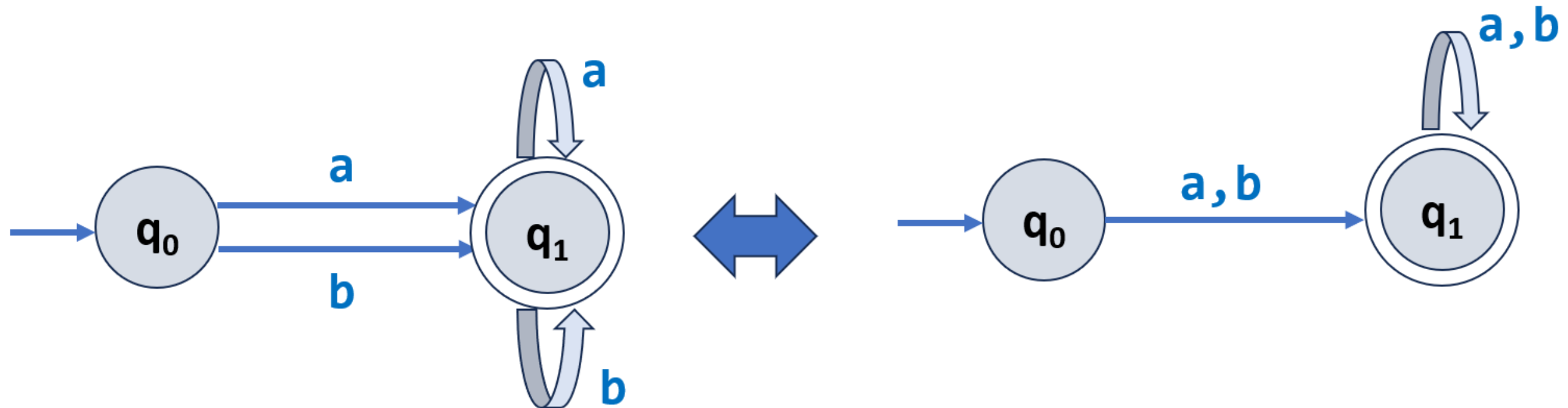
- Each finite automaton consists of a set of *states* connected by *transitions*.
- Symbols used in Constructing a Finite Automata as Transition Diagram:



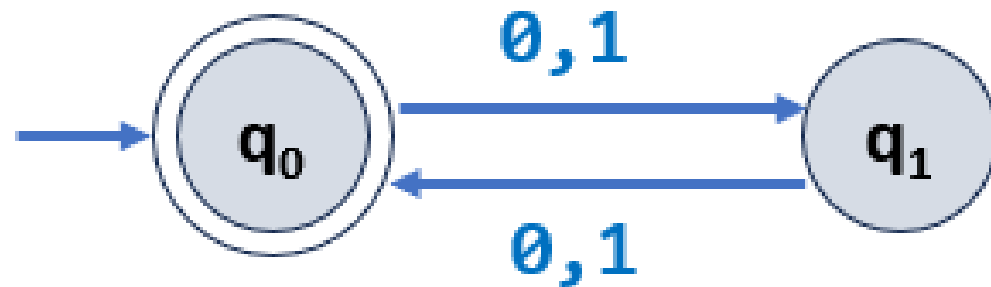
Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata

➤ The below Transition Diagrams are equivalent (or same):



DFA:



The automaton in the above figure can be represented as $(Q, \Sigma, \delta, q_0, F)$ where

- $Q = \{q_0, q_1\}$,
- $\Sigma = \{0, 1\}$,
- $q_0 = q_0$,
- $F = \{q_0\}$, and
- **Transition function δ** is such that

$$\delta(q_0, 0) = q_1, \delta(q_0, 1) = q_1, \text{ and } \delta(q_1, 0) = q_0, \delta(q_1, 1) = q_0.$$

Let's draw the state diagram of the following automaton $(Q, \Sigma, \delta, q_o, F)$

$$Q = \{q_1, q_2, q_3\}$$

$$\Sigma = \{0,1\},$$

δ is given in a tabular form below (i.e., **transition table**):

δ	0	1
$\rightarrow q_1$	q_1	q_2
q_2^*	q_3	q_2
q_3	q_2	q_2

q_1 is the initial state, and $F = \{q_2\}$.

What does it accept?

- In automata theory, a finite-state machine is called a deterministic finite automaton (DFA),
 1. If each of its transitions is uniquely determined by its source state and input symbol, and
 2. For every state, a transition is required for all the input symbols in Σ .

The Transition Function of a DFA is $\delta: Q \times \Sigma \rightarrow Q$

- In DFA, for each state there must be exactly one transition defined for each symbol in Σ . This is the “deterministic” part of DFA.

or

- In DFA, the transition for every state and input symbol is uniquely defined, there is no ambiguity or choice in the transitions.

or

- DFA is a finite-state machine that accepts or rejects a given string of symbols, by running through a state sequence uniquely determined by the string. *Deterministic* refers to the uniqueness of the computation run.

The Word (or string) acceptance by FA / FSA / FSM :

- A finite automaton does not accept as soon as it enters an accepting state.
- A finite automaton accepts if its run ends in an accepting state.

The Language acceptance by FA / FSA / FSM :

- The language accepted by a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is the set of all strings on Σ accepted by M .
- So, for an automaton to accept a particular language (e.g., $\{a^n b^m \mid n, m \geq 1\}$), it must not only accept every string in the language, but also reject every string not in the language.

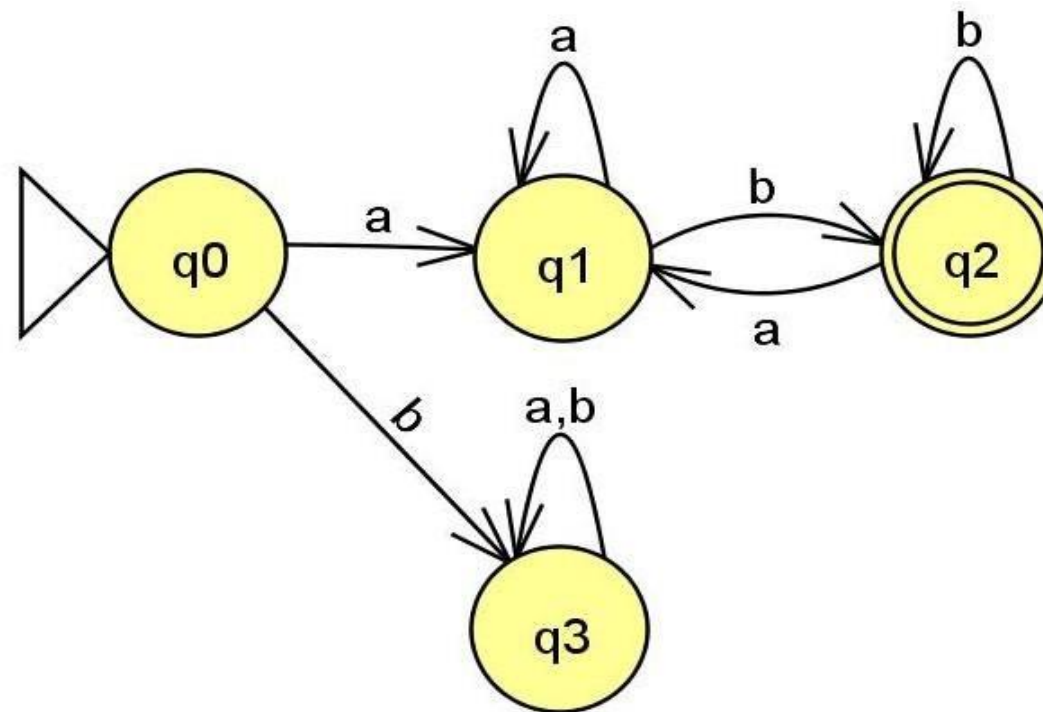
Computation or run of a DFA

- For an input string, DFA will create an unique ‘computation sequence’.
- A DFA accepts a string if run (or computation) leads to some accept state.

The Language of a DFA M is $L(M) = \{w \mid \text{run of } M \text{ on } w \text{ is accepting}\}.$

Computation or run of a DFA

➤ **Example:** Given $w = \text{aababb}$ and Machine M



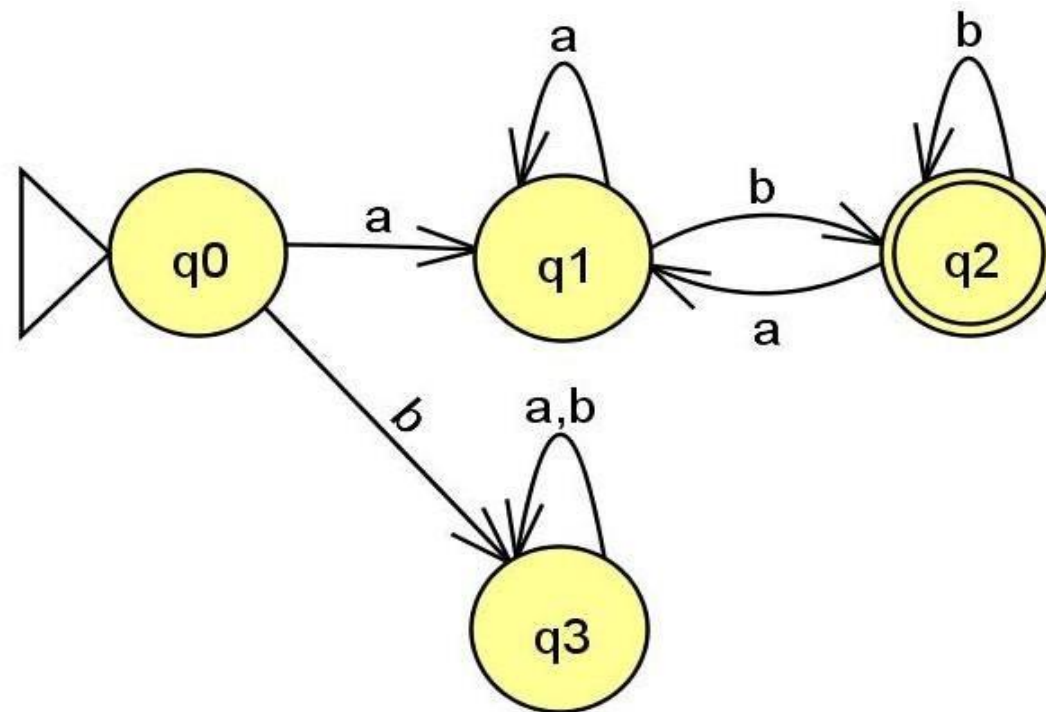
A run of M on w is

$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{b} q_2$

Accepted run/Computation because $q_2 \in F$

Computation or run of a DFA

➤ **Example:** Given $w = \text{aabbba}$ and Machine M



A run of M on w is

$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{b} q_2 \xrightarrow{b} q_2 \xrightarrow{a} q_1$

Rejected run/Computation because $q_1 \notin F$

Approach to Construct a DFA that recognises a language L:

- **Step-1:** Try to enumerate all the Strings in the language i.e.,
 - * Specify the minimal string and then the next minimal strings, ...
 - * Enumerate the strings in the order of increasing length
 - * If possible discover a pattern in the enumerated strings
- **Step-2:** Draw a DFA skeleton of the automata based on the minimal string or the pattern discovered.
- **Step-3:** Complete the DFA

Example 1:

Construct DFA for the Language of strings of length 2 , over $\Sigma = \{a,b\}$.

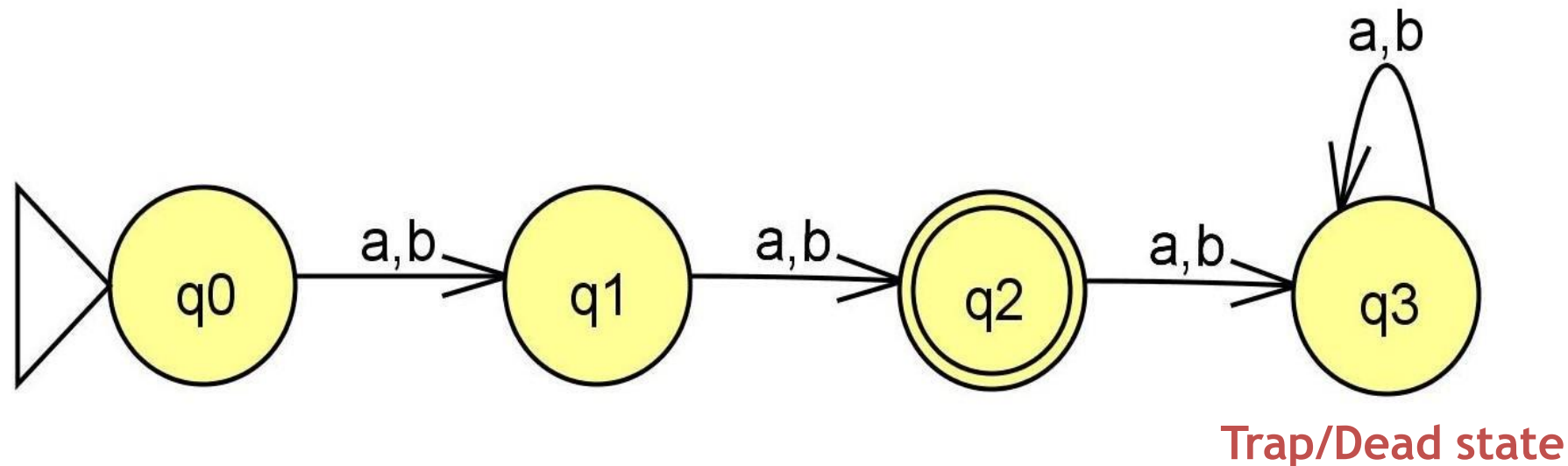
Solution :

Note Transition diagram is a directed graph.

Example 1:

Construct DFA for the Language of strings of length 2 , over $\Sigma = \{a,b\}$.

Solution : $L = \{aa, bb, ab, ba\}$



Note1: strings of length 1 are rejected in q_1 and strings of length >2 are rejected in q_3

Note2: For every state, transition is required for all the input symbols in Σ

Example 1a:

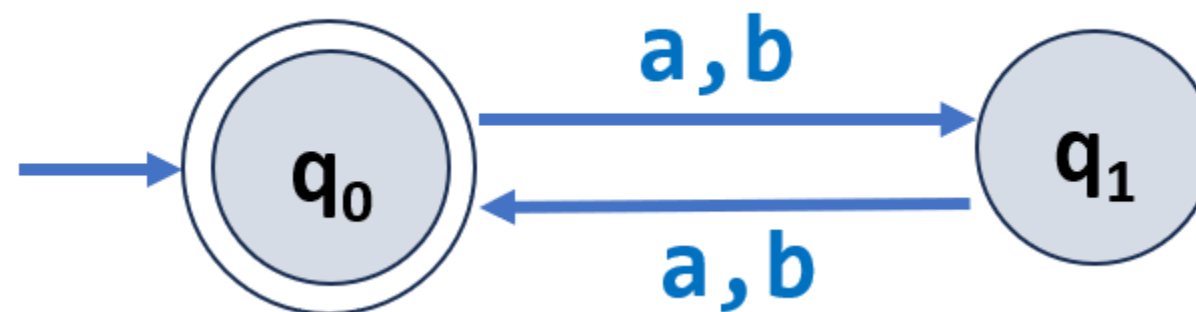
Construct DFA for the Language of strings of even length, over $\Sigma = \{a,b\}$.

Solution :

Example 1a:

Construct DFA for the Language of strings of even length, over $\Sigma = \{a, b\}$.

Solution : $L = \{\epsilon, aa, bb, ab, ba, aaaa, bbbb, abbb, \dots\}$



Example 1b:

Construct DFA for the Language of strings having even number of a's, over $\Sigma = \{a, b\}$.

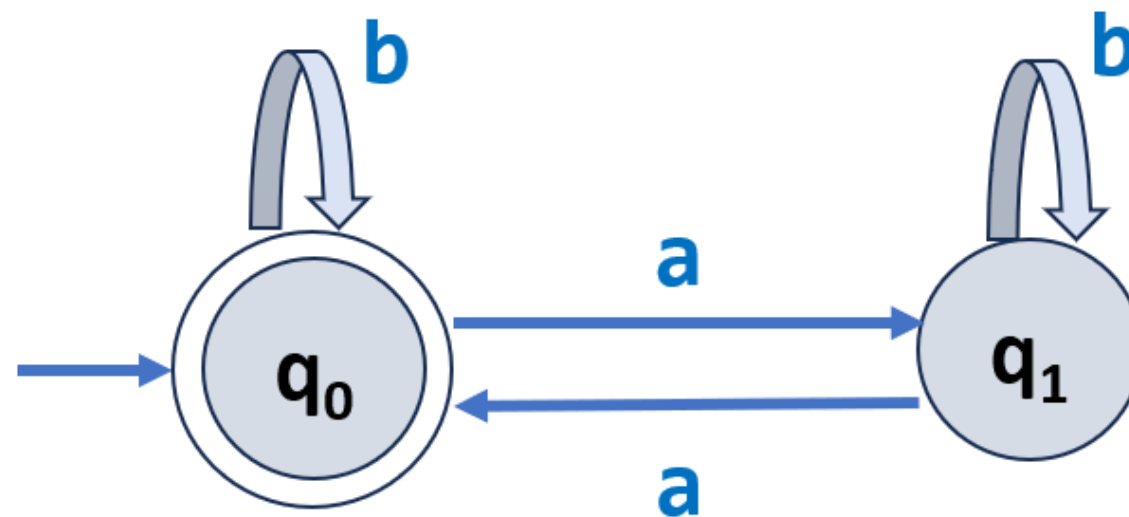
Solution :

Example 1b:

Construct DFA for the Language of strings having even number of a's, over $\Sigma = \{a, b\}$.

Solution :

$L = \{\epsilon, aa, aaaa, aaaaaa, aaaaaaaa, \dots, b, bb, bbb, bbbb, bbbbbb, \dots$
 $\quad aab, baa, aba, \dots \dots \}$



Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Exercise:

Construct DFA for the Language of strings having odd number of a's, over $\Sigma = \{a, b\}$.

Solution :

Example 2:

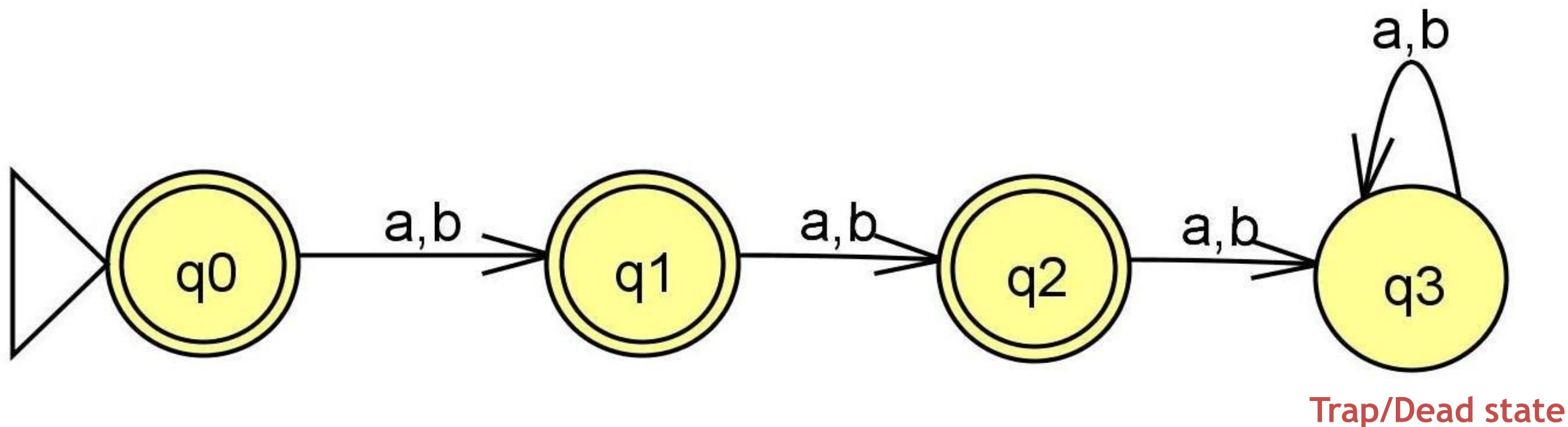
Construct DFA for the language $L = \{ w \mid w \in \{a, b\}^* \text{ and } |w| \leq 2 \}$

Solution :

Example 2:

Construct DFA for the language $L = \{ w \mid w \in \{a, b\}^* \text{ and } |w| \leq 2 \}$

Solution : $L = \{\epsilon, a, b, aa, bb, ab, ba\}$



Trap state is one from which we cannot escape, that is, it has either no transitions defined or transitions for all symbols goes to itself.

Example 3:

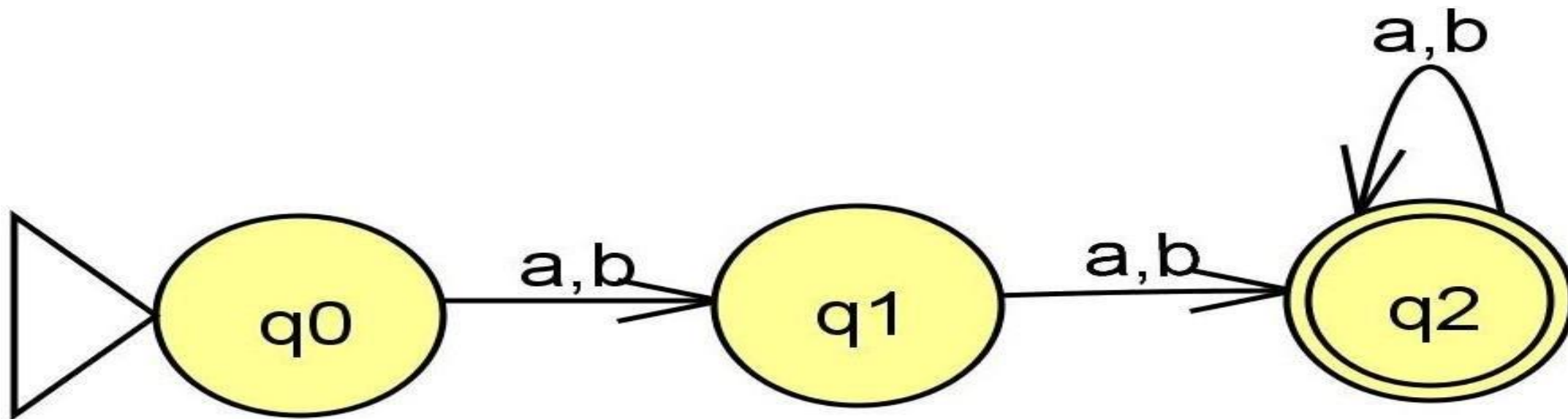
Construct DFA for the language $L = \{ w \mid w \in \{a, b\}^* \text{ and } |w| \geq 2 \}$

Solution :

Example 3:

Construct DFA for the language $L = \{ w \mid w \in \{a, b\}^* \text{ and } |w| \geq 2 \}$

Solution : $L = \{ aa, bb, ab, ba, aaa, aba, aab, \dots \}$



Example 4:

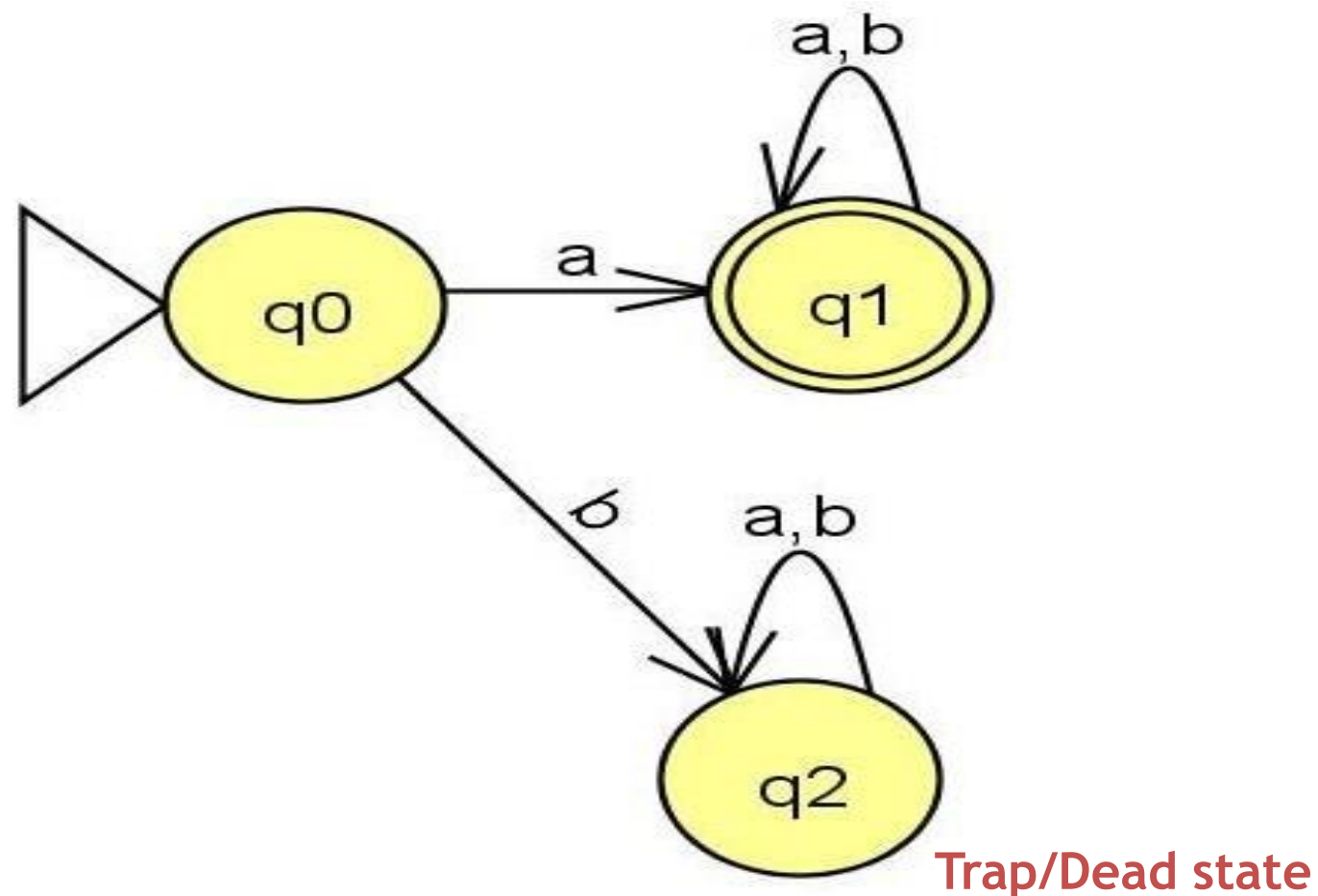
Construct DFA for the language of strings starts with a, over $\Sigma=\{a,b\}$.

Solution :

Example 4:

Construct DFA for the language of strings starts with a, over $\Sigma=\{a,b\}$.

Solution : $L = \{ a, aa, aaa, \dots, ab, abb, \dots, aba, aab, \dots \}$



Example 5:

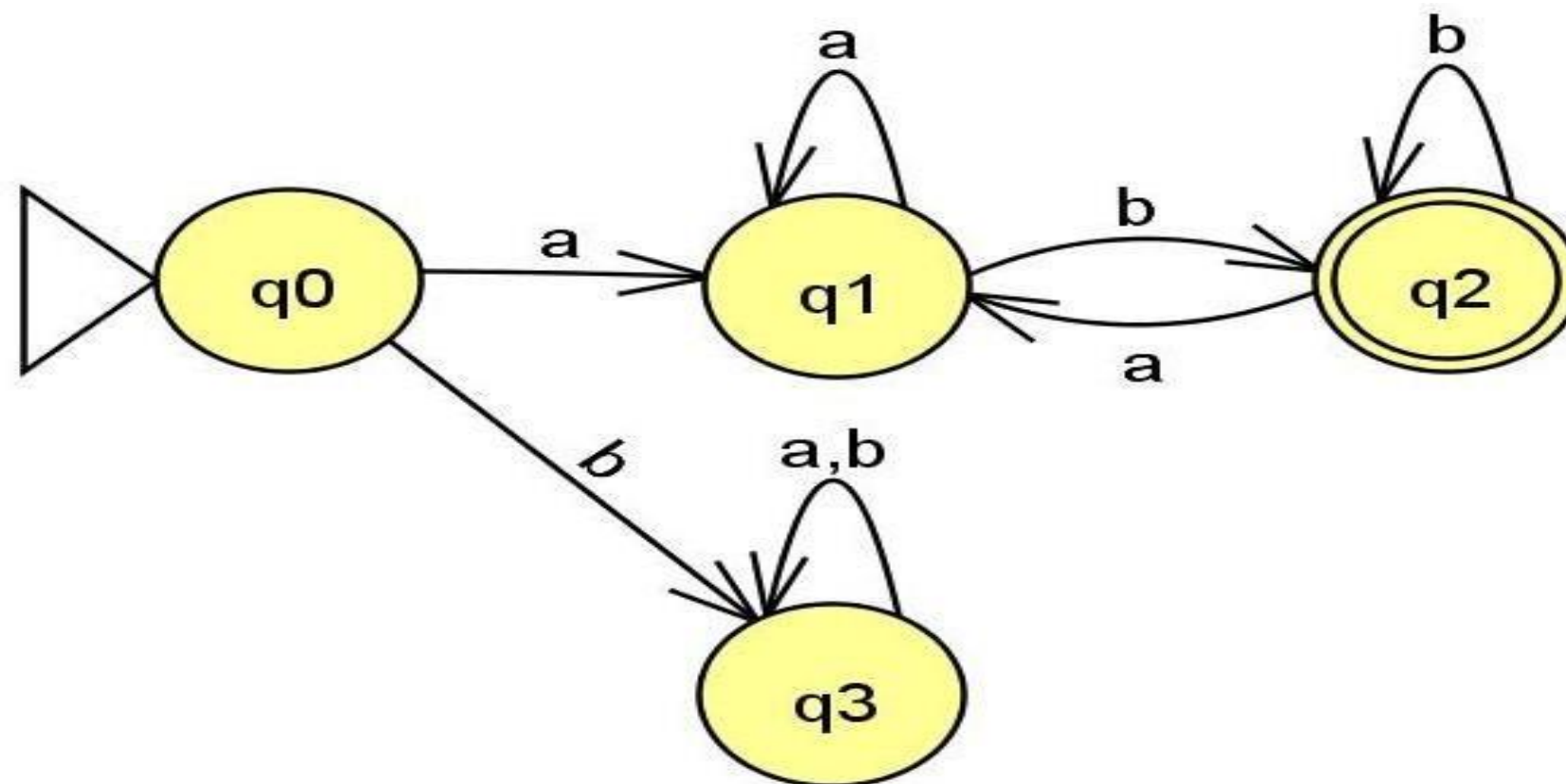
Construct DFA for the language of strings starts with a and ends with b, over $\Sigma = \{a, b\}$.

Solution:

Example 5:

Construct DFA for the language of strings starts with a and ends with b, over $\Sigma = \{a, b\}$.

Solution: $L = \{ ab, aab, abb, abab, aaab, abbb, aabb, \dots \}$



Trap/Dead state

Example 6:

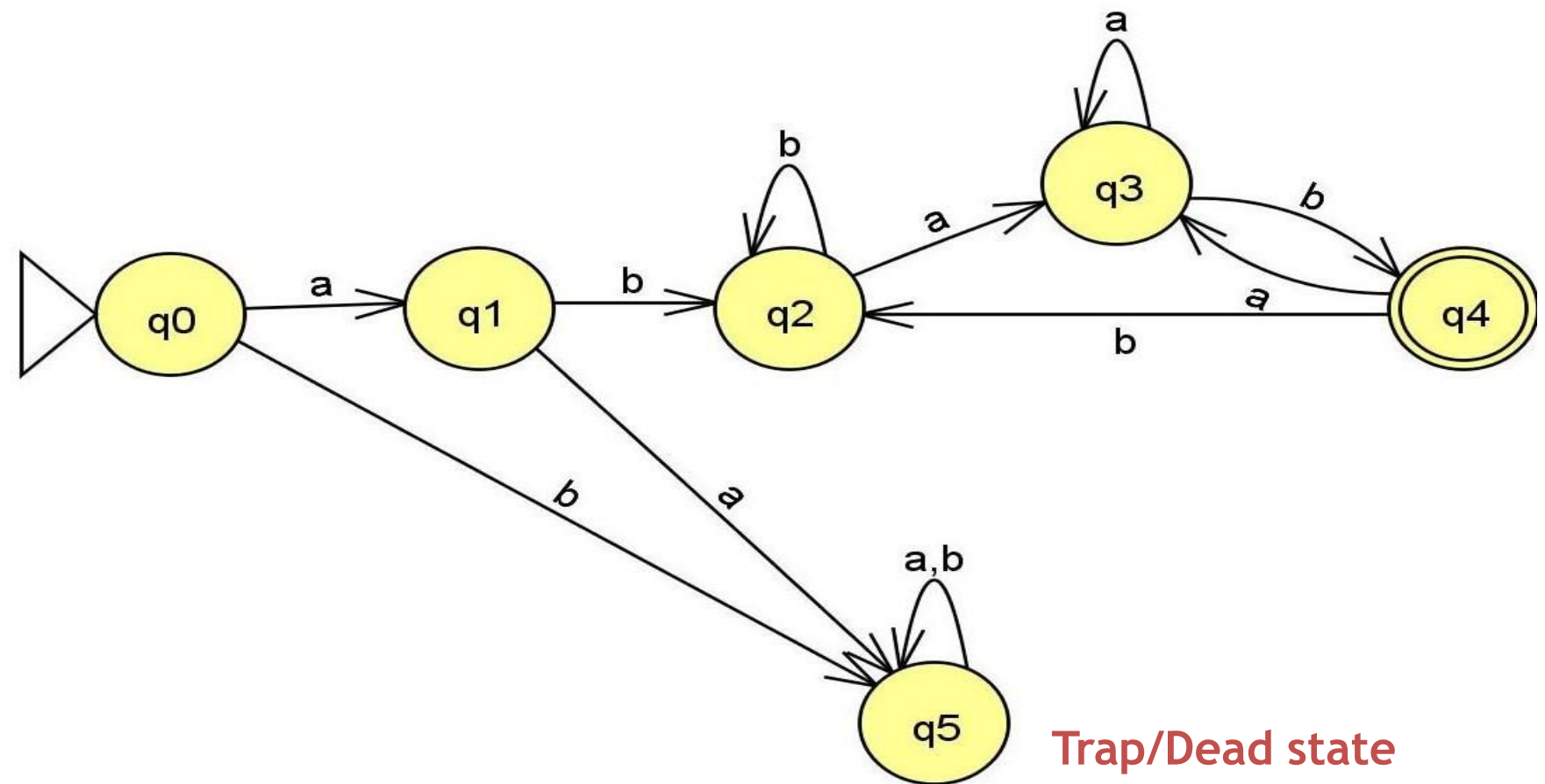
The language with strings over $\Sigma = \{a,b\}$ where every string starts with ab and ends with ab, over $\Sigma = \{a,b\}$.

Solution: $L = \{abab, abaab, abbab, \dots\}$ If minimal string in the language is abab then DFA is

Example 6:

The language with strings over $\Sigma = \{a,b\}$ where every string starts with ab and ends with ab , over $\Sigma = \{a,b\}$.

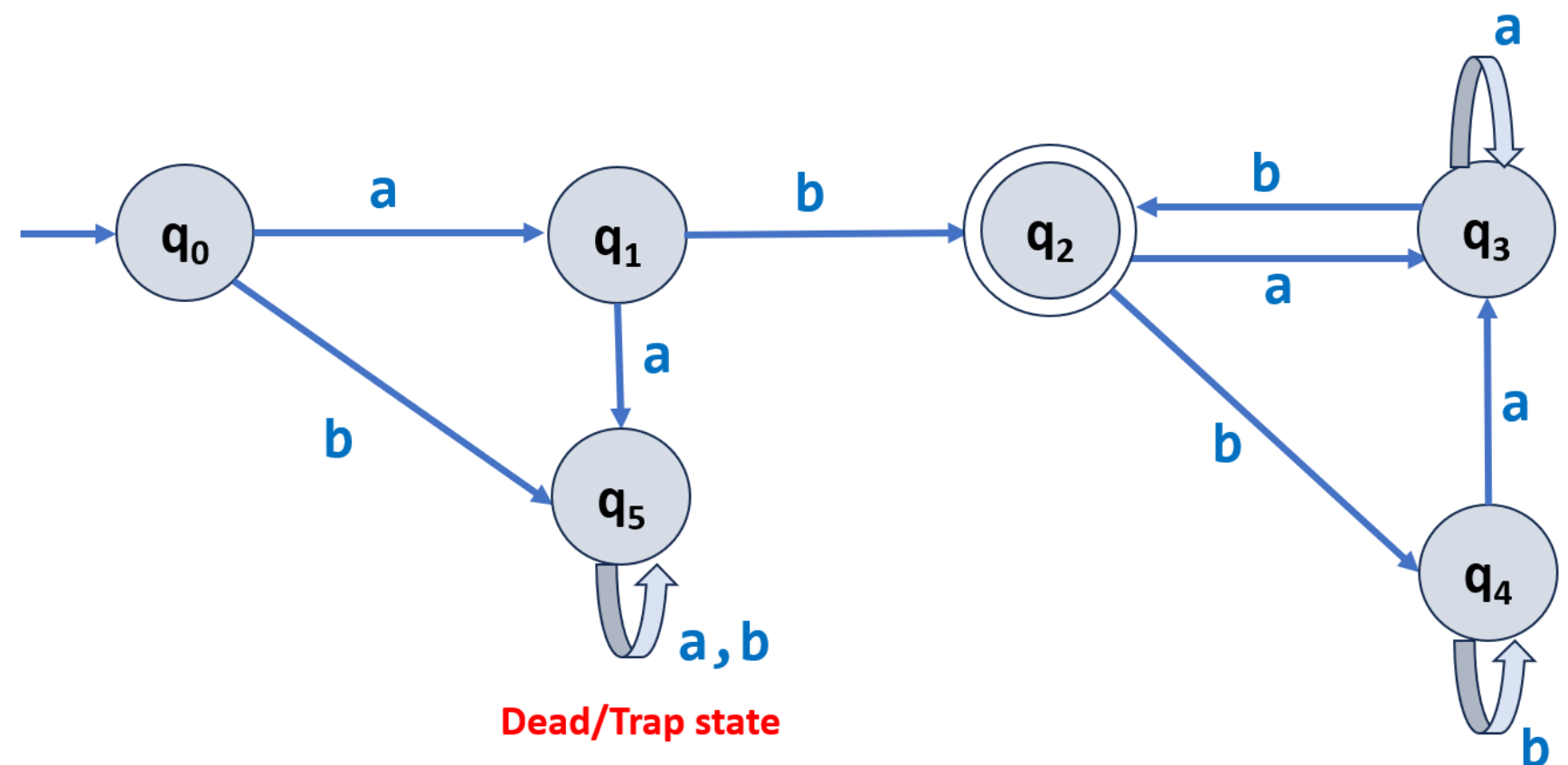
Solution: $L = \{abab, abaab, abbab, \dots\}$ If minimal string in the language is $abab$ then DFA is



Example 6a:

The language with strings over $\Sigma = \{a,b\}$ where every string starts with ab and ends with ab , over $\Sigma = \{a,b\}$.

Solution: $L = \{ab, abab, abaab, abbab, \dots\}$ If minimal string in the language is ab then DFA is



Example 7:

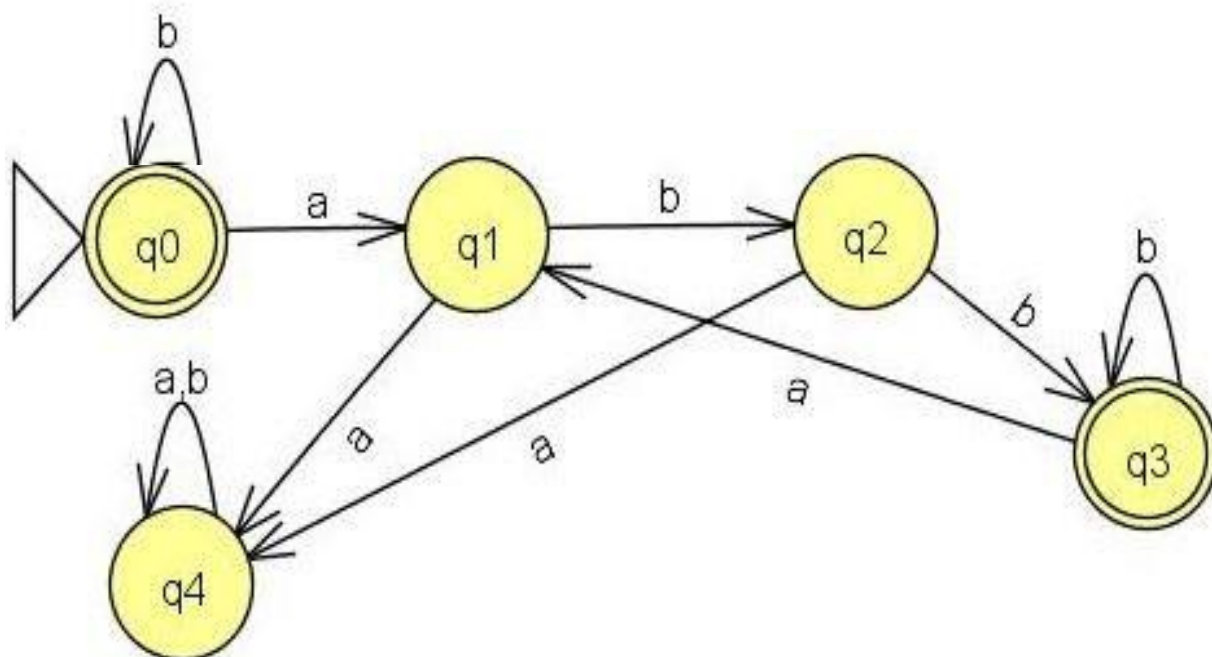
The language of strings over $\Sigma = \{a, b\}$, where every a in the string is followed by bb .

Solution :

Example 7:

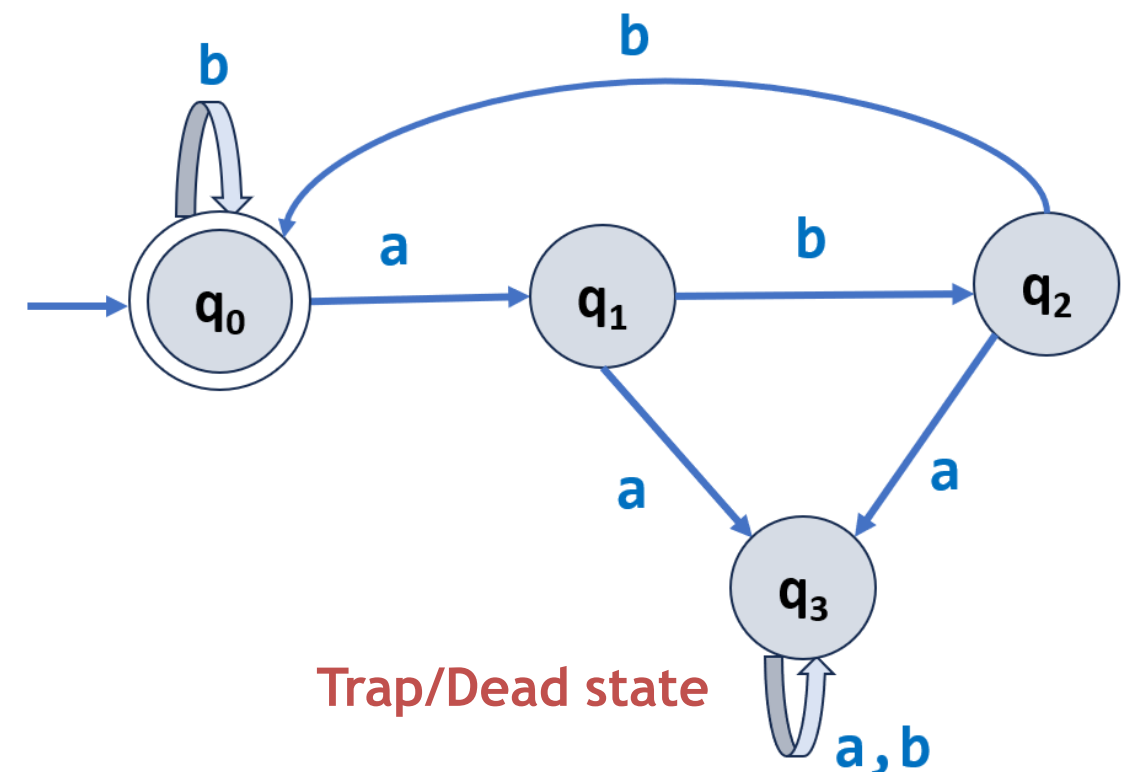
The language of strings over $\Sigma = \{a, b\}$, where every a in the string is followed by bb .

Solution : $L = \{\epsilon, b, bb, bbb, \dots, abb, babb, bbabb, babbb, \dots\}$



Trap/Dead state

or



Trap/Dead state

Example 7a:

The language of strings over $\Sigma = \{a, b\}$, where **every** **a** in the string is **never** followed by **bb**.

Solution :

Example 8:

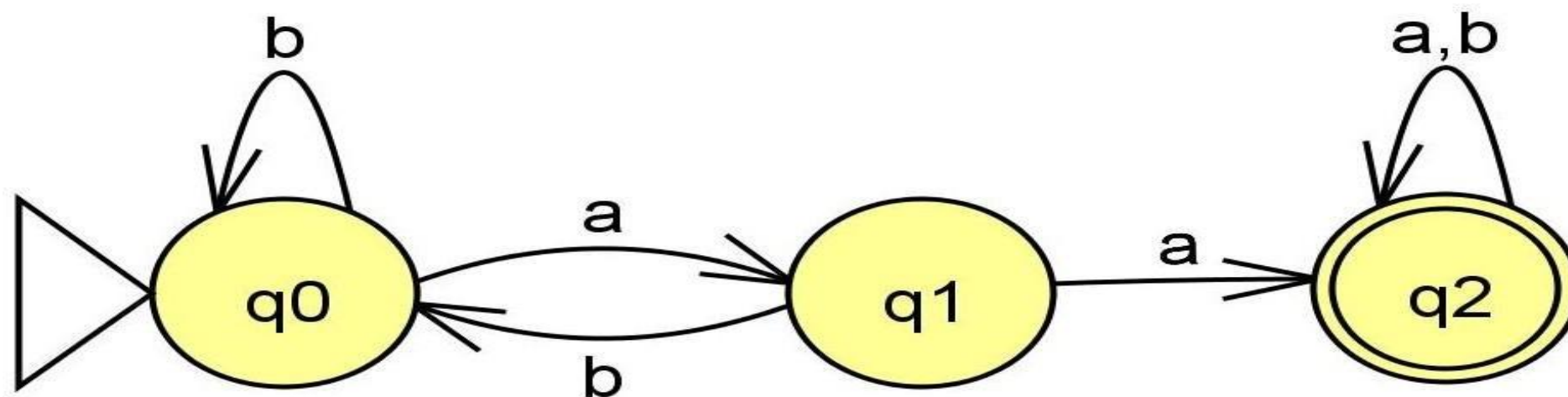
The language of strings over $\Sigma = \{a,b\}$ where every string must contain “aa” as the substring.

Solution :

Example 8:

The language of strings over $\Sigma = \{a, b\}$ where every string must contain “aa” as the substring.

Solution : $L = \{aa, aaa, aaaa, \dots, baa, aab, aabb, bbaa, baab, \dots\}$



Example 9:

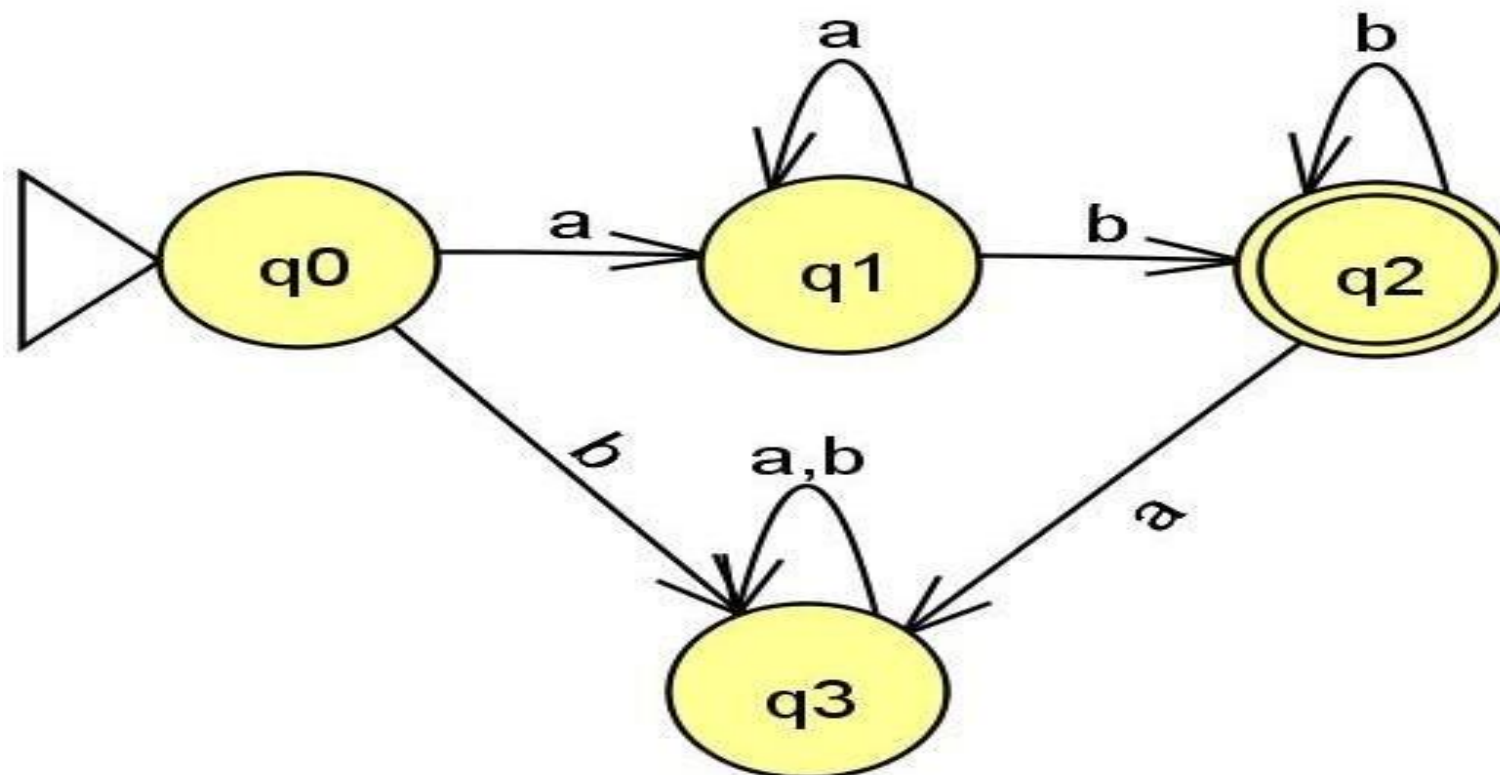
Construct DFA for the language of strings over $\Sigma = \{a,b\}$ of the form $a^n b^m$ where $n, m \geq 1$.

Solution:

Example 9:

Construct DFA for the language of strings over $\Sigma = \{a,b\}$ of the form $a^n b^m$ where $n, m \geq 1$.

Solution : $L = \{ ab, abb, aab, aabb, abbb, aaab, aaabbb, \dots \}$



Trap/Dead state

Example 10:

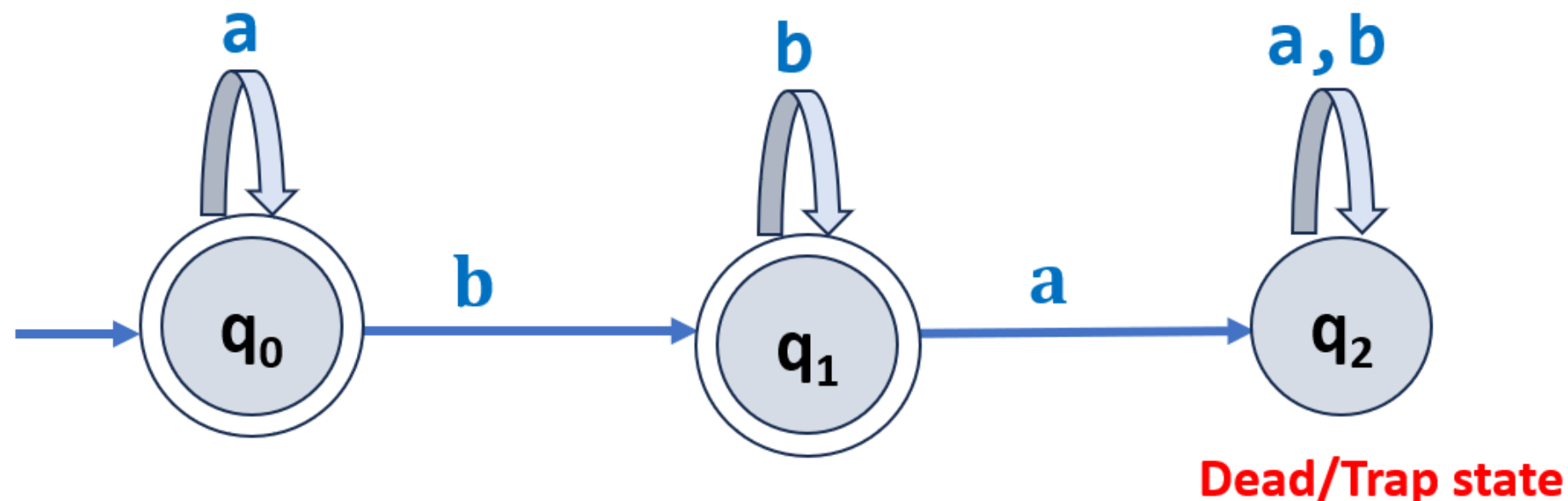
Construct DFA for the language of strings over $\Sigma = \{a,b\}$ of the form $a^n b^m$ where $n, m \geq 0$.

Solution :

Example 10:

Construct DFA for the language of strings over $\Sigma = \{a,b\}$ of the form $a^n b^m$ where $n, m \geq 0$.

Solution : $L = \{\epsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, ab, abb, aab, aabb, abbb, aaab, aaabbb, \dots\}$



Example 11:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where $n_a(w) \bmod 2 = 0$.

Solution : $L = \{\epsilon, aa, aaaa, aaaaaa, \dots, b, bb, bbb, \dots, aab, aba, baa, \dots\}$

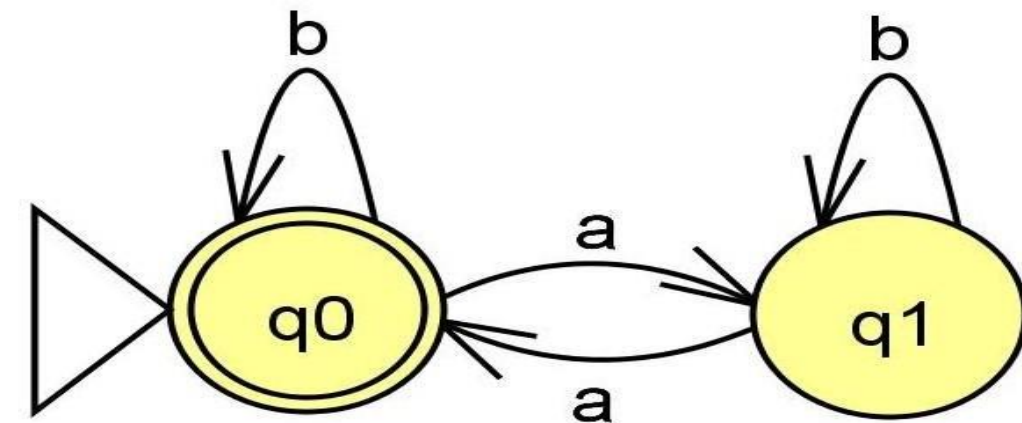
Example 11:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where $n_a(w) \bmod 2 = 0$.

Solution : $L = \{\epsilon, aa, aaaa, aaaaaa, \dots, b, bb, bbb, \dots, aab, aba, baa, \dots\}$

Explanation:

- In the DFA the states should represent the information about the string processed so far.
- In this example, states have to represent only the number of a's seen so far is even or odd, so only need two states q_0 and q_1 . One state to represent “even” and another state to represent “odd”.
- Before reading any symbol, the number of a's processed so far is 0, which is even. Hence, the initial state q_0 should represent even. We want the DFA to accept the strings in L (strings with even number of a's). Hence, we should choose q_0 to also represent our accepting state.



Example 12:

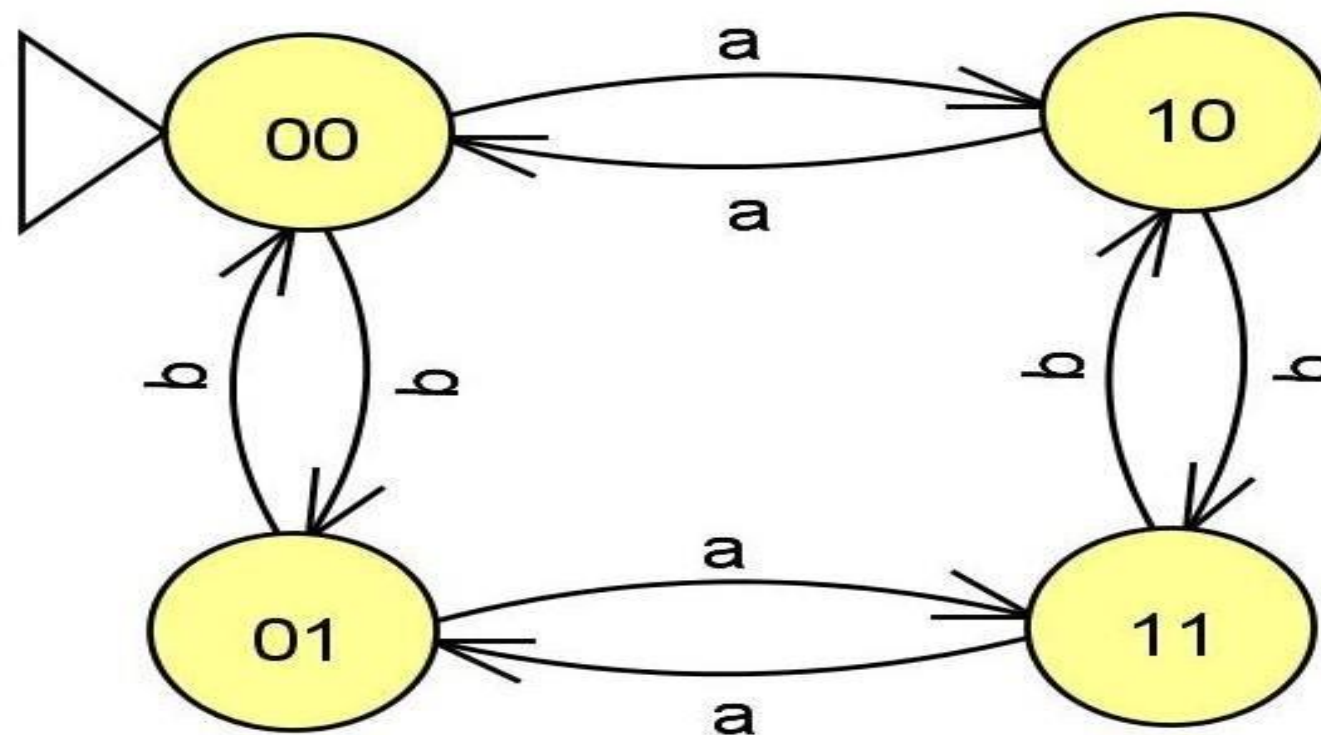
Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 2 = 0$ and $n_b(w) \bmod 2 = 0$

Solution:

Example 12:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 2 = 0$ and $n_b(w) \bmod 2 = 0$

Solution:

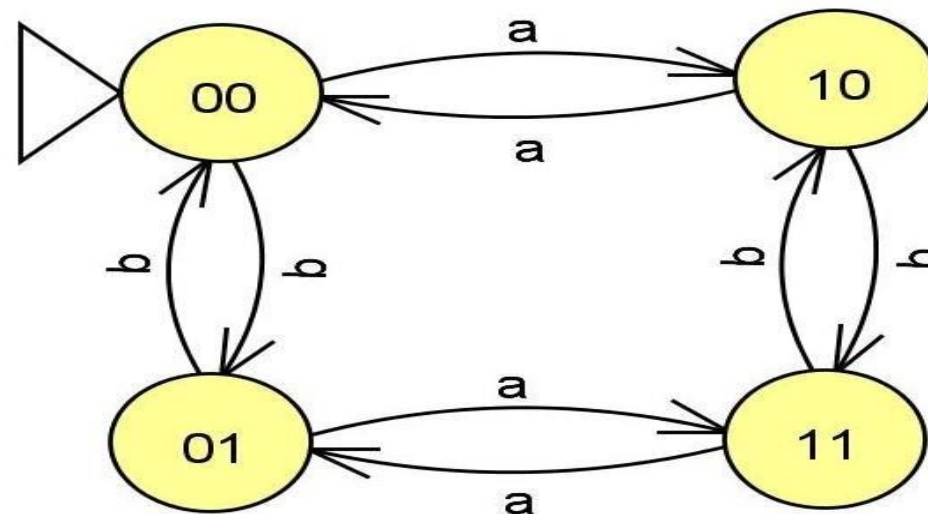


This state represents, seen so far odd number of a's and even number of b's.

Note: State 00 is the final state.

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Note:

- **State 00 is final state** for strings having **Even number of a's** and **Even number of b's**. (i.e., $n_a(w) \bmod 2 = 0$ and $n_b(w) \bmod 2 = 0$)

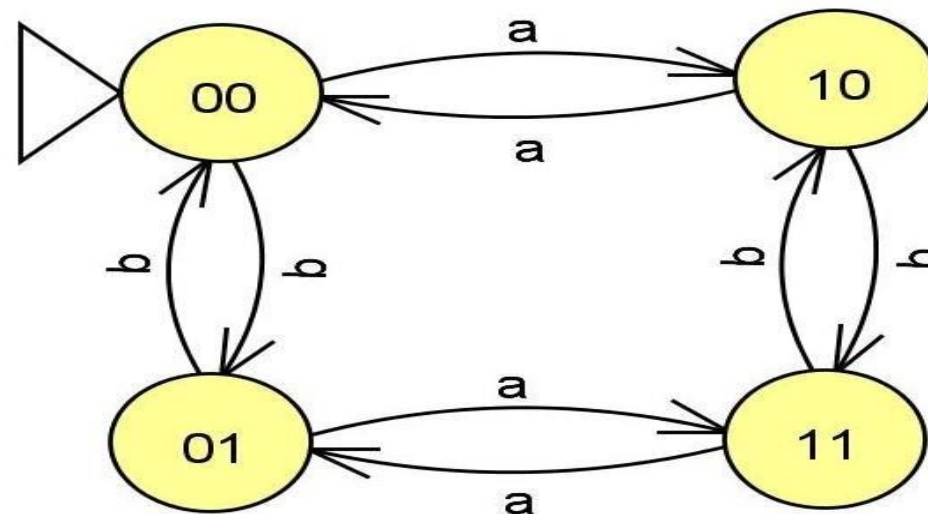
$L = \{ \epsilon, aa, aaaa, aaaaaa, \dots,$
bb, bbbb, bbbbbb, ...,
aabb, bbaa, abab, baba, abba, baab, ... }

- **State 10 is final state** for strings having **odd number of a's** and **Even number of b's**. (i.e., $n_a(w) \bmod 2 = 1$ and $n_b(w) \bmod 2 = 0$)

$L = \{ a, aaa, aaaaa, aaaaaa, \dots,$
abb, bab, bba, abbbb, babbb, ...,
aaabb, bbaaa, ... }

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Note:

- **State 01 is final state** for strings having **Even number of a's** and **odd number of b's**. (i.e., $n_a(w) \bmod 2 = 0$ and $n_b(w) \bmod 2 = 1$)

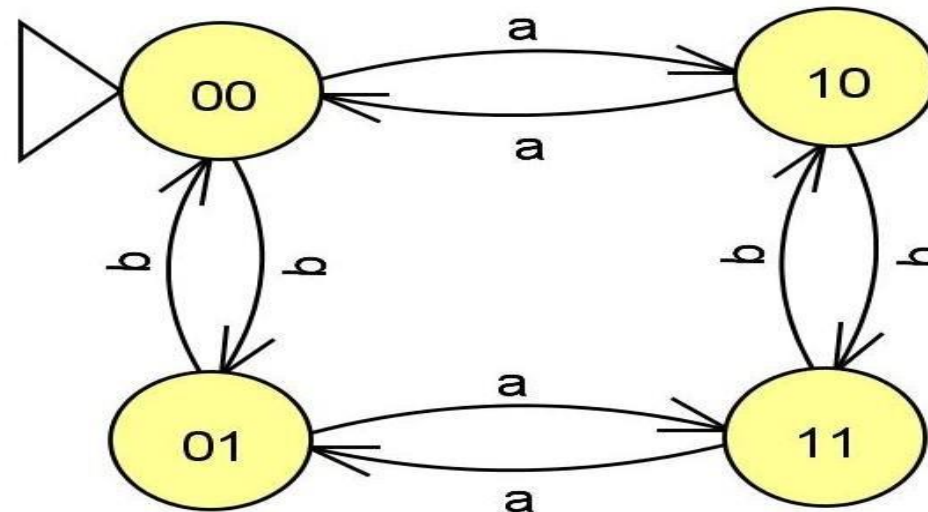
$L = \{ b, bbb, bbbbb, bbbbbb, \dots,$
 $baa, aba, aab, baaaa, abaaa, \dots, bbbbaa, aabbb, \dots \}$

- **State 11 is final state** for strings having **Odd number of a's** and **odd number of b's**. (i.e., $n_a(w) \bmod 2 = 1$ and $n_b(w) \bmod 2 = 1$)

$L = \{ ab, aaabbb, aaaaabbbb, aaaaaabbbbb, \dots,$
 $ba, bbbbaa, bbbbaaaaa, bbbbbbbaaaaaa, \dots,$
 $ababab, ababababab, \dots, bababa, bababababa, \dots, aabbba, abbbba, \dots \}$

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Note:



1st bit is related to symbol b's remainders [i.e., $n_b(w) \bmod k$]

2nd bit is related to symbol a's remainders [i.e., $n_a(w) \bmod k$]

In case of **mod 2** - Remainder is **0** or **1**

- Remainder 0 means Even number of a's or b's
- Remainder 1 means Odd number of a's or b's

In case of **mod 3** - Remainder is **0** or **1** or **2**

Example 13:

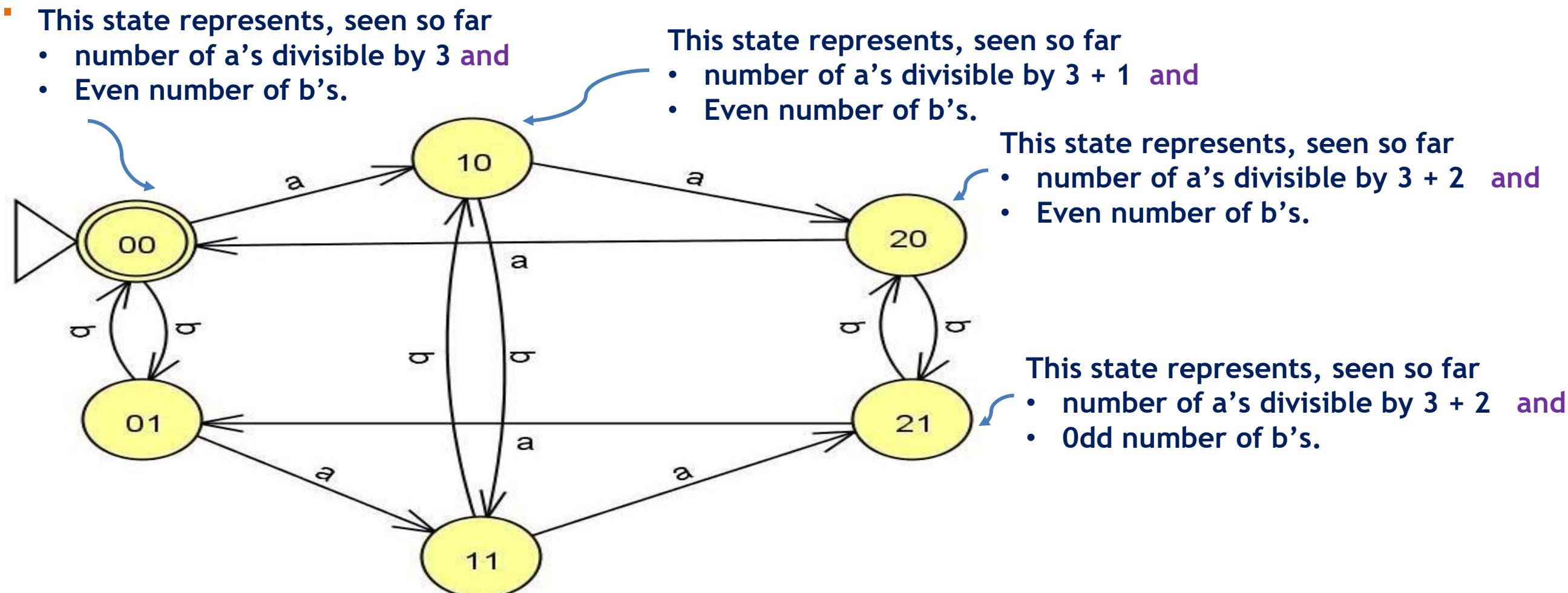
Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3=0$ and $n_b(w) \bmod 2=0$.

Solution :

Example 13:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 0$ and $n_b(w) \bmod 2 = 0$.

Solution :



Example 14:

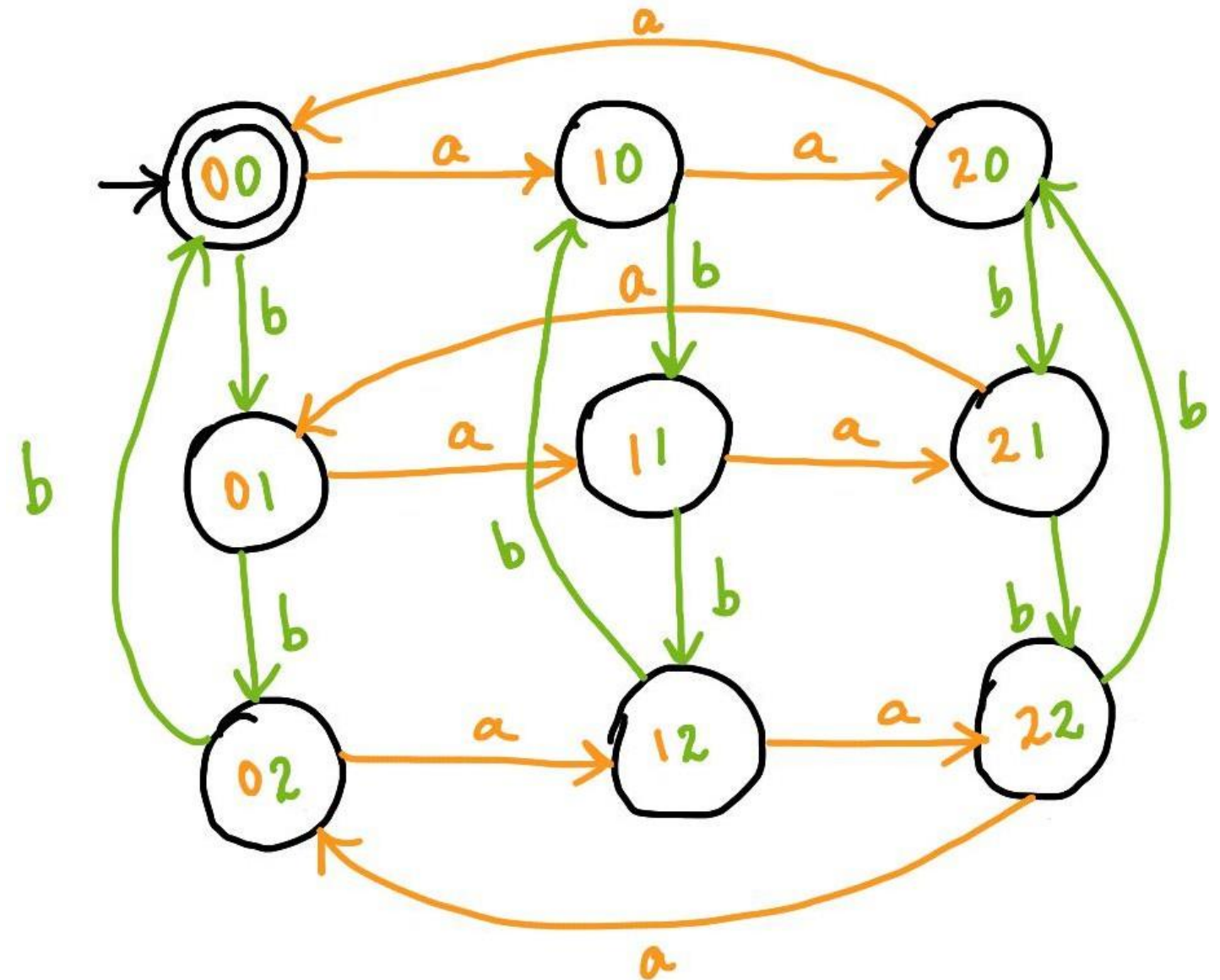
Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3=0$ and $n_b(w) \bmod 3=0$

Solution :

Example 14:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 0$ and $n_b(w) \bmod 3 = 0$

Solution :



Example 15:

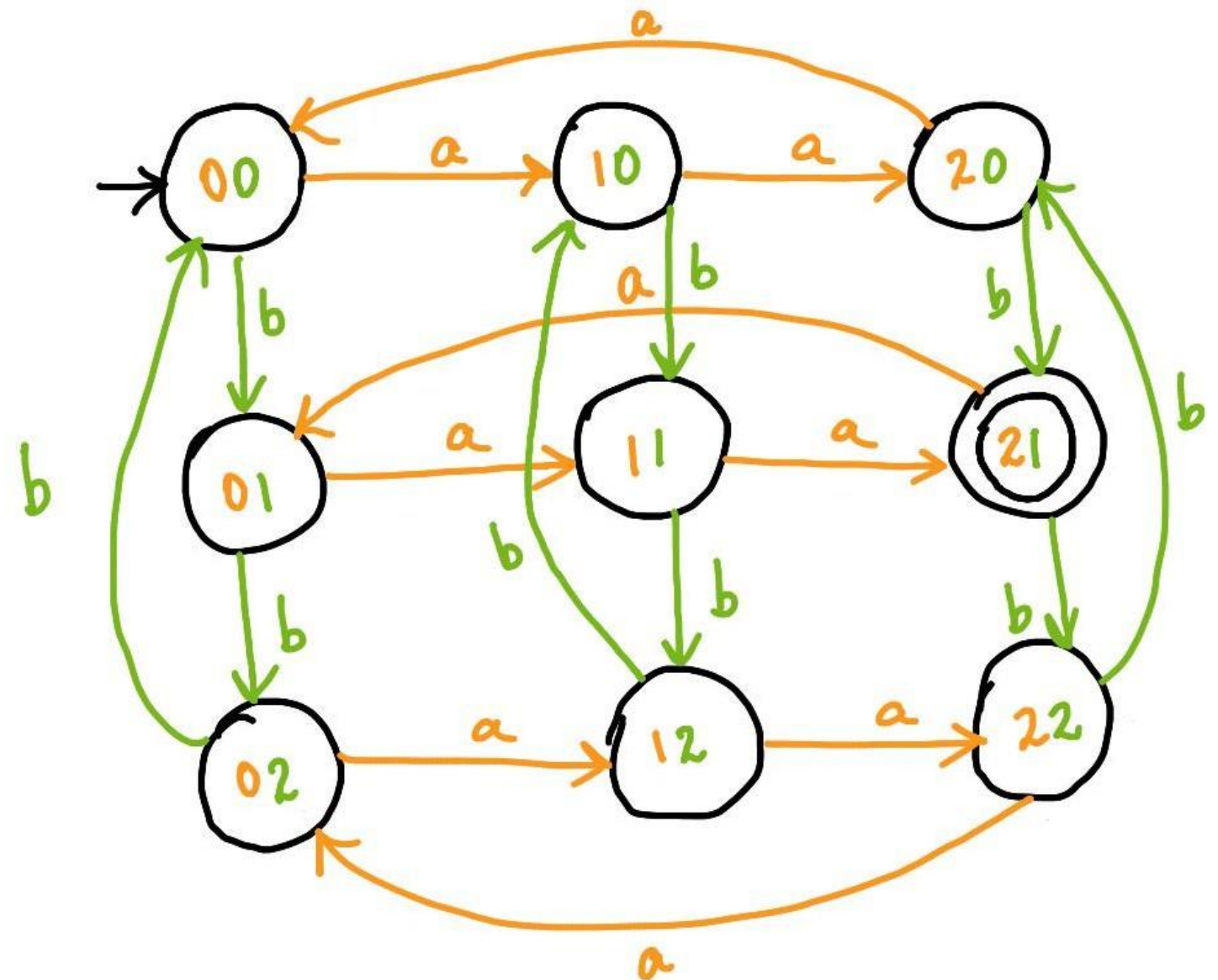
Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 2$ and $n_b(w) \bmod 3 = 1$

Solution :

Example 15:

Construct DFA for the language of strings over $\Sigma=\{a,b\}$ where, $n_a(w) \bmod 3 = 2$ and $n_b(w) \bmod 3 = 1$

Solution :



Example 16:

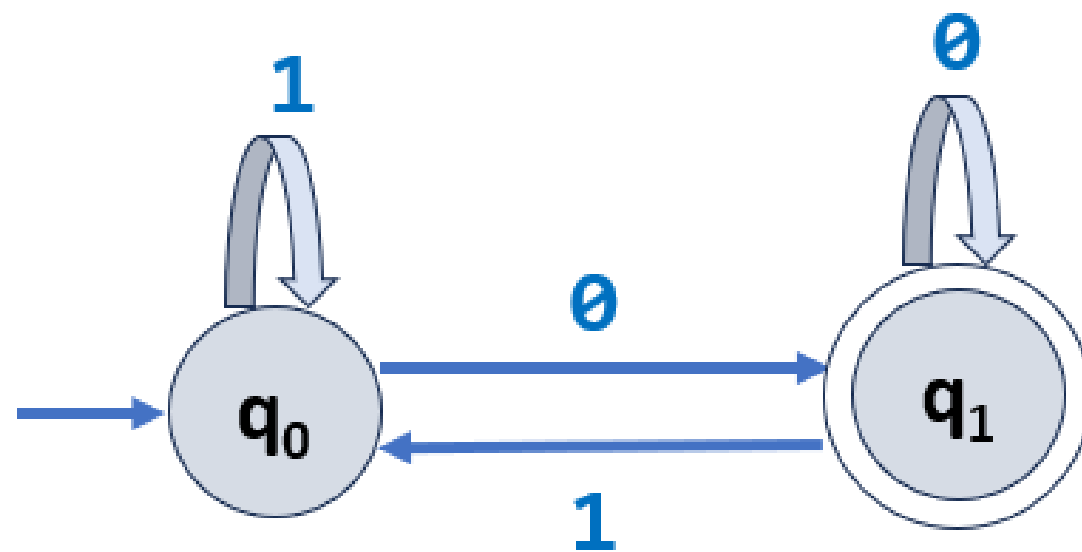
Construct DFA for a binary number divisible by 2 and $\Sigma = \{0,1\}$. i.e., $w \bmod 2 = 0$)

Solution :

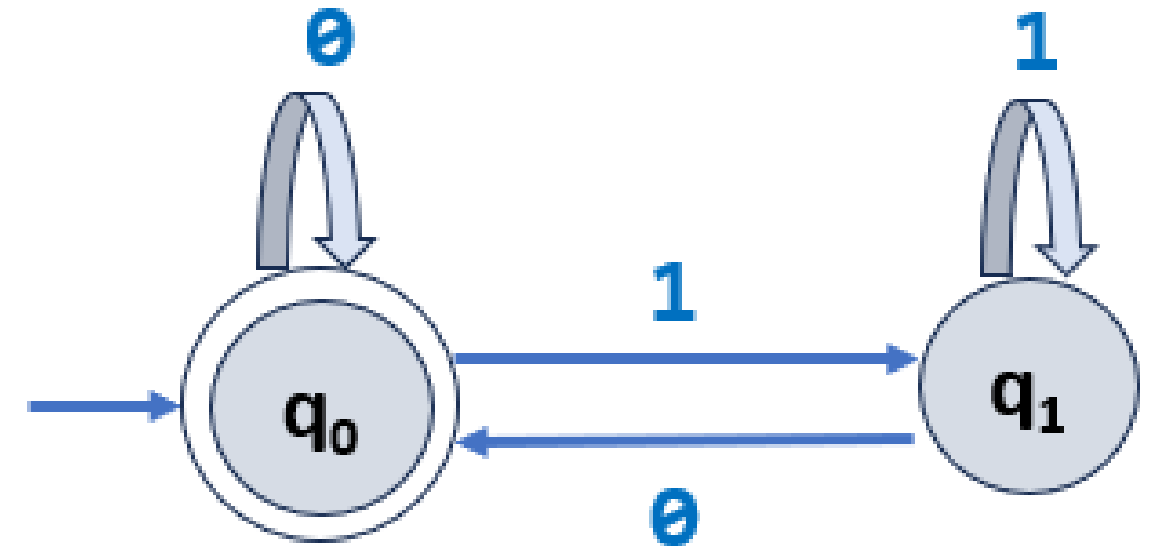
Example 16:

Construct DFA for a binary number divisible by 2 and $\Sigma = \{0,1\}$. (i.e., $w \bmod 2 = 0$)

Solution : $L = \{ 0, 10, 100, 110, 1000, 1010, \dots \}$



or



Example 16a:

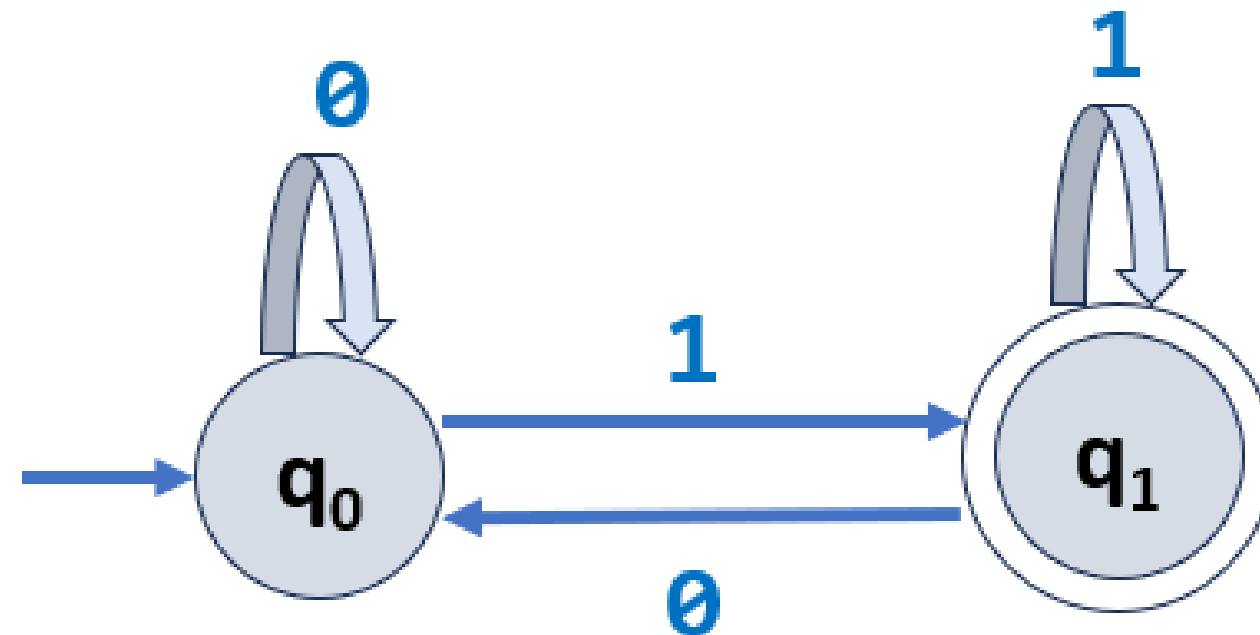
Construct DFA for a binary number where $w \bmod 2 = 1$

Solution :

Example 16a:

Construct DFA for a binary number where $w \bmod 2 = 1$

Solution :



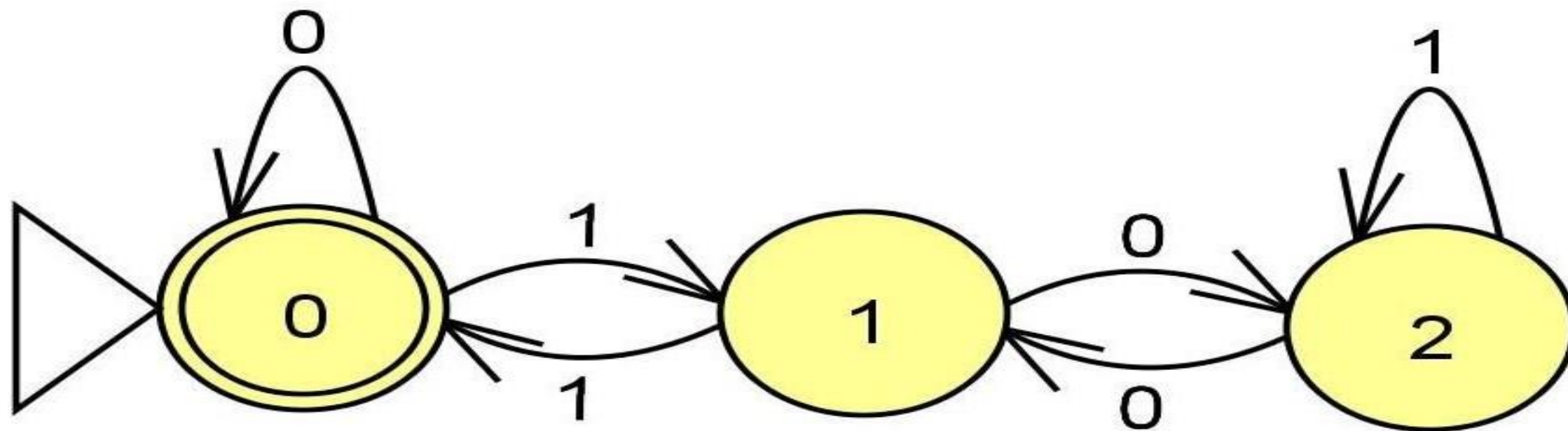
Example 17:

Construct a DFA for binary number divisible by 3. (i.e., $w \bmod 3 = 0$).

Solution :

Example 17:
Construct a DFA for binary number divisible by 3. (i.e., $w \bmod 3 = 0$).

Solution :



Example 18:

Construct a DFA for binary number divisible by 4. (i.e., $w \bmod 4 = 0$).

Solution :

Exercise:

1. Construct DFAs to recognize C-language
 - Identifiers,
 - Integers
 - Floating point numbers.
2. Construct DFAs to recognize C-language keywords
 - for
 - while
 - switch
3. Construct a DFA to recognize all the relational operators in C-language.

Exercise:

4. Construct DFAs for the validation of

- PAN number
- Aadhar number
- Email-id
- Phone number

5. ...

Limitations of DFA (or FSMs):

1. No stored program concept - the machine is the computation. No auxiliary memory.
2. A DFA is not powerful enough to recognize many context-free languages because a DFA can't count. But counting is not enough - consider a language of palindromes, containing strings of the form ww_R . Such a language requires more than an ability to count; it requires a stack.

Limitations of DFA (or FSMs): cont...

Non-regular languages are those for which no DFA exists. These languages have patterns or structures that cannot be recognized by a finite automaton.

Which languages CANNOT be described by any RE or DFA?

- Bit strings with equal number of 0s and 1s.
- Nested parenthesis matching.
- Decimal strings that represent prime numbers.
- Genomic strings that are Watson-Crick complemented palindromes.
- Many more. . . .

Note:

- Every NFA has an equivalent DFA (accepting the same language).
- The languages accepted by Finite State Machines (DFA, NFA, ϵ -NFA ...) are referred to as Regular languages (can be described by regular expressions).
- Regular Expression is a concise way to describe a set of strings.
- For any DFA, there exists a regular expression to describe the same set of strings; for any regular expression, there exists a DFA that recognizes the same set.
- DFA that accepts nothing has no final states.
- There is a unique minimal DFA for every regular language.



THANK YOU

Preet Kanwal and Prakash C O

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 1 - Deterministic Finite Acceptor/Automata



Criteria	DFA	NFA
State transition	Each input symbol leads to exactly one state	One input symbol can lead to one, more or no states



Automata Formal Languages & Logic

Unit 1 – NFA and ϵ -NFA

Prakash C O

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1: NFA and ϵ -NFA

Prakash C O

Department of Computer Science & Engineering

Formal Definition of an NFA

NFA can be represented by a 5-tuple $M=(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states.
- Σ is a finite non-empty set of input symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states ($F \subseteq Q$).

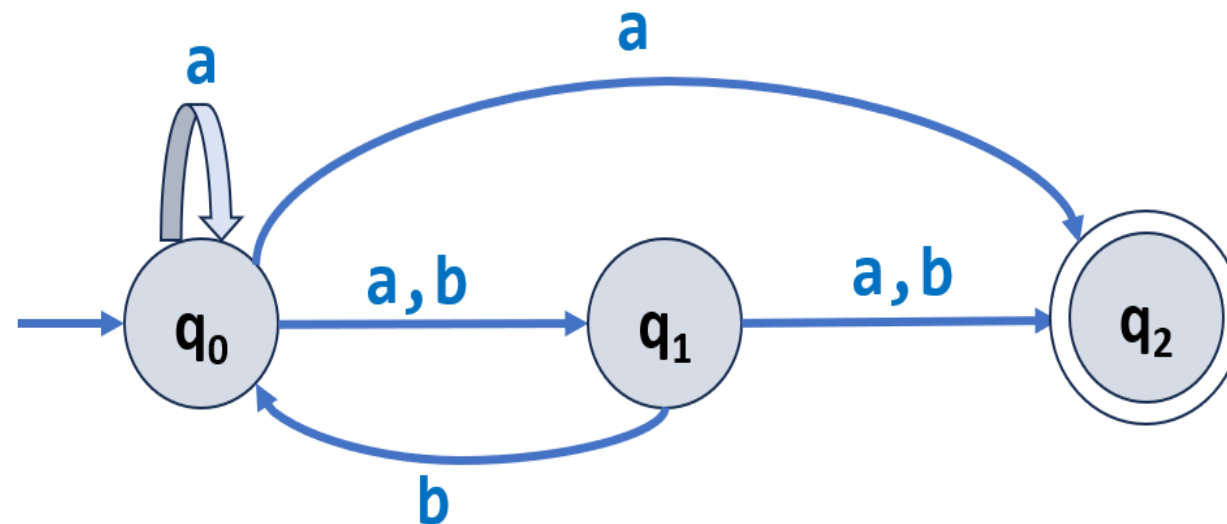
Why NFA?

- NFAs were introduced in 1959 by Michael O. Rabin and Dana Scott, who also showed their equivalence to DFAs.
- NFAs are used in the implementation of Regular Expressions (i.e., Thompson's construction is used to convert RE to NFA).
- Constructing an NFA to recognize a given language is sometimes much easier than constructing a DFA for that language.
- **DFAs have been generalized to nondeterministic finite automata (NFA). NFA provides a nice way of ‘abstraction of information’.**

The Transition Function of an NFA is $\delta: Q \times \Sigma \rightarrow 2^Q$

- Here the power set (i.e., $P(Q)=2^Q$) has been taken because in case of an NFA, from a state, transition can occur to any combination of Q states.

- Example:



$$2^Q = 2^3 = \{ \{\}, \{q_0\}, \{q_1\}, \{q_2\}, \\ \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \\ \{q_0, q_1, q_2\} \}$$

- In an NFA, a (state, input_symbol) combination may lead to several states simultaneously.

$$\delta(q_0, a) = \{q_0, q_1, q_2\}$$

$$\delta(q_2, a) = \{\}$$

Why NFA is called Non-deterministic?

- In automata theory, a finite-state machine is called a deterministic finite automaton (DFA), if
 - each of its transitions is uniquely determined by its source state and input symbol, and
 - reading an input symbol is required for each state transition.
- A nondeterministic finite automaton (NFA), does not need to obey the above restrictions of DFA. In particular, every DFA is also an NFA. NFA may have several arrows of the same label starting from a state.
- A Finite Automata(FA) is said to be non-deterministic if there is more than one possible transition from a state on the same input symbol.

Computation in NFA

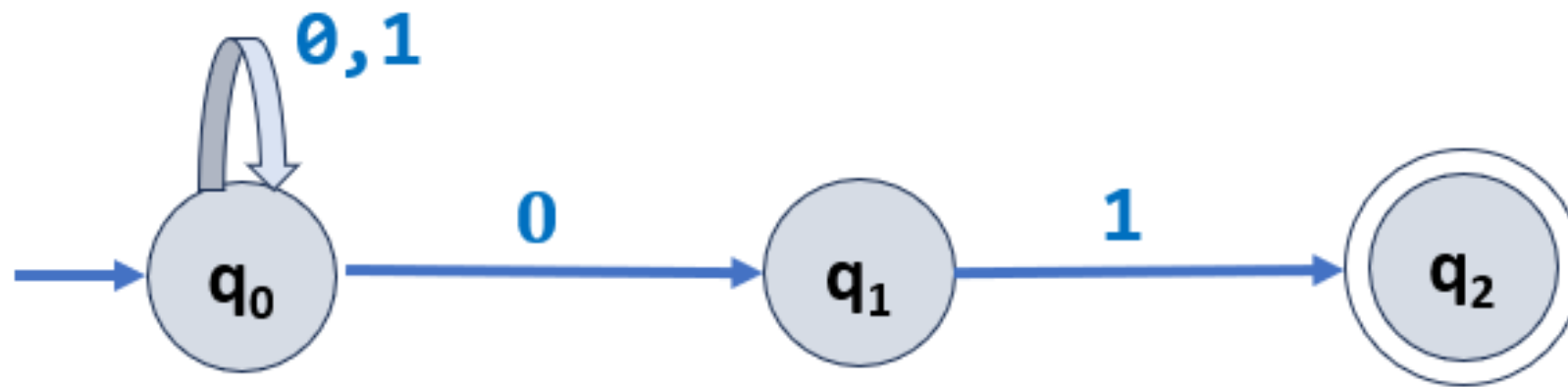
- For an input string, NFA will create a ‘computation tree’ rather than a ‘computation sequence’ in case of DFA.

An NFA accepts a string if any one of the paths in the tree leads to some accept state.

- Nondeterminism in NFA causes multiple runs. There can be several or no runs for a given input word.

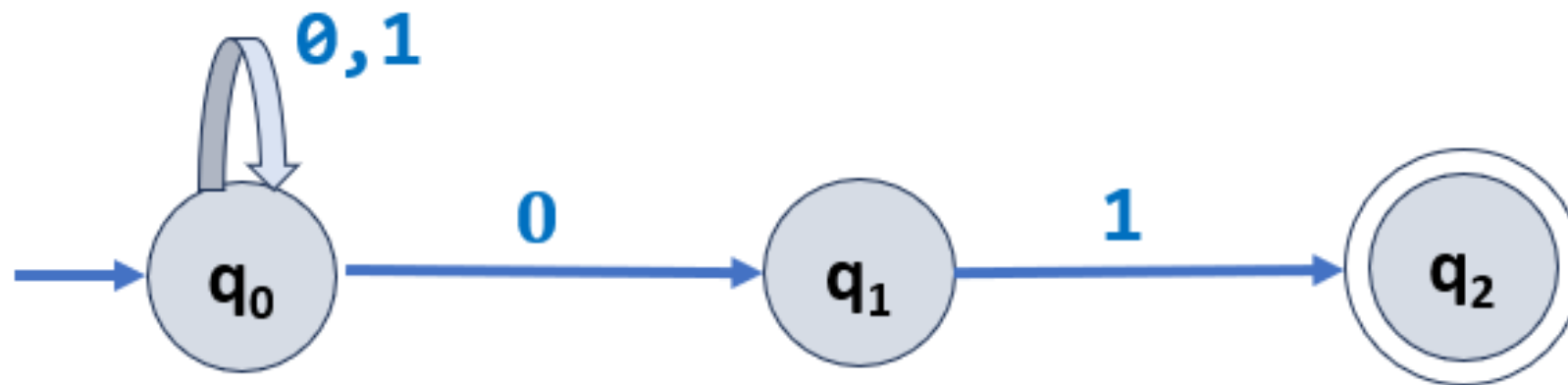
If there is at least one run that leads to an accepting state, the machine will accept the input string.

➤ **Example 1:** Consider $L = \{ x \in \{0, 1\}^* \mid \text{where } x \text{ ends with } 01 \}$



Let 10101 be an input word. The following are potential runs.

➤ **Example 1:** Consider $L = \{ x \in \{0, 1\}^* \mid \text{where } x \text{ ends with } 01 \}$



Let 10101 be an input word. The following are potential runs.

$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0}$ **stuck**

Unfinished runs are unacceptable

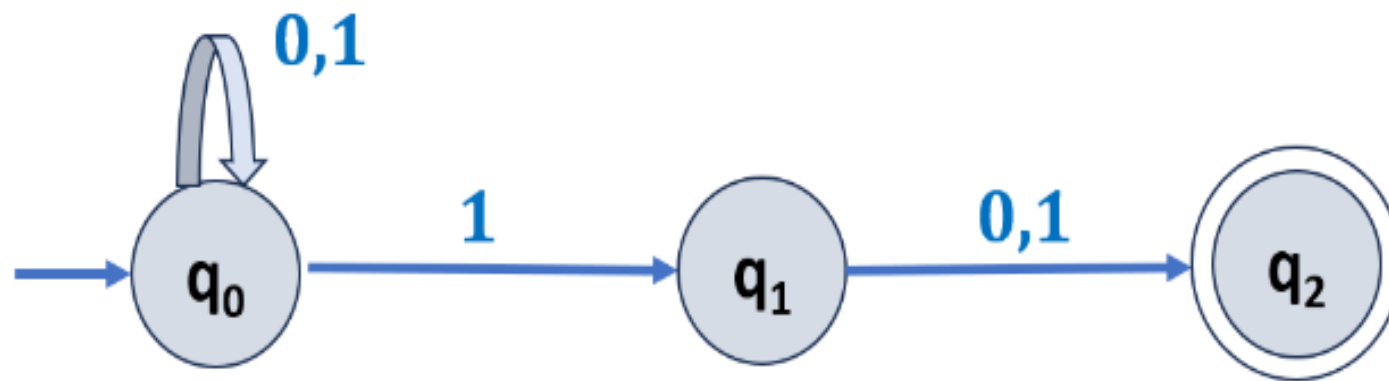
$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$

Accepted run/Computation

$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0$

Rejected run/Computation

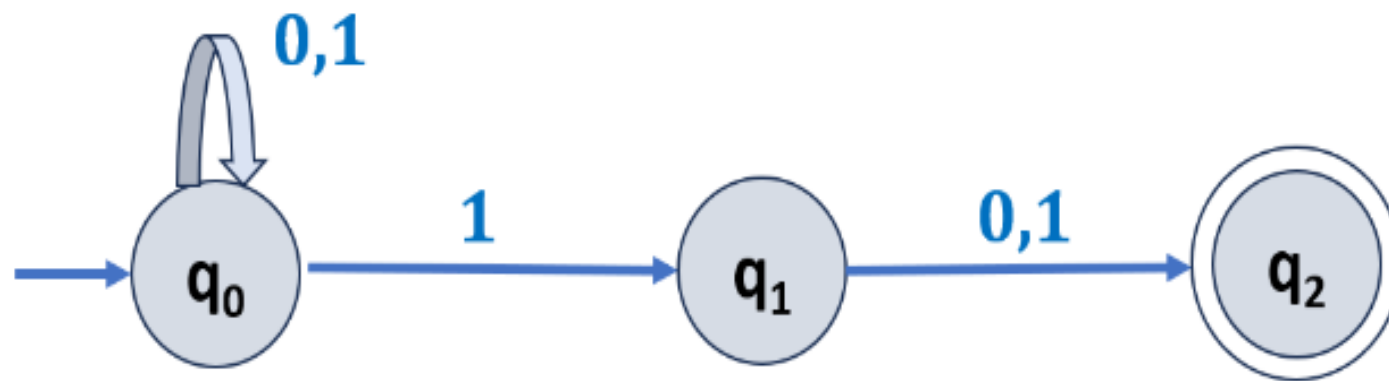
➤ **Example 2:** Consider $L = \{x \in \{0, 1\}^* \mid \text{2}^{\text{nd}} \text{ symbol from the right is } 1\}$



Computation tree for the input string 11010

- The NFA consumes a string of input symbols, one by one. In each step, **whenever two or more transitions are applicable**, it "clones" itself into appropriately many copies, each one following a different transition.
-
- If no transition is applicable, the current copy is in a dead end (or stuck), it "dies". If, after consuming the complete input, any of the copies is in an accept state, the input is accepted, else, it is rejected.

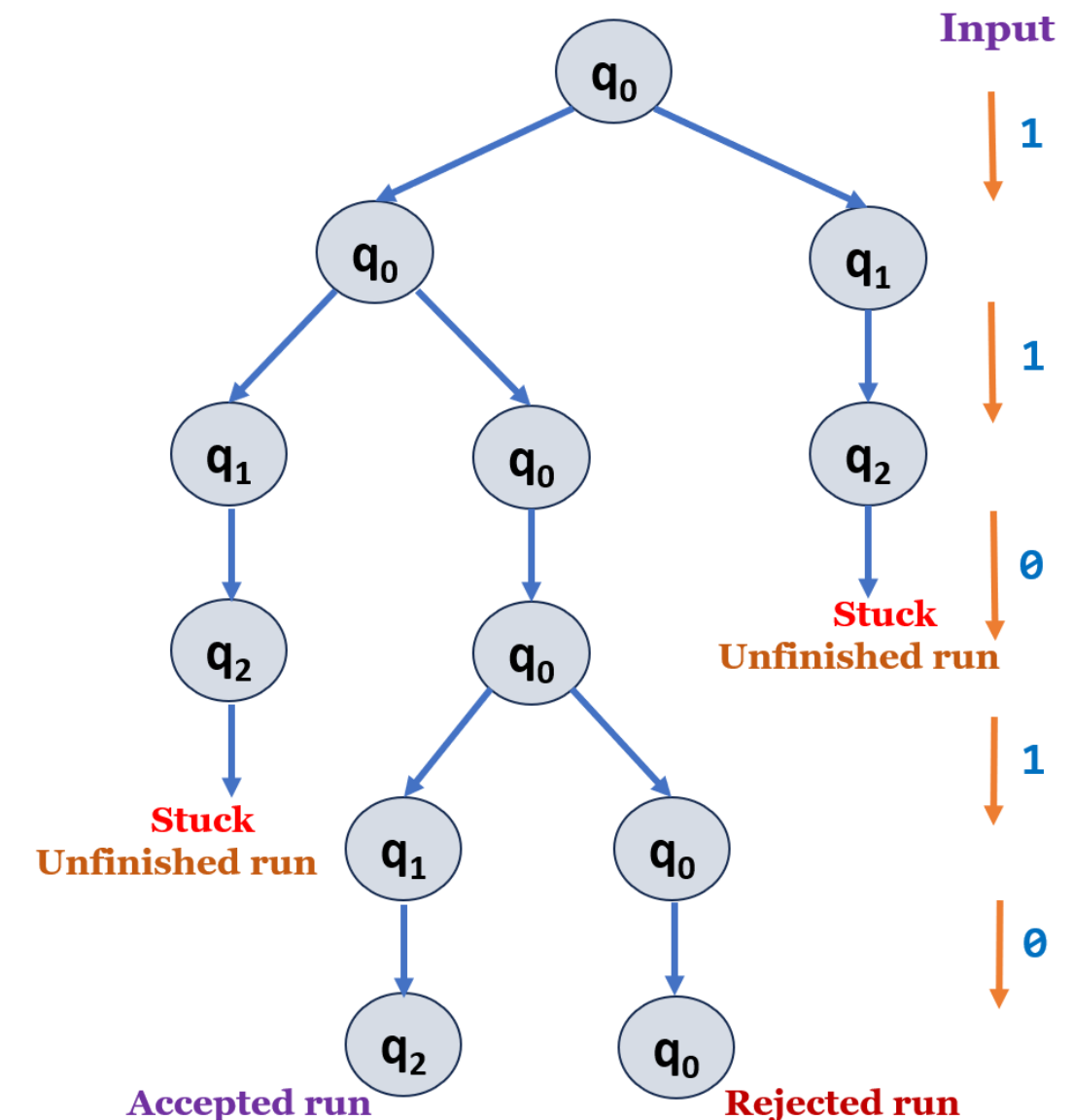
➤ **Example 2:** Consider $L = \{x \in \{0, 1\}^* \mid \text{2}^{\text{nd}} \text{ symbol from the right is } 1\}$



Computation tree for the input string 11010

➤ The NFA consumes a string of input symbols, one by one. In each step, **whenever two or more transitions are applicable**, it "clones" itself into appropriately many copies, each one following a different transition.

➤ If no transition is applicable, the current copy is in a dead end (or stuck), it "dies". If, after consuming the complete input, any of the copies is in an accept state, the input is accepted, else, it is rejected.



NFA Construction:

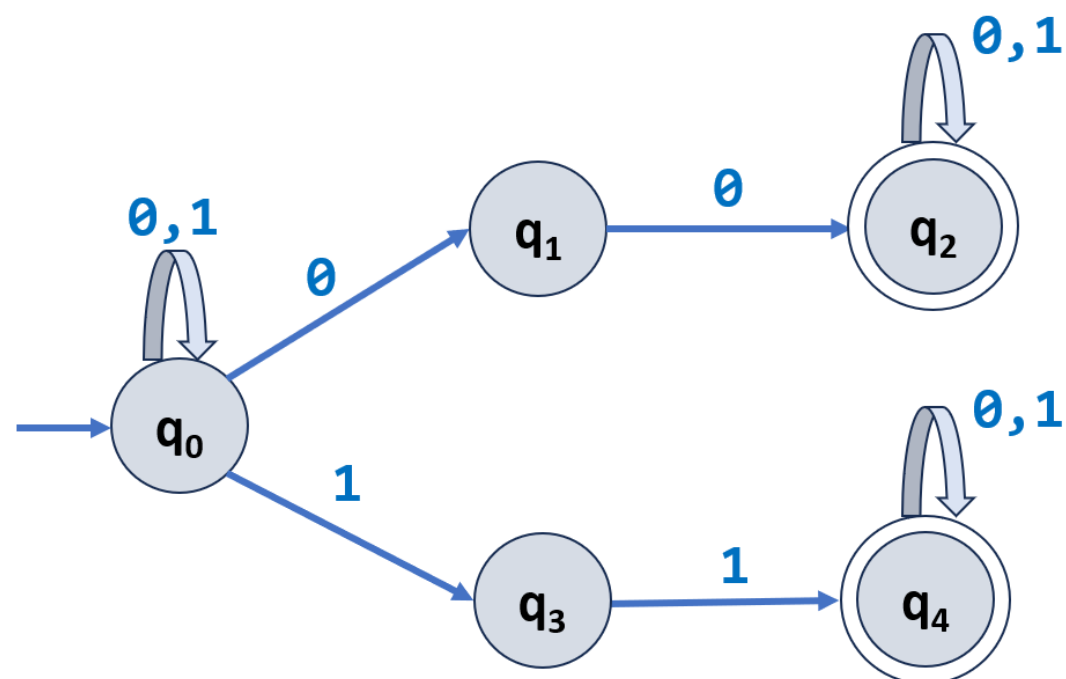
Example 1: Construct NFA for the language of binary strings having at least a pair of '00' or a pair of '11'

Solution:

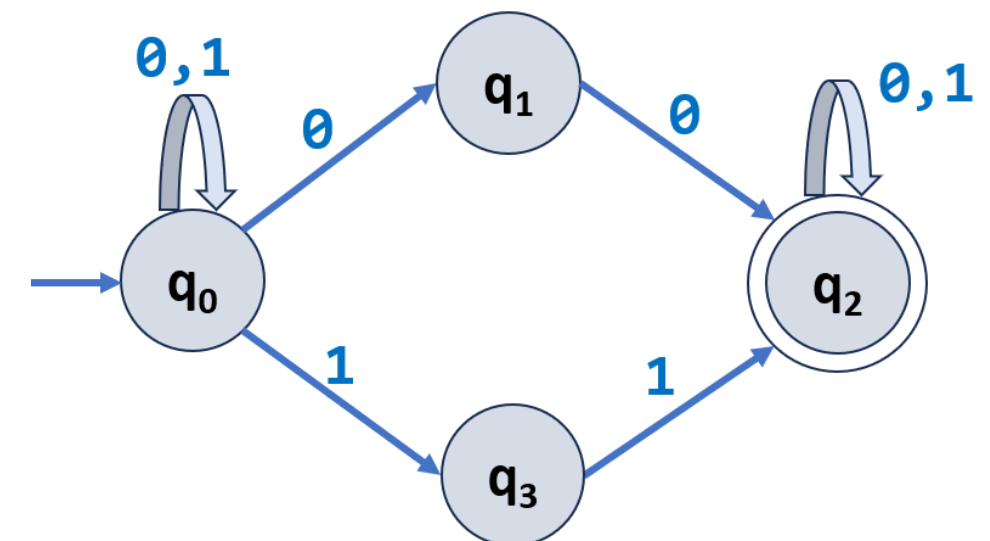
NFA Construction:

Example 1: Construct NFA for the language of binary strings having at least a pair of '00' or a pair of '11'

Solution:



or

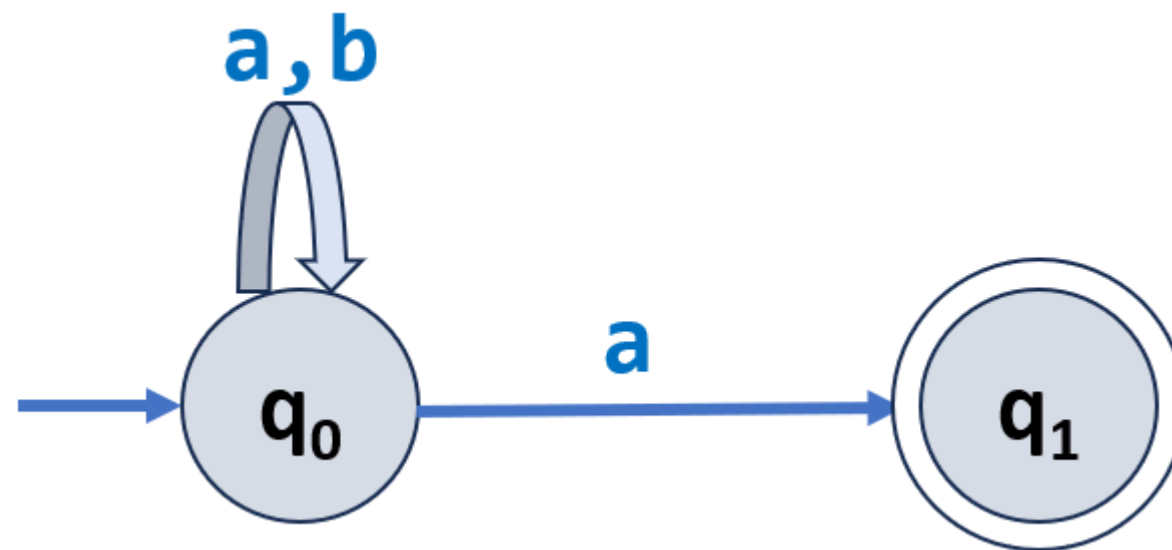


Example 2: Construct NFA for the language of strings ends with a, over $\Sigma = \{a, b\}$

Solution:

Example 2: Construct NFA for the language of strings ends with a, over $\Sigma = \{a, b\}$

Solution: $L = \{ a, aa, ba, bba, baa, aaa, aba, aaaa, aaba, abba, baba, bbba, \dots \}$

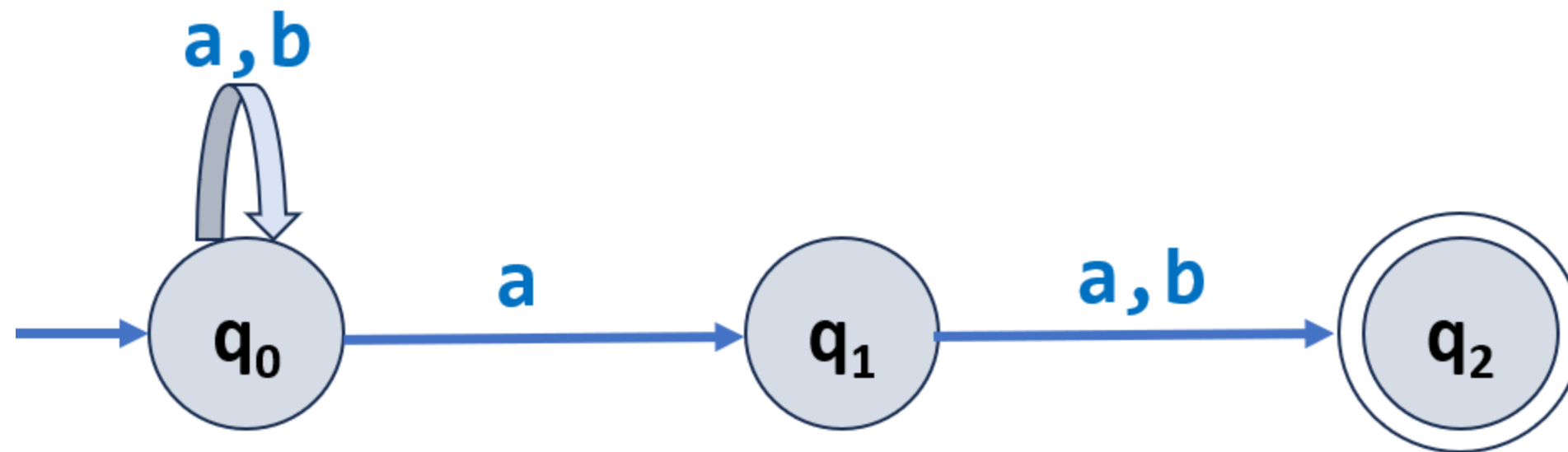


Example 3: Construct NFA for the language of strings, where the second symbol from the RHS of a string is 'a' and $\Sigma = \{a, b\}$

Solution:

Example 3: Construct NFA for the language of strings, where the second symbol from the RHS of a string is 'a' and $\Sigma = \{a, b\}$

Solution: $L = \{ \text{aa, ab, aab, baa, bab, aaa, aaaa, aaab, abab, baab, bbab, ...} \}$



Example 4: Design an NFA in which all the string contain a substring 1110
and $\Sigma = \{0, 1\}$

Solution:

Example 5: Design an NFA with $\Sigma = \{0, 1\}$ accepts all string in which the third symbol from the right end is always 0.

Solution:

DFA and NFA Equivalence

- **NFAs and DFAs are equivalent in that if a language is recognized by an NFA, it is also recognized by a DFA and vice versa.**
- **The establishment of NFA and DFA equivalence is important and useful.**
 - **It is useful because constructing an NFA to recognize a given language is sometimes much easier than constructing a DFA for that language.**
 - **It is important because NFAs can be used to reduce the complexity of the mathematical work required to establish many important properties in automata theory.**
 - **For example, it is much easier to prove closure properties of regular languages using NFAs than DFAs.**

Formal Definition of an ε -NFA (or λ -NFA)

ε -NFA can be represented by a 5-tuple $M=(Q, \Sigma, \delta, q_0, F)$ where

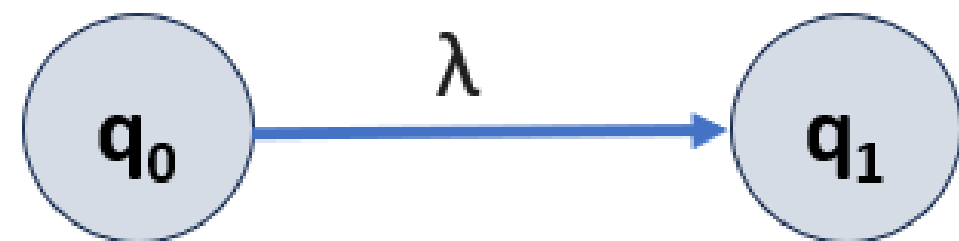
- Q is a finite set of states.
- Σ is a finite non-empty set of input symbols called the alphabet.
- δ is the transition function where $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$ **or** $\delta: Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states ($F \subseteq Q$).

ϵ -transitions (or λ -transitions) (or Null moves):

- Let us add another feature to our automaton.
- Let us allow it to jump states without reading inputs. We will call such moves ϵ -transitions (or λ -transitions) .



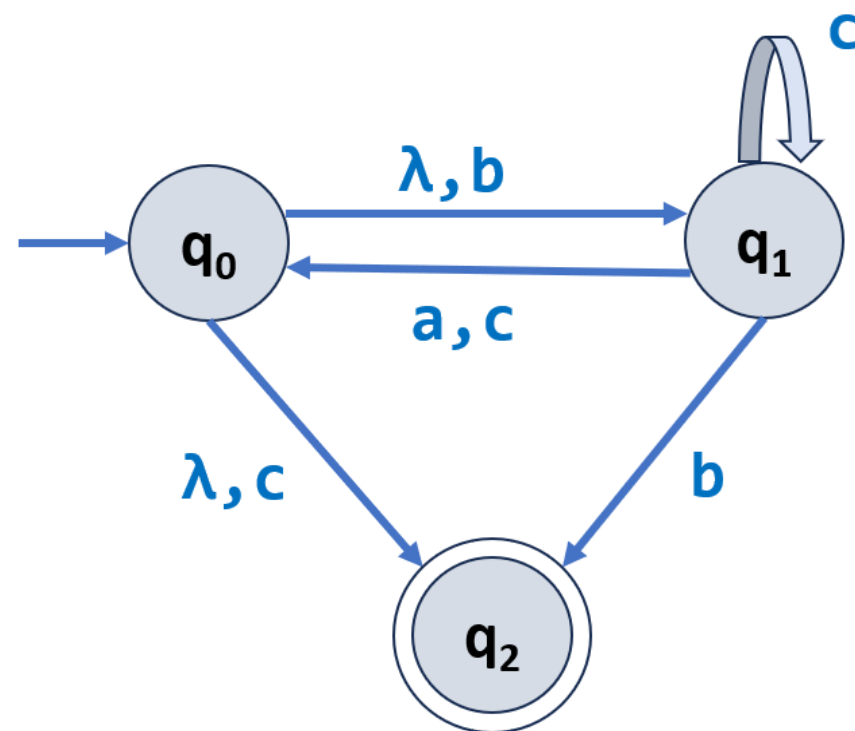
or



ϵ -closure (or λ -closure) of a state or set of states:

- For a state q , let ϵ -closure(q) (or λ -closure(q)) denote the set of states that are reachable from q by ϵ -transitions (or λ -transitions) including the state q itself.

Example 1:



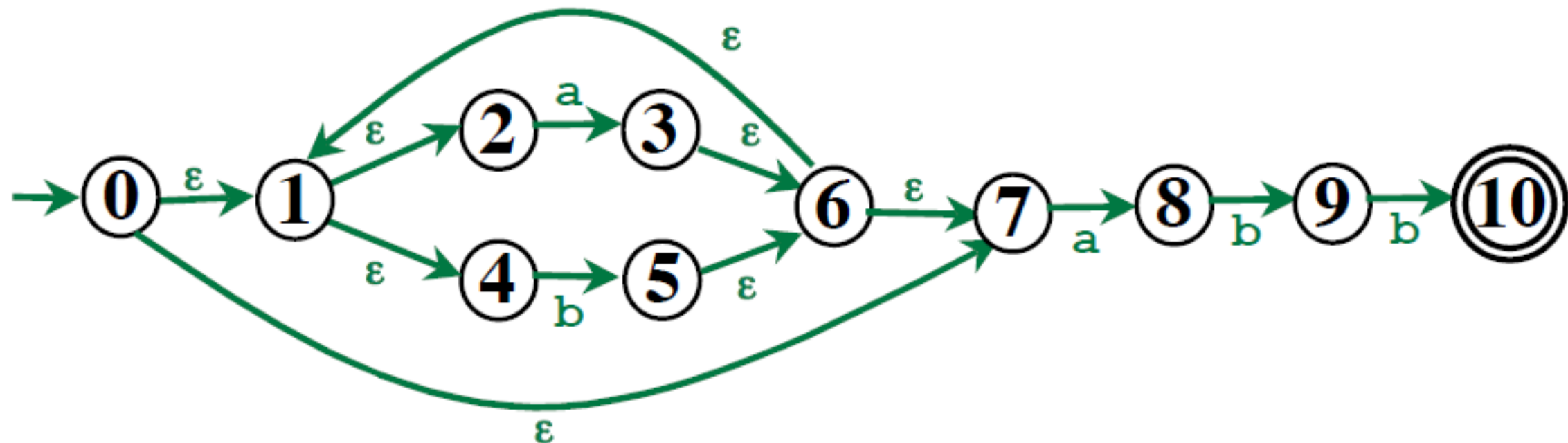
$$\lambda\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\lambda\text{-closure}(q_1) = \{q_1\}$$

$$\lambda\text{-closure}(q_2) = \{q_2\}$$

ϵ -closure (or λ -closure) of a state or set of states:

➤ Example 2:

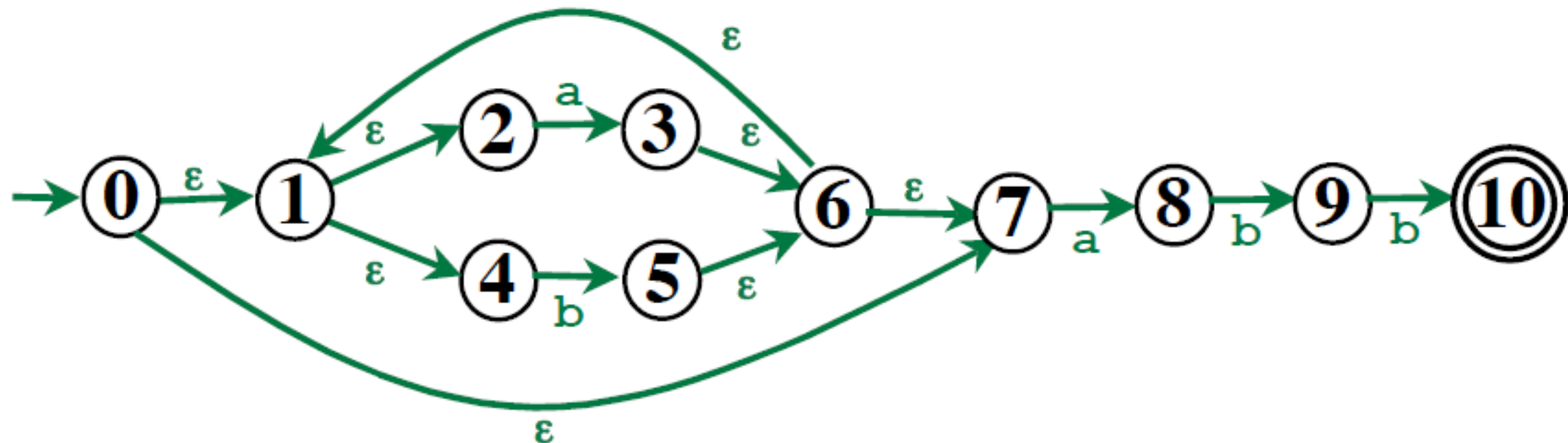


ϵ -closure(0) =

ϵ -closure(6) =

ϵ -closure (or λ -closure) of a state or set of states:

➤ Example 2:

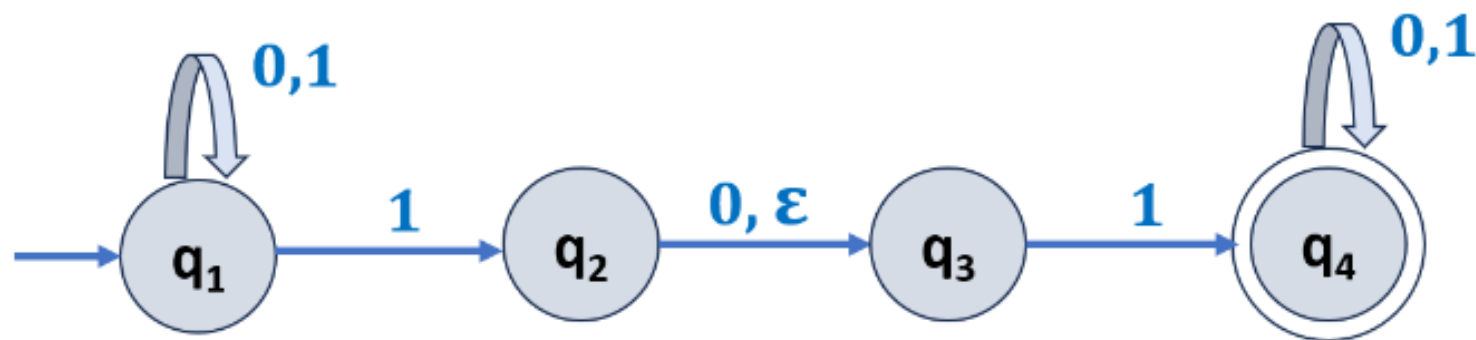


$$\epsilon\text{-closure}(0) = \{ 0 \ 1 \ 2 \ 4 \ 7 \}$$

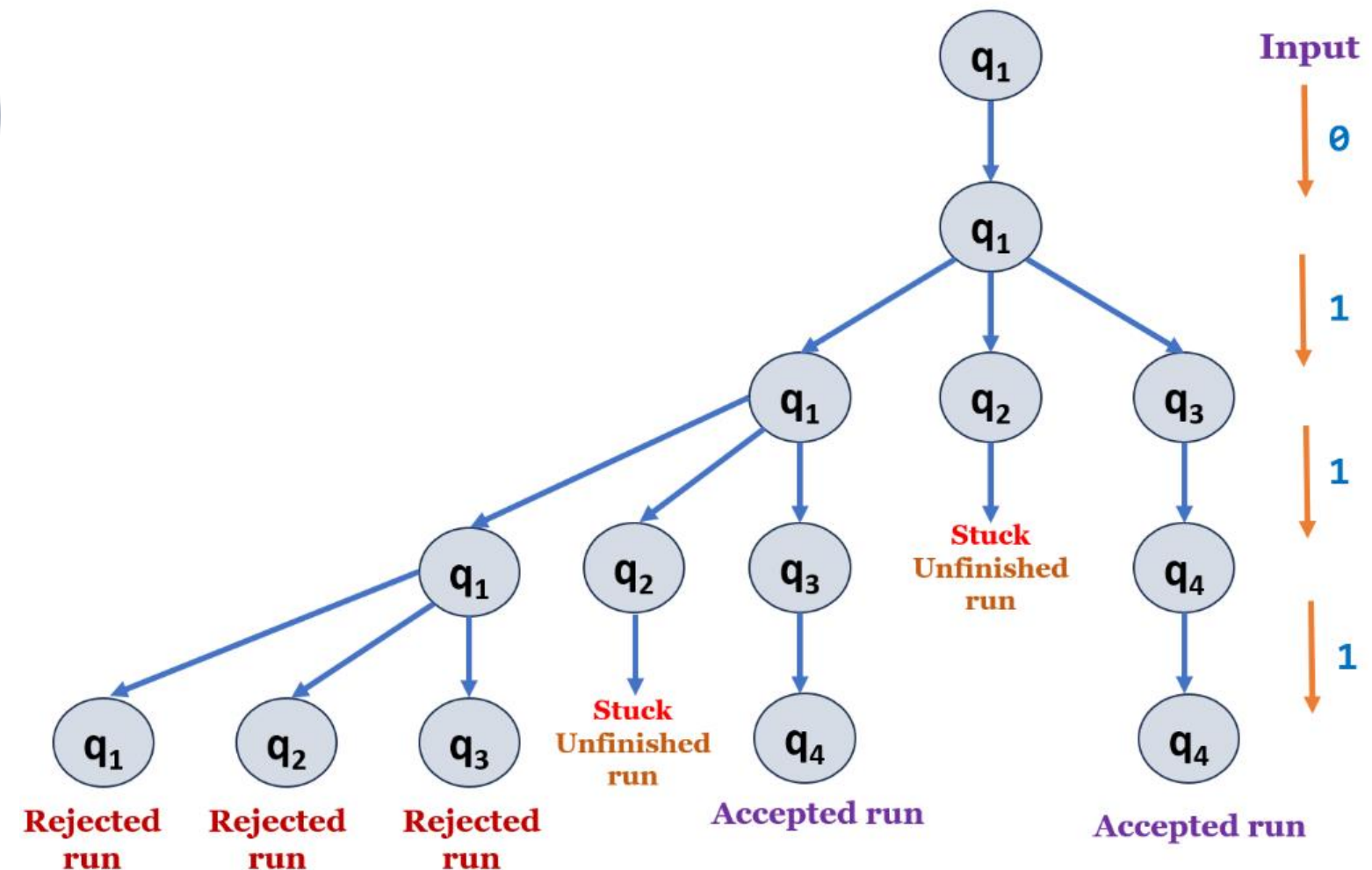
$$\epsilon\text{-closure}(6) = \{ 6 \ 7 \ 1 \ 2 \ 4 \}$$

Computation or Run in an ϵ -NFA

Example: Consider $L = \{ x \in \{0, 1\}^* \mid \text{where } x \text{ contains the substring } 101 \text{ or } 11 \}$

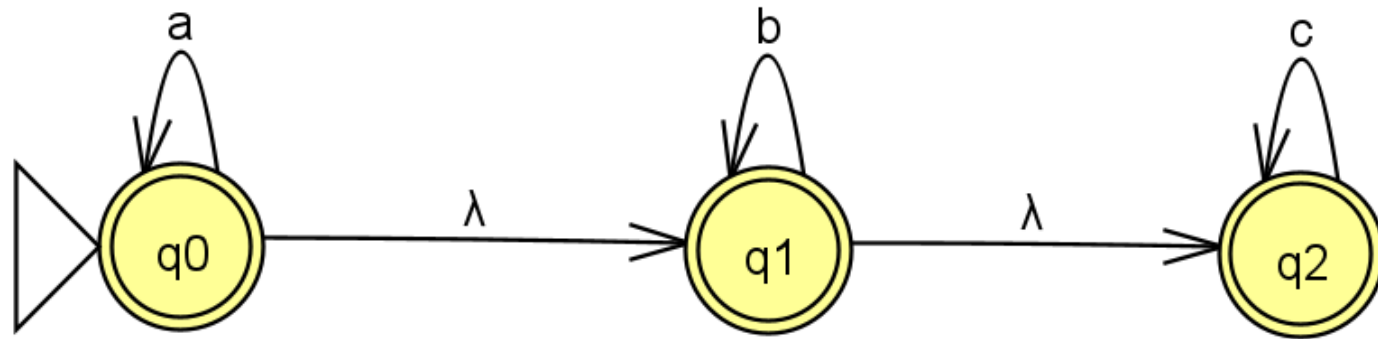


Computation tree for the input string 0111



Computation or Run in an ϵ -NFA

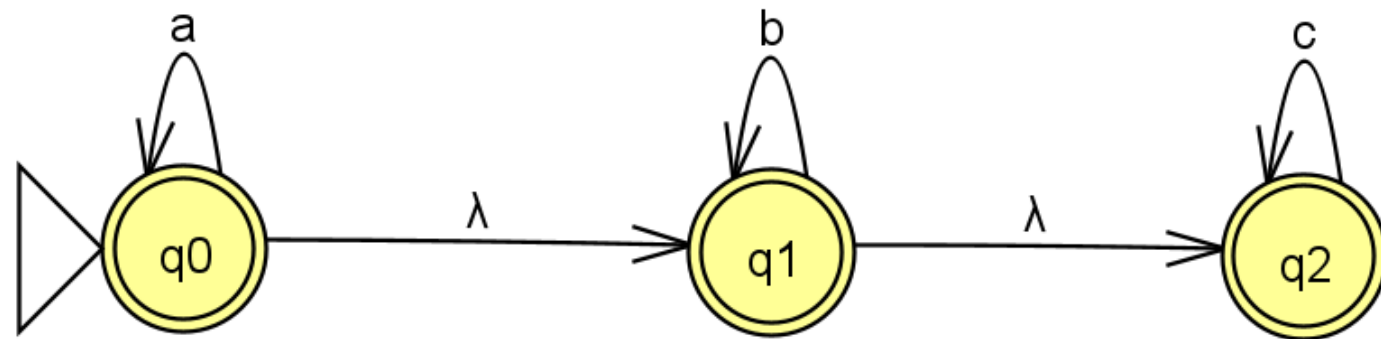
Exercise: Consider the language of strings over $\Sigma = \{a, b, c\}$ of the form $a^n b^m c^k$ where $n, m, k \geq 0$.



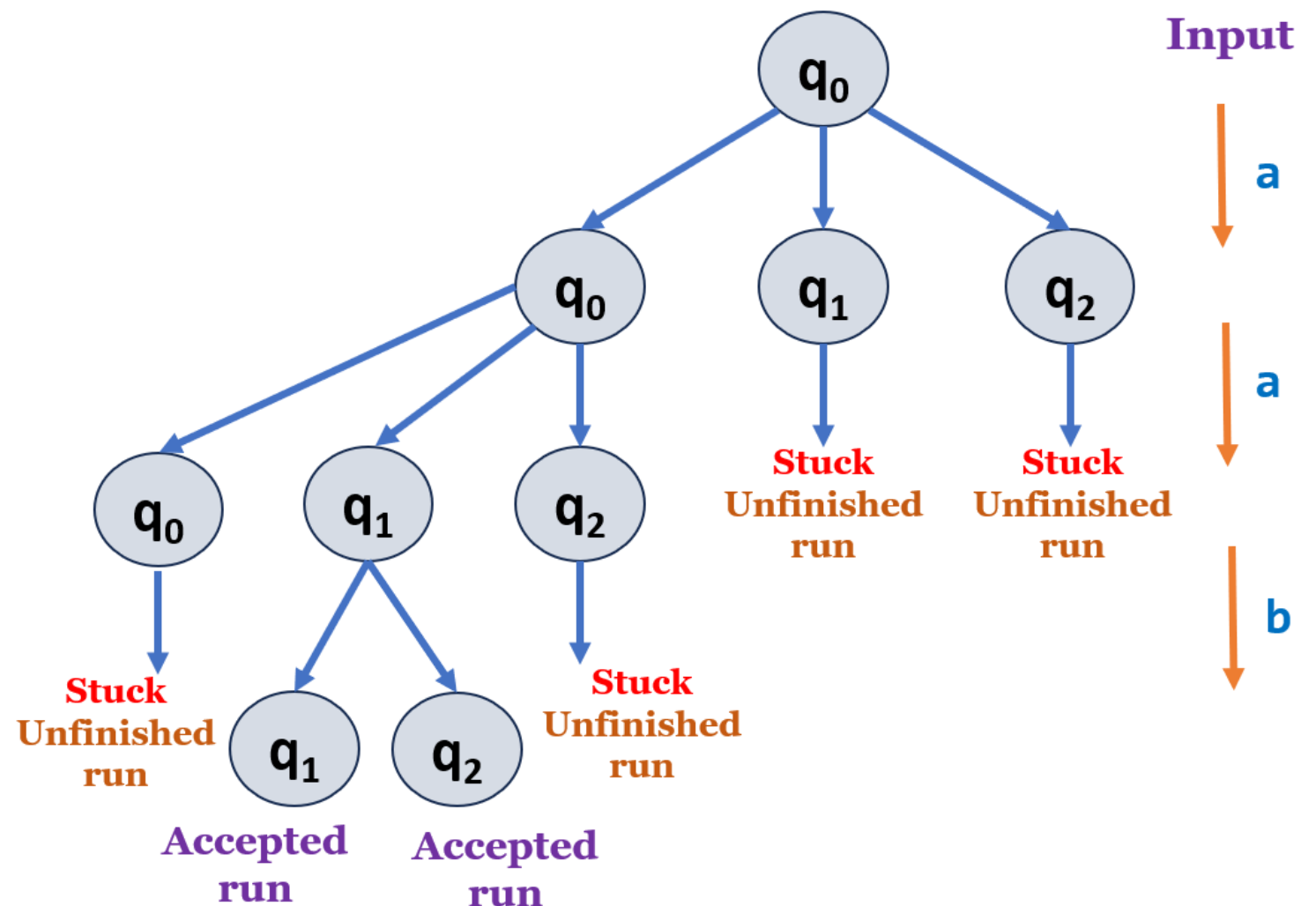
Construct Computation tree for the input string aab.

Computation or Run in an ϵ -NFA

Exercise: Consider the language of strings over $\Sigma = \{a, b, c\}$ of the form $a^n b^m c^k$ where $n, m, k \geq 0$.



Computation tree for the input string aab.

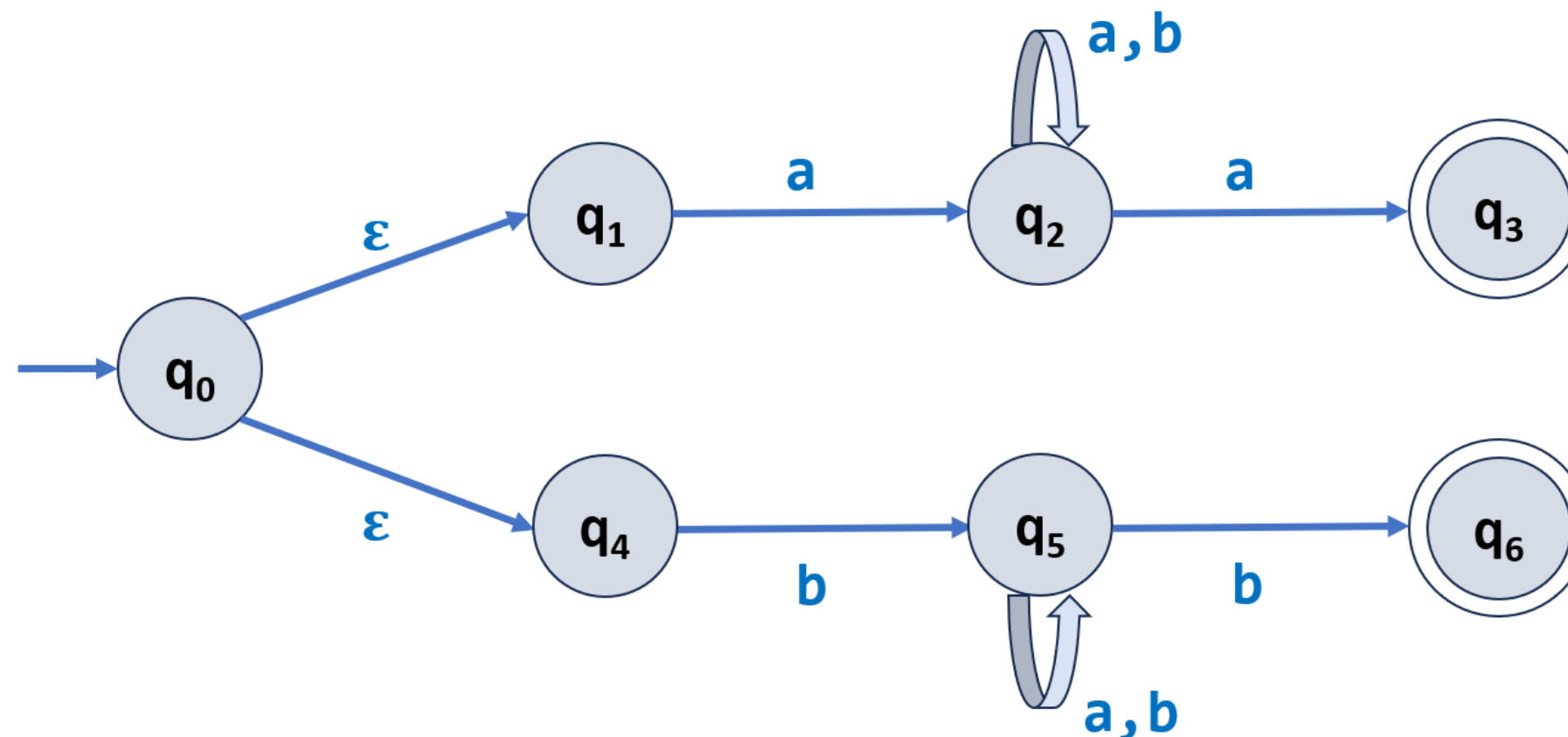


Example 1: Construct ε -NFA for a language of strings that begins and ends with the same symbol, and $\Sigma = \{a, b\}$

Solution:

Example 1: Construct ϵ -NFA for a language of strings that begins and ends with the same symbol, and $\Sigma = \{a, b\}$

Solution: $L = \{aa, bb, aba, bab, aaa, bbb, \dots\}$



Example 2: Construct ε -NFA that recognizes integers without leading zeros.

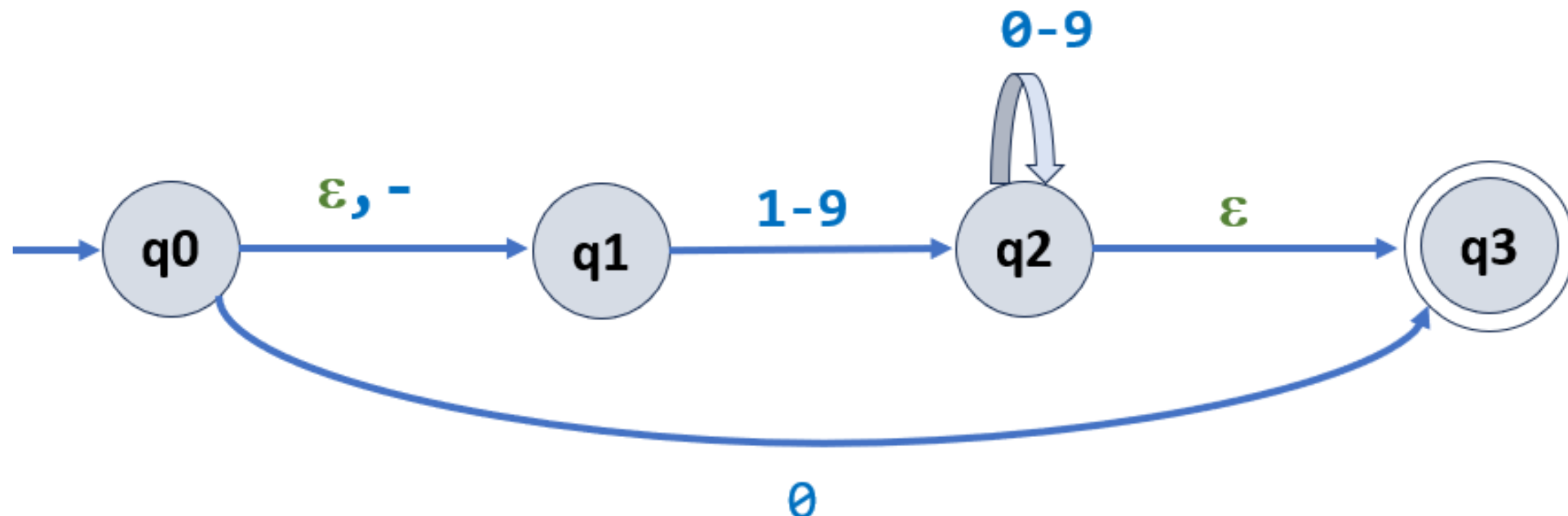
$$\Sigma = \{ -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$$

Solution: Example integers: $\dots, -2, -1, 0, 1, 2, \dots, 101, 102, \dots$

Example 2: Construct ϵ -NFA that recognizes integers without leading zeros.

$$\Sigma = \{ -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$$

Solution: Example integers: $\dots, -2, -1, 0, 1, 2, \dots, 101, 102, \dots$

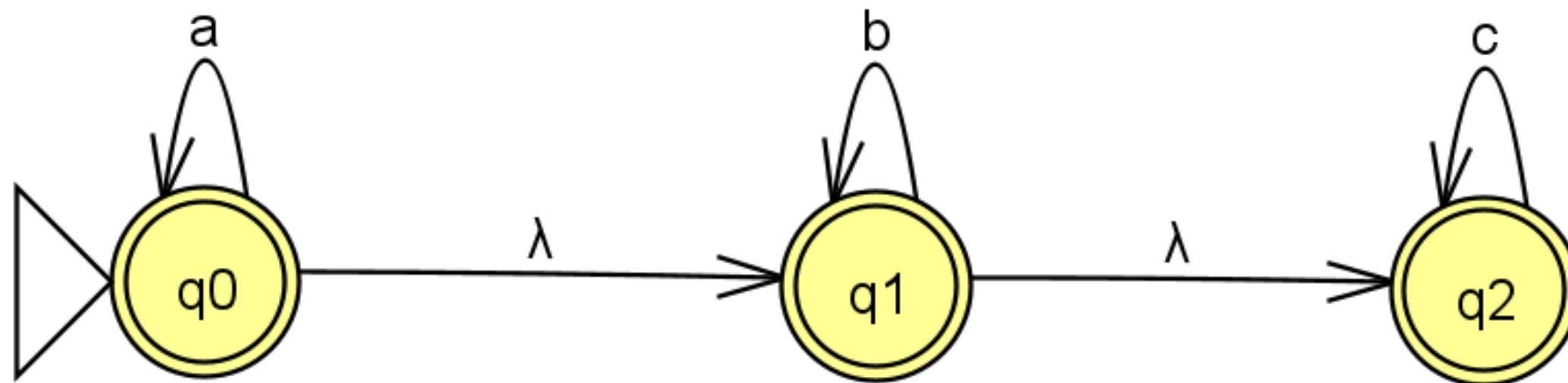


Example 3: Construct λ -NFA for the language $L = \{ a^n b^m c^p \mid n, m, p \geq 0 \}$

Solution: $L = \{ \epsilon, a, aa, aaa, aaaa, \dots, b, bb, bbb, bbbb, \dots, c, cc, ccc, cccc, \dots, ab, bc, ac, \dots, abc, \dots \}$

Example 3: Construct λ -NFA for the language $L = \{ a^n b^m c^p \mid n, m, p \geq 0 \}$

Solution: $L = \{ \epsilon, a, aa, aaa, aaaa, \dots, b, bb, bbb, bbbb, \dots, c, cc, ccc, cccc, \dots, ab, bc, ac, \dots, abc, \dots \}$



- DFA may be cumbersome to specify for complex systems and may even be unknown, but NFA provides a nice way of ‘abstraction of information’.
- Any language that can be accepted by a DFA can also be accepted by an NFA, i.e., Every NFA has an equivalent DFA (accepting the same language).
- DFA’s, NFA’s, and ϵ -NFA’s all accept exactly the same set of languages: the regular languages.
- NFA are often more convenient to design than DFA and may have fewer states compared to DFA.
- In real-time, only Deterministic Finite Automata (DFAs) can be implemented.



THANK YOU

Prakash C O

Department of Computer Science & Engineering
coprakasha@pes.edu

- Using subset construction method, every NFA can be translated to a DFA that recognizes the same language. DFAs, and NFAs as well, recognize exactly the set of regular languages.
- NFAs have been generalized in multiple ways, e.g., ϵ -NFA, finite-state transducers, pushdown automata, alternating automata, ω -automata, and probabilistic automata.

Note:

- "Deterministic" means "if you put the system in the same situation twice, it is guaranteed to make the same choice both times".
- "Non-deterministic" means "not deterministic", or in other words, "if you put the system in the same situation twice, it might or might not make the same choice both times".
- A non-deterministic finite automaton (NFA) can have multiple transitions out of a state. This means there are multiple options for what it could do in that situation. It is not forced to always choose the same one; on one input, it might choose the first transition, and on another input it might choose the same transition.
- Here you can think of "situation" as "what state the NFA is in, together with what symbol is being read next from the input". Even when both of those are the same, a NFA still might have multiple matching transitions that can be taken out of that state, and it can choose arbitrarily which one to take. In contrast, a DFA only has one matching transition that can be taken in that situation, so it has no choice -- it will always follow the same transition whenever it is in that.



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

NFA $\rightarrow$ DFA (Subset Construction)

- The initial state is the start state, plus all states reachable from the start state via λ -transitions (Called λ -closure).
- Transition from a state S on character a is found by following all possible transitions on a for each state in S , then taking the set of states reachable from there by λ -transitions.
- Accepting states are any set of states where some state in the set is an accepting state.

NFA -> DFA (Subset Construction)

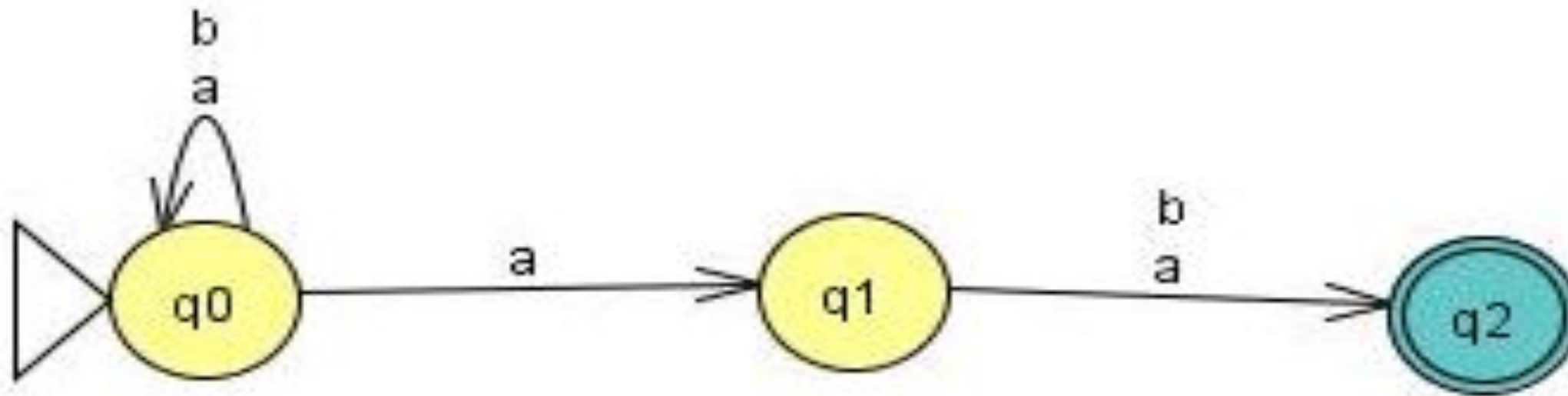
- In converting an NFA to a DFA, the DFA's states correspond to set of NFA states.
- In the worst-case, the construction can result in a DFA that is exponentially larger than the original NFA .
- Minimization of a DFA ensures that the resulting DFA (after minimization) has the least possible states. The advantages of having a minimal DFA are: Faster Execution: The more the number of states the more time the DFA will take to process a string, hence minimization ensures faster execution.

NFA $\rightarrow$ DFA (Subset Construction)

Given an NFA with states Q , inputs Σ , transition function δ_N , state state q_0 , and final states F , construct equivalent DFA with:

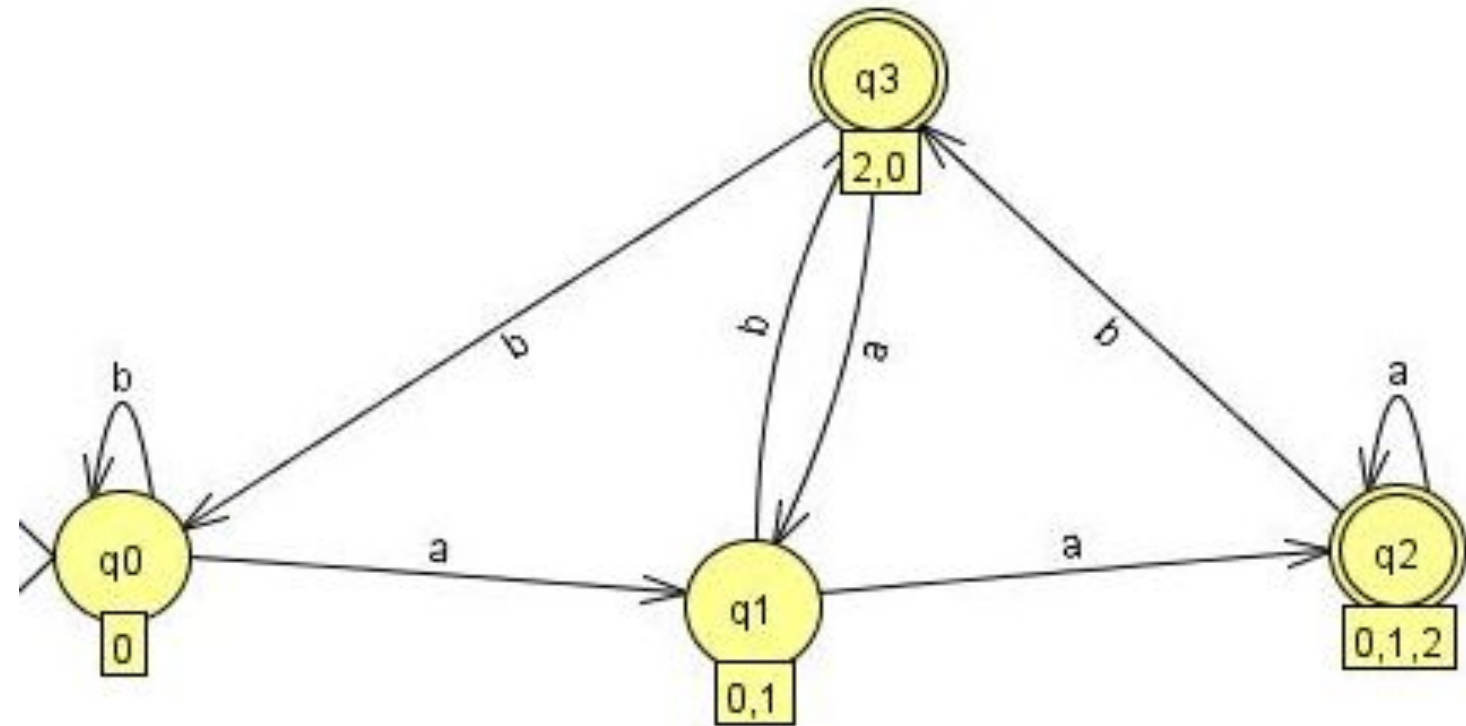
- * States 2^Q (Set of subsets of Q).
- * Inputs Σ .
- * Start state $\{q_0\}$.
- * Final states = all those with a member of F .
- The DFA states have names that are sets of NFA states.
- But as a DFA state, an expression like $\{p,q\}$ must be read as a single symbol, not as a set.

Example 1: Convert the following NFA, $\Sigma=\{a,b\}$, $L=\{\text{Strings where the second symbol from RHS is } a\}$ to DFA.



Solution:

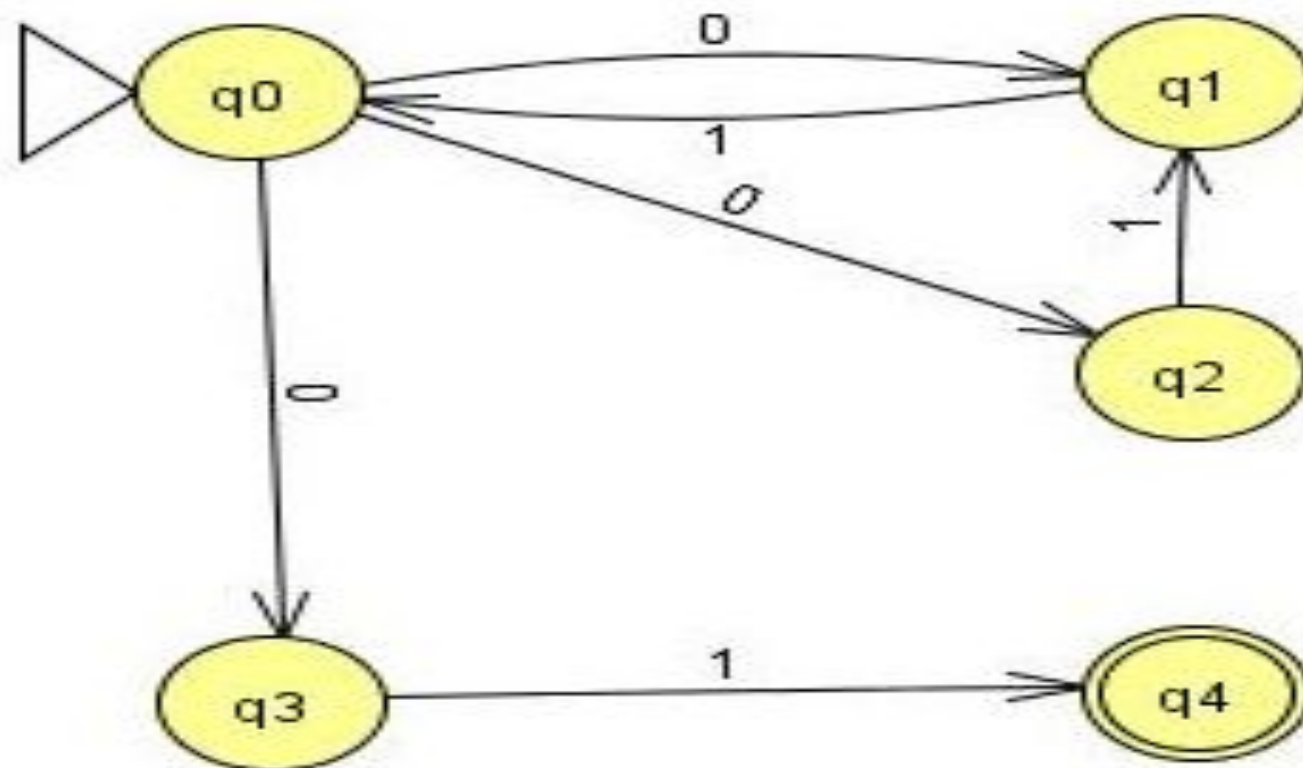
	a	b
q0	{q0,q1}	q0
{q0q1}	{q0,q1,q2}	{q0,q2}
{q0,q1,q2}	{q0,q1,q2}	{q0,q2}
{q0,q2}	{q0q1}	q0



States= [{q0}, {q0q1},{q0q1q2},{q1q2}]

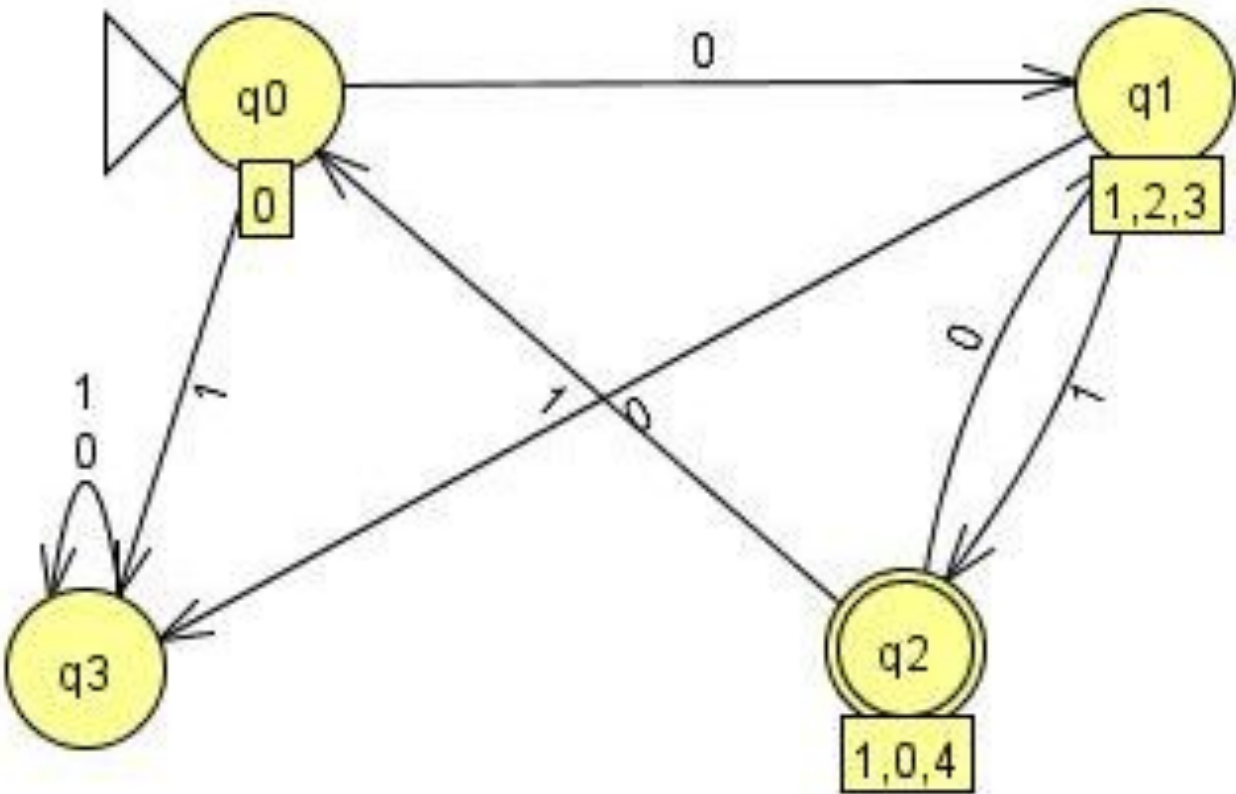
Can rename the states as $q0=q0$, $\{q0q1\}=q1$, $\{q0,q1,q2\}=q2$, $\{q0,q2\}=q3$

Example 2: Convert the following NFA to DFA.



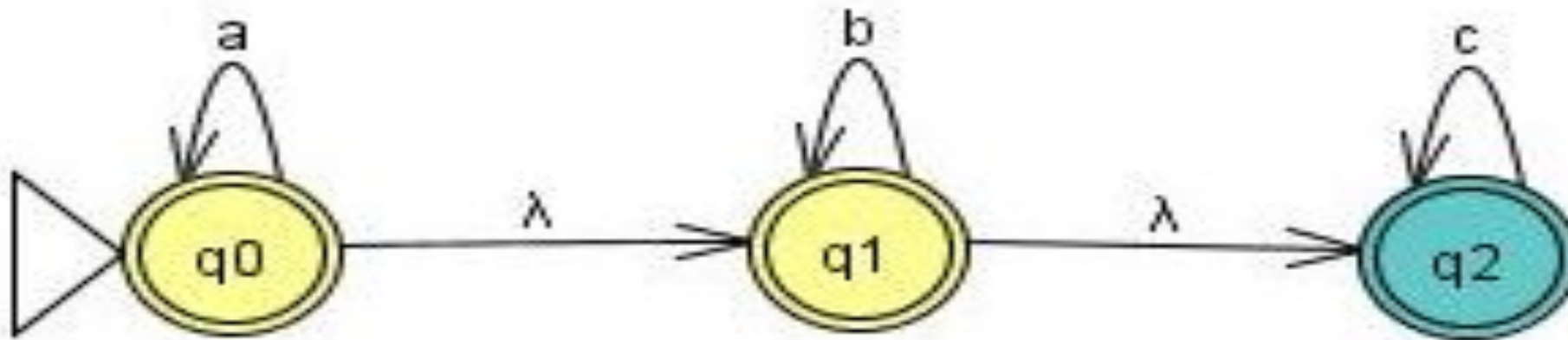
Solution:

	0	1
---> q0	{q1,q2,q3}	Φ
{q1,q2,q3}	Φ	{q0,q1,q4}
*{q0,q1,q4}	{q1,q2,q3}	q0
Φ	Φ	Φ



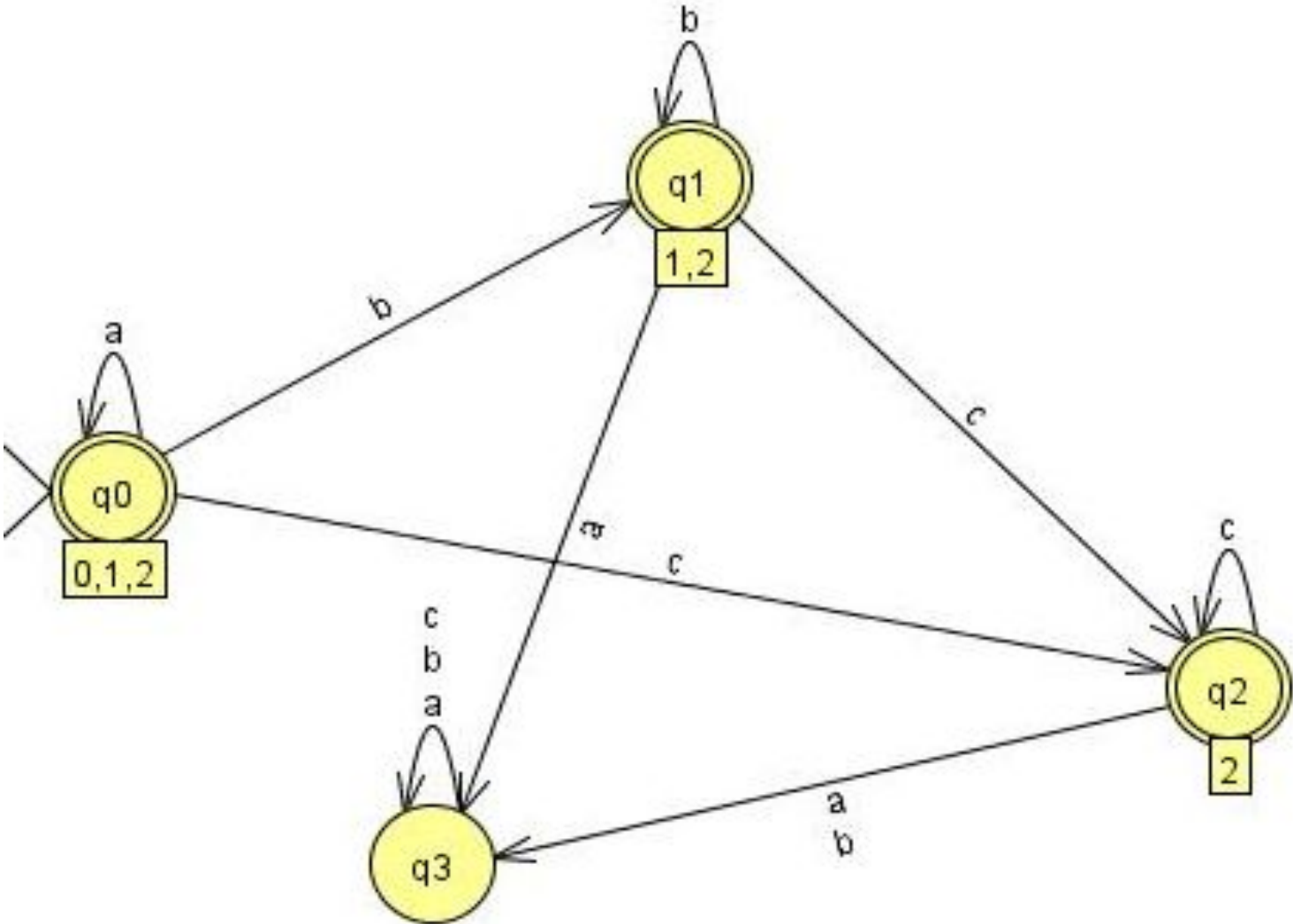
States: [{q0},{q0q1q2},{q0q1q4}, Φ]
Rename the states as q0=q0,q0q1q2=q1, q0q1q4=q2, Φ = q3

Example 3: Convert the following λ -NFA $L=\{a^n b^m c^k, n,m,k \geq 0\}$ to DFA.



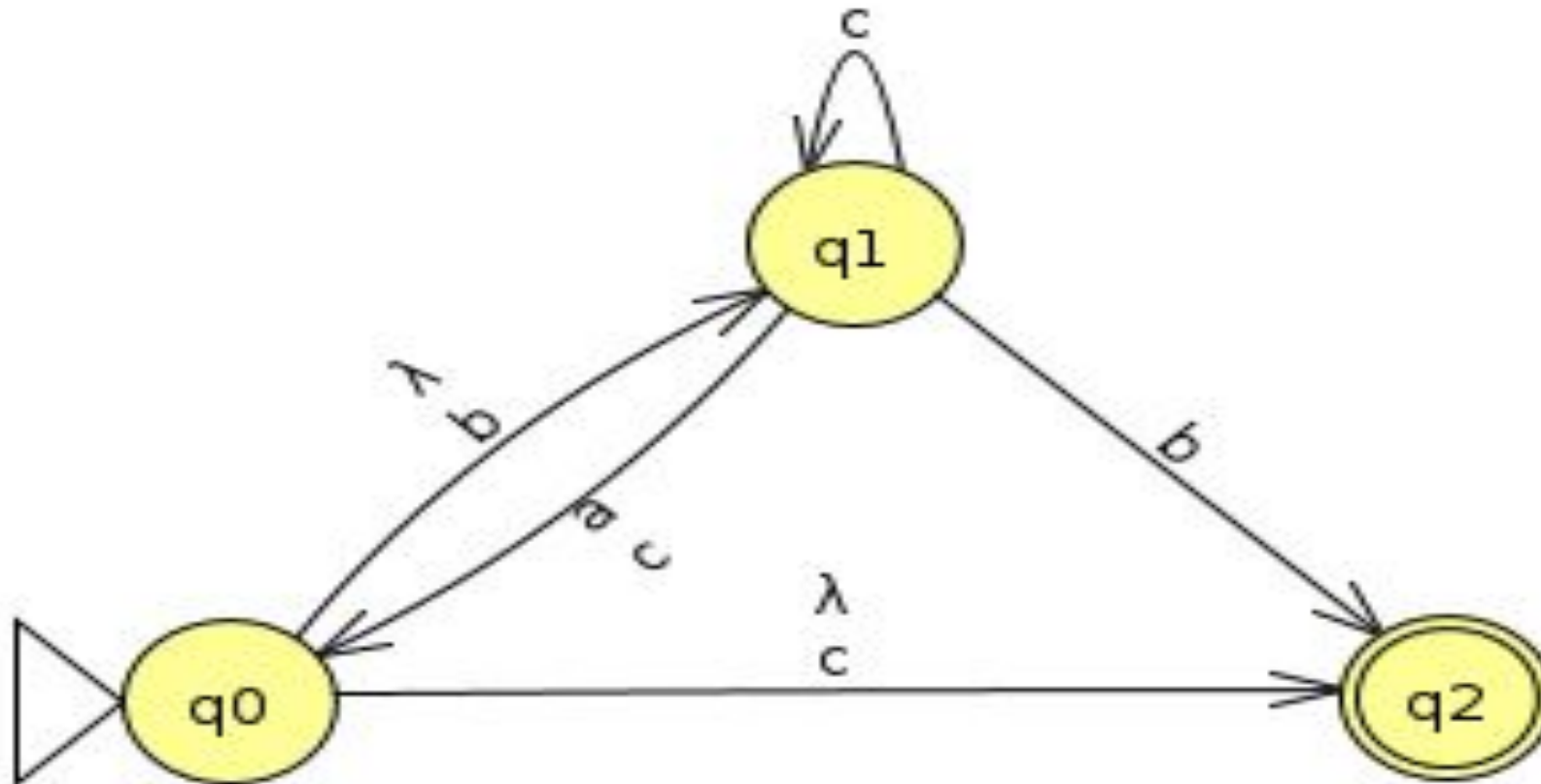
Solution:

	a	b	c
$\rightarrow^* \{q_0q_1q_2\}$	$\{q_0q_1q_2\}$	$\{q_1q_2\}$	q_2
$\{q_1q_2\}$	Φ	$\{q_1q_2\}$	q_2
$\{q_2\}$	Φ	Φ	q_2
Φ	Φ	Φ	Φ



Rename the states as:
 $\{q_0q_1q_2\}$ as q_0 , $\{q_1q_2\}$ as q_1 , q_2 as

Example 4: Convert the following λ -NFA to DFA.

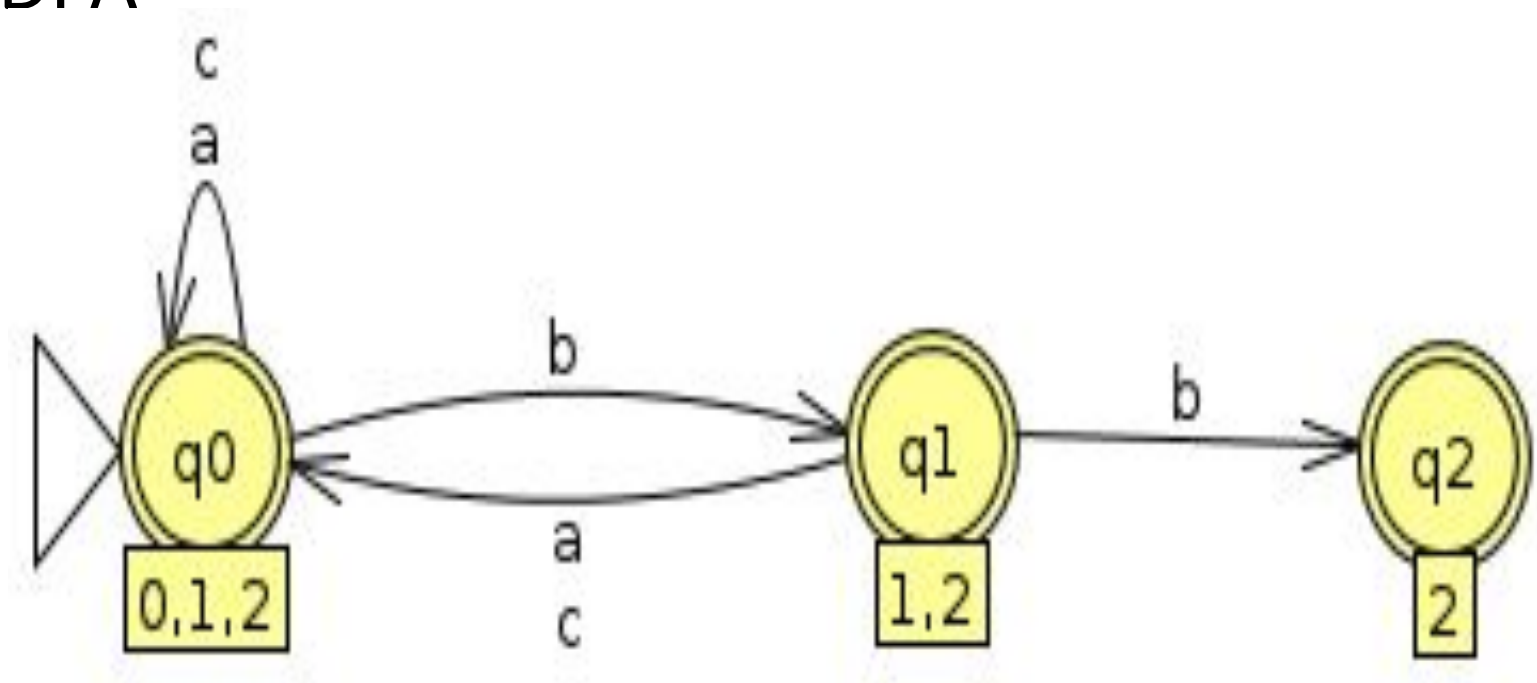


Solution:

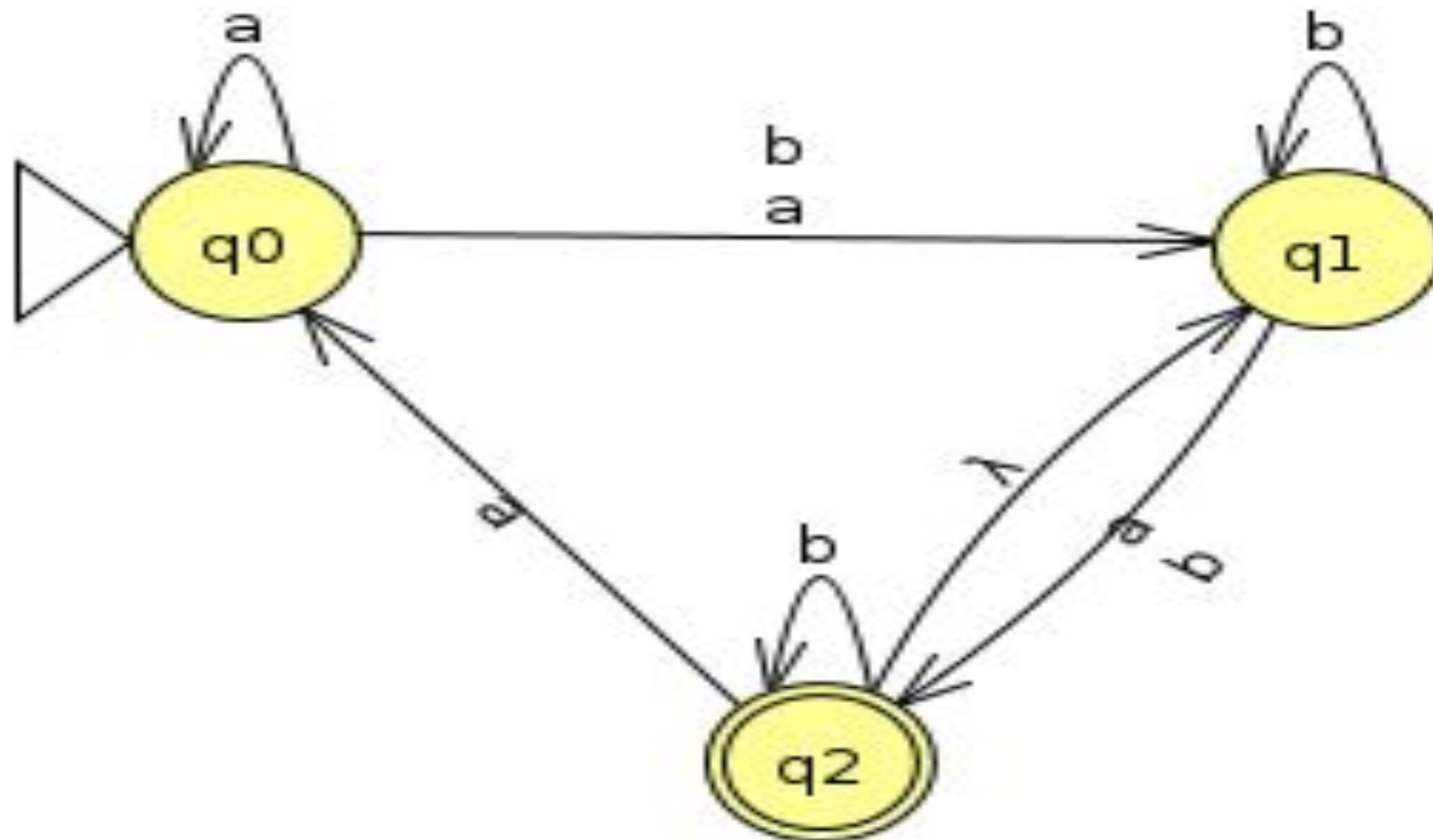
Transition Table

	a	b	c
-> *{q0q1q2}	{q0q1q2}	{q1q2}	{q0q1q2}
*{q1q2}	{q0q1q2}	q2	Φ
*q2	Φ	Φ	Φ
Φ	Φ	Φ	Φ

DFA



Example 5: Convert the following λ -NFA to DFA.

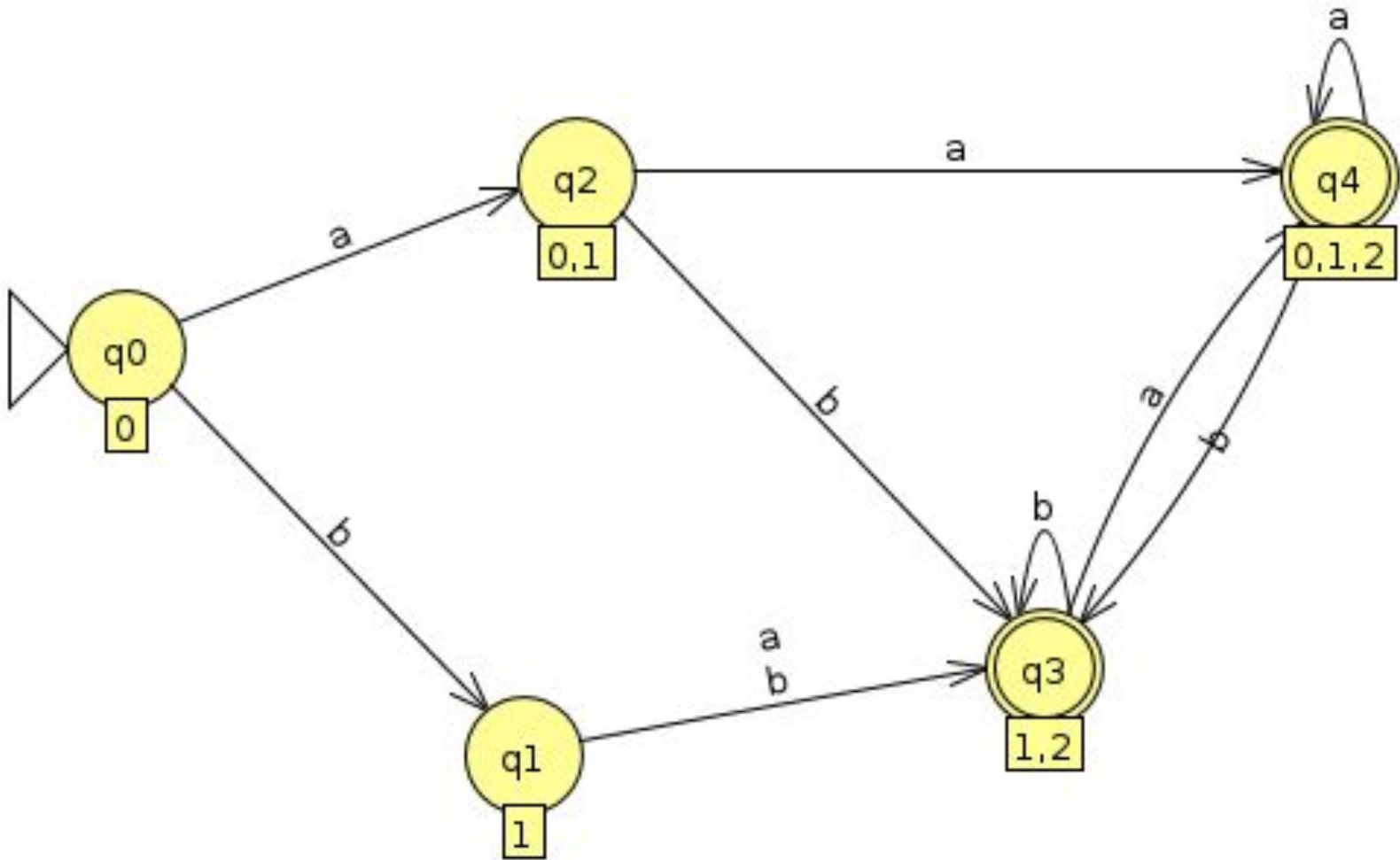


Solution:

Transition Table

	a	b
->q0	{q0q1}	q1
q1	{q1q2}	{q1q2}
{q0q1}	{q0q1q2}	{q0q2}
*{q1q2}	{q0q1q2}	{q1q2}
*{q0q1q2}	{q0q1q2}	{q1q2}

DFA





THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 1

Preet Kanwal

Department of Computer Science & Engineering

Minimization of DFA (Table Filling Algorithm)

- In converting an NFA to a DFA, the DFA's states correspond to set of NFA states.
- In the worst-case, the construction can result in a DFA that is exponentially larger than the original NFA .
- Minimization of a DFA ensures that the resulting DFA (after minimization) has the least possible states.
- The advantages of having a minimal DFA are: **Faster Execution:** The more the number of states the more time the DFA will take to process a string, hence minimization ensures faster execution.

Minimization of DFA (Table Filling Algorithm)

Step I : Eliminate Unreachable States

Step II : Mark Distinguishable Pair of States
(Final, Non-Final States)

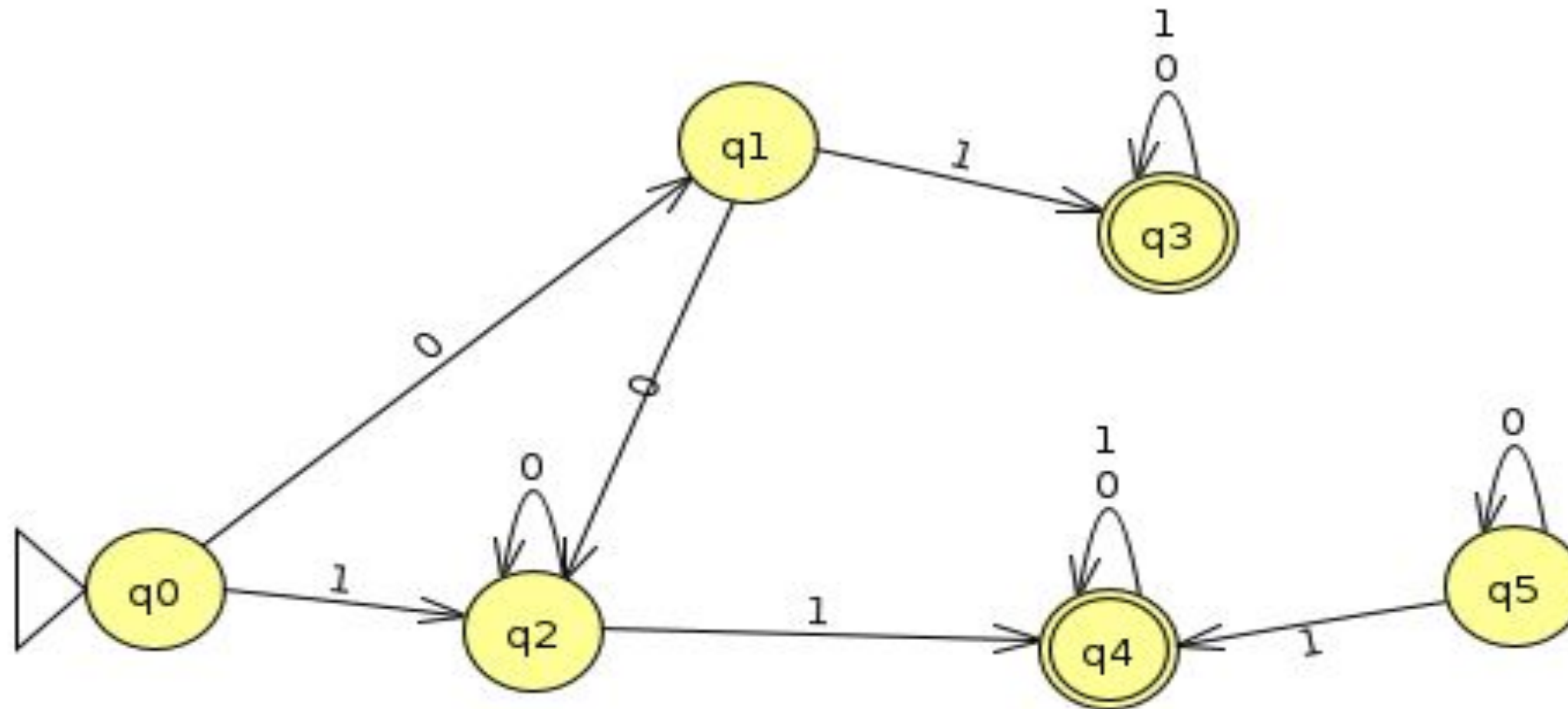
Step III : If there are any Unmarked pairs (M, N) make the following Check :

if $\delta(M, a) = X$ and $\delta(N, a) = Y$ && (X, Y) is marked then,
Mark (M, N) also.

Repeat Step III until no new states can be marked.

Step IV : Combine all the unmarked pairs and make them a single state in the minimized DFA.

Example 1: Minimize the following DFA.

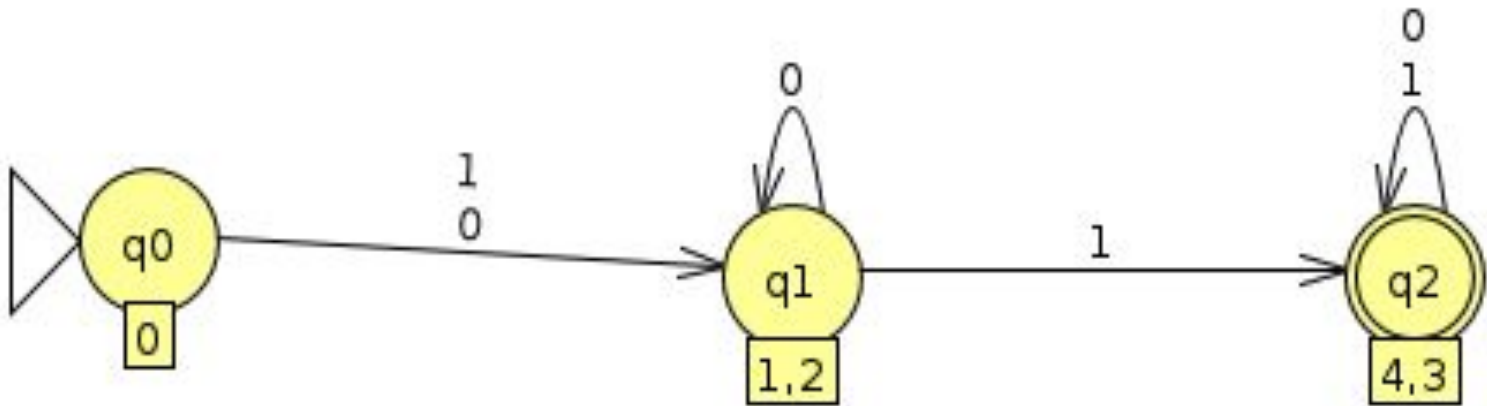


Solution:

Final Table

q1	X			
q2	X			
*q3	X	X	X	
*q4	X	X	X	
	q0	q1	q2	* q3

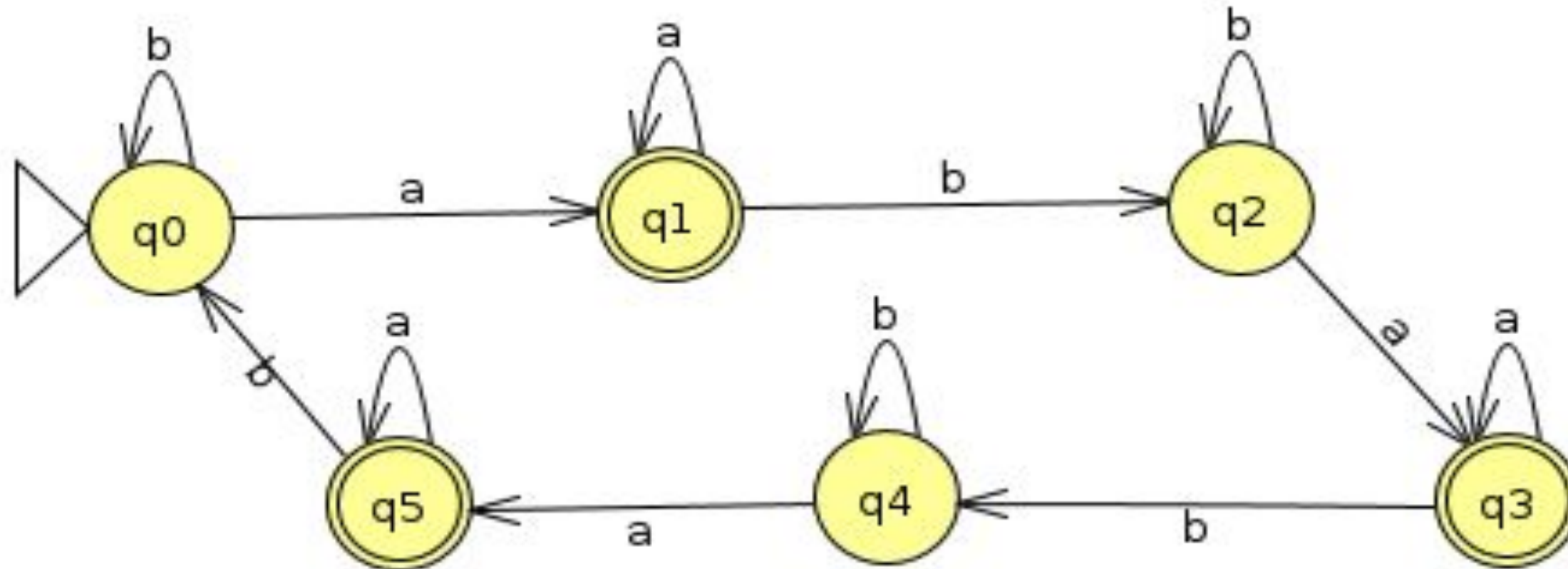
Minimized DFA



States merged :

- {q1, q2}
- {q3, q4}

Example 2 : Minimize the following DFA.

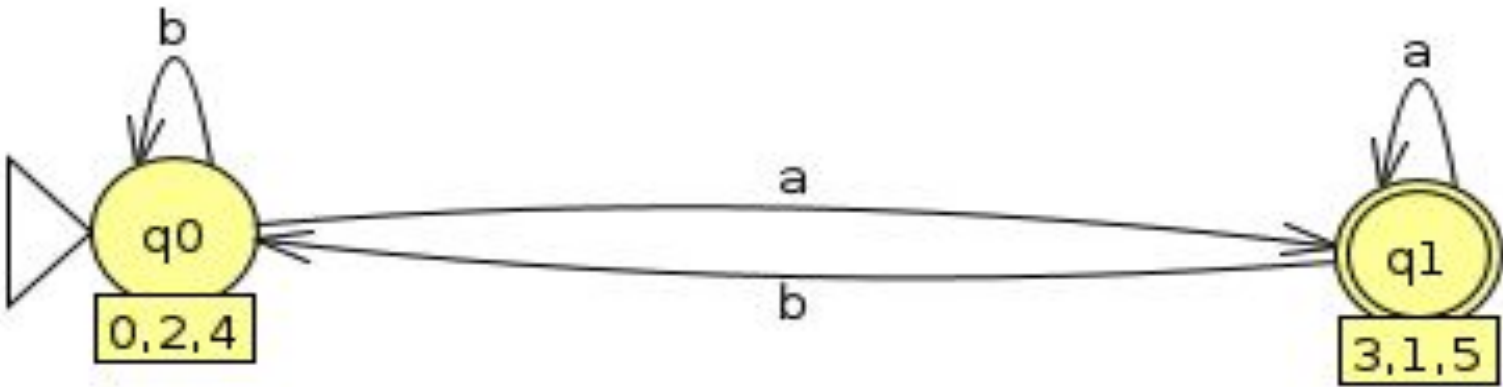


Solution:

Final Table

*q1	X				
q2		X			
*q3	X		X		
q4		X		X	
*q5	X		X		X
	q0	*q1	q2	*q3	q4

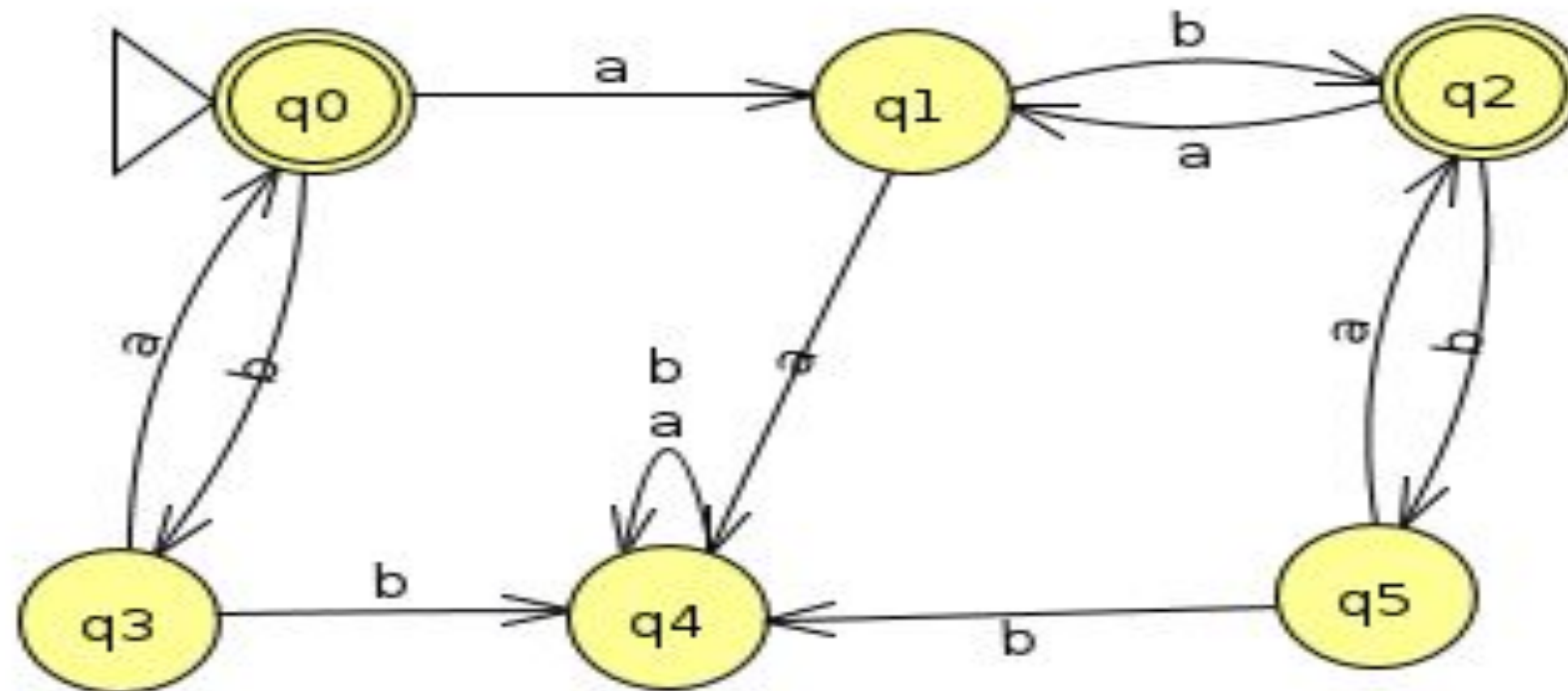
Minimized DFA



States merged :

- {q0,q4}, {q0, q2} and {q2, q4}
- {q1,q5}, {q1,q3} and {q3,q5}

Example 3: Minimize the following DFA.



Automata Formal Languages and Logic

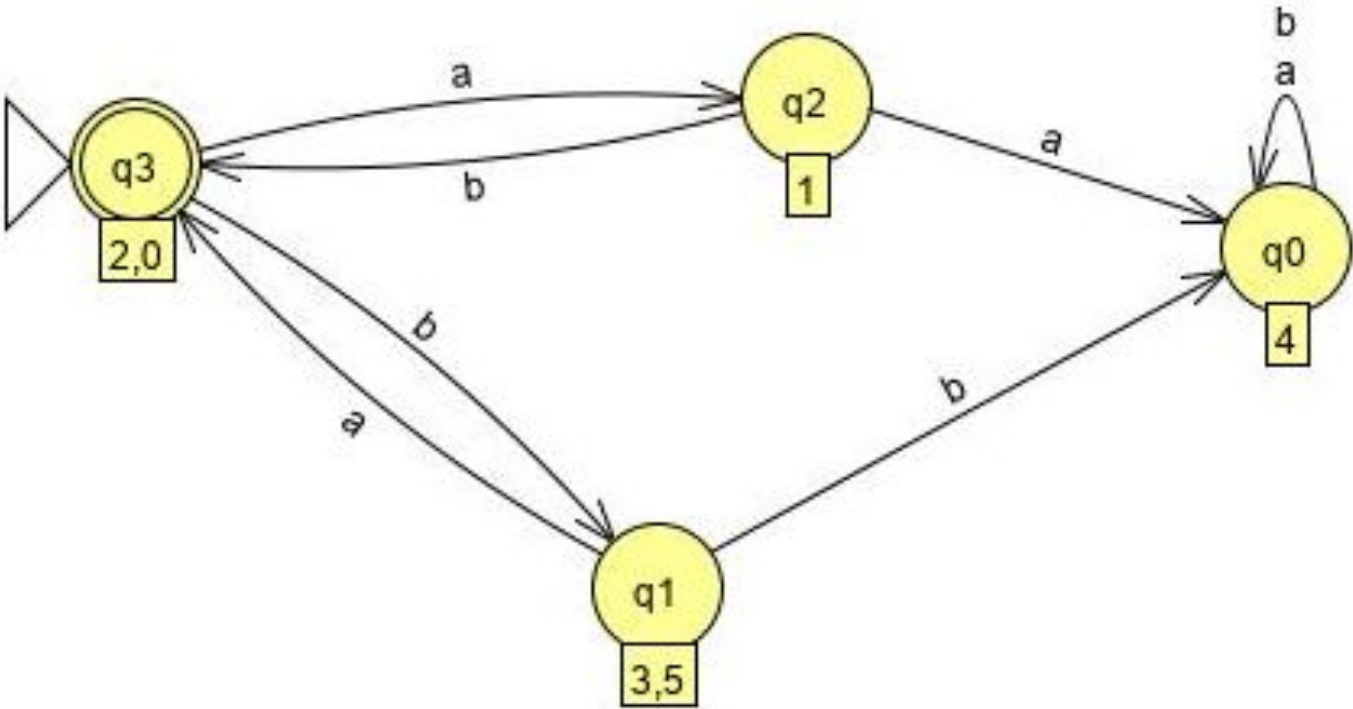
Unit 1 - Minimization of DFA

Solution:

Final Table

q1	X				
q2		X			
q3	X	X	X		
q4	X	X	X		
q5	X	X	X		X
	q0	q1	q2	q3	q4

Minimized DFA



- States merged :
- {q0, q2}
 - {q3, q5}



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Regex is an algebraic way to describe regular languages.

Regular Expression

Atoms

- A
Character/symbol
- ϕ ($L = \{\}$)
- λ ($L = \{\lambda\}$)

Operators

- (R)
- R^*
- $R_1.R_2$ or R_1R_2
- $R_1 + R_2$ or $R_1 | R_2$

Example

$a b^* c + d$

Automata Formal Languages and Logic

Unit 2 - Example of Regular Expressions

Example

$$\underbrace{(a \ b^* \ c)}_{R_1} + \underbrace{d}_{R_2}$$

or

Example

$(a b^* c) + d$

Starting with
an 'a'

Followed by 0 or
more no. of b's

Ending with
a 'c'

Example

$(a b^* c) + d$

String that
contains only d

Starting with
an 'a'

Followed by 0 or
more no. of b's

Ending with
a 'c'

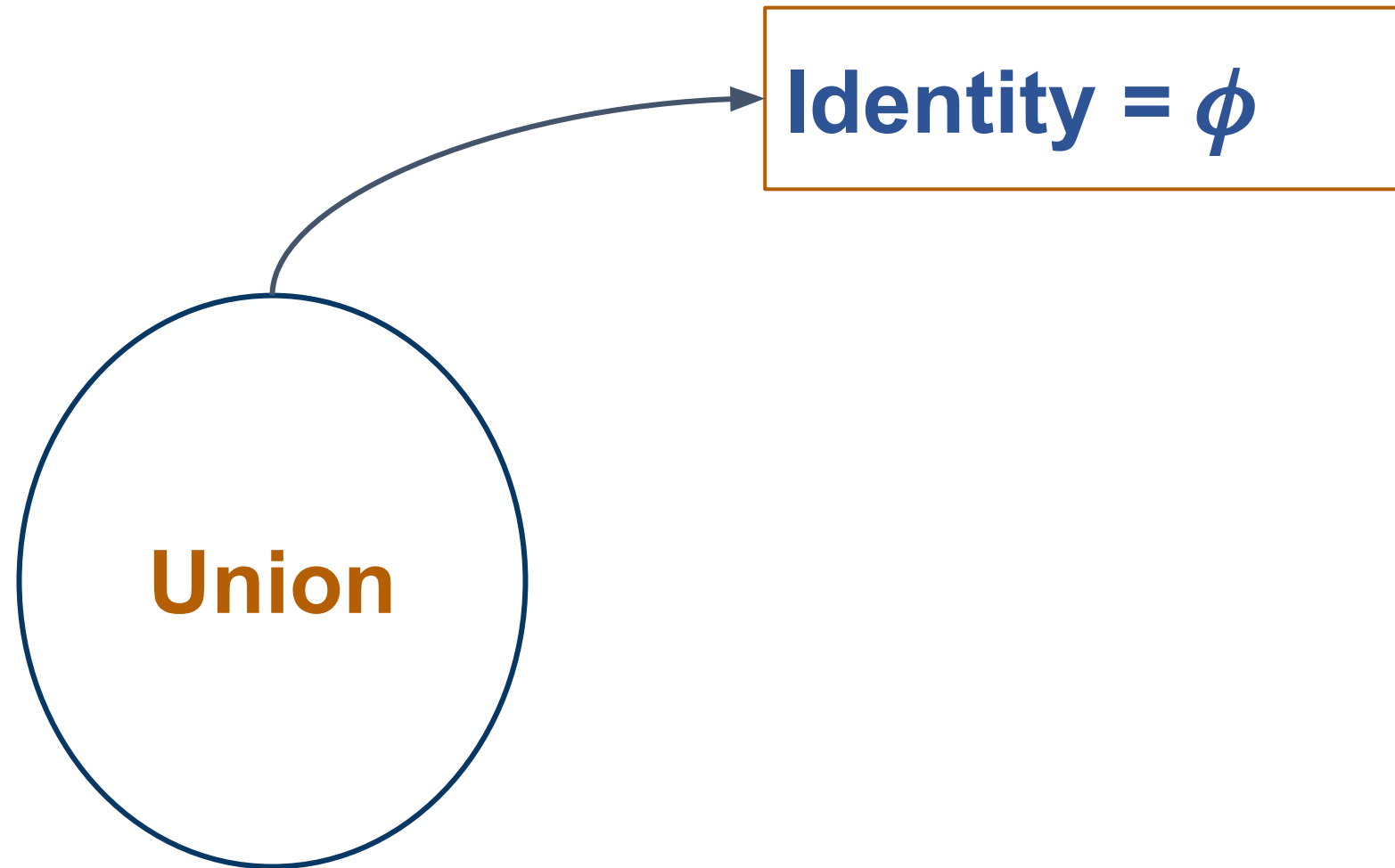
Example

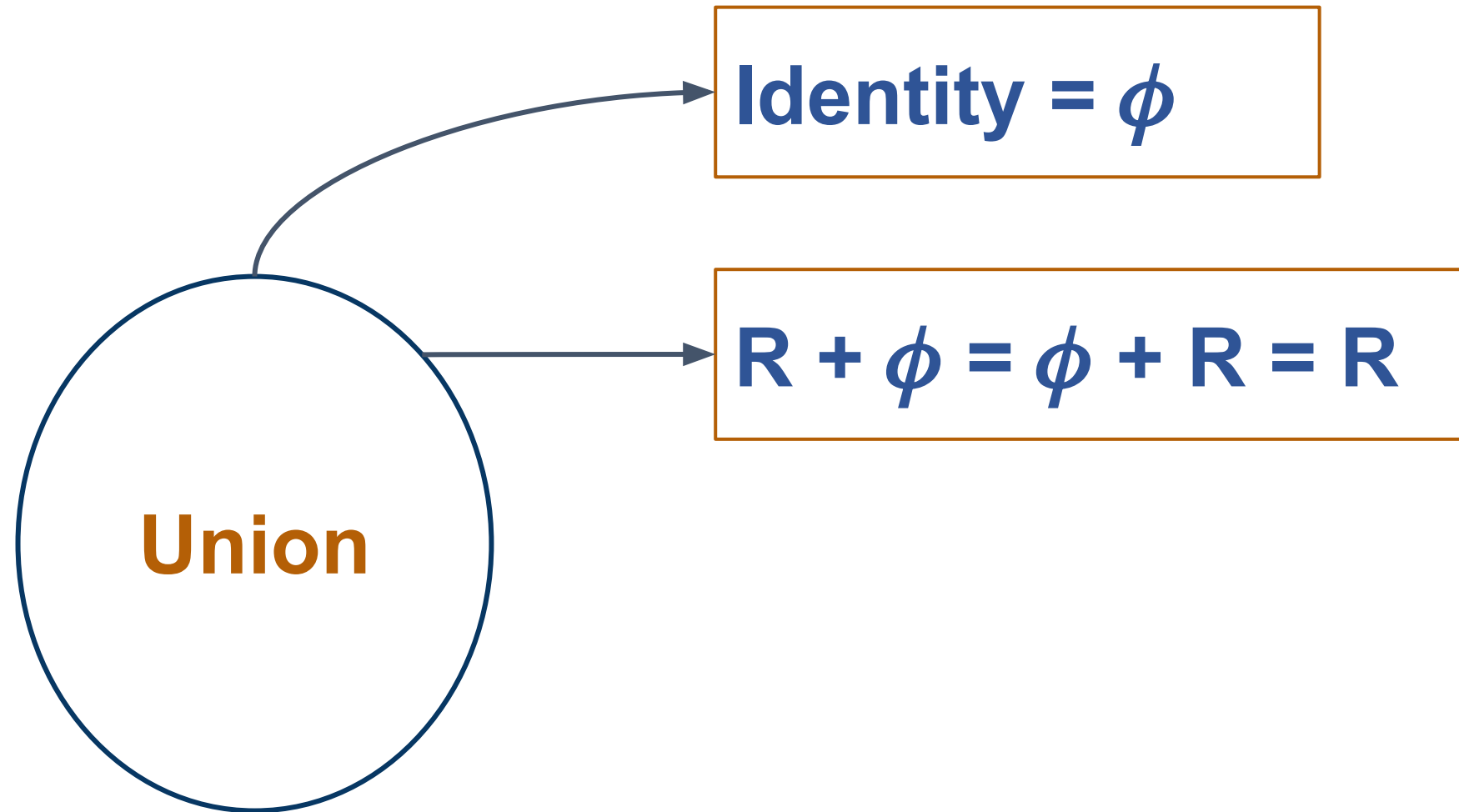
$(a \ b^* \ c) + d$

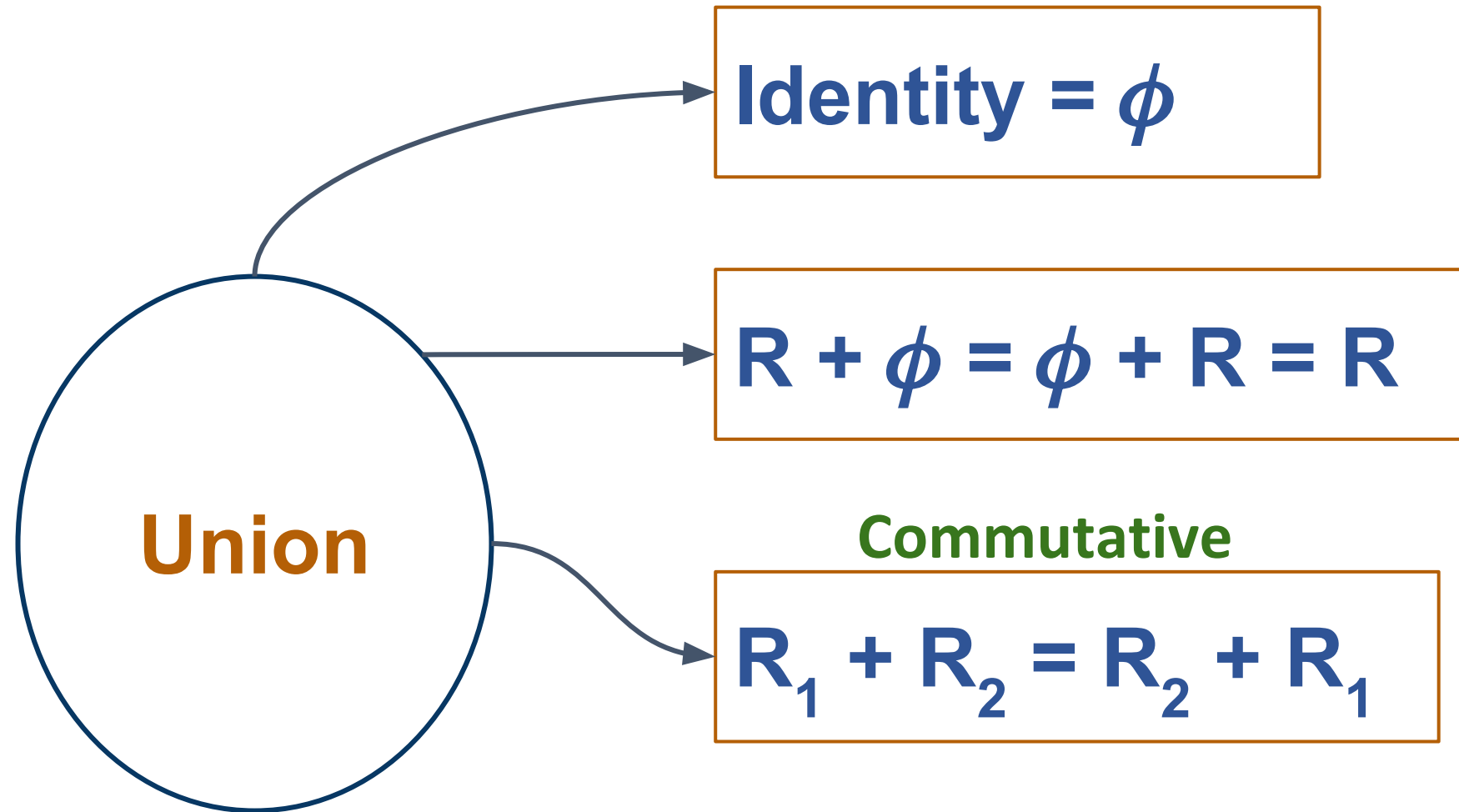
$L = \{ ac, \\ abc, \\ abbb\dots c, \\ d \}$

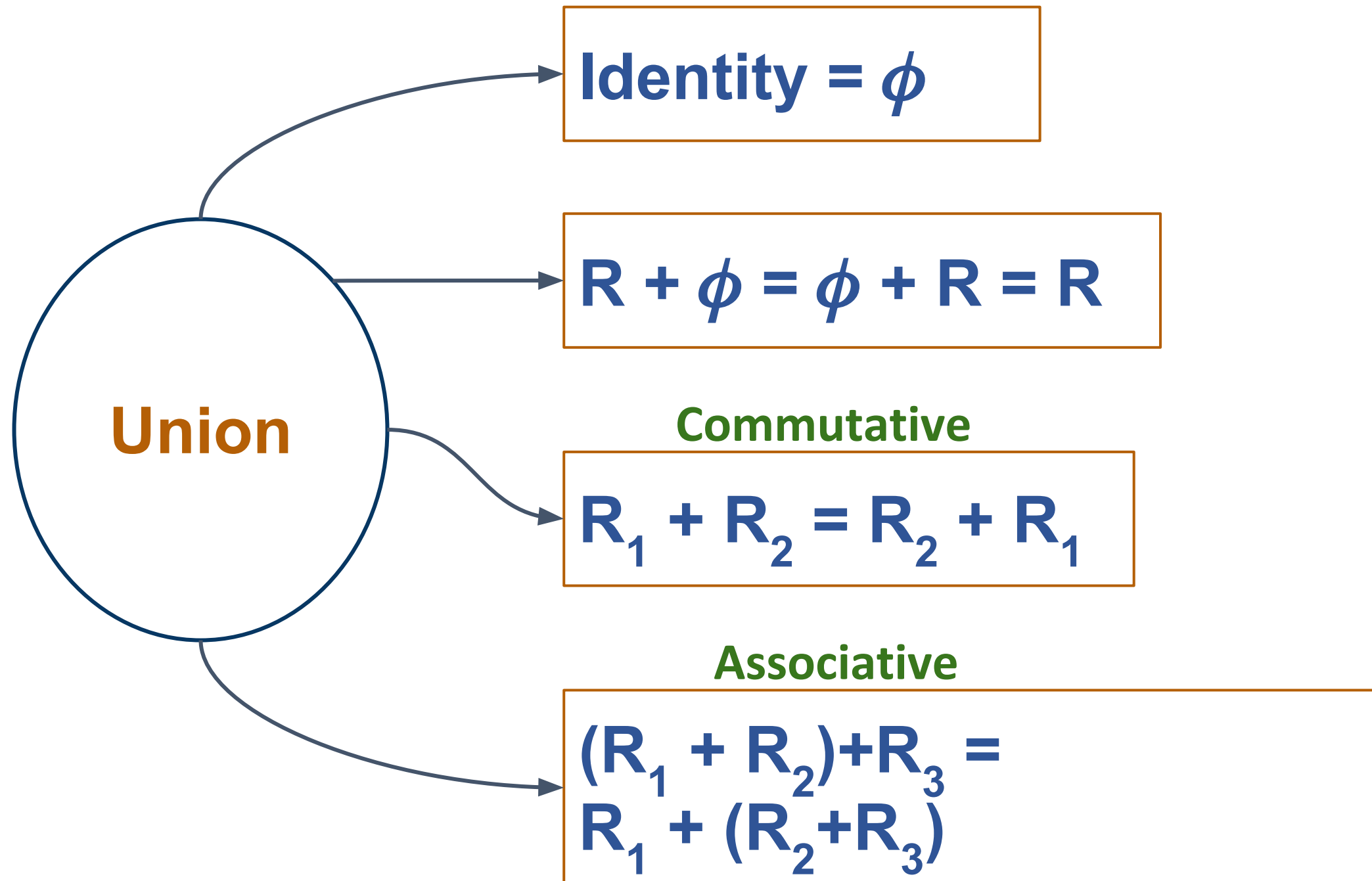
Automata Formal Languages and Logic

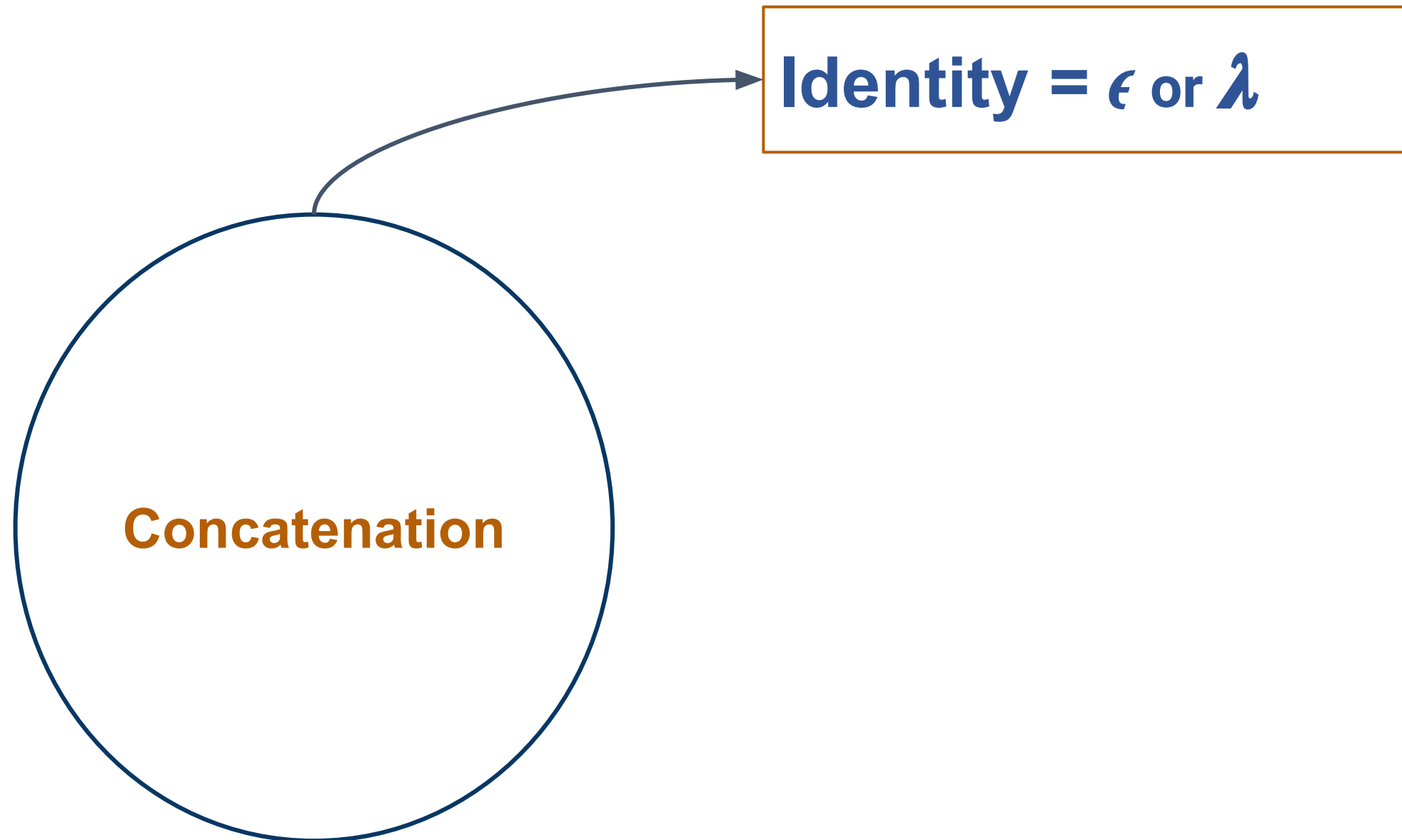
Unit 2 - Algebraic Laws of Regular Expressions

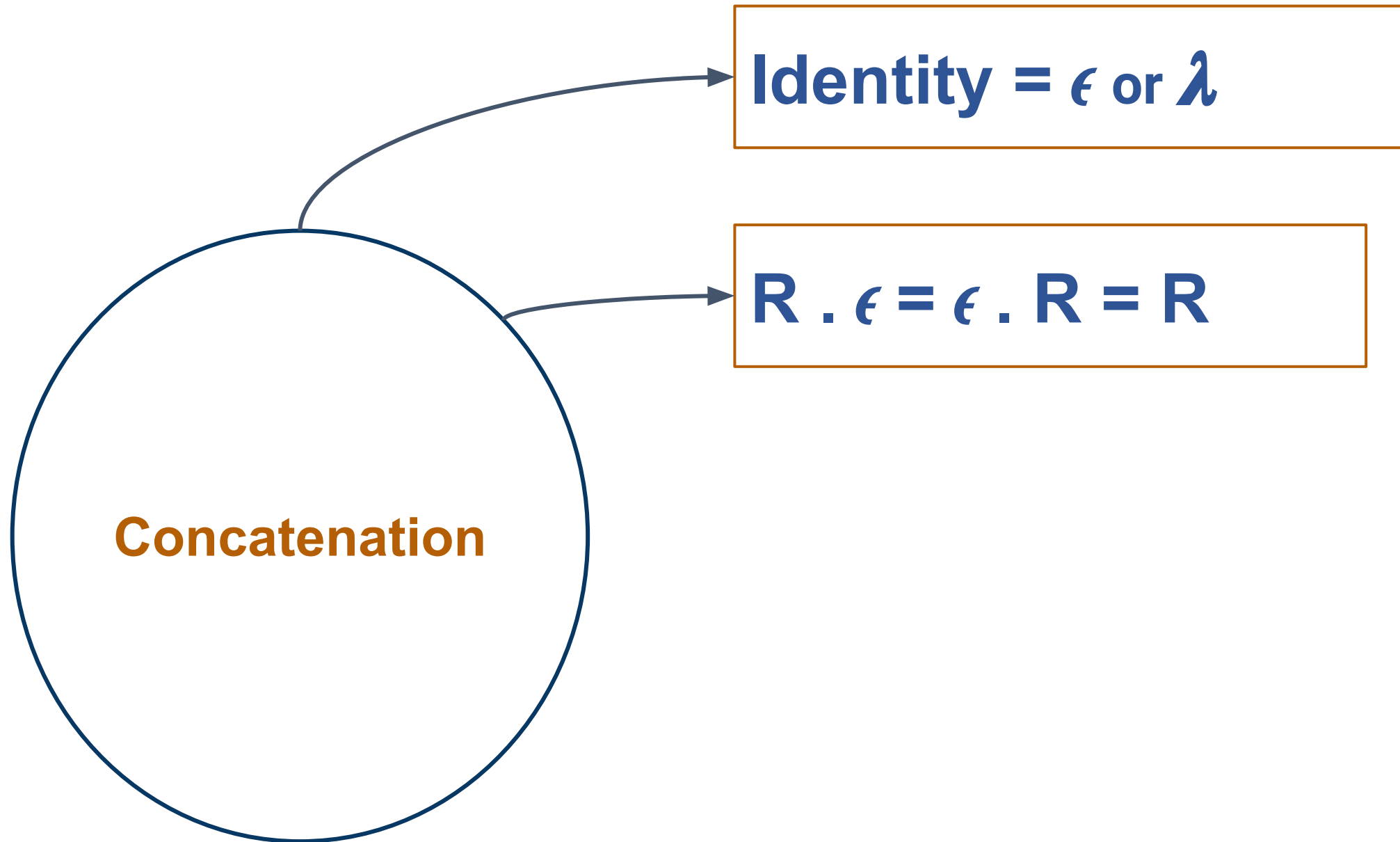






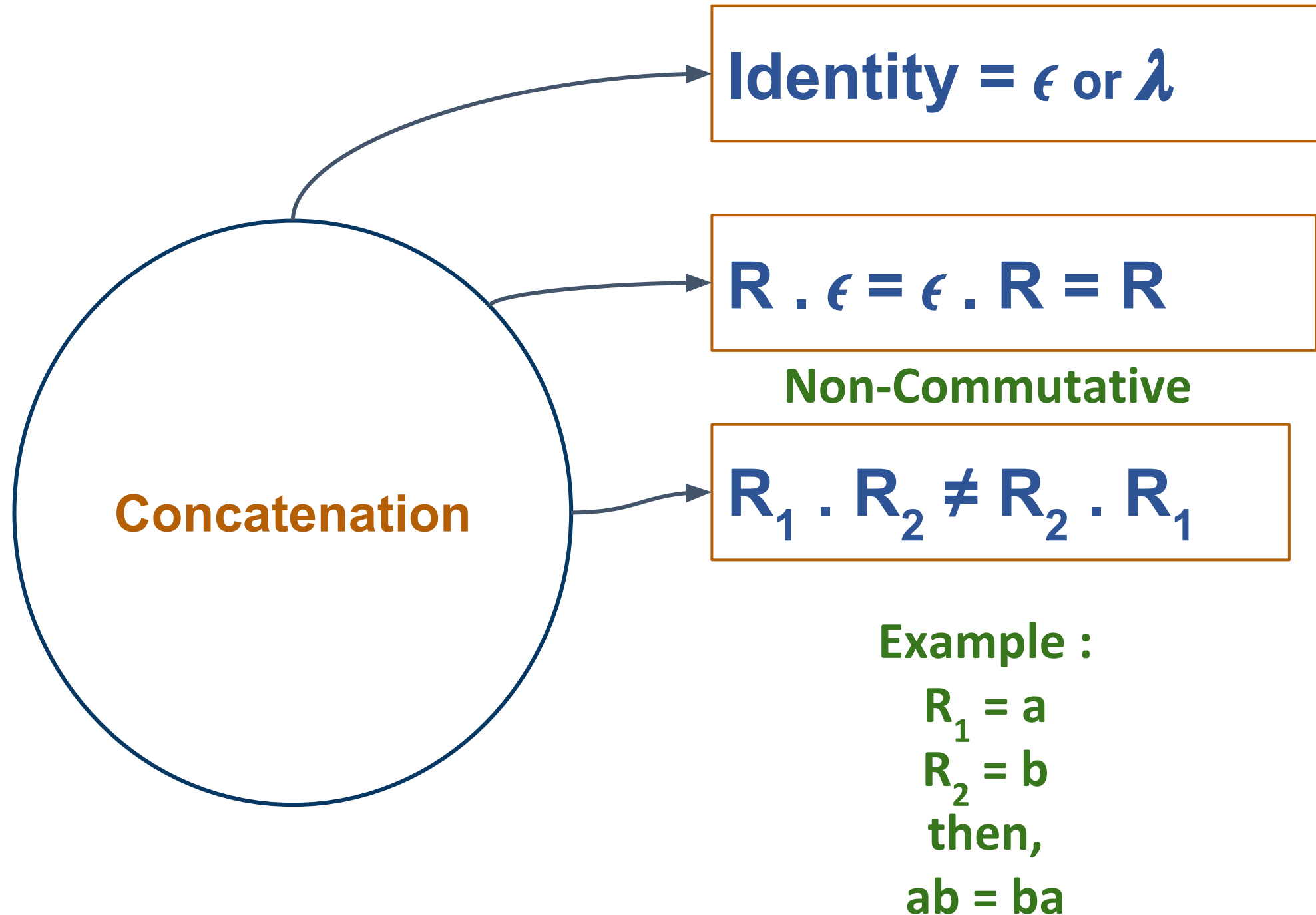


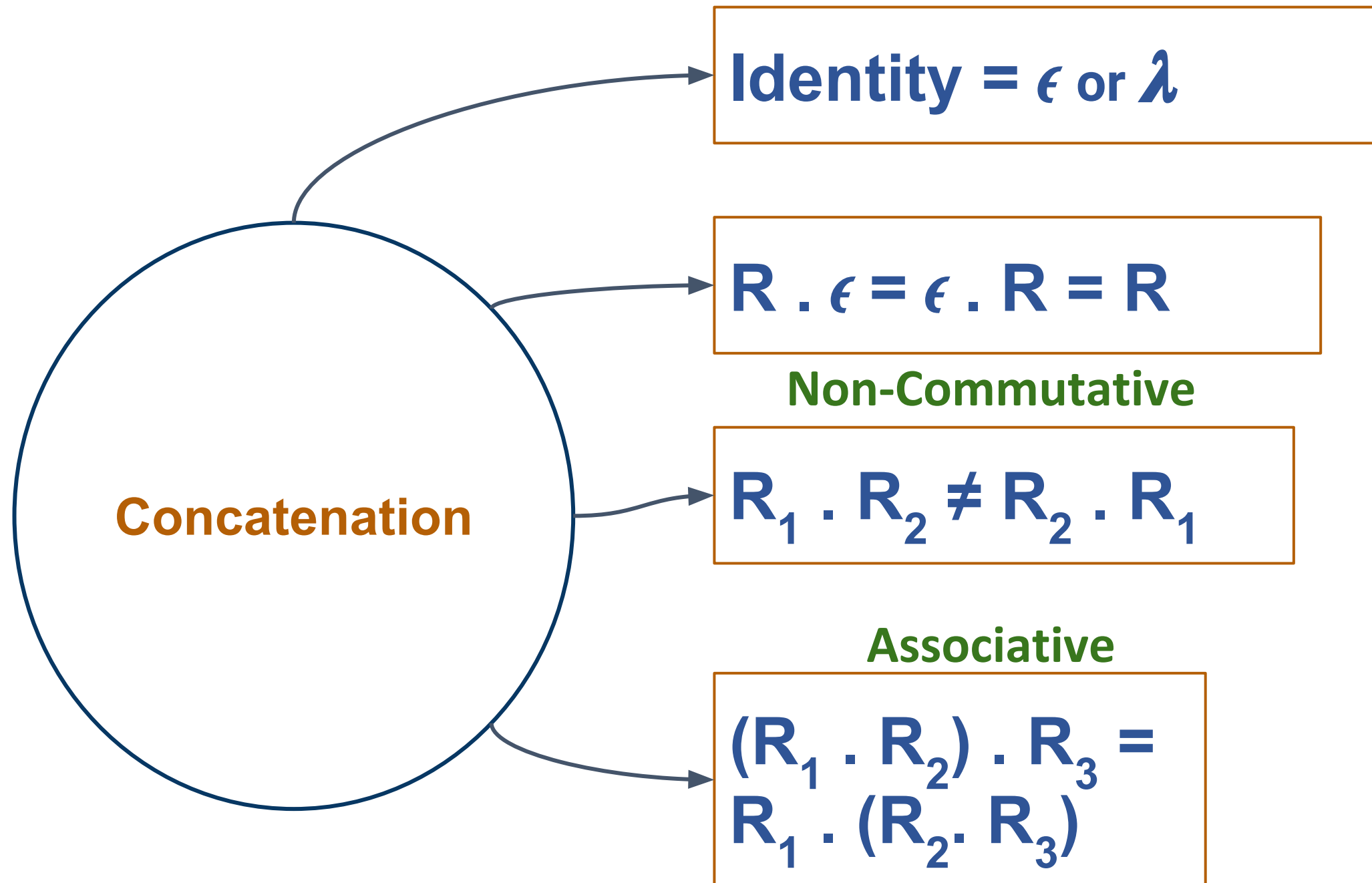




$$R \cdot \phi = \phi \cdot R = \phi$$

Let L_1, L_2 be languages, then the concatenation $L_1 \circ L_2 = \{w \mid w = xy, x \in L_1, y \in L_2\}$. If $L_2 = \emptyset$, then there is no string $y \in L_2$ and so there is no possible w such that $w = xy$. Thus for any L_1 , we'll have $L_1 \circ \emptyset = \emptyset$.





Closure

$$\epsilon^* = \epsilon$$

$$\phi^* = \epsilon$$

$$(R^*)^* = R^*$$

$$(R + \epsilon)^* = R^*$$

$$R^+ = RR^* = R^*R$$

$$(R^+ + \epsilon) = R^*$$

$$\epsilon + RR^* = \epsilon + R^* = R^*$$

Closure

$$\begin{aligned} & (a + b)^* \\ &= (a^* + b^*)^* \\ &= (a^* b^*)^* \\ &= (a + b^*)^* \\ &= (a^* + b)^* \\ &= a^*(ba^*)^* \\ &= b^*(ab^*)^* \end{aligned}$$

Construct a Regular Expression for a given language L



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Regex is an algebraic way to describe regular languages.

Regular Expression

Atoms

- A
Character/symbol
- ϕ ($L = \{\}$)
- λ ($L = \{\lambda\}$)

Operators

- (R)
- R^*
- $R_1.R_2$ or R_1R_2
- $R_1 + R_2$ or $R_1 | R_2$

Example

$a b^* c + d$

Example

$$\underbrace{(a \ b^* \ c)}_{R_1} + \underbrace{d}_{R_2}$$

or

Example

$(a b^* c) + d$

Starting with
an 'a'

Followed by 0 or
more no. of b's

Ending with
a 'c'

Example

$(a b^* c) + d$

String that
contains only d

Starting with
an 'a'

Followed by 0 or
more no. of b's

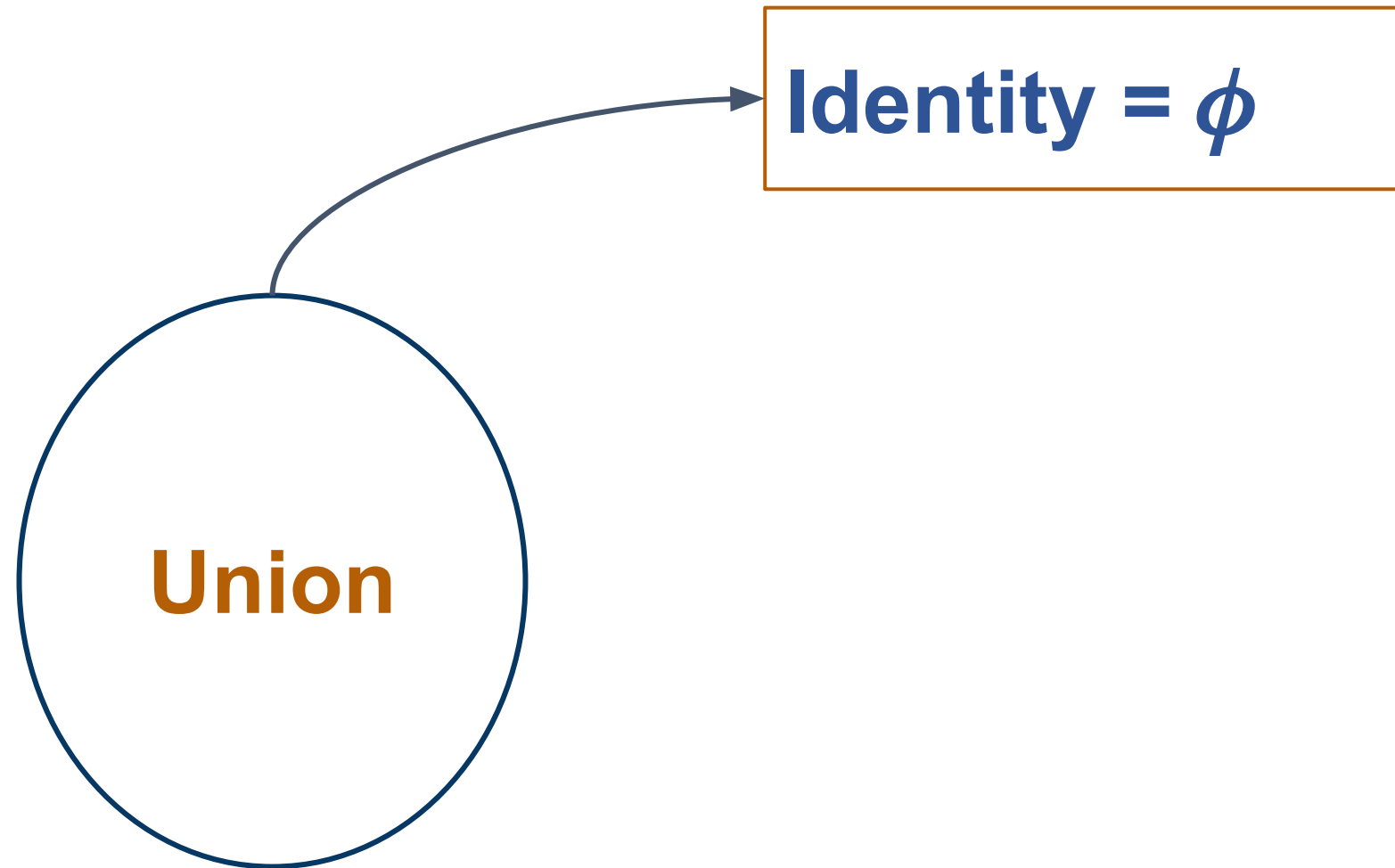
Ending with
a 'c'

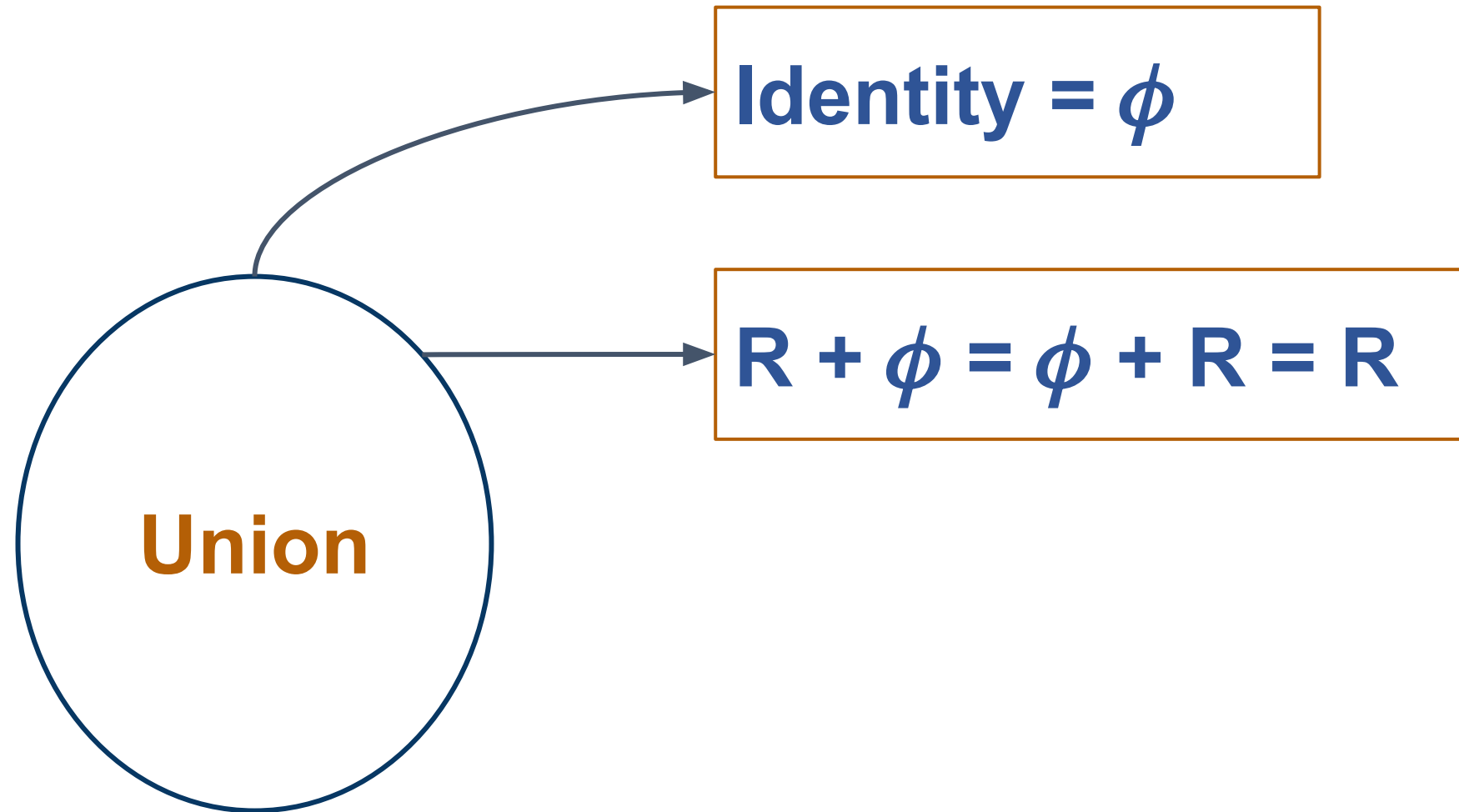
Example

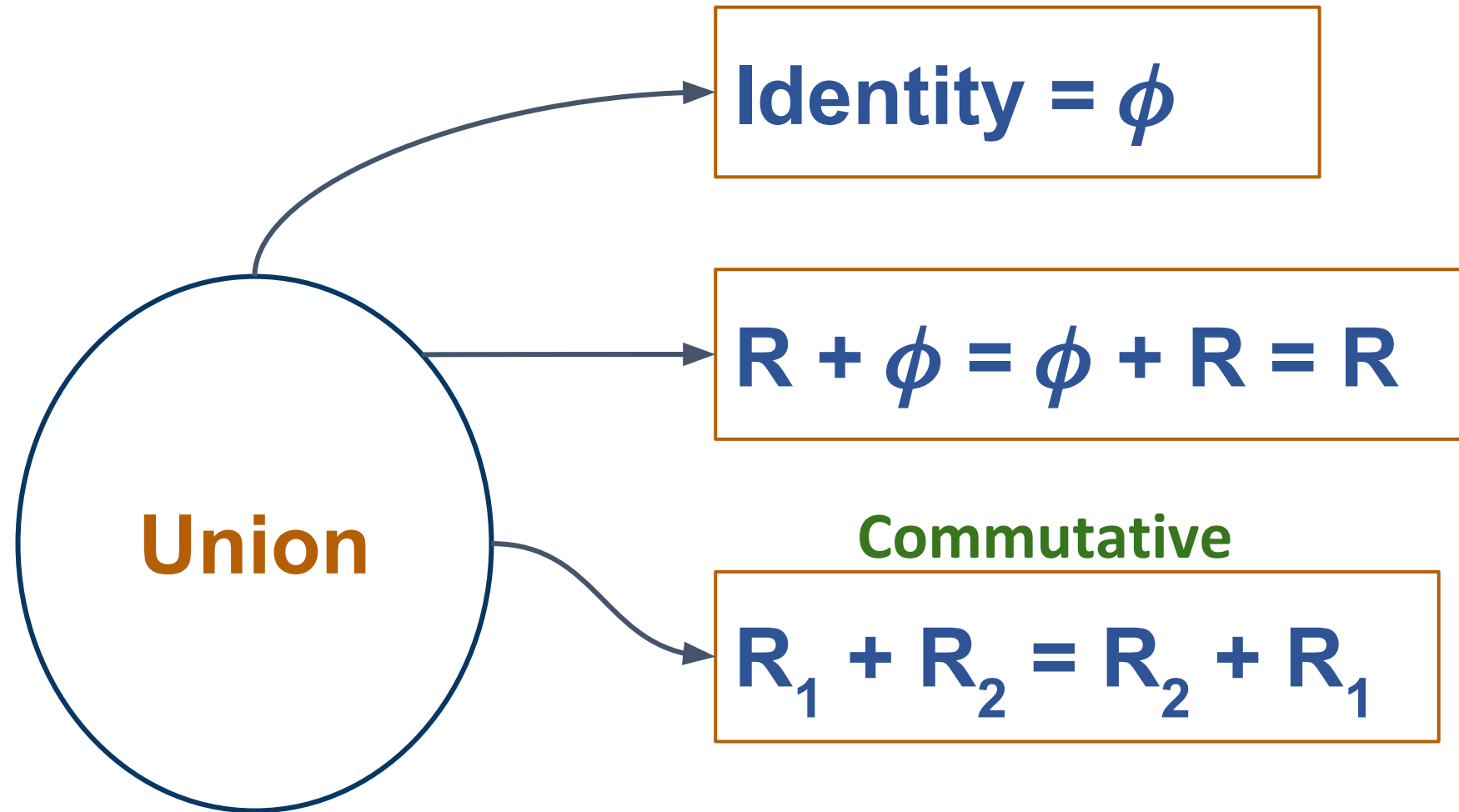
$$(a \ b^* \ c) + d$$
$$L = \{ ac, \\ abc, \\ abbb\dots c, \\ d \}$$

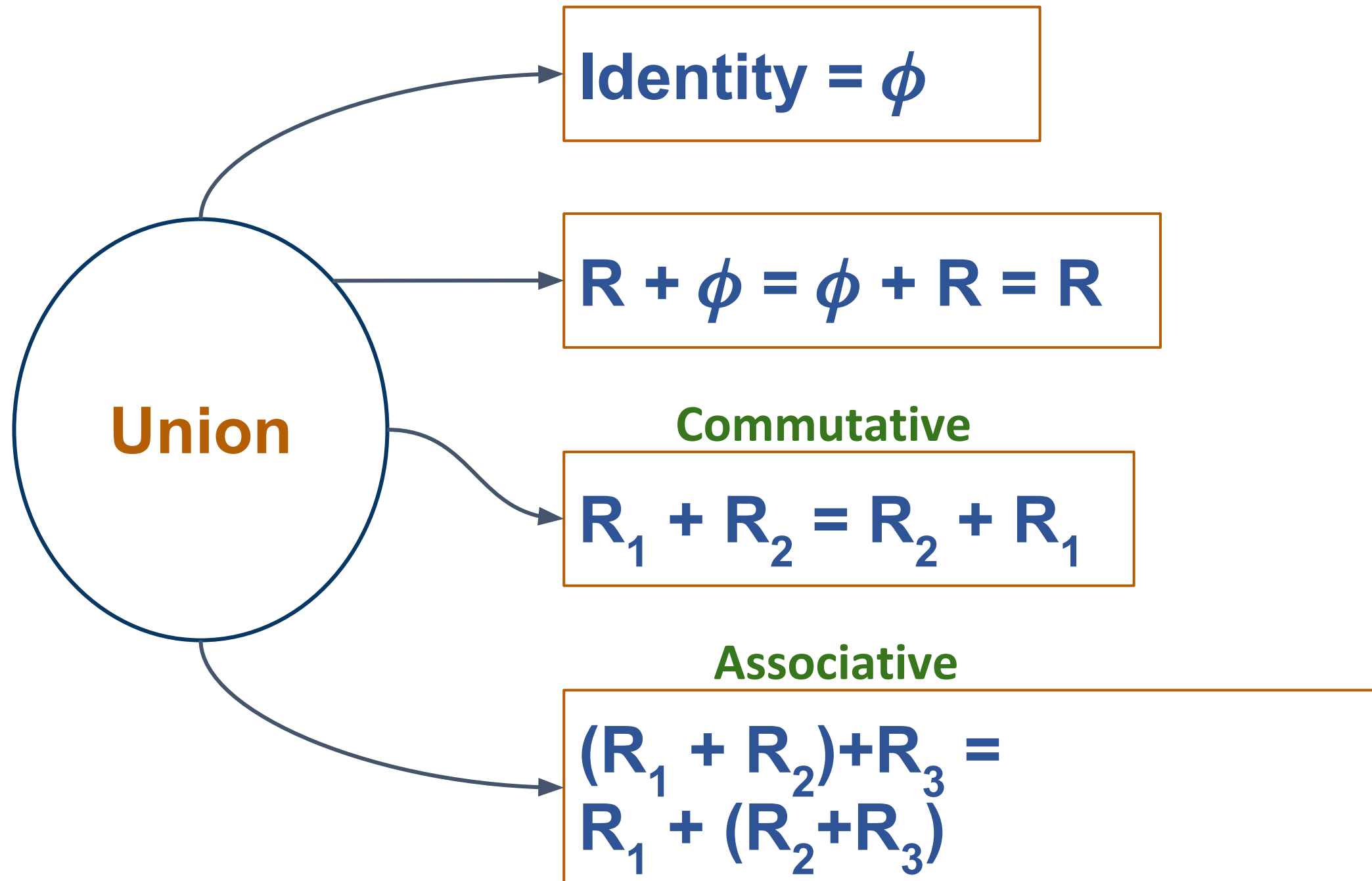
Automata Formal Languages and Logic

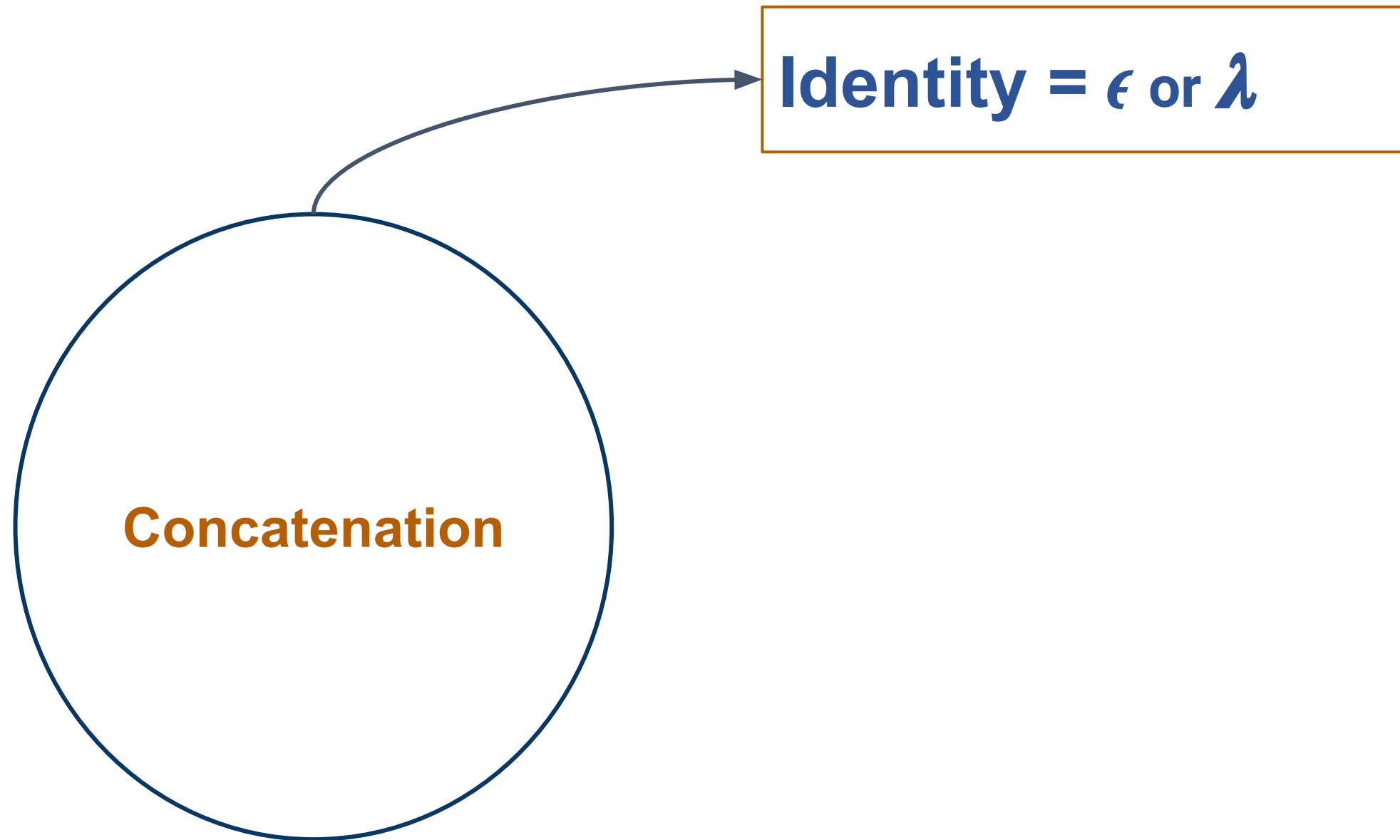
Unit 2 - Algebraic Laws of Regular Expressions

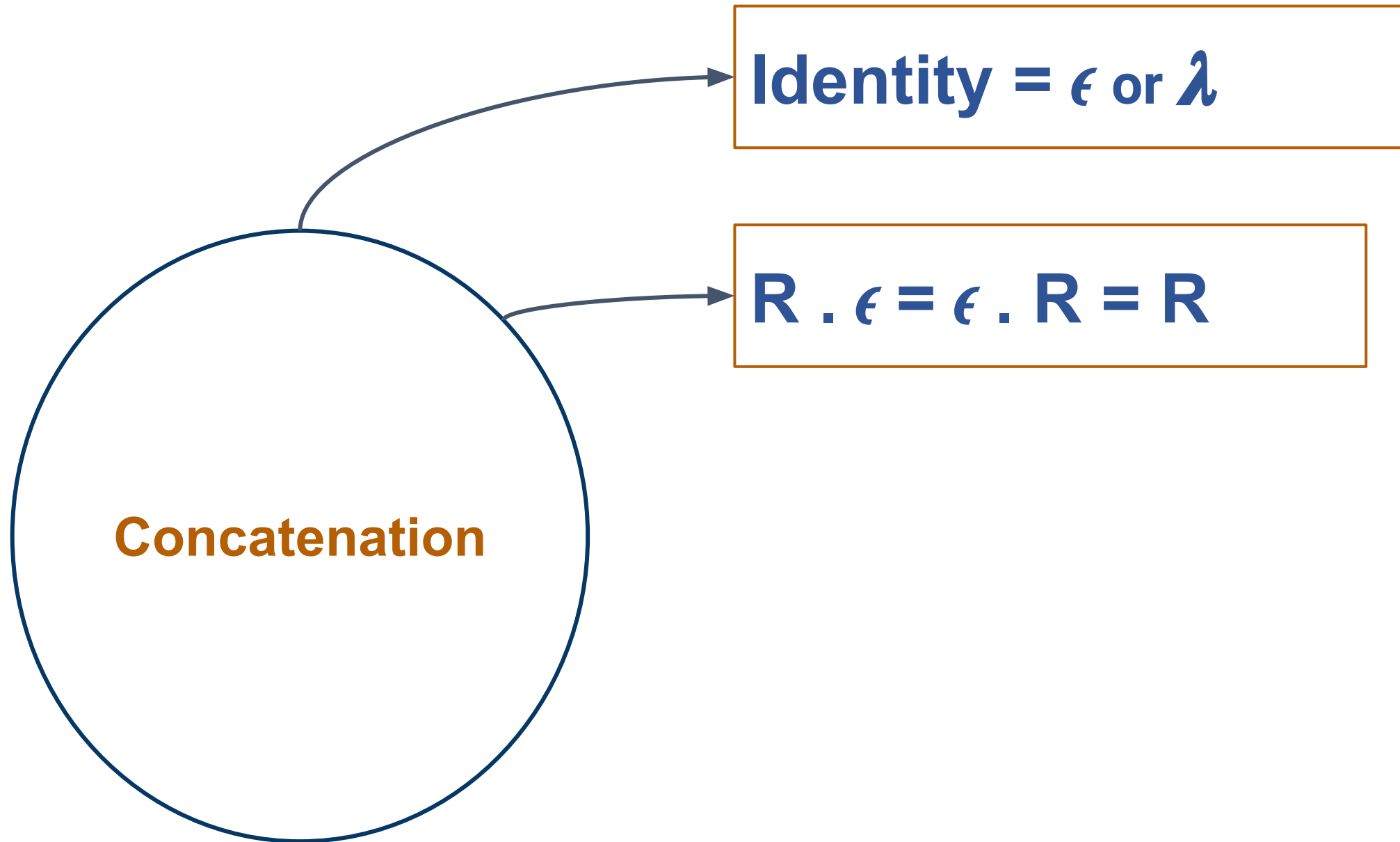






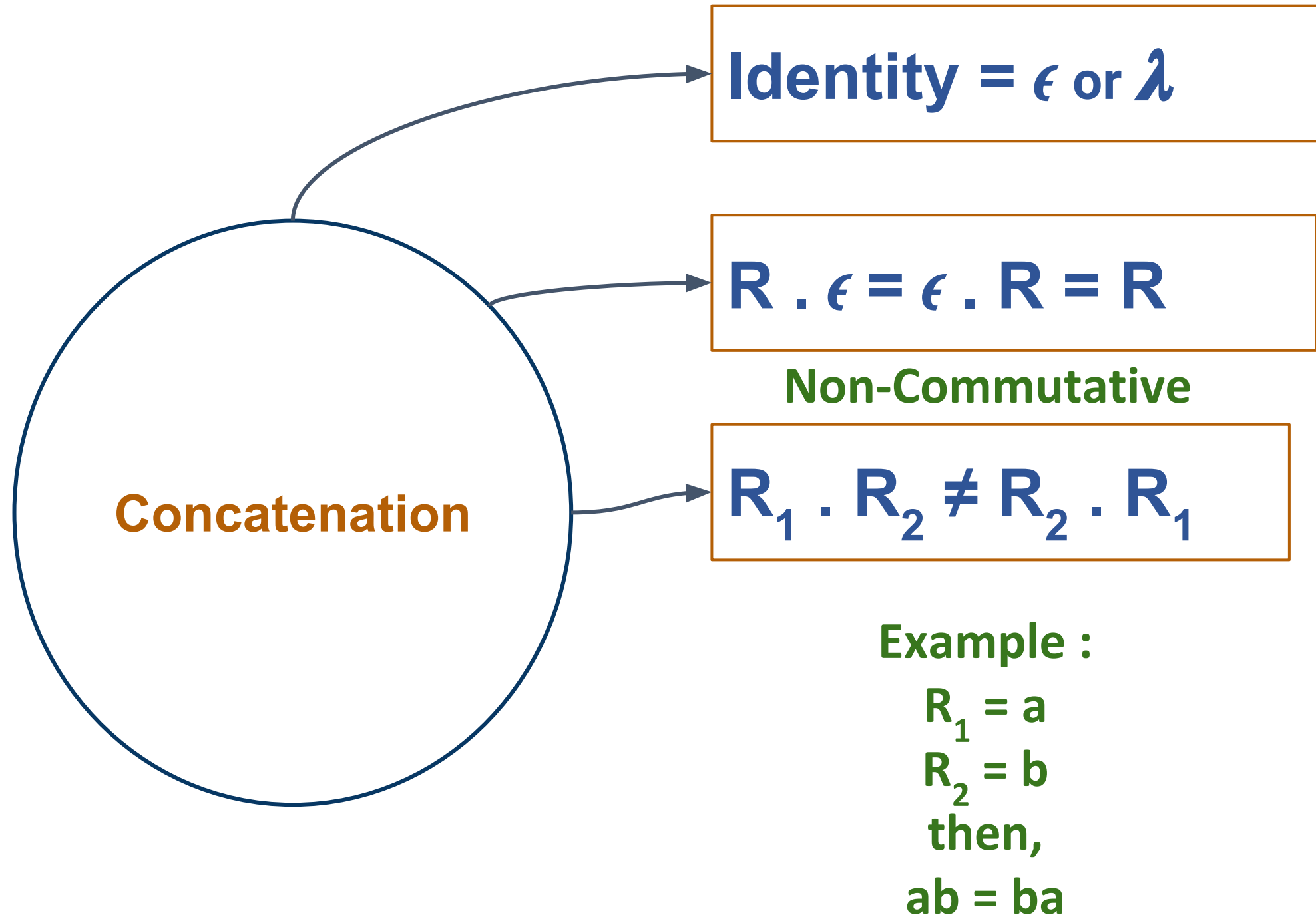


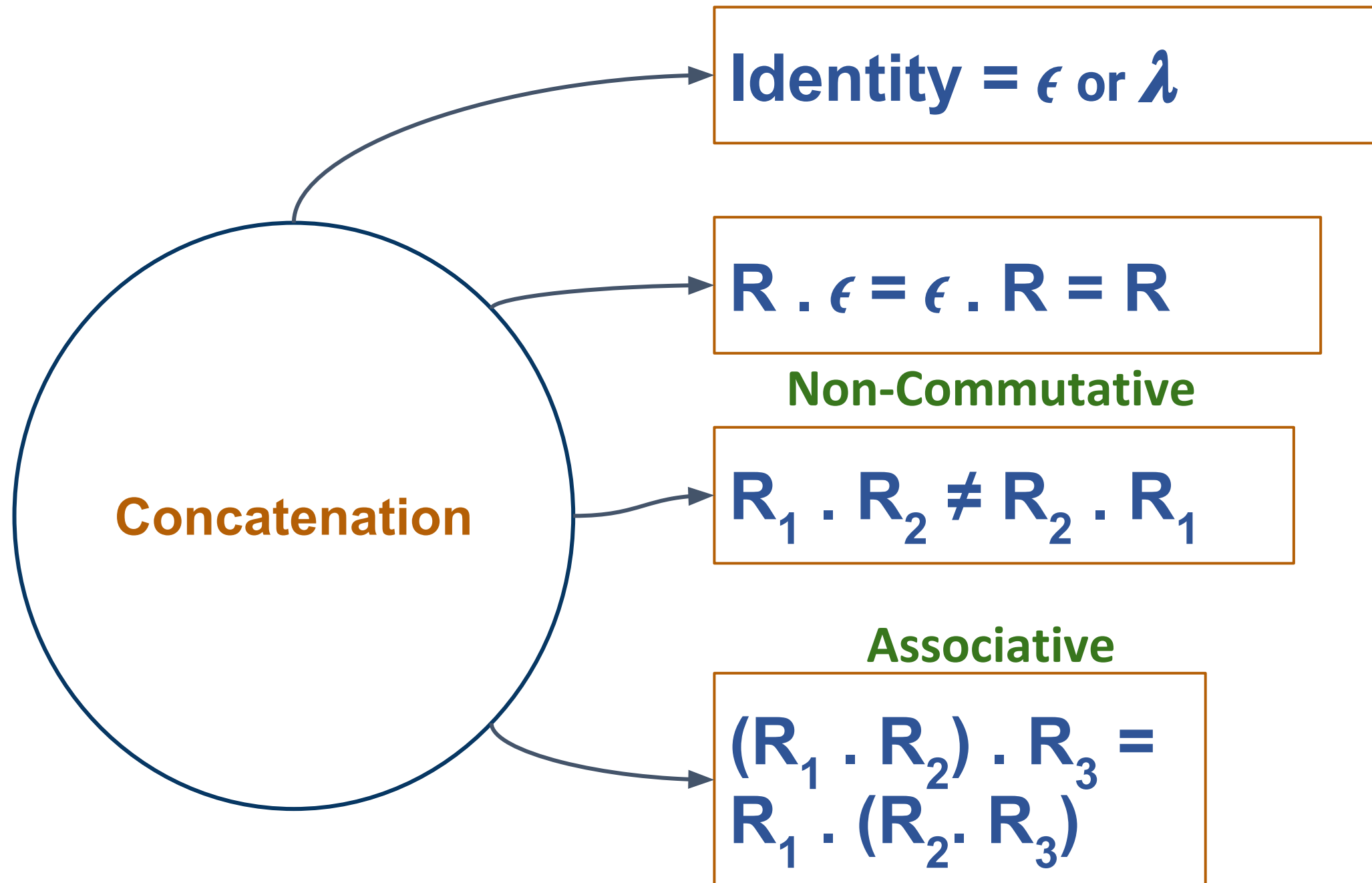




$$R \cdot \phi = \phi \cdot R = \phi$$

Let L_1, L_2 be languages, then the concatenation $L_1 \circ L_2 = \{w \mid w = xy, x \in L_1, y \in L_2\}$. If $L_2 = \emptyset$, then there is no string $y \in L_2$ and so there is no possible w such that $w = xy$. Thus for any L_1 , we'll have $L_1 \circ \emptyset = \emptyset$.





Closure

$$\epsilon^* = \epsilon$$

$$\phi^* = \epsilon$$

$$(R^*)^* = R^*$$

$$(R + \epsilon)^* = R^*$$

$$R^+ = RR^* = R^*R$$

$$(R^+ + \epsilon) = R^*$$

$$\epsilon + RR^* = \epsilon + R^* = R^*$$

Closure

$$\begin{aligned} & (a + b)^* \\ &= (a^* + b^*)^* \\ &= (a^* b^*)^* \\ &= (a + b^*)^* \\ &= (a^* + b)^* \\ &= a^*(ba^*)^* \\ &= b^*(ab^*)^* \end{aligned}$$

Construct a Regular Expression for a given language L



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Regular Expression in Practice

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



Special characters

1. .

Special characters

1. .

(dot)

matches any single character except newline

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



Special characters

1. .
2. *

Special characters

1. .

2. *

(Star)

0 or more repetitions of preceding regex

Special characters

1. .
2. *

Example :

a^*

Strings matched :

0 or more no. of a's

Example : $\lambda, a, aa, aaa, aaaa, aaaaa, \dots$

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



Special characters

1. .
2. *
3. +

Special characters

1. .
2. *
3. +

Example :

a+

Strings matched :

1 or more no. of a's

Example : a,aa,aaa,aaaa,aaaaa....

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



Special characters

1. .
2. *
3. +
4. ?

Special characters

1. .
2. *
3. +
4. ?

Example:

ab?c

matches abc or ac

Special characters

1. .
2. *
3. +
4. ?
5. []

Character Class

Matches any single character

Special characters

1. .
2. *
3. +
4. ?
5. []

Example

[cat]

String matched :

c

or a

or t

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^

^ (caret) :

Example:

Example: ^abc matches "a" at the start of the string.

[^abc]: This pattern matches any character except a or b or c.

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$

\$ (dollar) :

Example :

- > **abc\$** matches "c" at the end of a line.
- > **^\$** matches the empty string.

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$
8. { } (Syntax - $R\{m,n\}$)

Example of { }

$a\{0, \}$ - same as a^*

$a\{1, \}$ - same as a^+

$a\{2,3\}$ matches "aa" or "aaa".

$a\{3\}$ matches aaa only

Special characters

1. .
2. *
3. +
4. ?
5. []
6. ^
7. \$
8. { } (Syntax - $R\{m,n\}$)
9. \d - matches any digit between [0-9]
 - a. \d{9} matches any 9 digits
 - b. \d{2,3} matches 2 or 3 digits

We will discuss how to construct regex for validating the following :

- PAN Card
- Adhaar Card
- Mobile Number
- Date
- Email Address

- **PAN CARD:**

The valid PAN Card number must satisfy the following conditions:

1. It should be 10 characters long.
2. The first five characters should be any upper case alphabets.
3. The next four-characters should be any number from 0 to 9.
4. The last(tenth) character should be any upper case alphabet.
5. It should not contain any white spaces.

PAN CARD Regex - $^[A-Z]\{5\}[0-9]\{4\}[A-Z]\$$

Input: str = "BNZAA2318J"

Output: true

Explanation:

The given string satisfies all the above mentioned conditions.

Input: str = "23ZAABN18J"

Output: false

Explanation:

The given string does not start with upper case alphabets, therefore it is not a valid PAN Card number.

- **AADHAR NUMBER:**

The valid Aadhar number must satisfy the following conditions:

1. It should have 12 digits.
2. It should not start with 0 and 1.
3. It should not contains any alphabet and special characters.
4. It should have white space after every 4 digits.

AADHAR Number Regex - `^[2-9]{1}\d{3}\s{4}[" "]\s{4}$`

Input: str = "3675 9834 6012"

Output: true

Explanation:

The given string satisfies all the above mentioned conditions. Therefore it is a valid Aadhar number.

Input: str = "3675 9834 6012 8"

Output: false

Explanation:

The given string contains 13 digits. Therefore it is not a valid Aadhar number.

- **INDIAN MOBILE NUMBER:**

The valid Mobile number must satisfy the following conditions:

- It is a 10 digits number
- The first digit should contain number between 6 to 9.
- The rest 9 digit can contain any number between 0 to 9.
- The mobile number can have 11 digits also by including 0 at the starting.
- The mobile number can be of 13 digits also by including +91 at the starting

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



MOBILE Number Regex - `^(0|"+91")?[6-9]\d{9}$`

Dates in the year 2020

- It is a leap year
- Let the format be DD-MM-YY

Months with 31 days are : Jan, Mar, May, July, Aug, Oct, Dec
: [1, 3, 5, 7, 8, 10, 12]

Months with 30 days are : Apr, June, Sep, Nov
: [4, 6, 9, 11]

Month with 29 days : Feb
: [2]

- Year : 2020

Dates in the year 2020

Regex:

```
(  
(0[0-9])|([10-31]"-"(0[13578]|1[0|2])  
|  
(0[0-9])|([10-30]"-"(0[469]|11)  
|  
(0[0-9])|([10-29]"-"02"  
)"-"2020"
```


Email Address:

An email is a string (a subset of ASCII characters) separated into two parts by @ symbol. a "personal_info" and a domain, that is personal_info@domain.

The length of the personal_info part may be up to 64 characters long and domain name may be up to 253 characters.

The personal_info part contains the following ASCII characters.

- Uppercase (A-Z) and lowercase (a-z) English letters.
- Digits (0-9).
- Characters ! # \$ % & ' * + - / = ? ^ _ ` { | } ~
- Character . (period, dot or fullstop) provided that it is not the first or last character and it will not come one after the other.

The domain name [for example com, org, net, in, us, info] part contains letters, digits, hyphens, and dots.

Automata Formal Languages and Logic

Unit 2 - Regular Expression in Practice



Email Address:

Example of valid email id

- mysite@ouearth.com
- my.ownsite@ouearth.org
- mysite@you.net

Example of invalid email id

- mysite.ouearth.com [@ is not present]
- mysite@.com.my [tld (Top Level domain) can not start with dot "."]
- @you.me.net [No character before @]

**$$^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)^*@[a-zA-Z0-9-]+(\.[a-zA-Z0-9-]+)^*\.$$
$$(((0-9)\{1,3\})|([a-zA-Z]\{2,3\})|(com|edu|info|museum|name))\$$$**

Regular Expression in practice: Examples

Example 2: Simple URL Validator

Regular Expression in practice: Examples

Example 2: Simple URL Validator

$^((ht|f)tp(s?))\:\/\/([0-9a-zA-Z\-.]+[a-zA-Z]{2,6}(\:[0-9]+)?(\/\S^*))?\$$

Regular Expression in practice: Examples

Example 5: -mail addresses Validation

Regular Expression in practice: Examples

Example 5: -mail addresses Validation

$$^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)^*@[a-zA-Z0-9-]+(\. [a-zA-Z0-9-]+)^*\.
(([0-9]{1,3}) | ([a-zA-Z]{2,3}) | (com | edu | info | museum | name))\$$$

Regular Expression in practice: Examples

Example 6: Ip Address validation:

Regular Expression in practice: Examples

Example 6: Ip Address validation:

```
^(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9])\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9]|0)\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[1-9]|0)\.(25[0-5]|2[0-4][0-9]|[0-1]{1}[0-9]{2}|[1-9]{1}[0-9]{1}|[0-9])$
```




THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

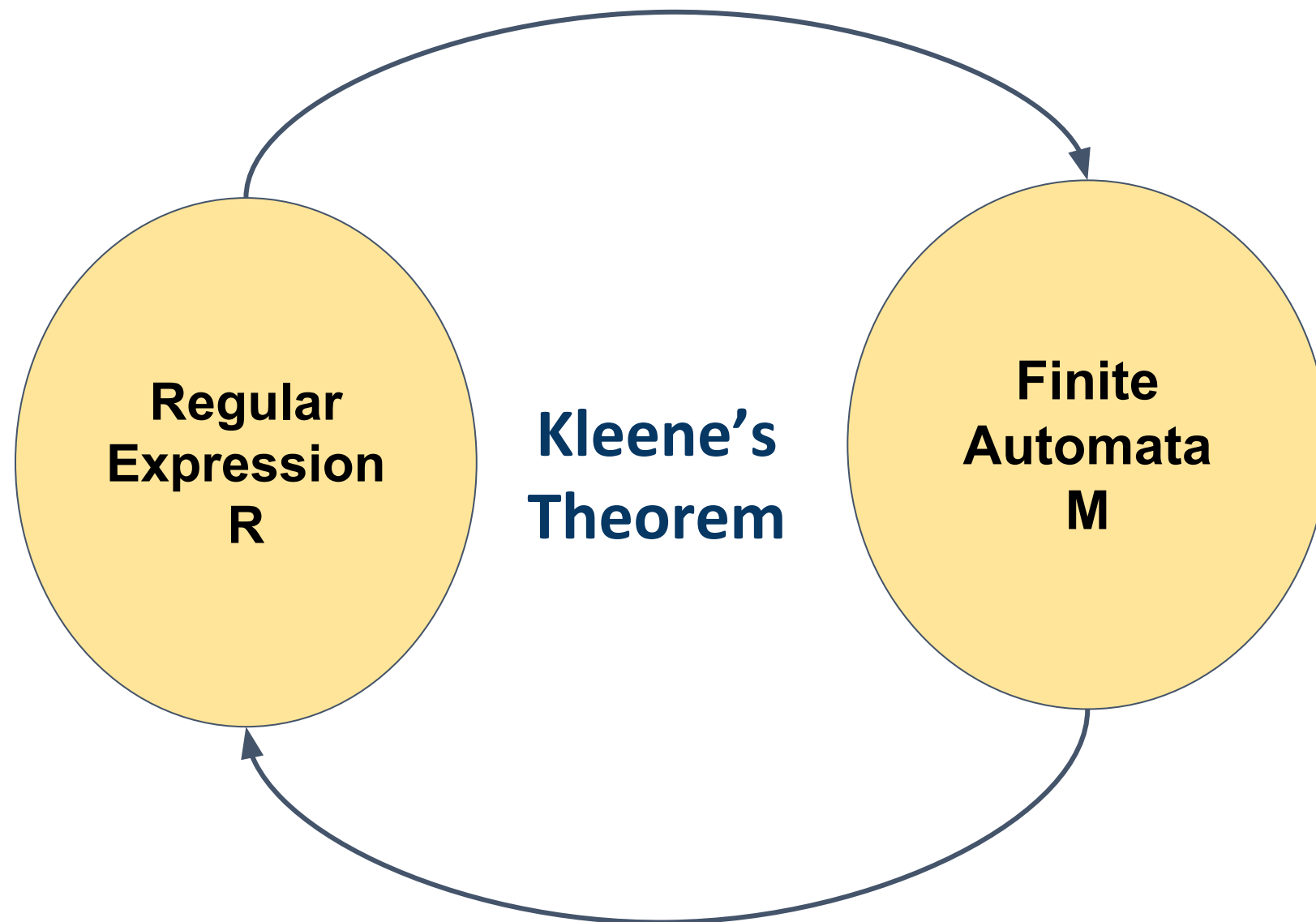
Automata Formal Languages & Logic

Unit 2

Preet Kanwal

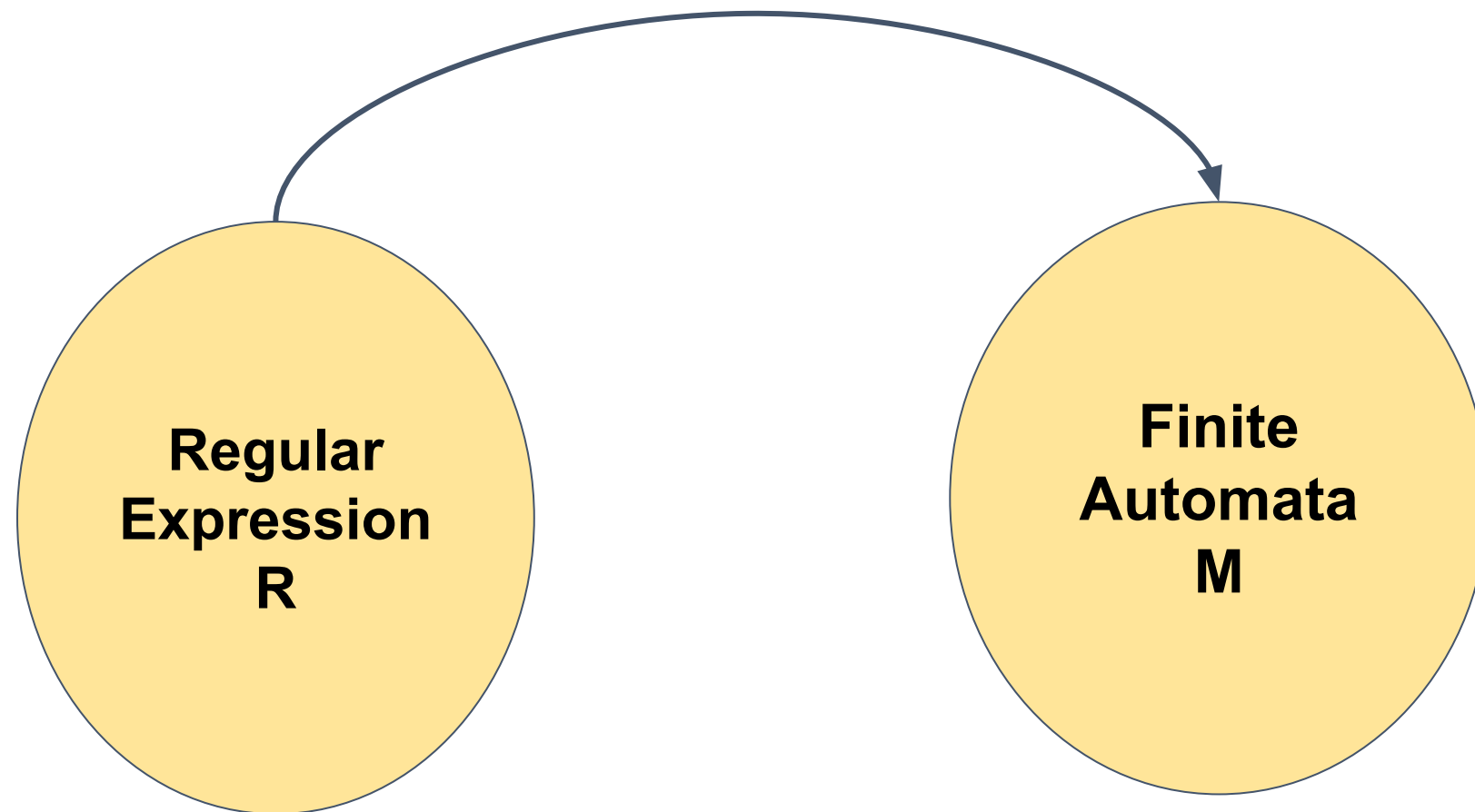
Department of Computer Science & Engineering

Thompson's Construction Algorithm

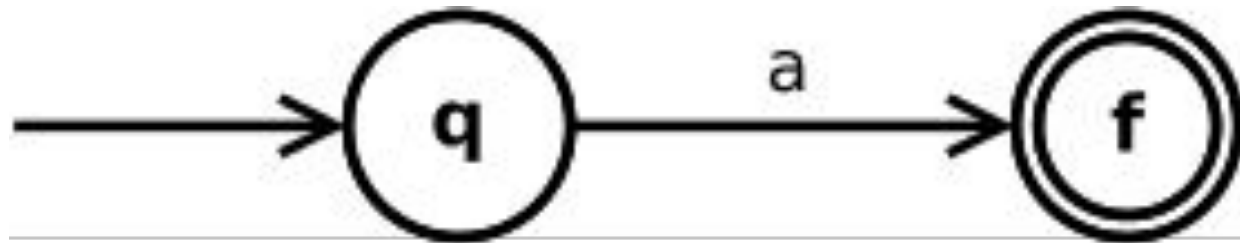
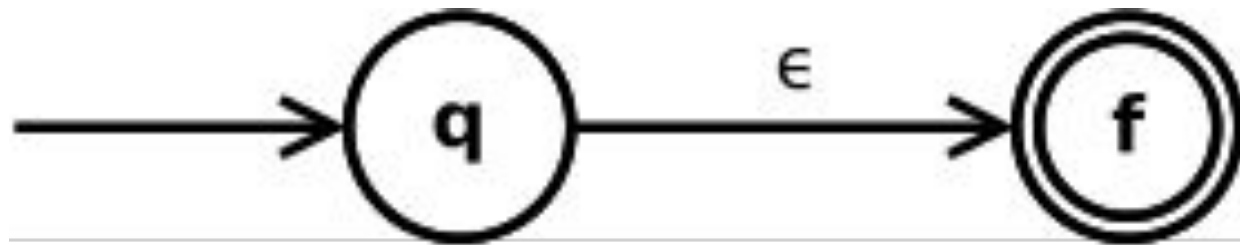


State Elimination Algorithm

Thompson's Construction Algorithm

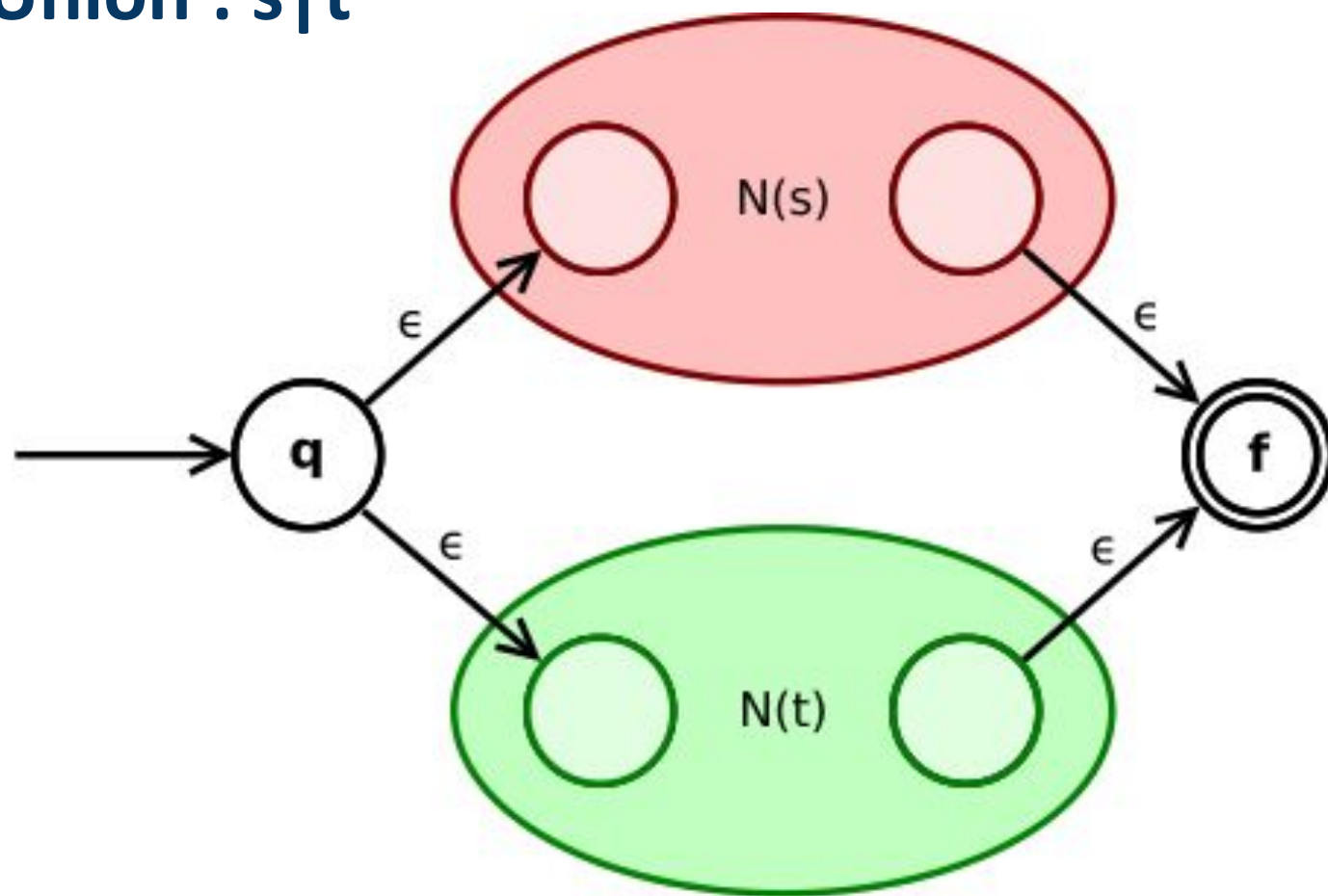


Thompson's Construction Algorithm



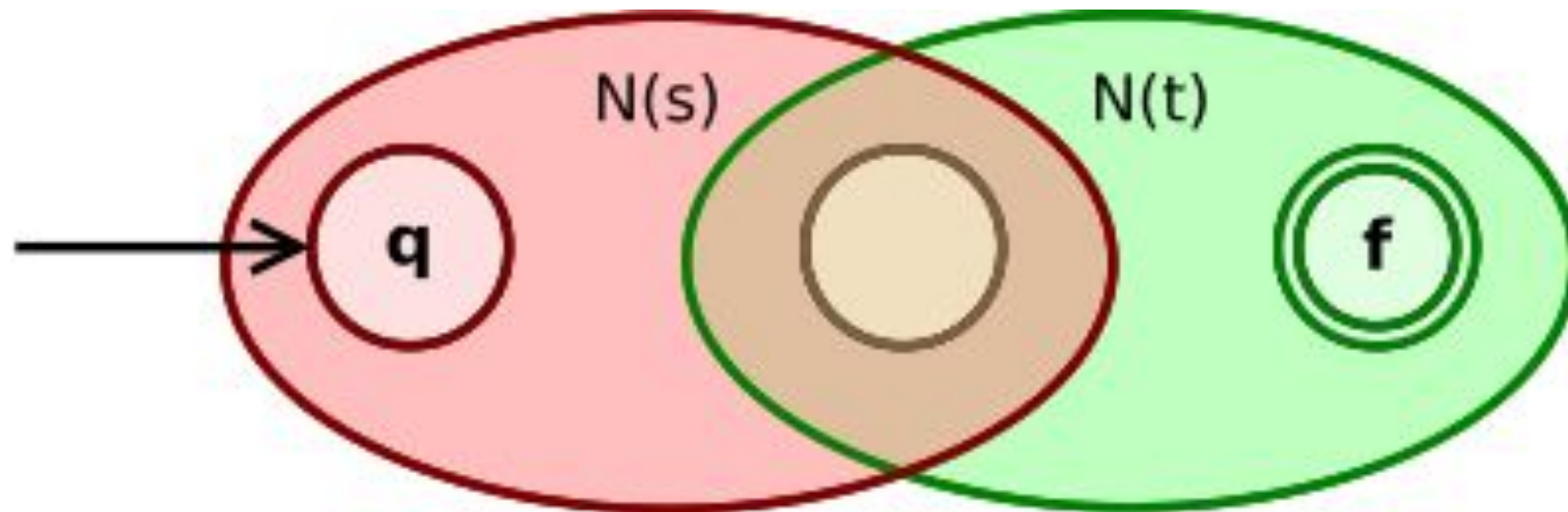
Thompson's Construction Algorithm

Union : $s|t$



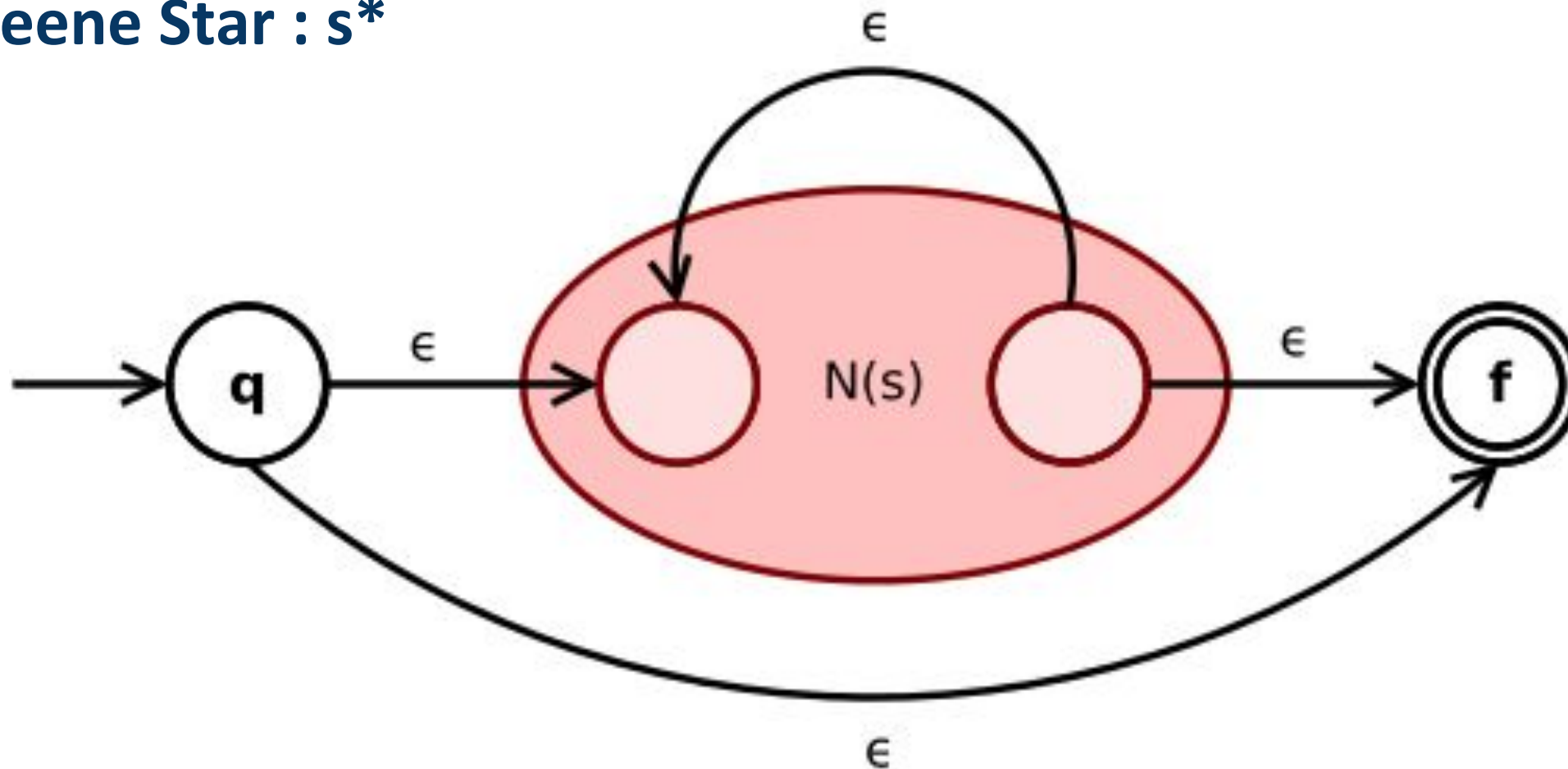
Thompson's Construction Algorithm

Concatenation : $s.t$



Thompson's Construction Algorithm

Kleene Star : s^*

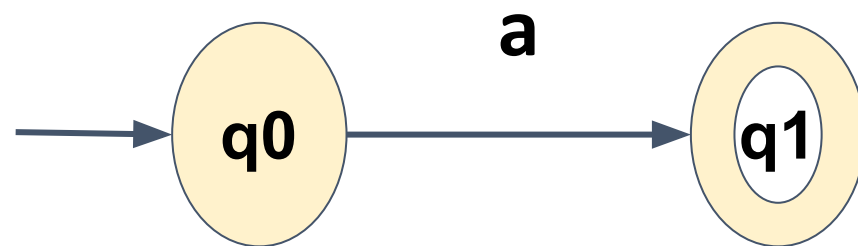


Construct Finite Automata for the following Regex :

$(ab+c)^*$

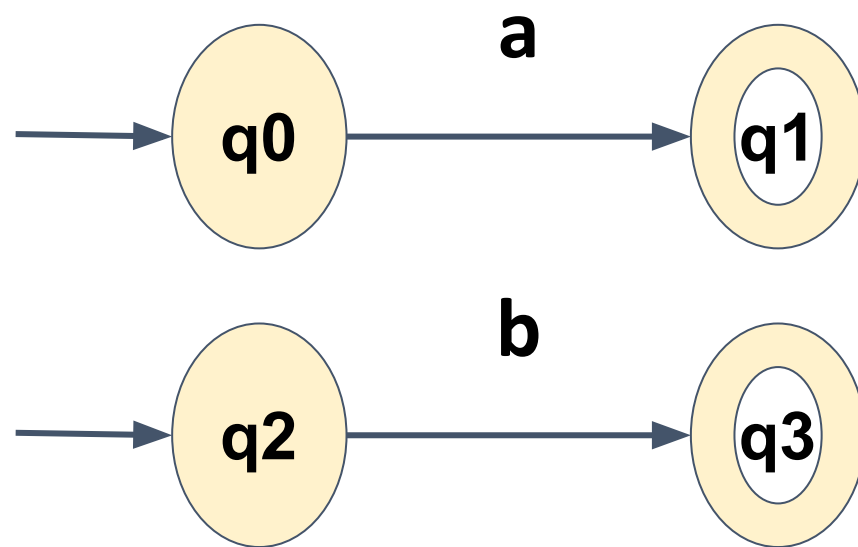
Construct Finite Automata for the following Regex :

$(ab+c)^*$



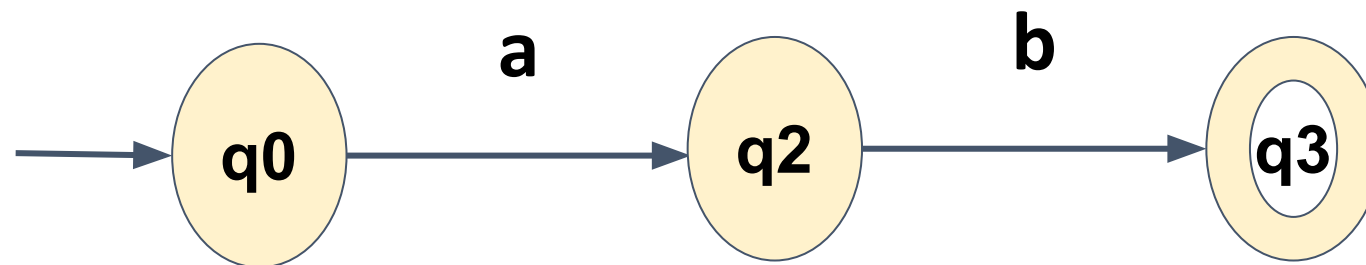
Construct Finite Automata for the following Regex :

$(ab+c)^*$



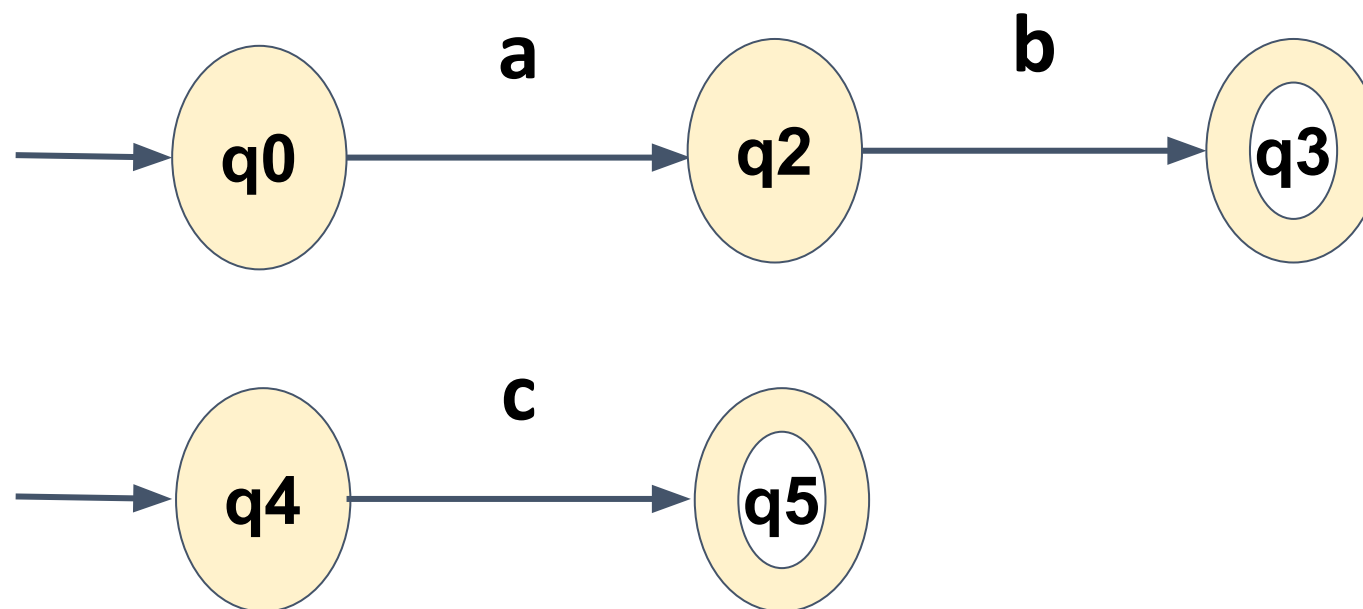
Construct Finite Automata for the following Regex :

(ab+c)*



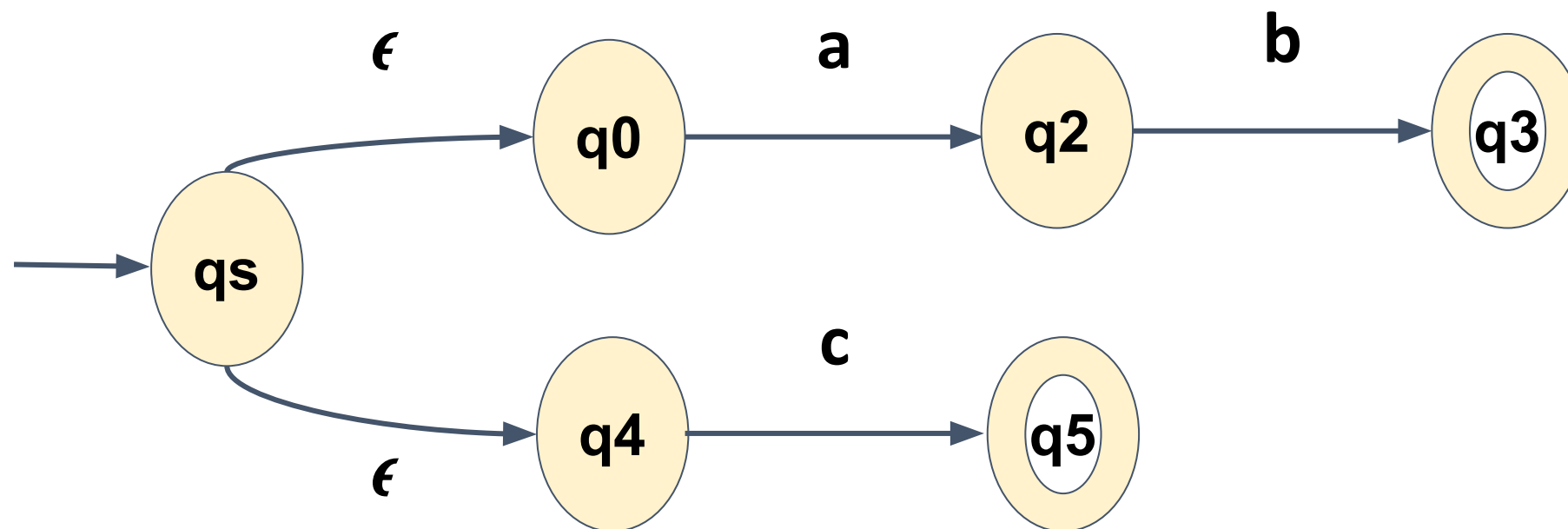
Construct Finite Automata for the following Regex :

$(ab+c)^*$



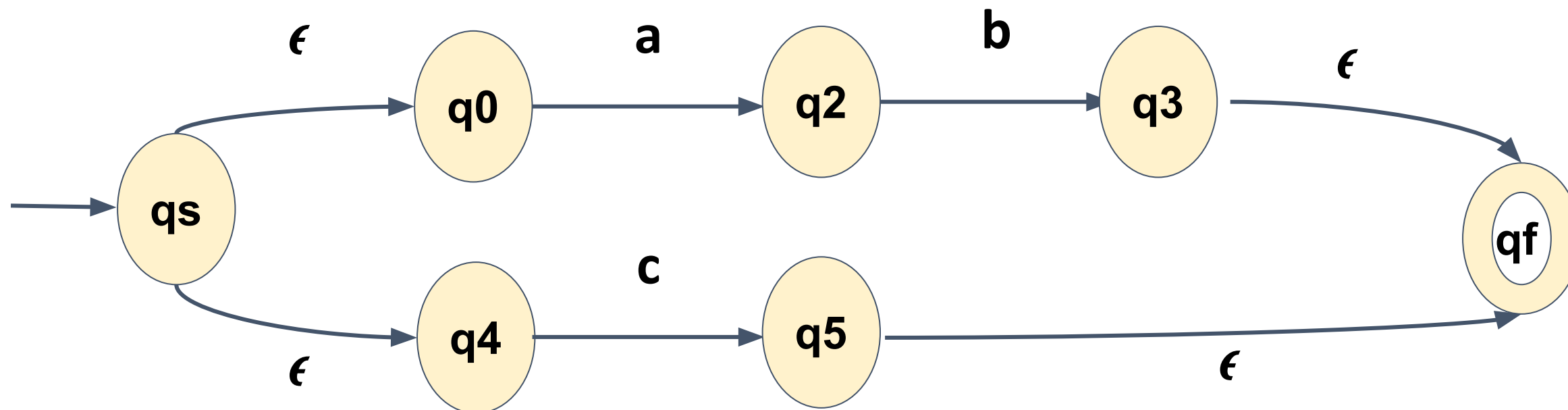
Construct Finite Automata for the following Regex :

$(ab+c)^*$

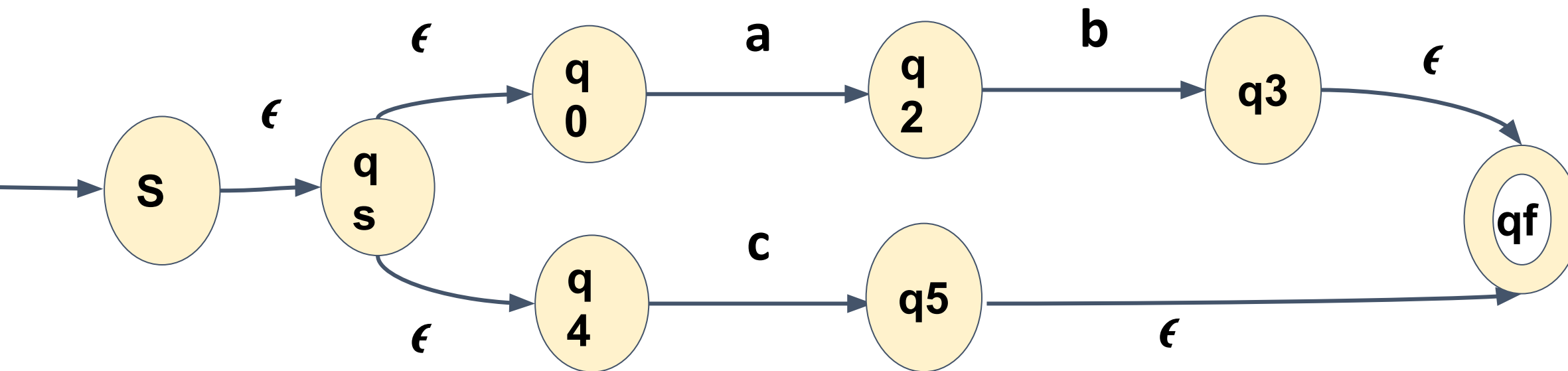


Construct Finite Automata for the following Regex :

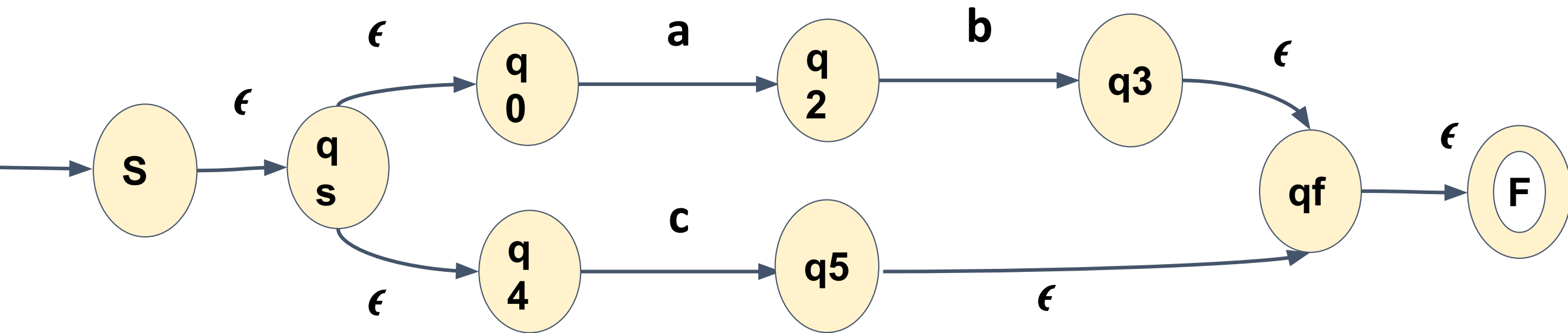
$(ab+c)^*$



$(ab+c)^*$



$(ab+c)^*$

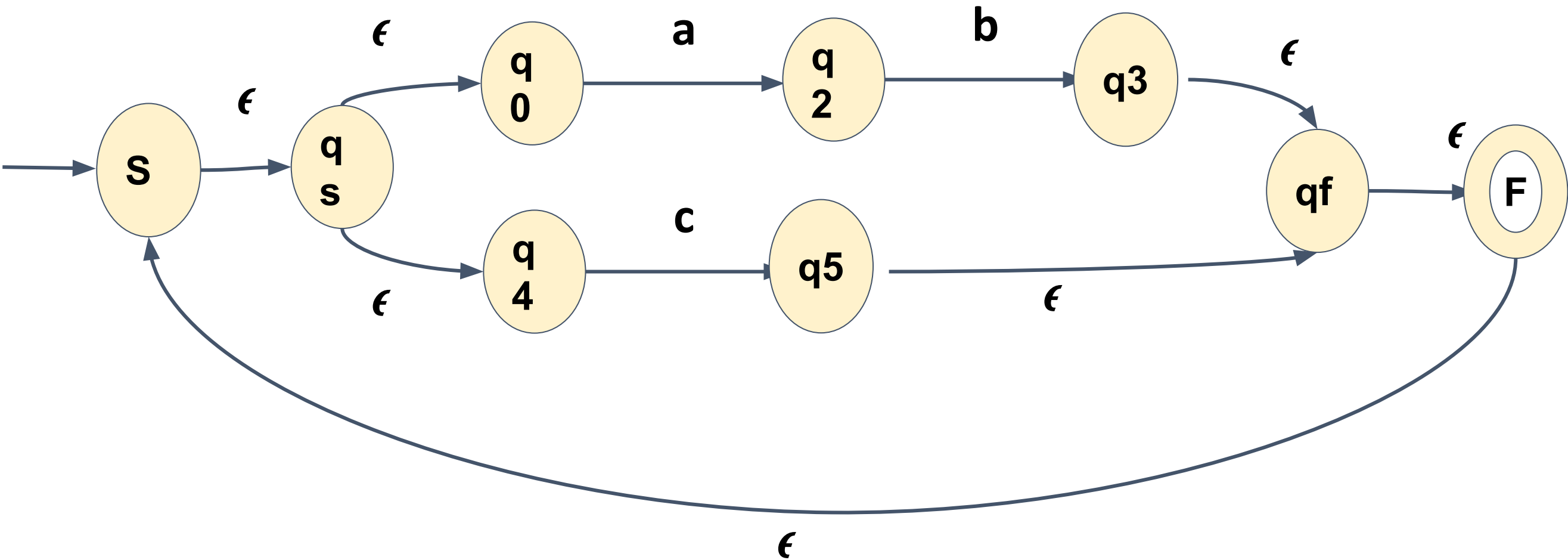


Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata



$(ab+c)^*$

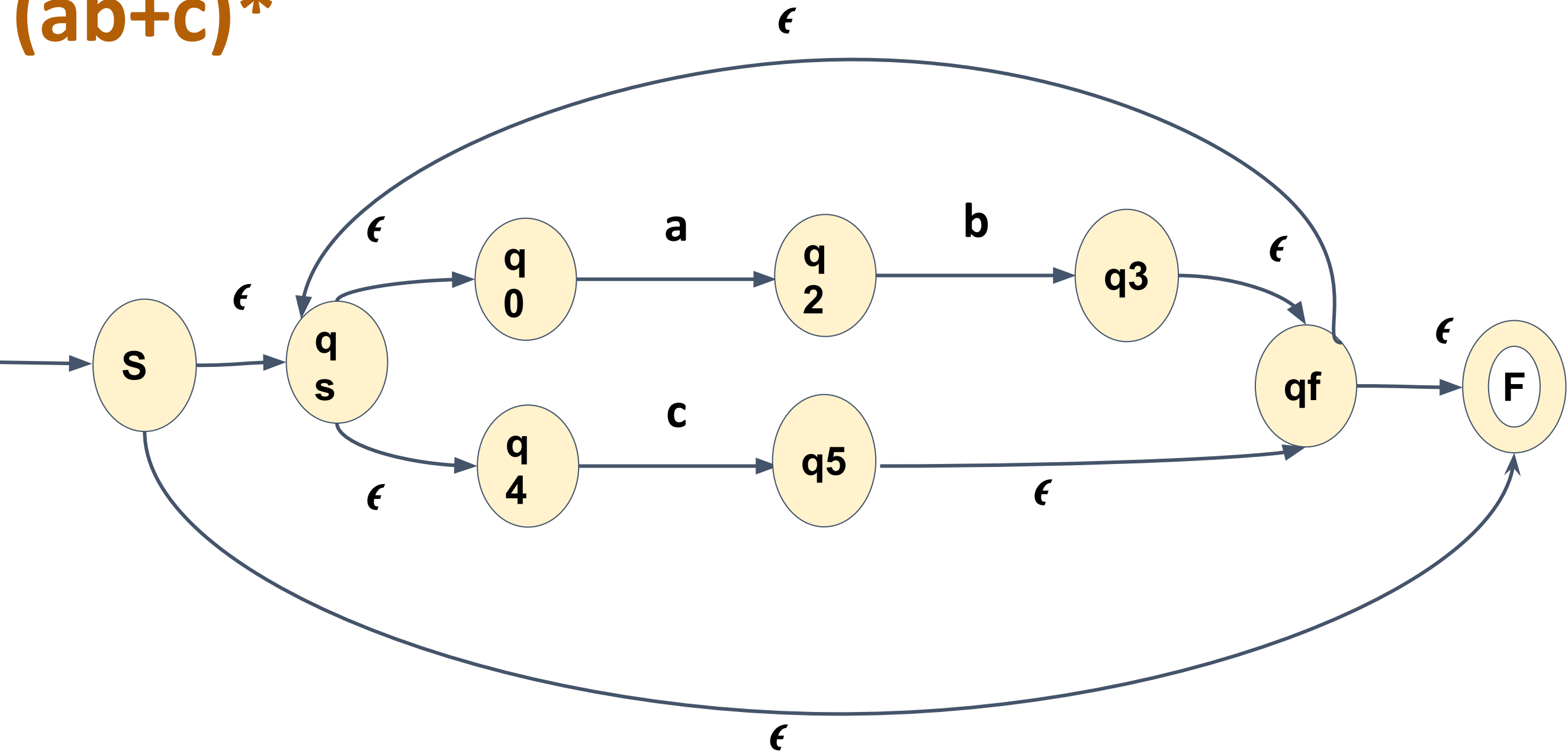


Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata



$(ab+c)^*$



Automata Formal Languages and Logic

Unit 2 - Regular Expression to Finite Automata



Construct Finite Automata for given Regex



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

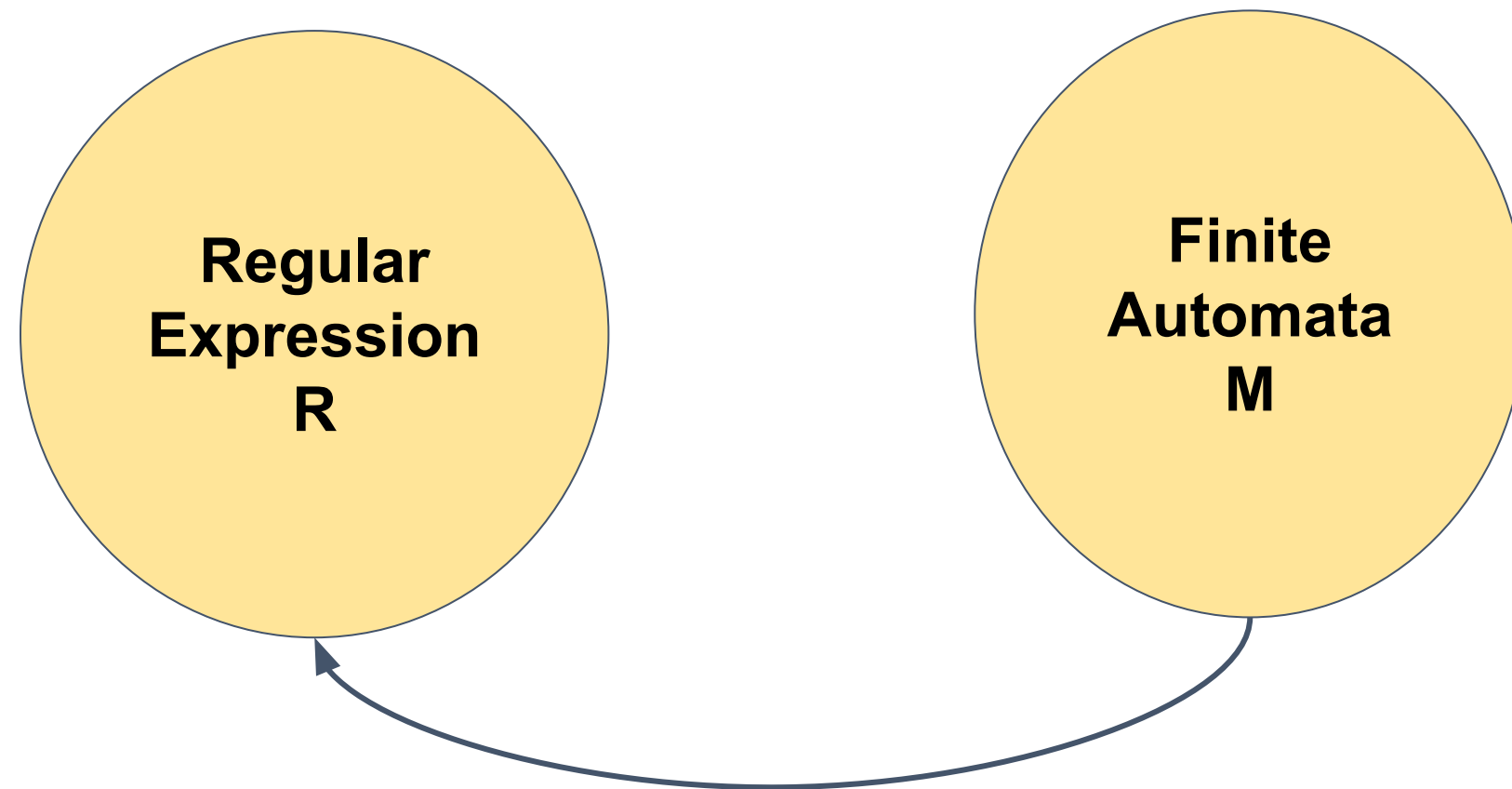
Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Mechanical Engineering



State Elimination Algorithm

State Elimination Method

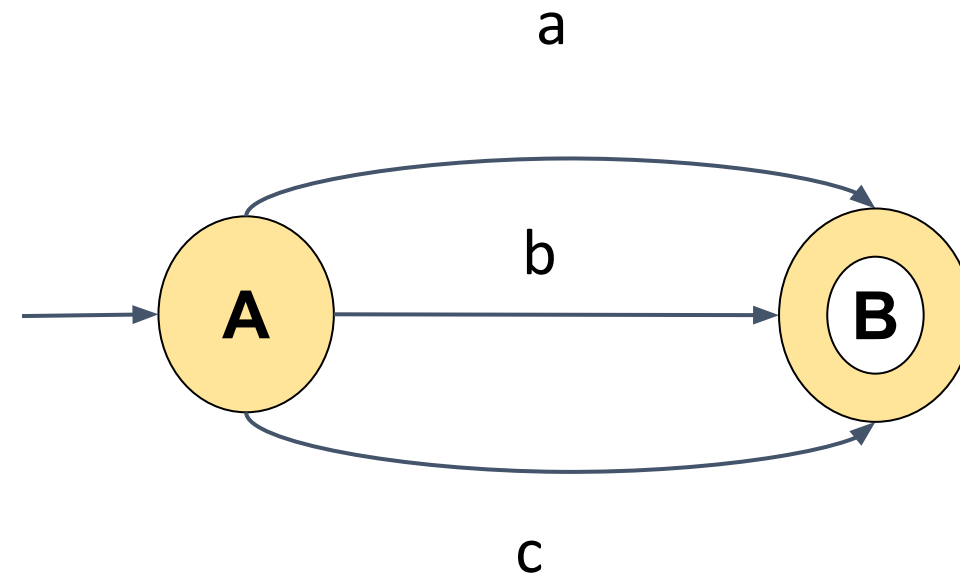
Start with an FA for the language L.

- Add a new start state q_s and accept state q_f to the FA.
- Add ϵ -transitions from each original accepting state to q_f , then mark them as not accepting.
 - Repeatedly remove states other than q_s and q_f from the FA by “shortcutting” them until only two states remain: q_s and q_f .
- The transition from q_s to q_f is then a regular expression for the FA.

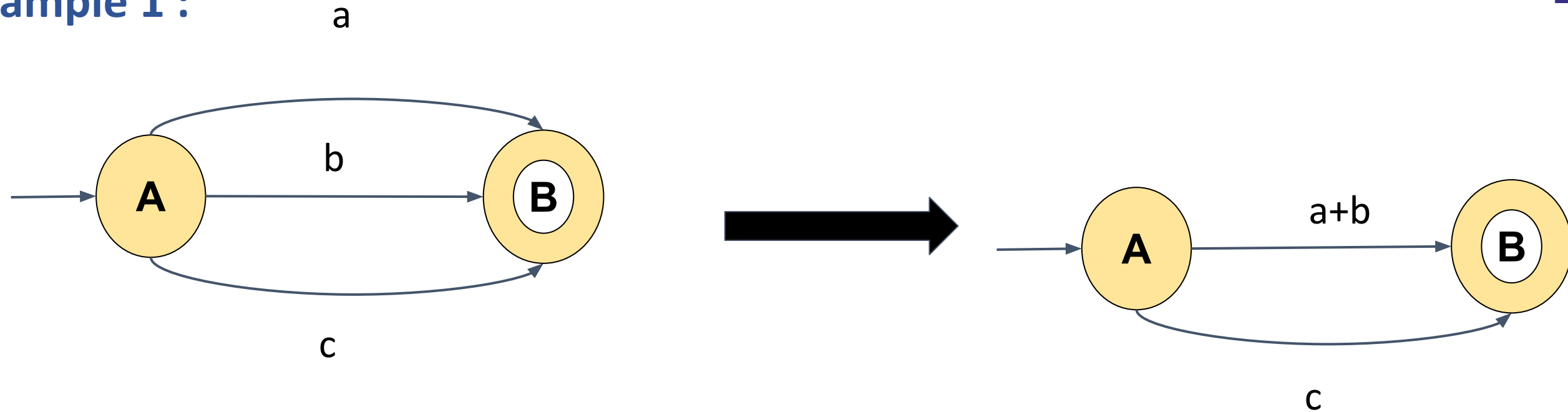
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

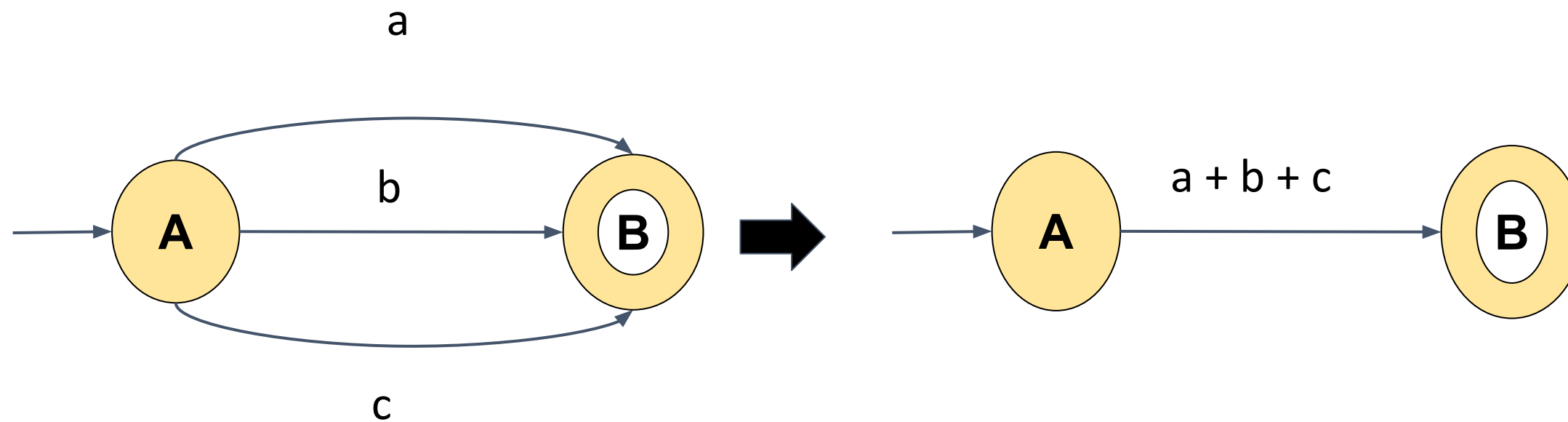
Example 1 :



Example 1 :



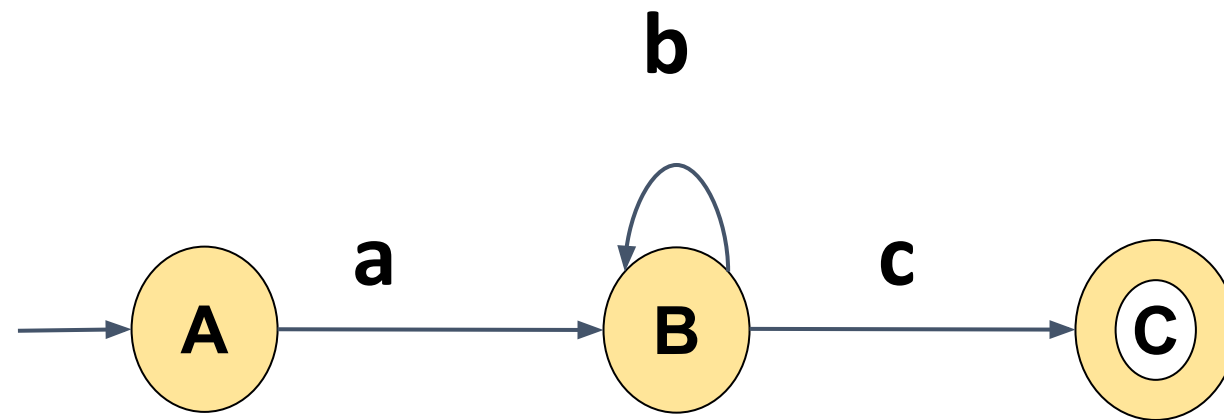
Example 1 :



Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

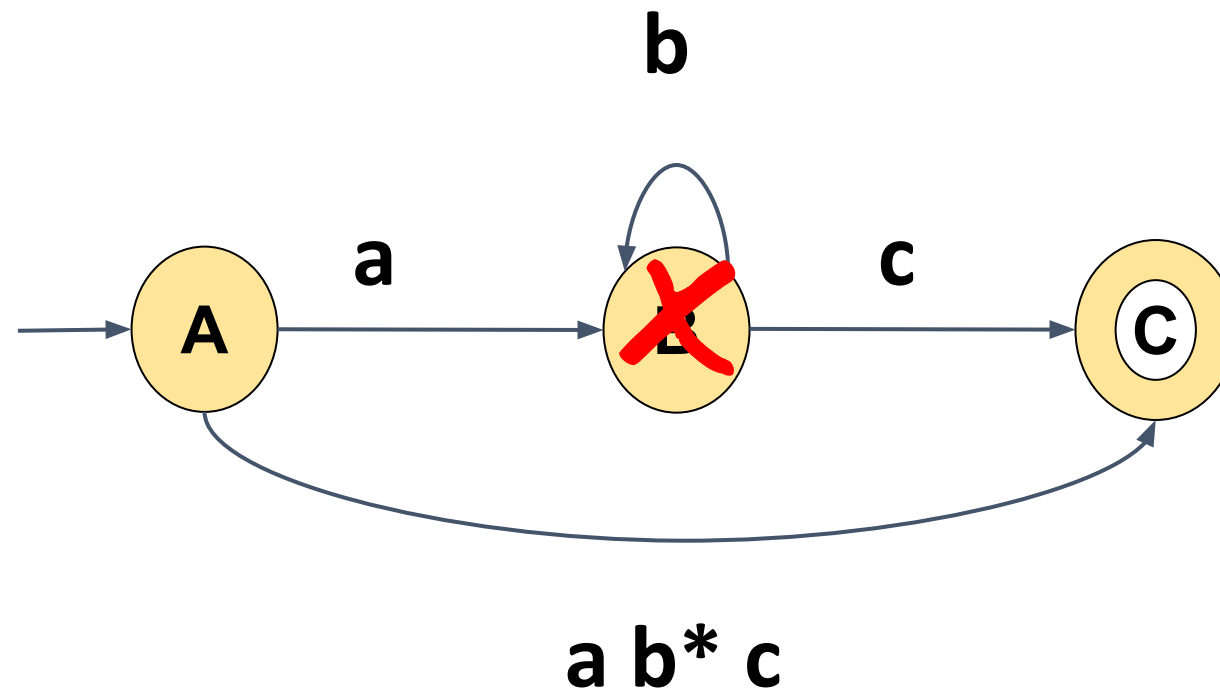
Example 2 :



Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 2 :

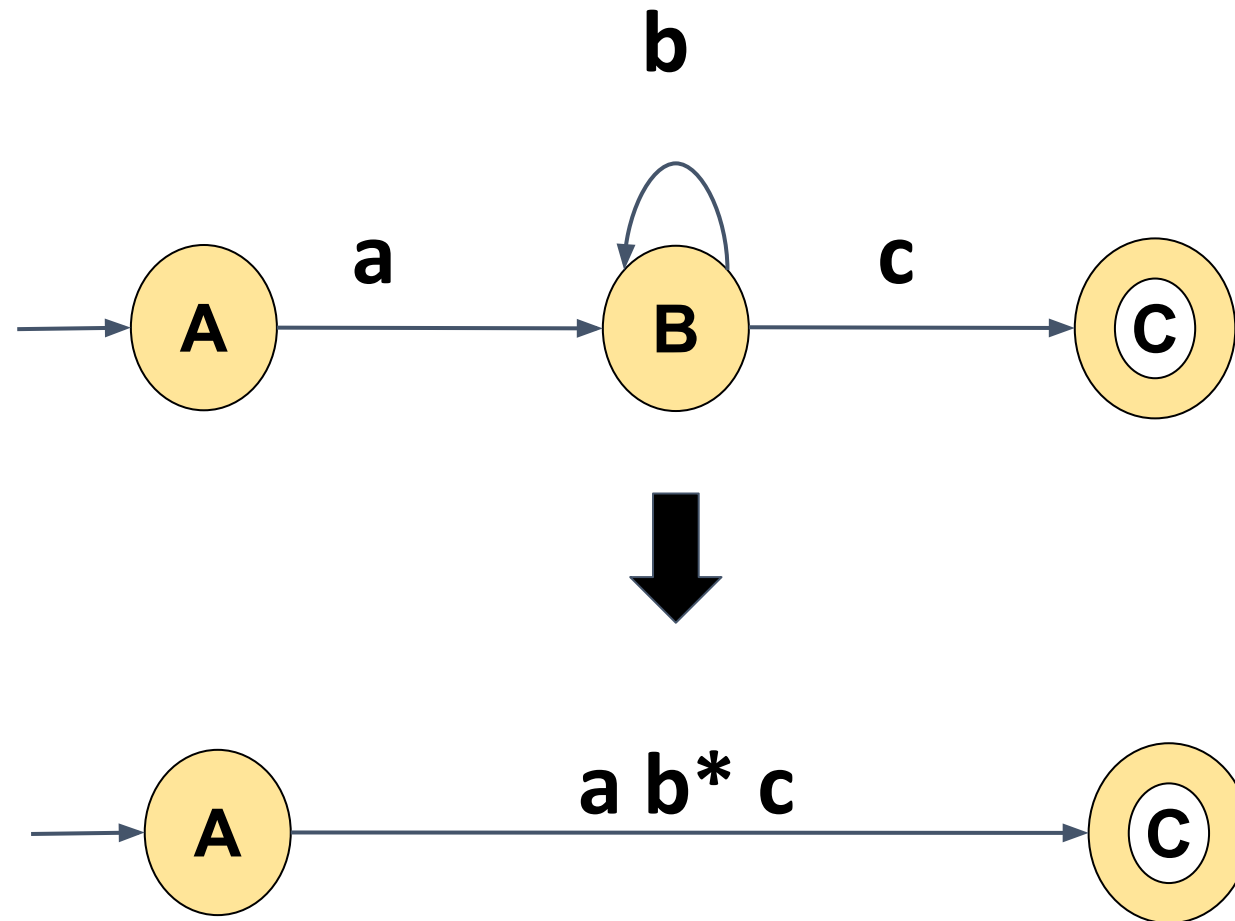


Eliminate B

Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

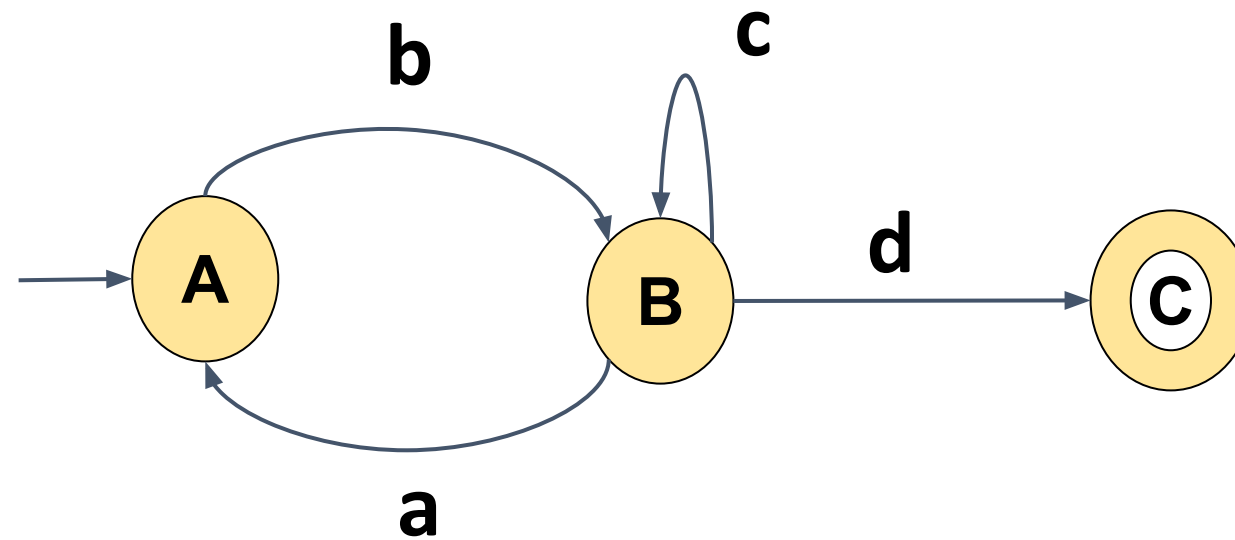
Example 2 :



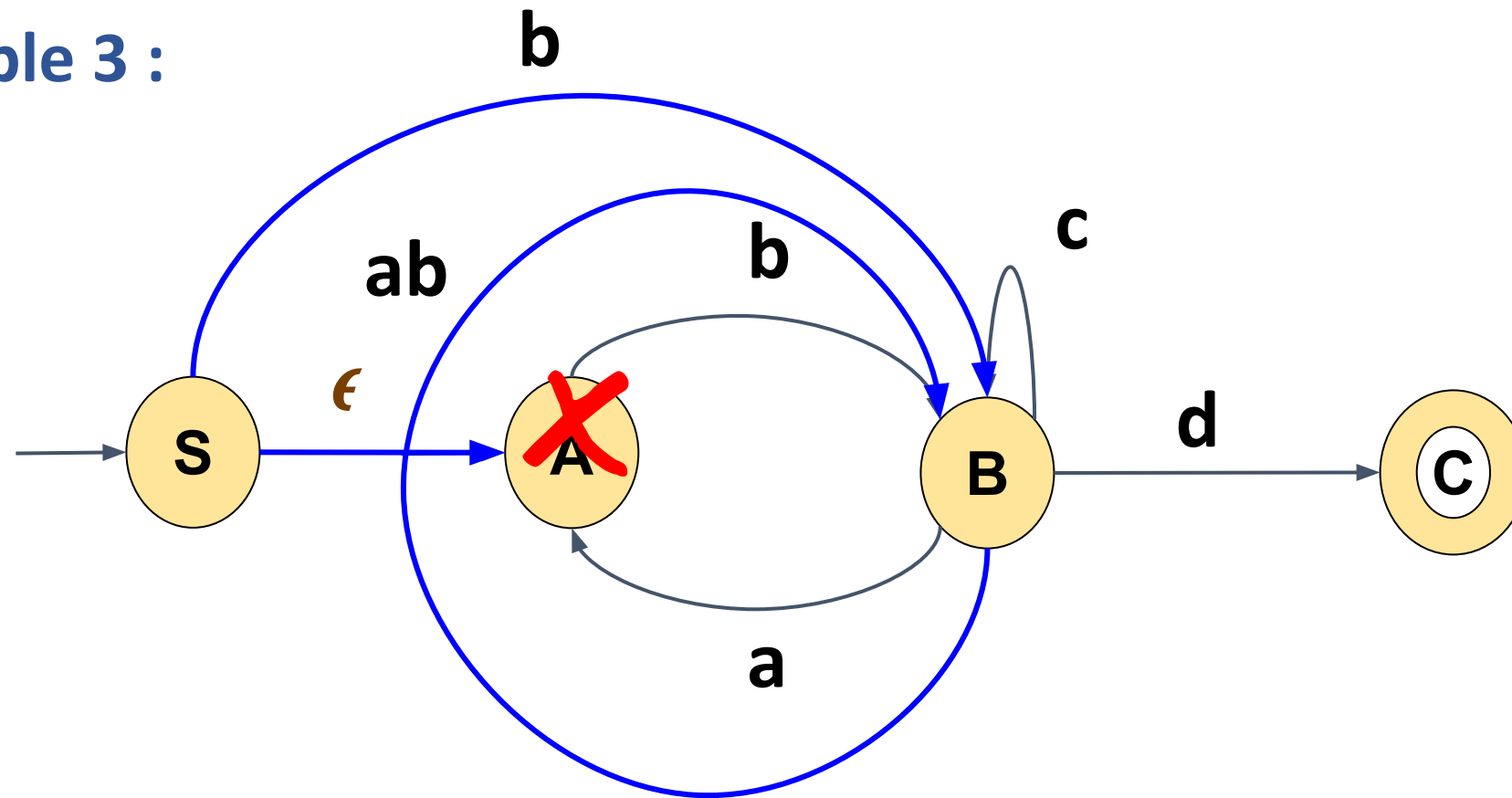
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 3 :

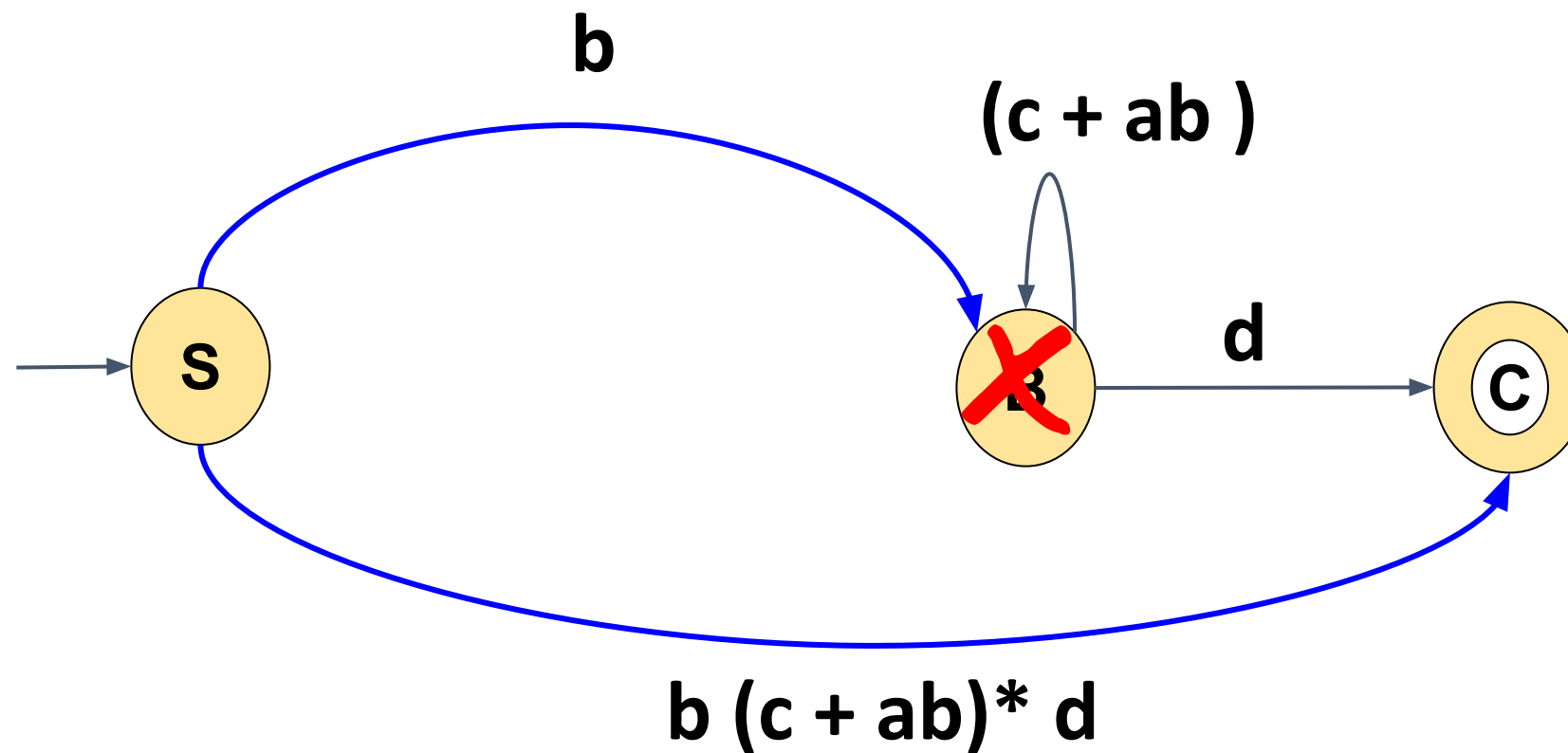


Example 3 :



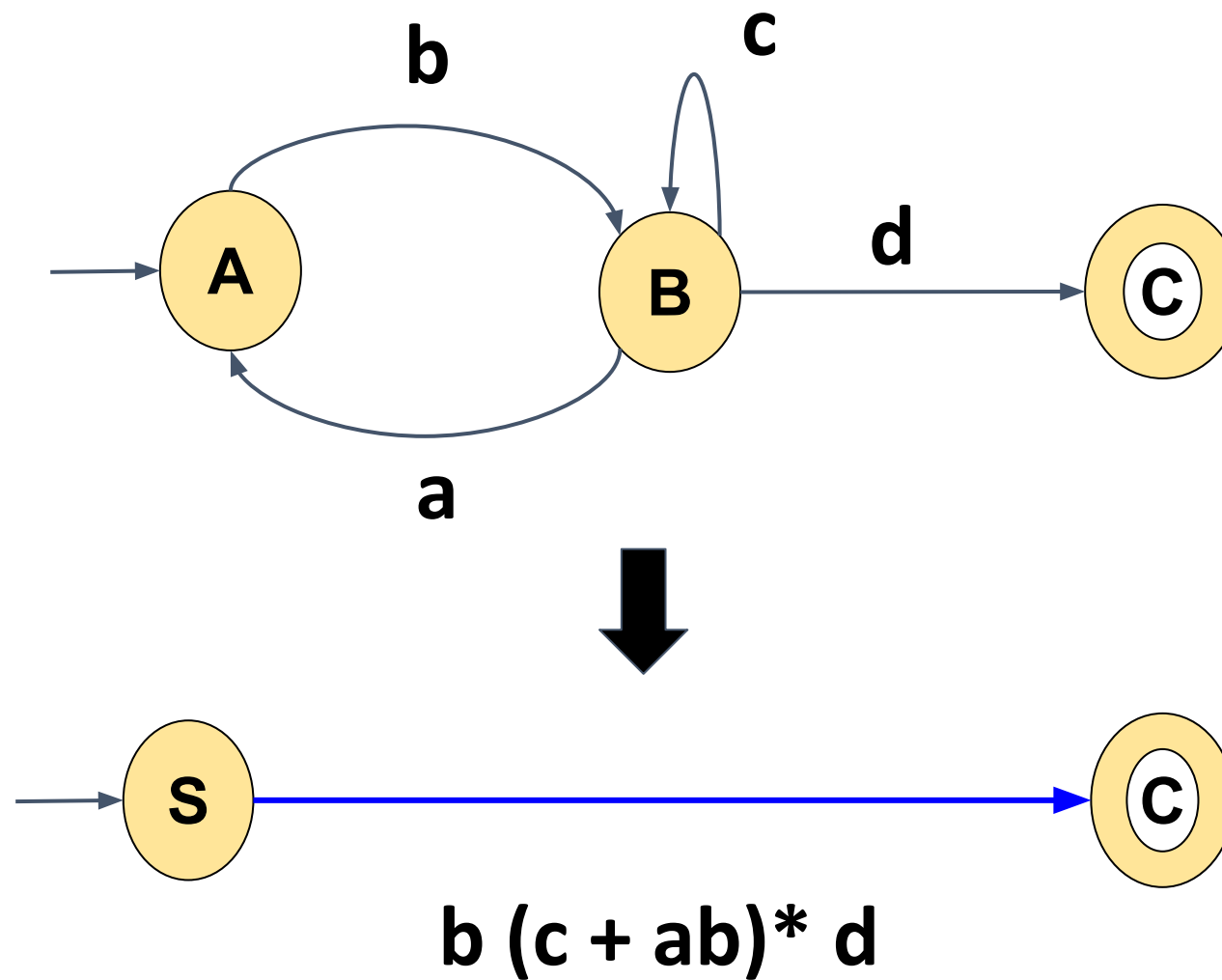
1. Eliminate A

Example 3 :



1. Eliminate A
2. Eliminate B

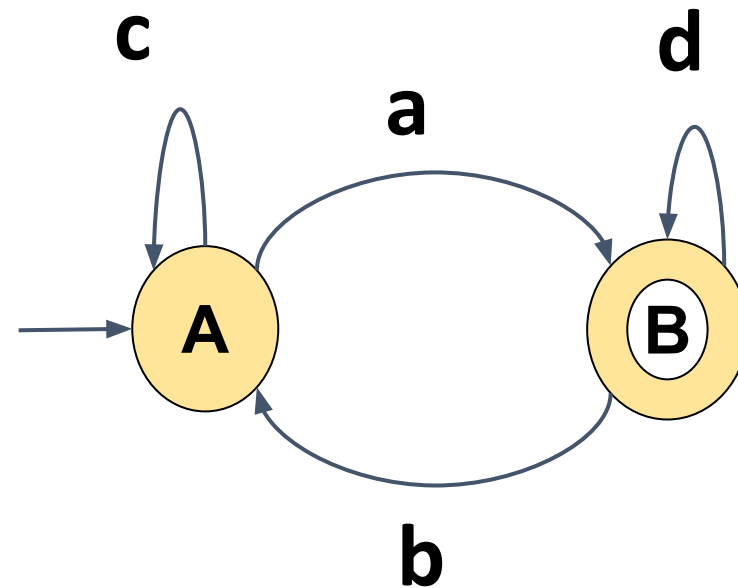
Example 3 :



Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

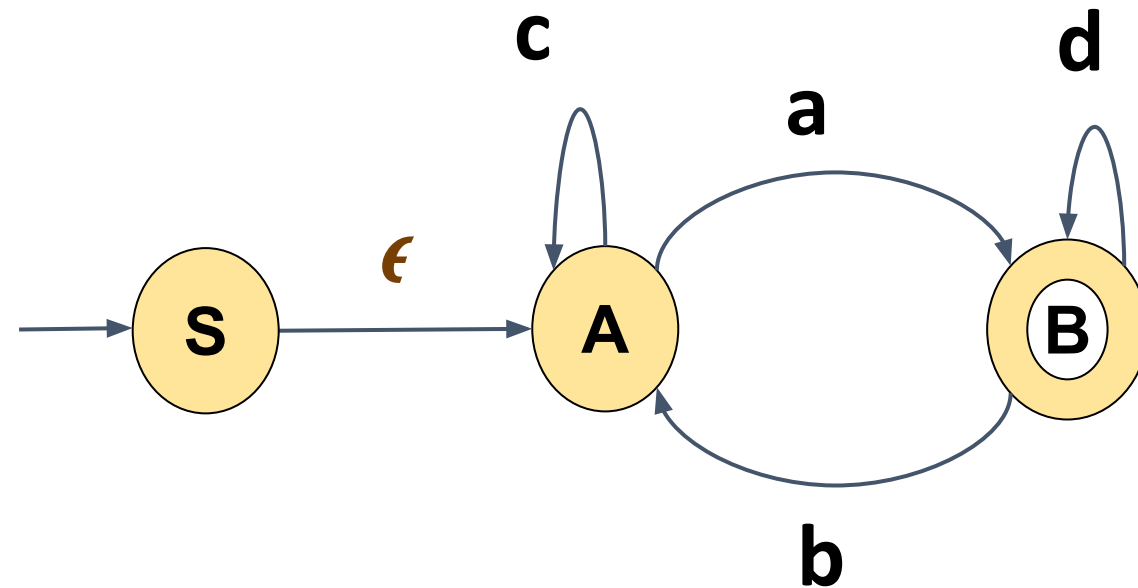
Example 4 :



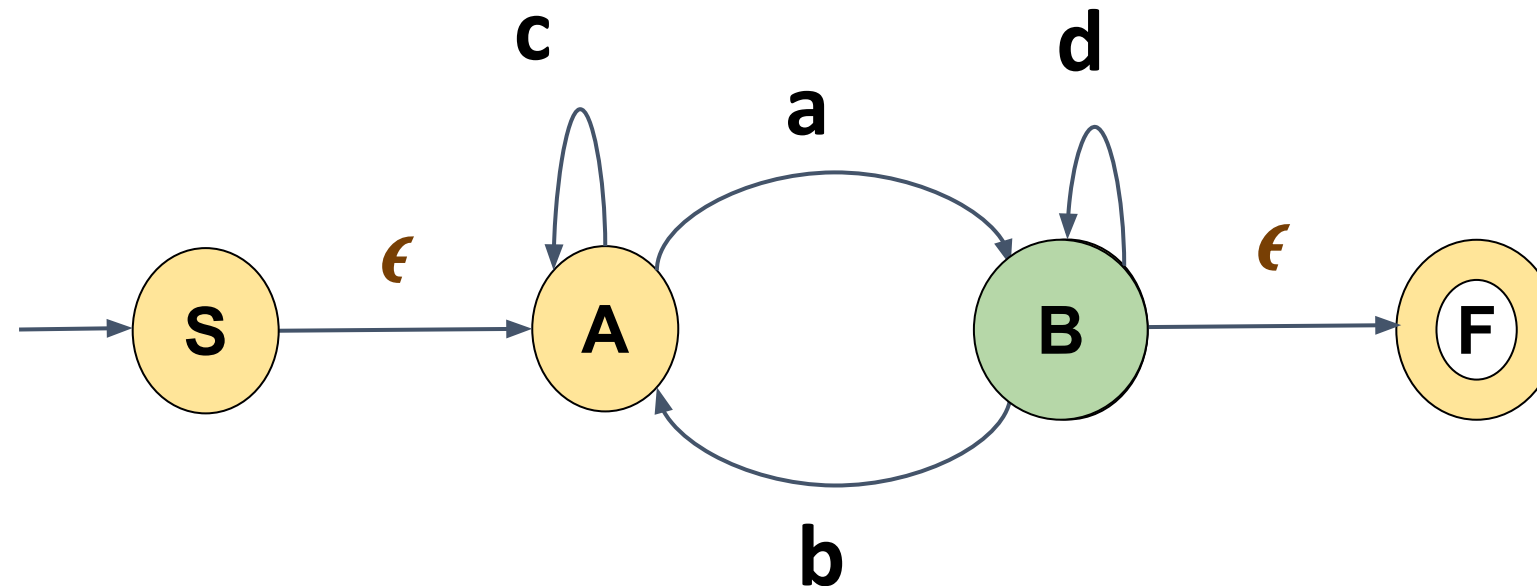
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 4 :

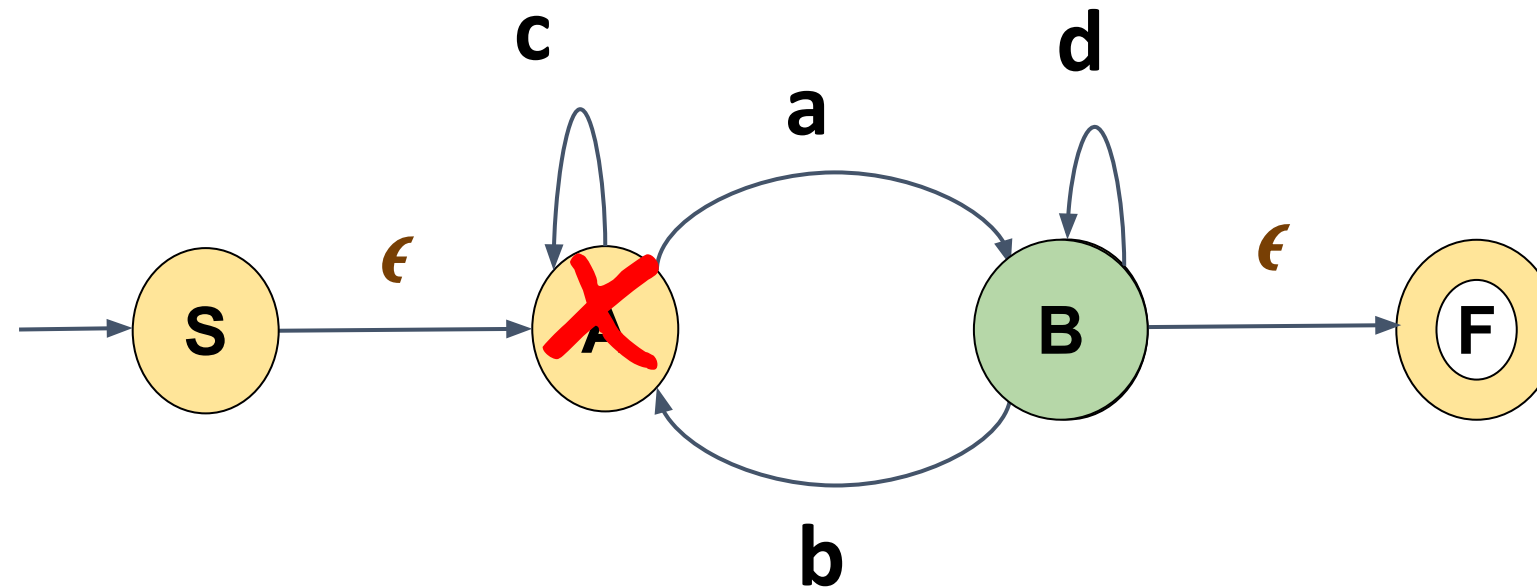


Example 4 :



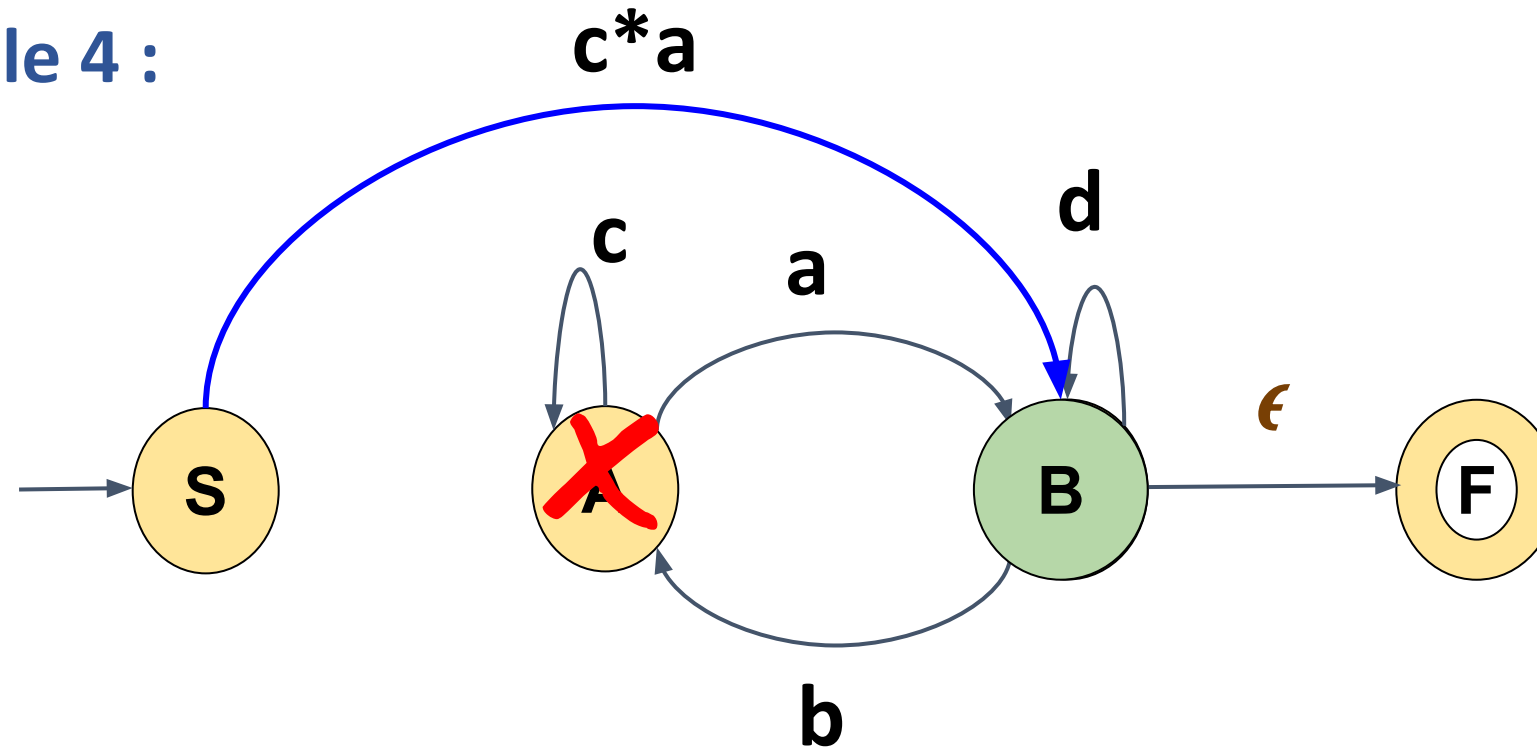
A new Final state F is introduced as there is an Outgoing edge to the existing Final State B.

Example 4 :



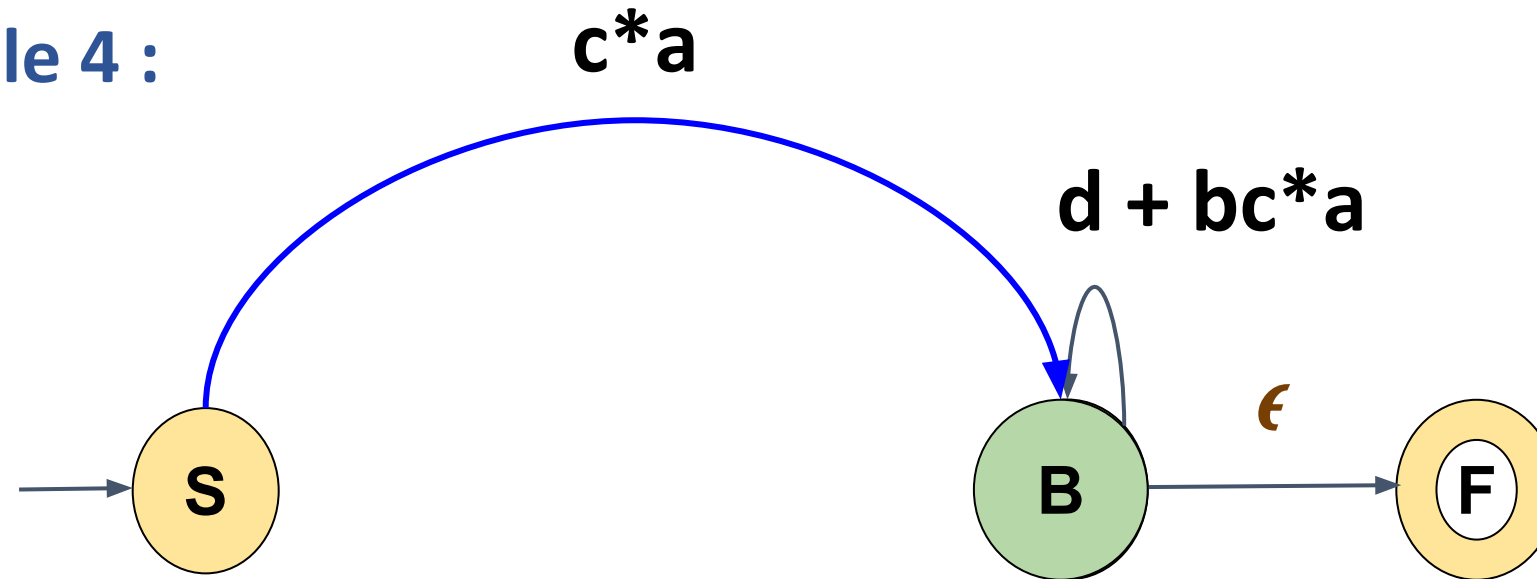
1. Eliminate A

Example 4 :



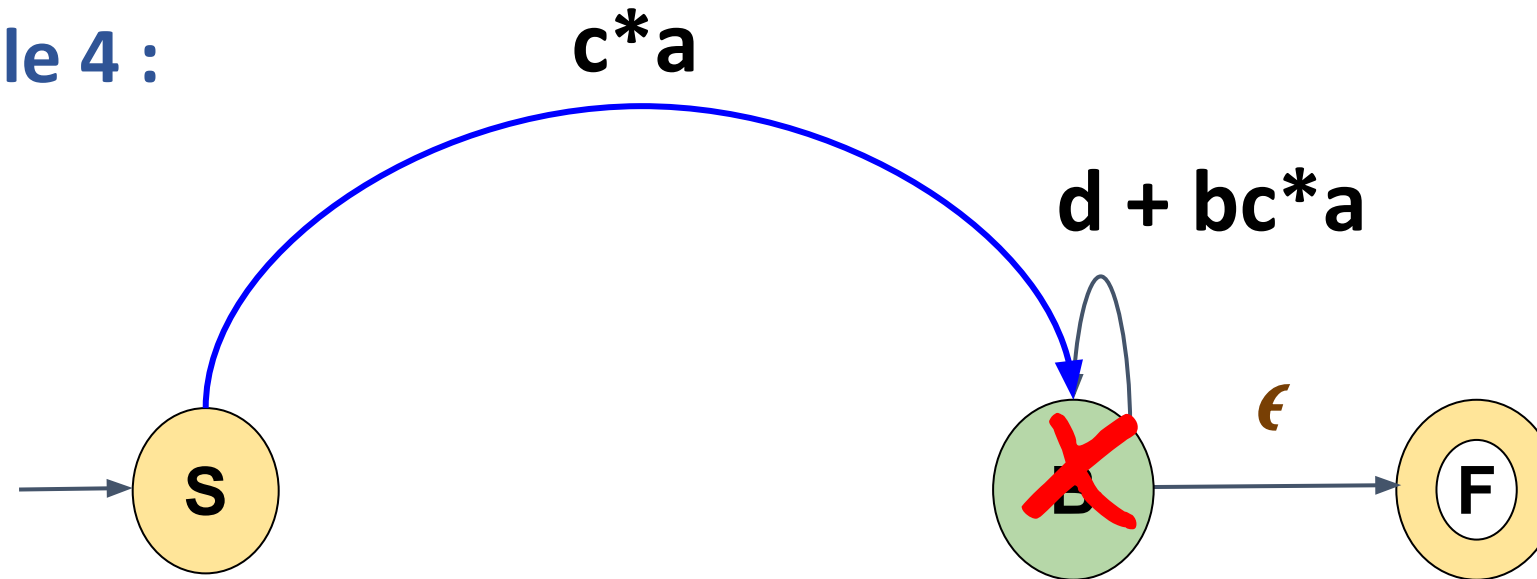
1. Eliminate A

Example 4 :



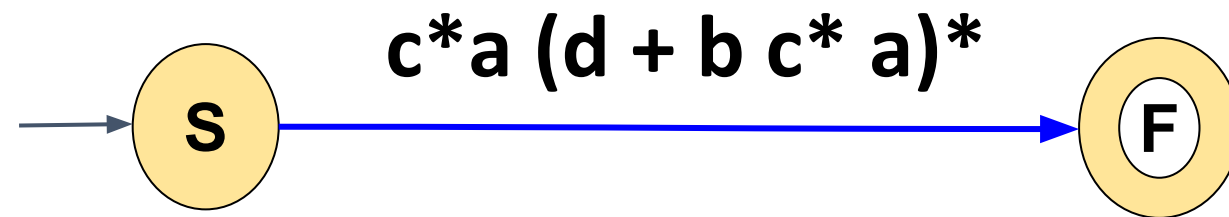
1. Eliminate A

Example 4 :



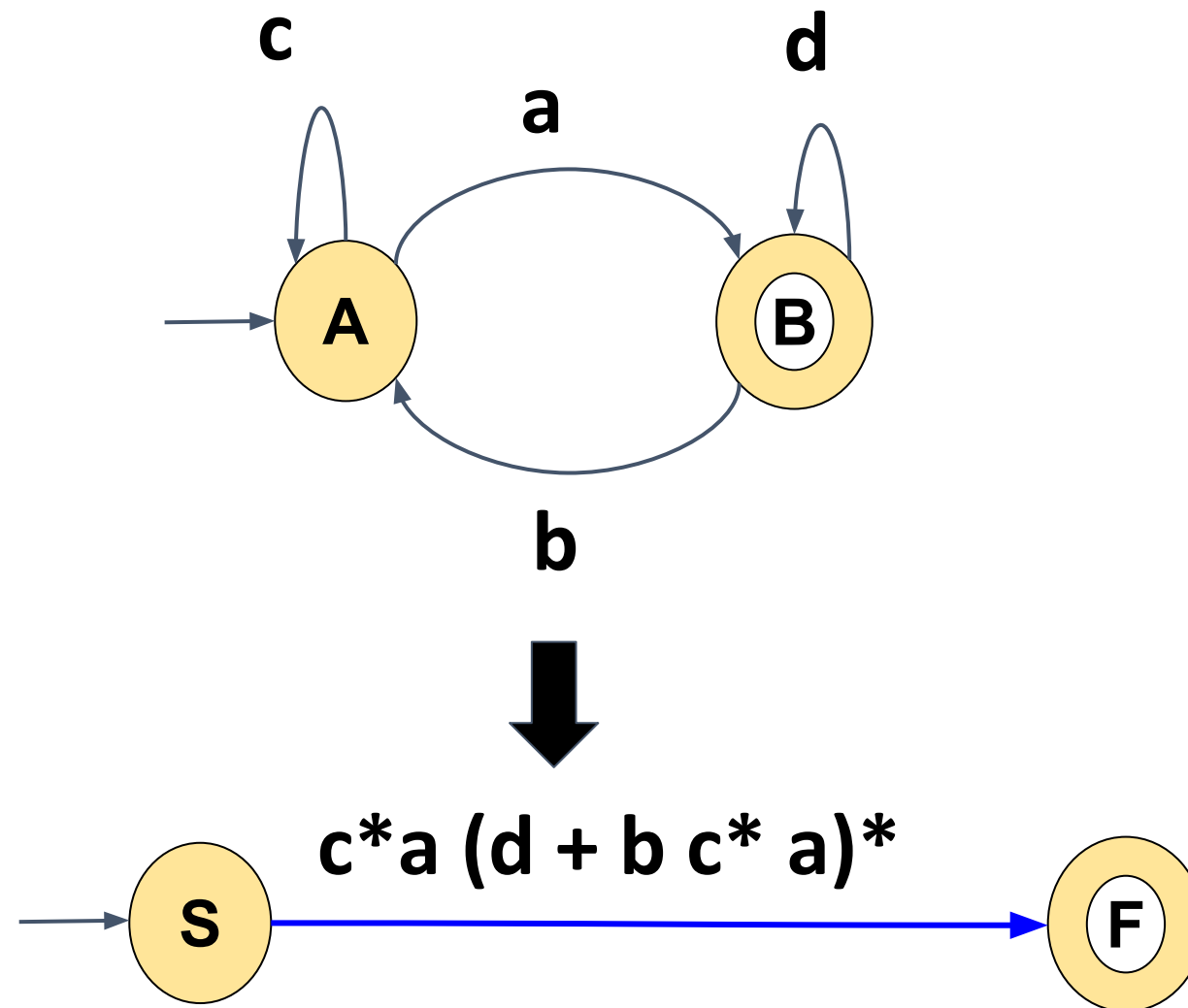
1. Eliminate A
2. Eliminate B

Example 4 :



1. Eliminate A
2. Eliminate B

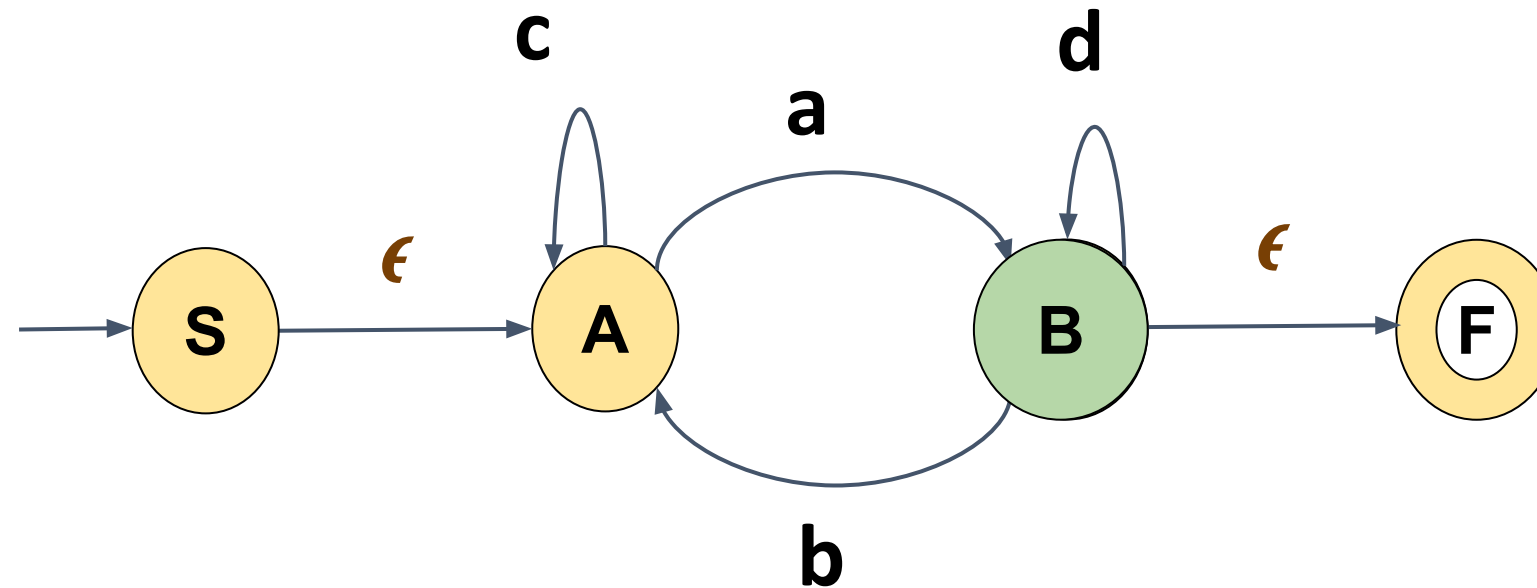
Example 4 :



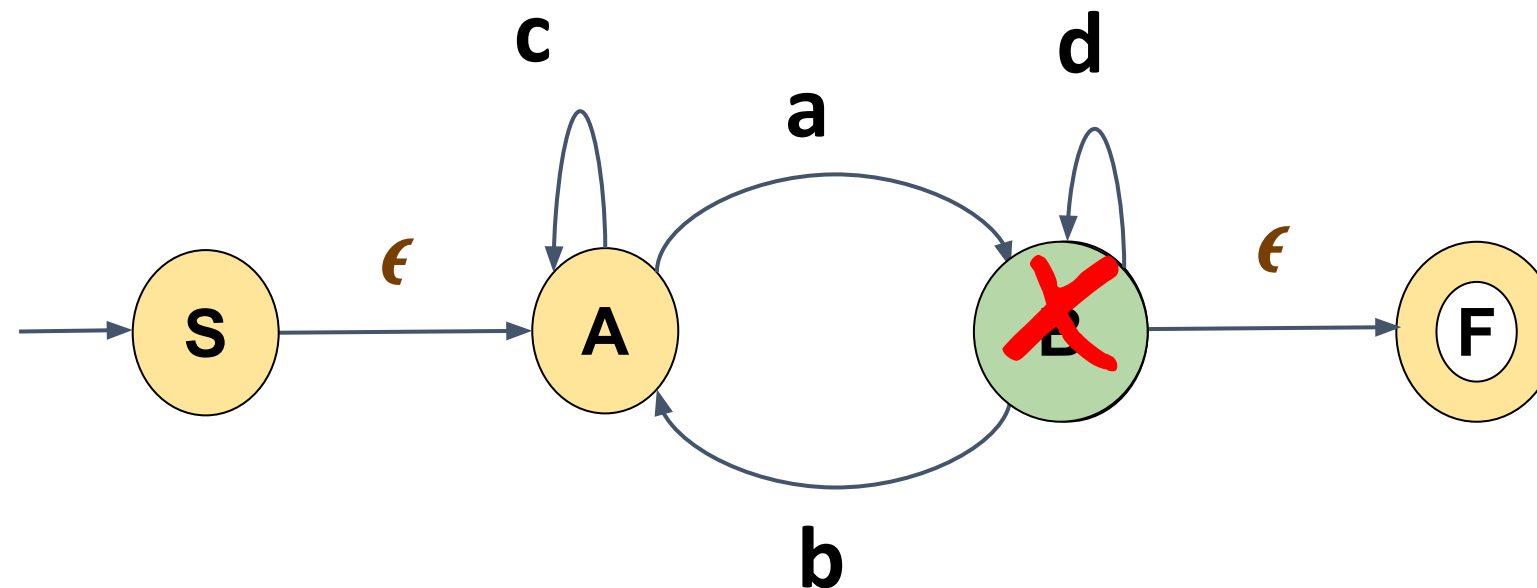
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 4 :

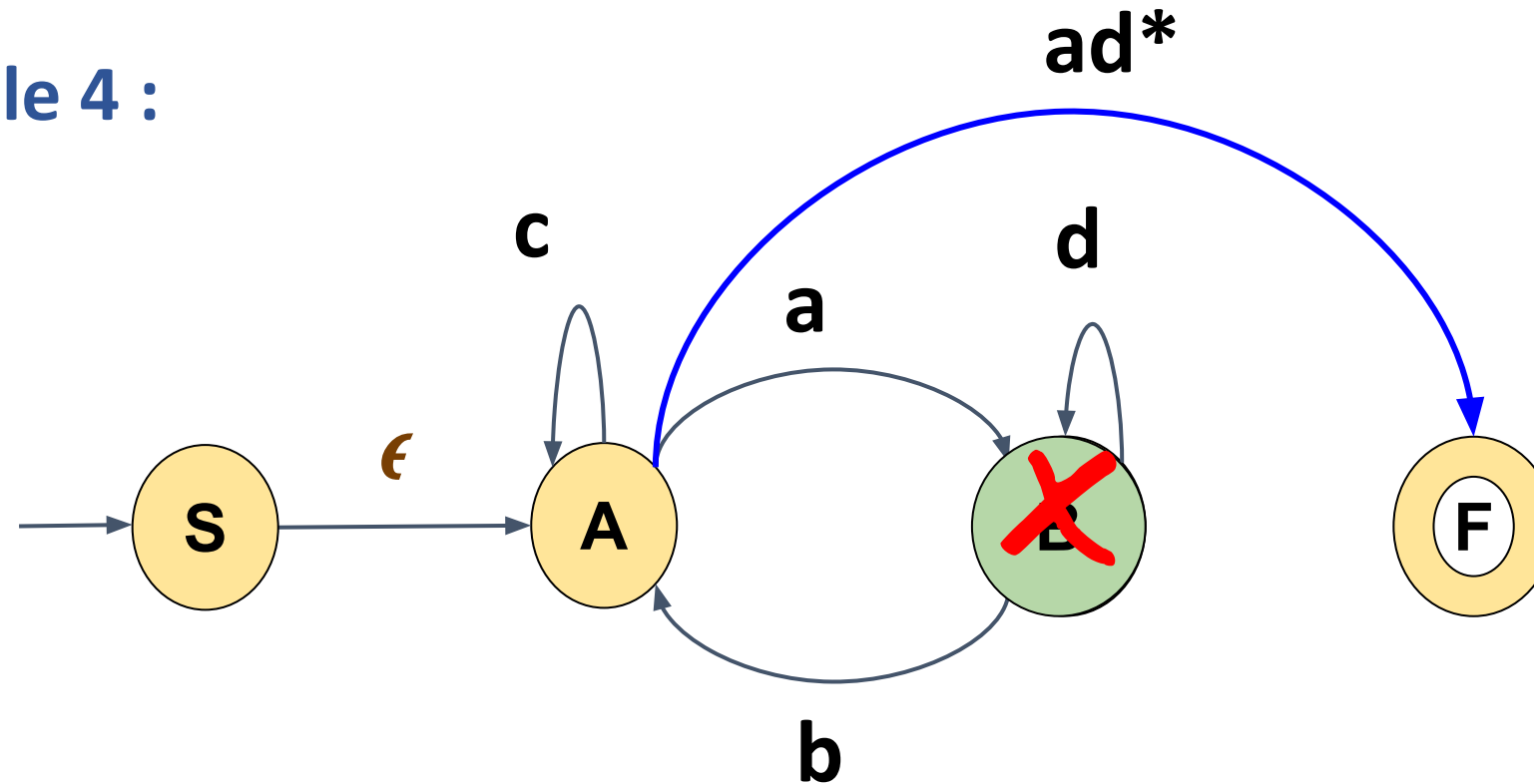


Example 4 :



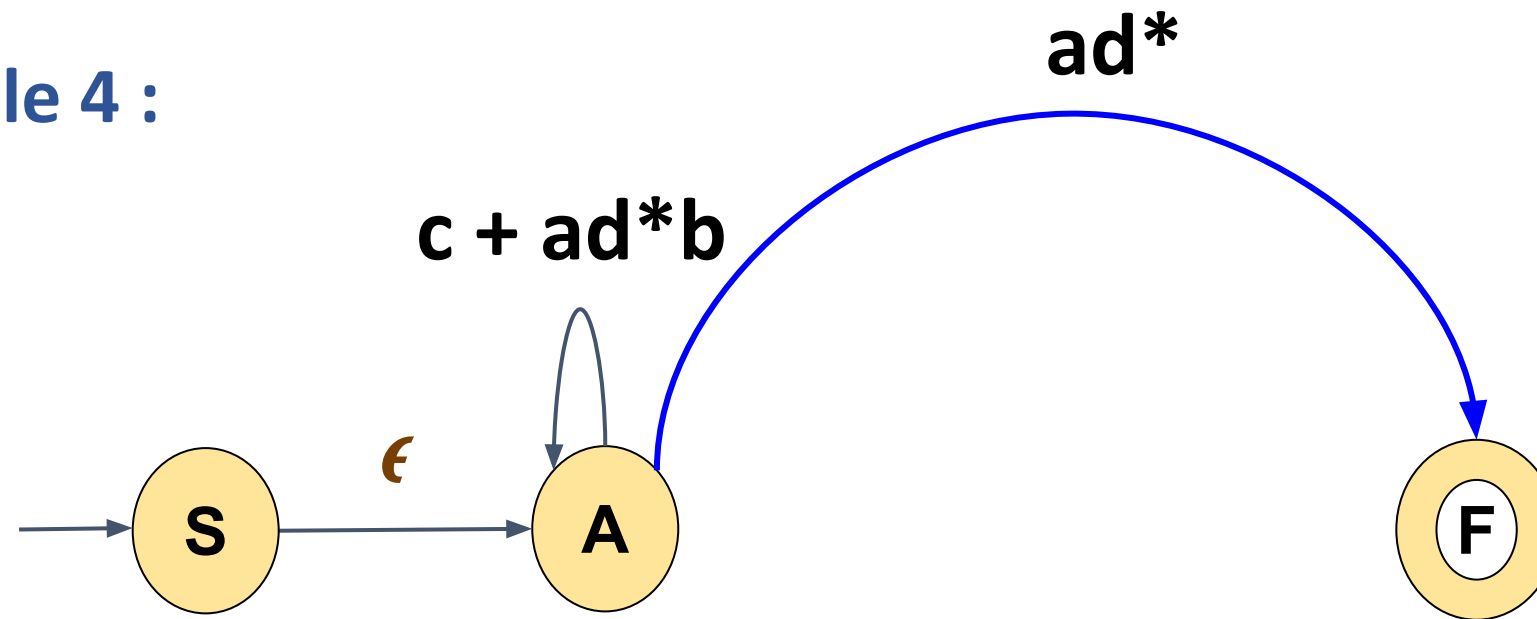
1. Eliminate B

Example 4 :



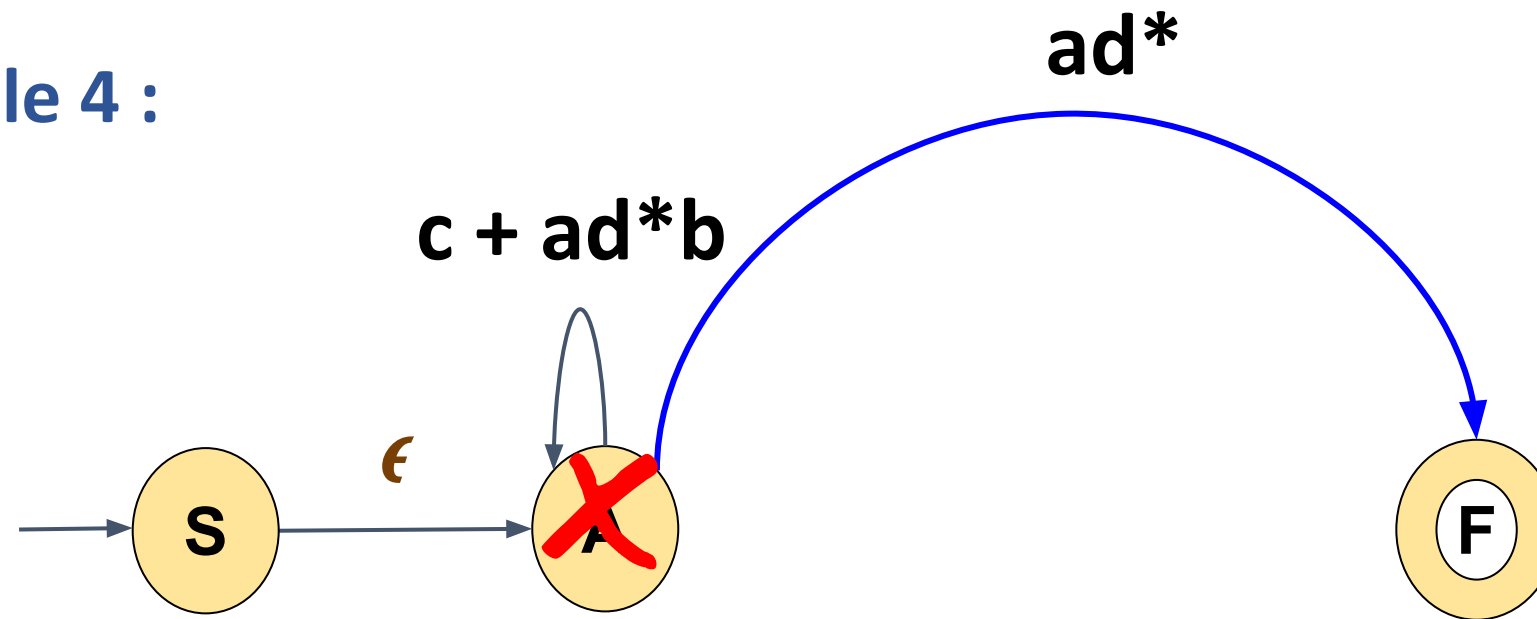
1. Eliminate B

Example 4 :



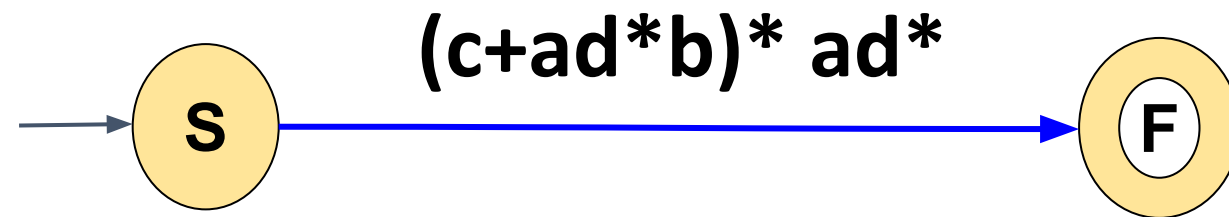
1. Eliminate B

Example 4 :



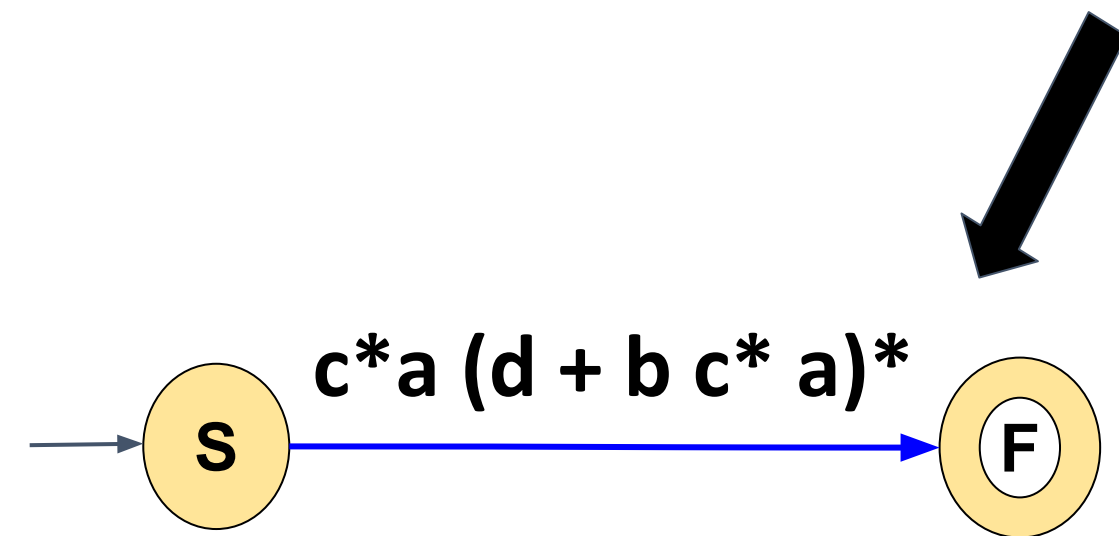
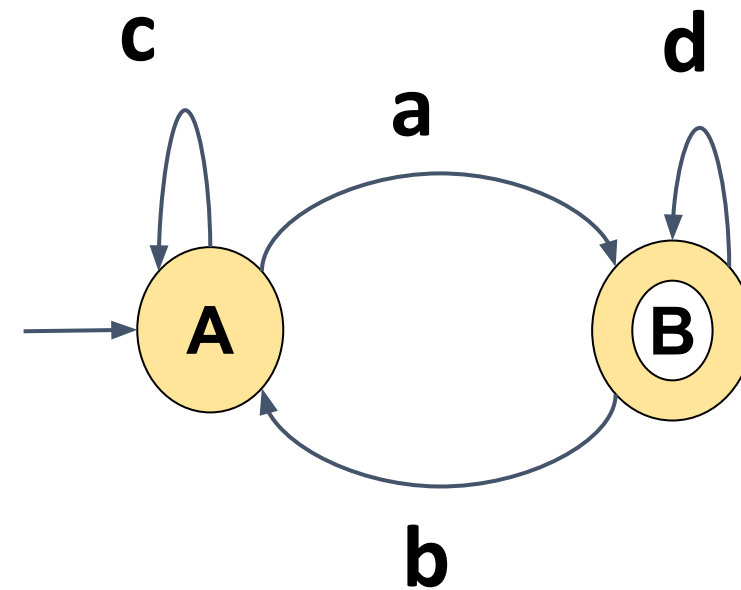
1. Eliminate B
2. Eliminate A

Example 4 :

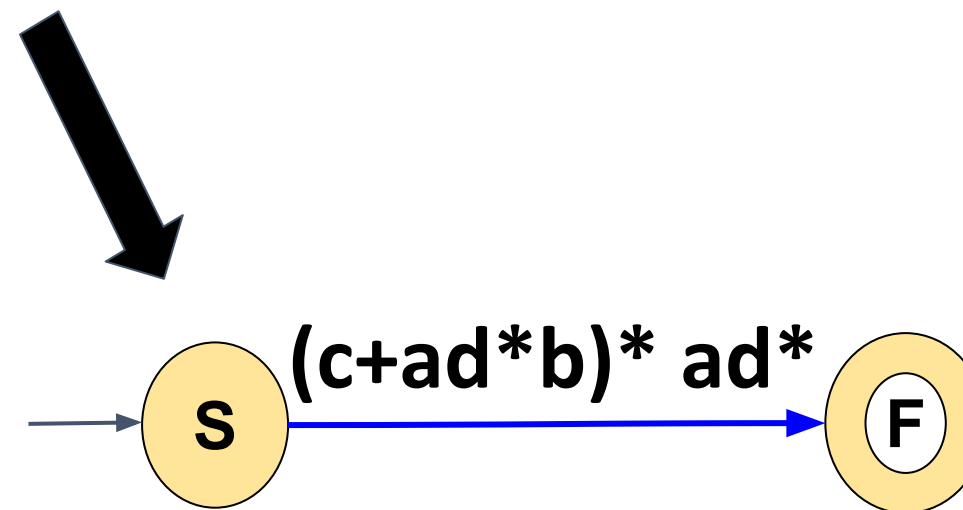


1. Eliminate B
2. Eliminate A

Example 4 :



1. Eliminate A
2. Eliminate B

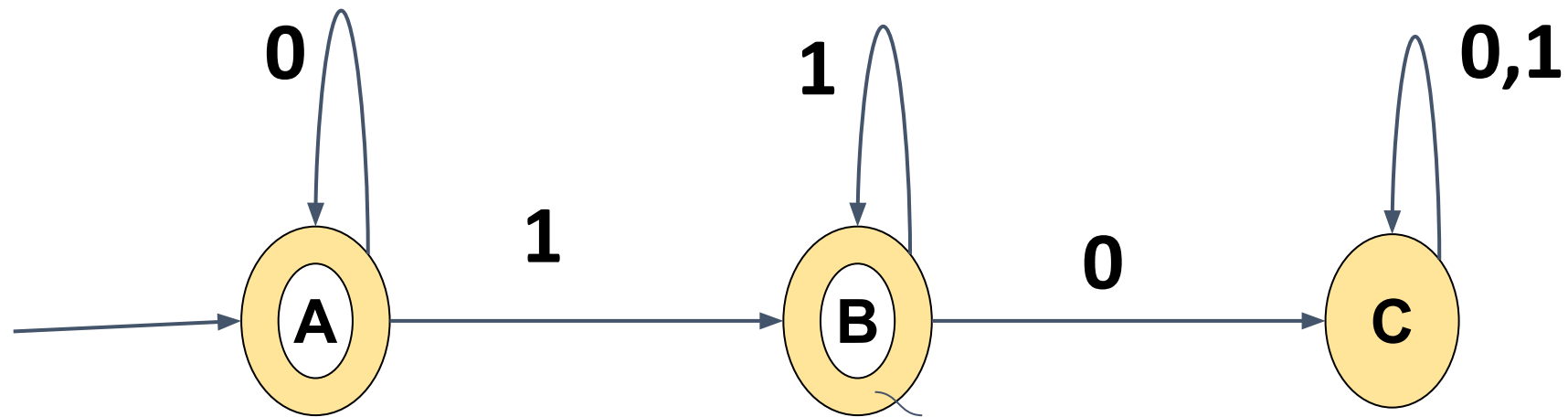


1. Eliminate B
2. Eliminate A

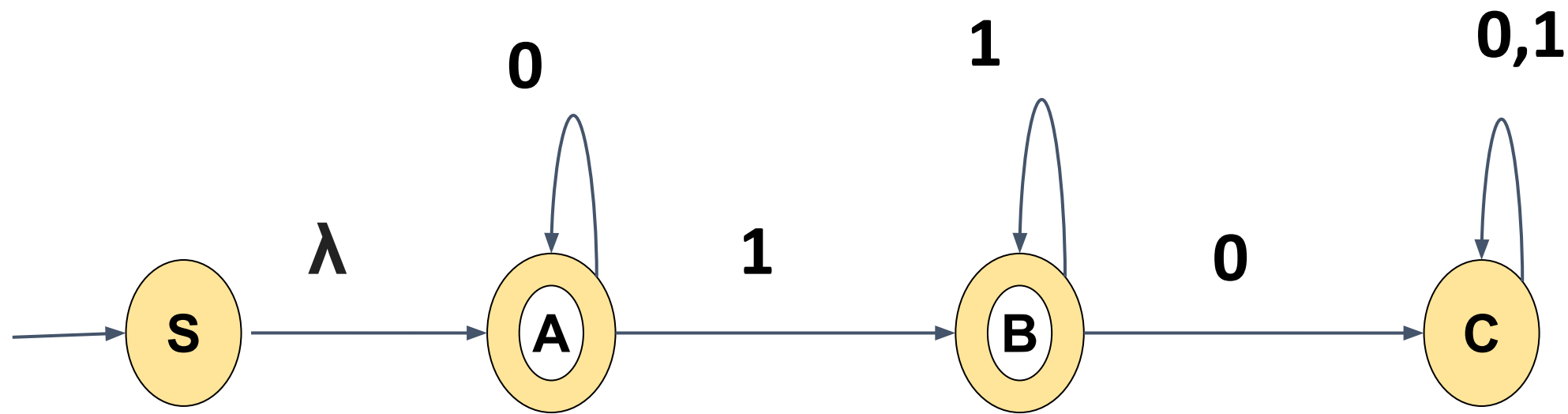
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 5 :

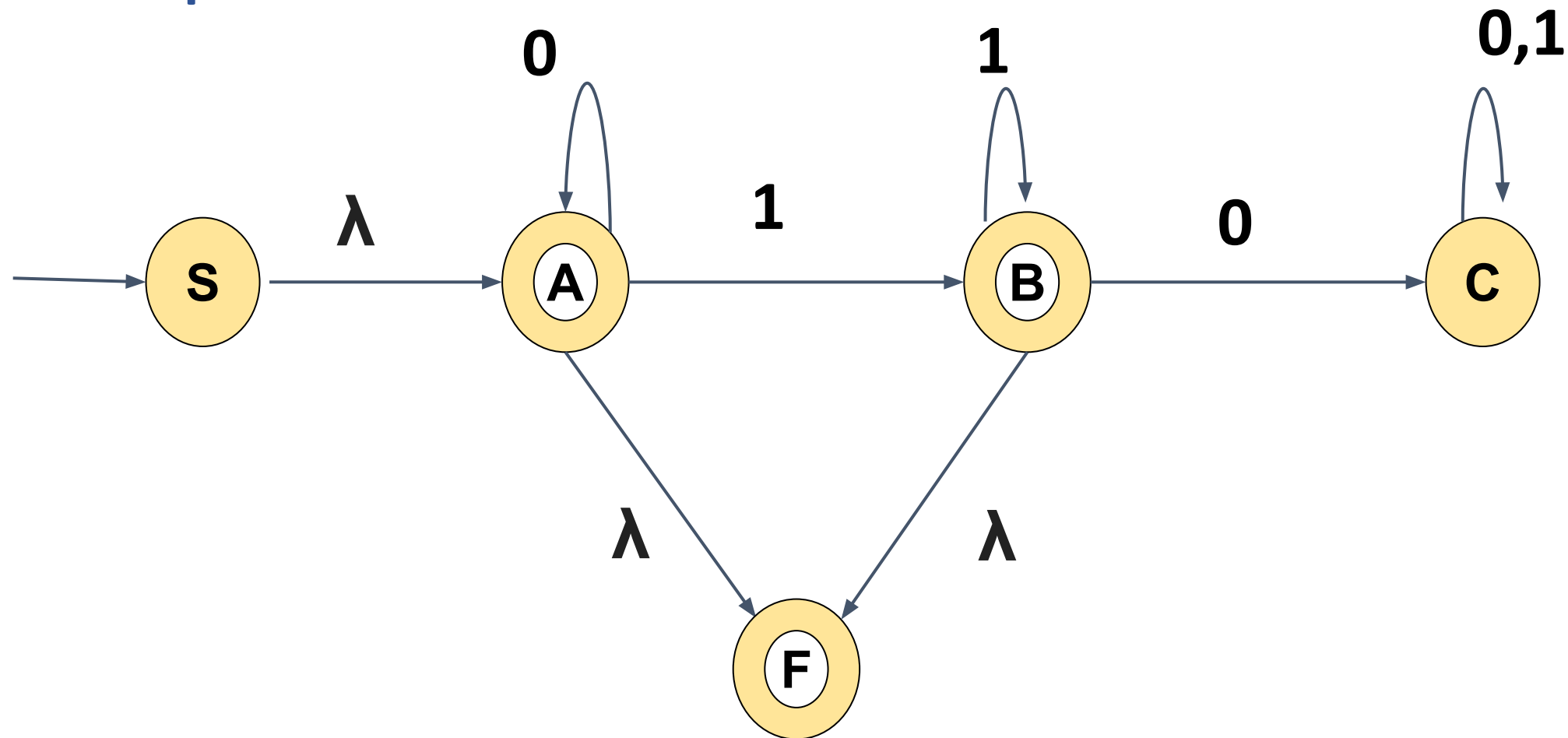


Example 5 :



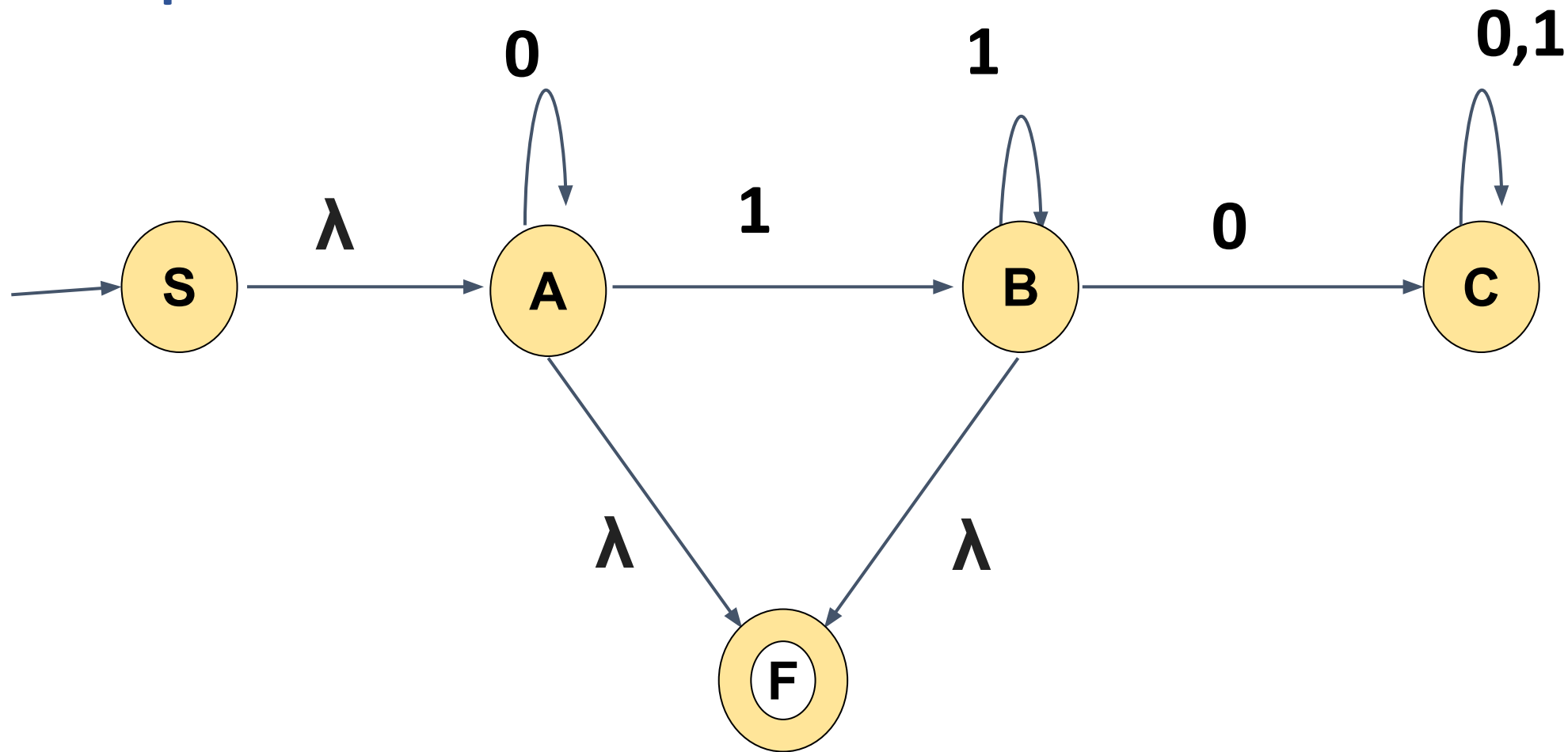
A new start state (S) is introduced as there is an incoming edge to the existing start state

Example 5 :



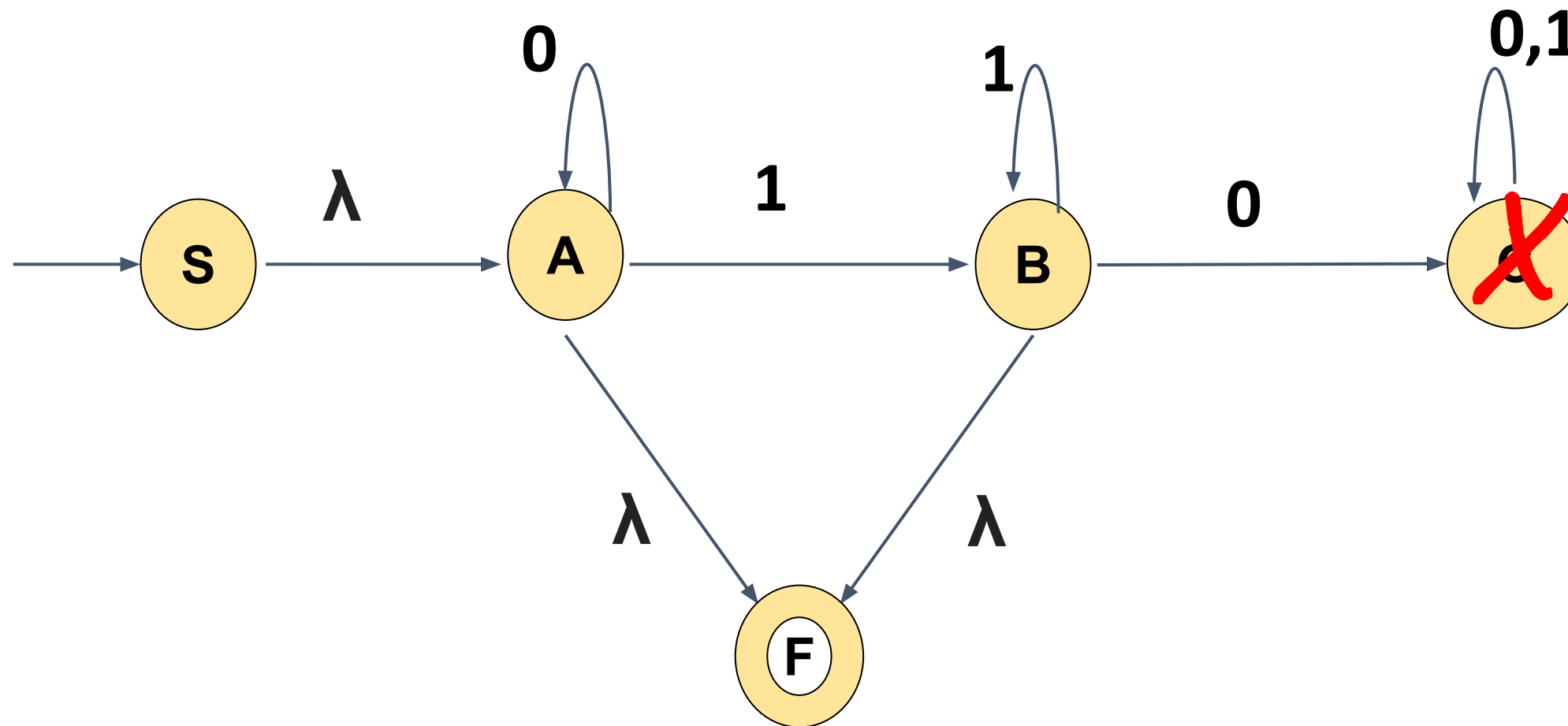
A new final state (F) is introduced as there is an outgoing edge from the existing final state and we must have a single accepting state

Example 5 :



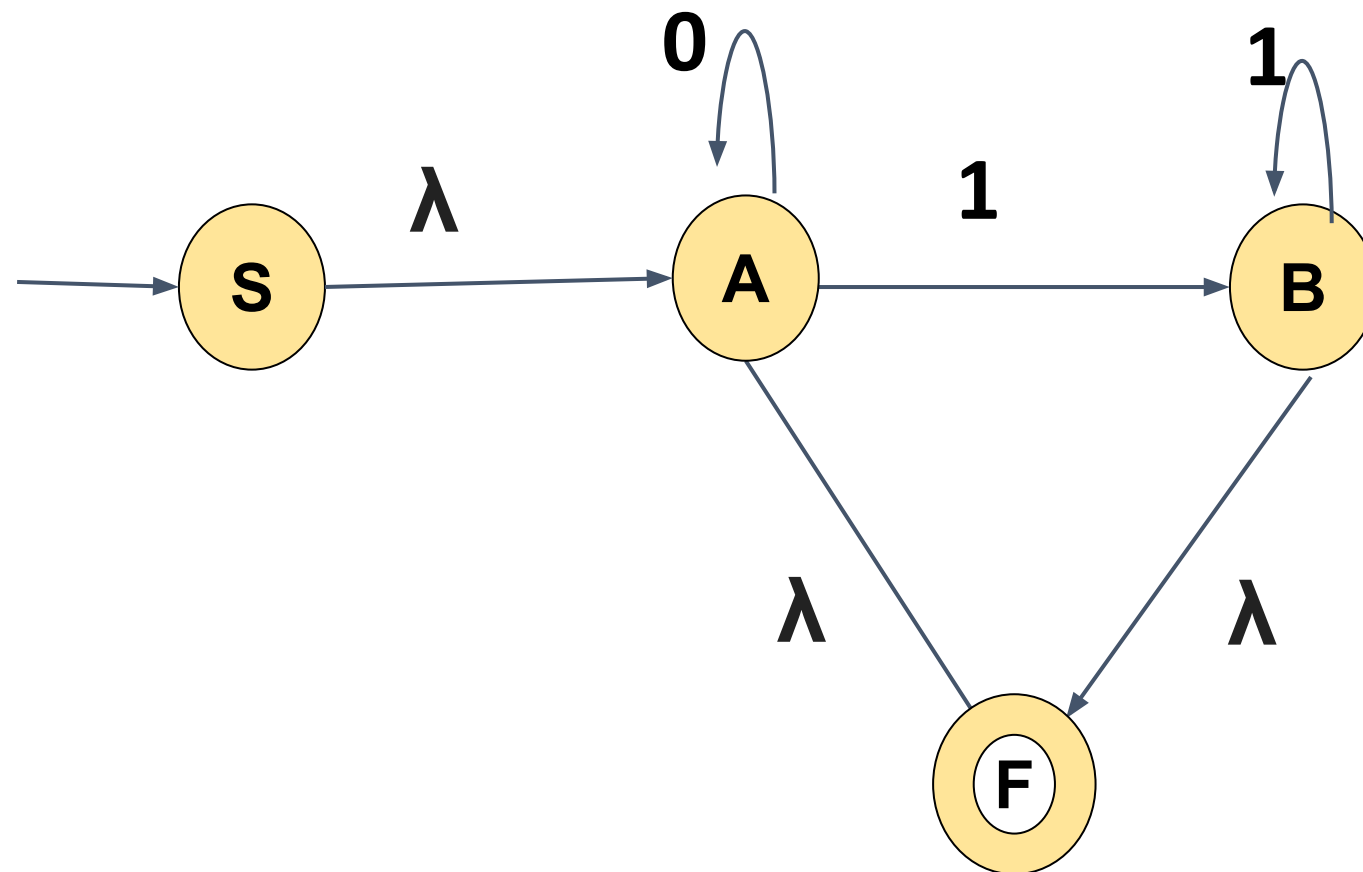
Previous final state is made as non final state

Example 5 :



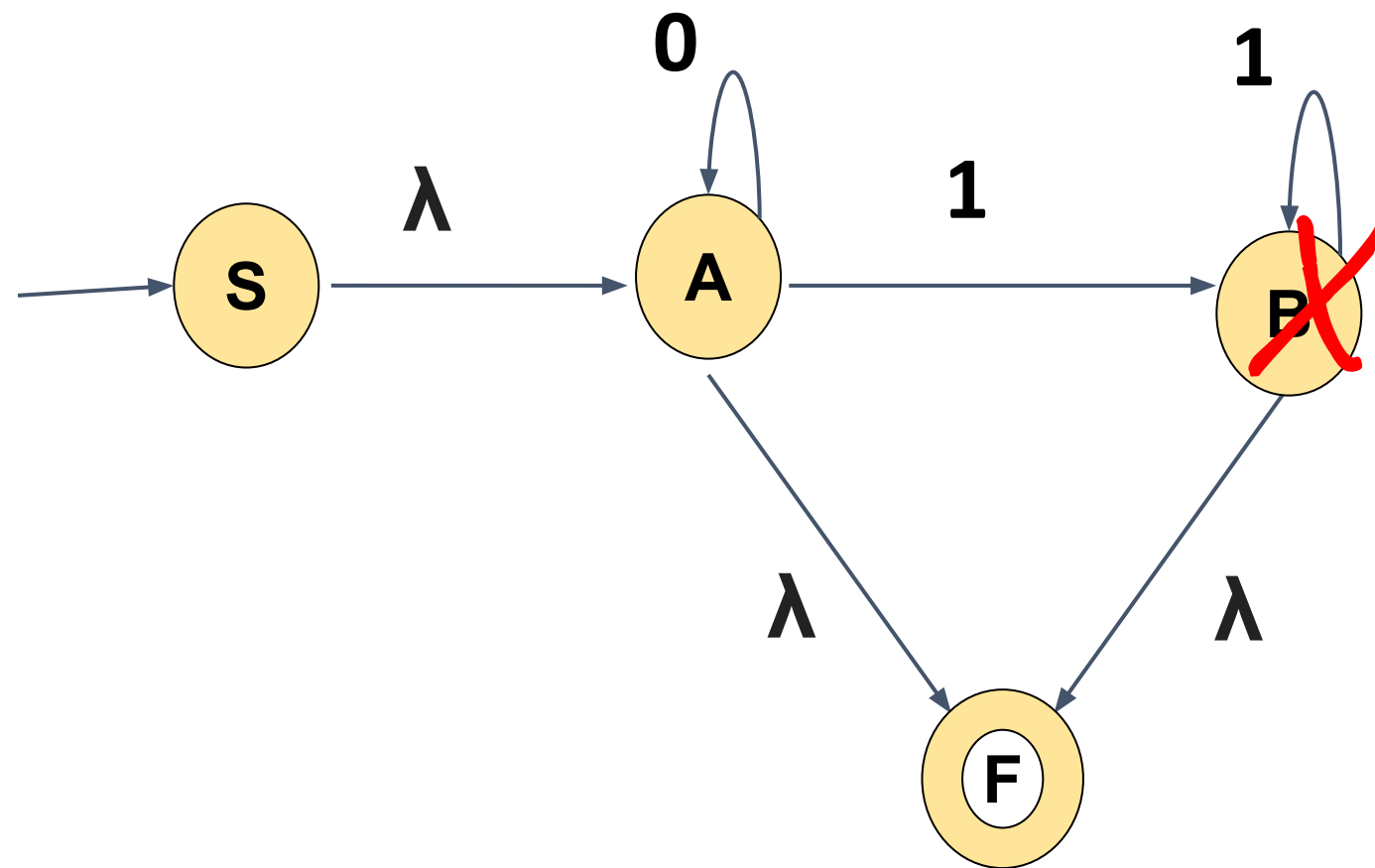
1. Eliminate C

Example 5 :



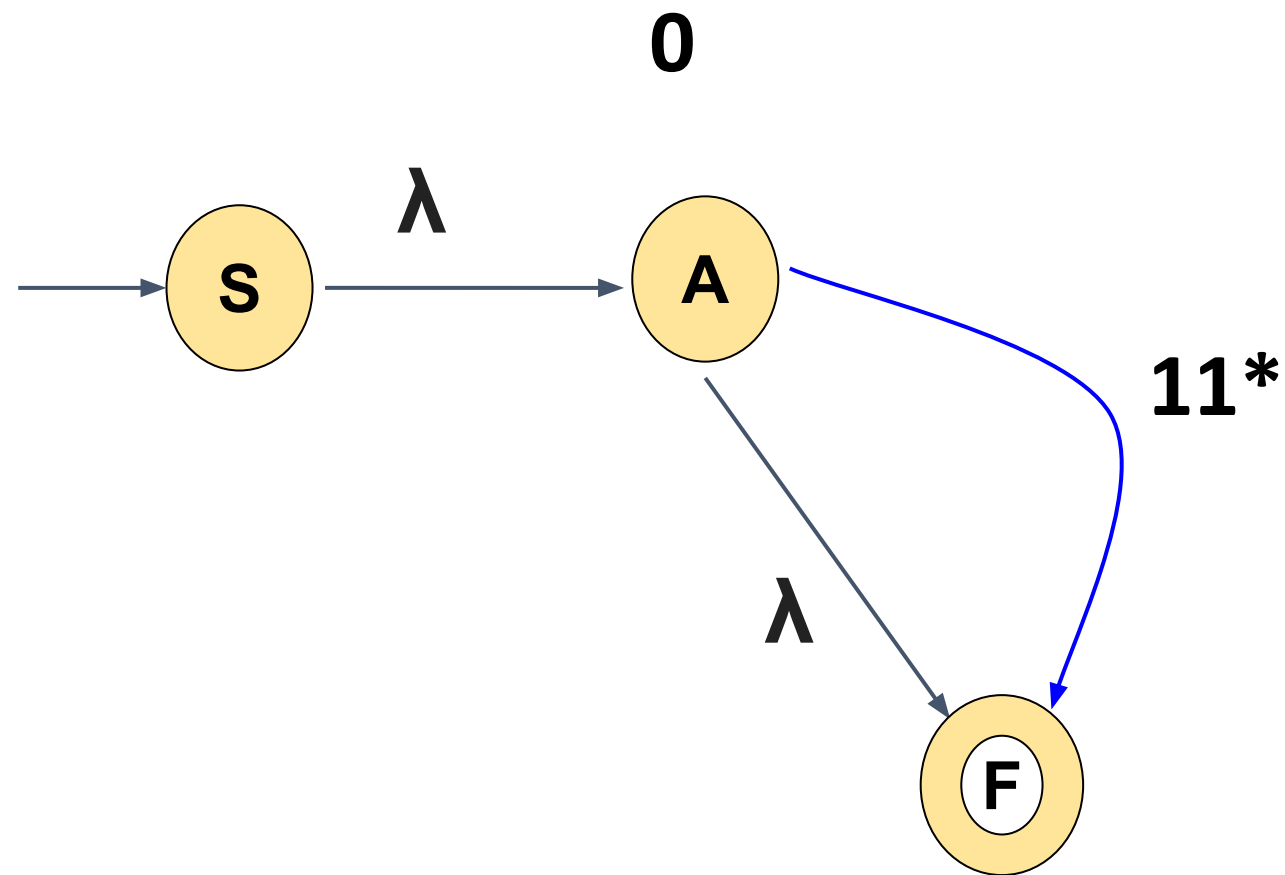
1. Eliminate C

Example 5 :



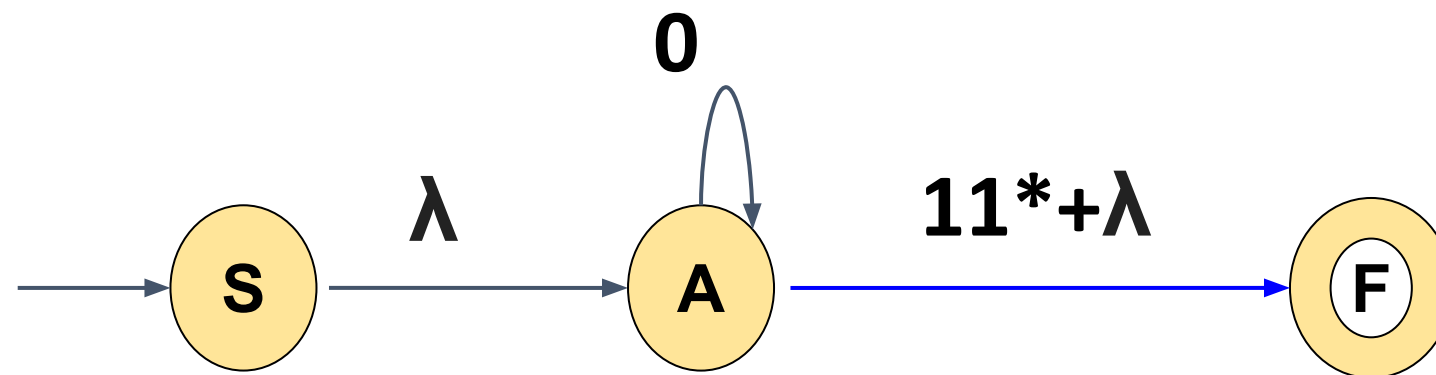
1. Eliminate C
2. Eliminate B

Example 5 :



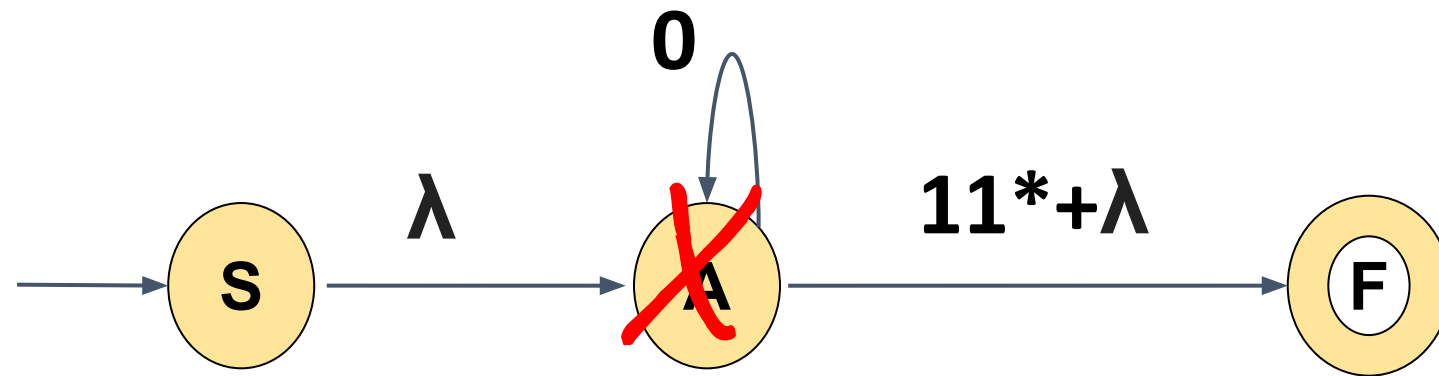
1. Eliminate C
2. Eliminate B

Example 5 :



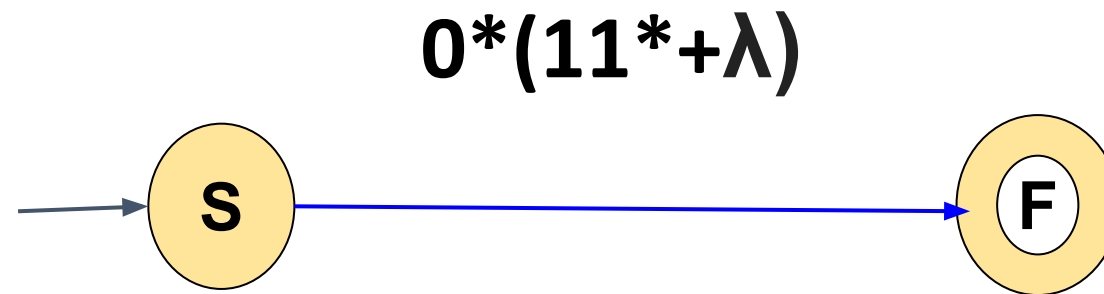
1. Eliminate C
2. Eliminate B

Example 5 :



1. Eliminate C
2. Eliminate B
3. Eliminate A

Example 5 :

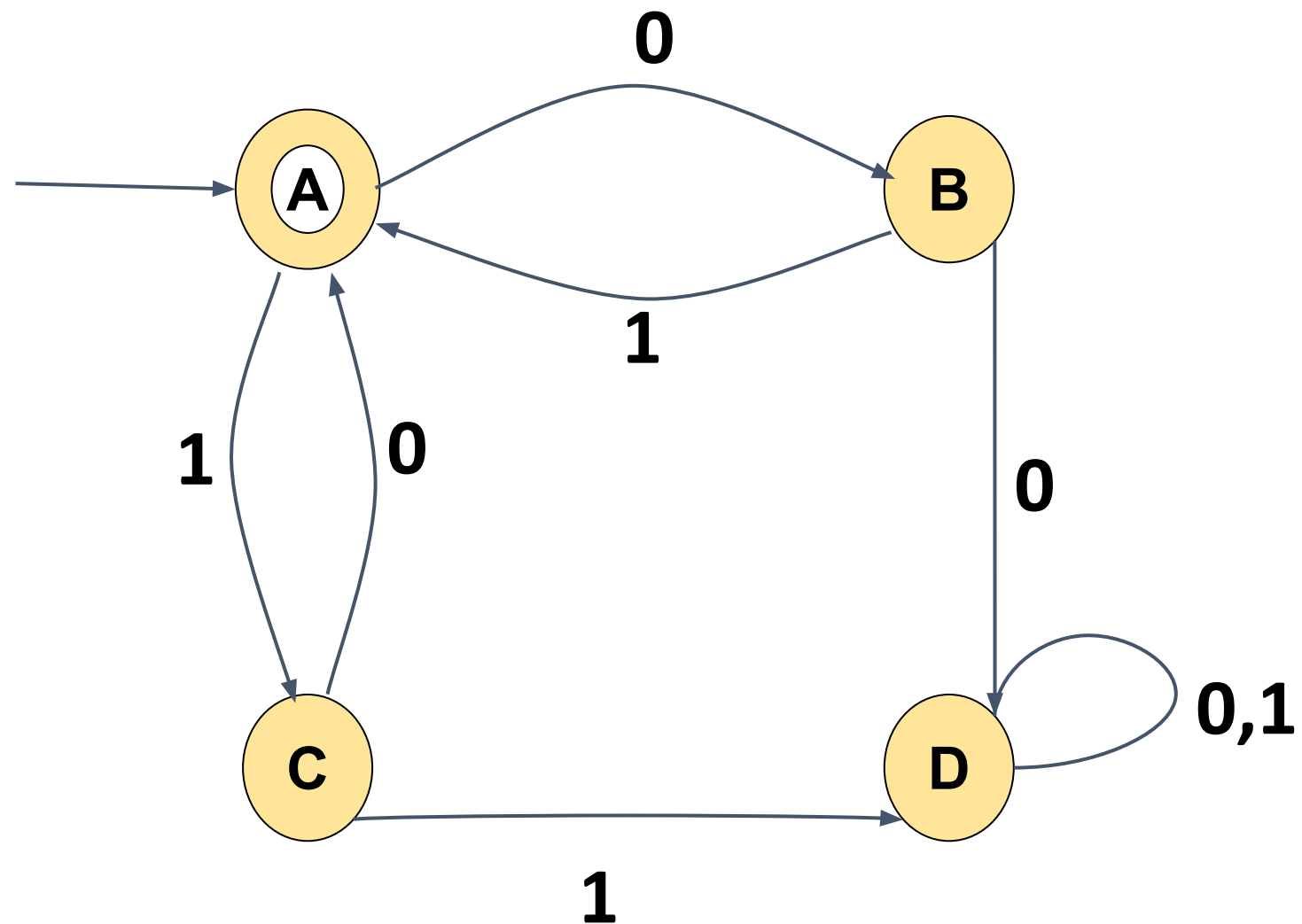


$$= 0(11^*+\lambda)$$

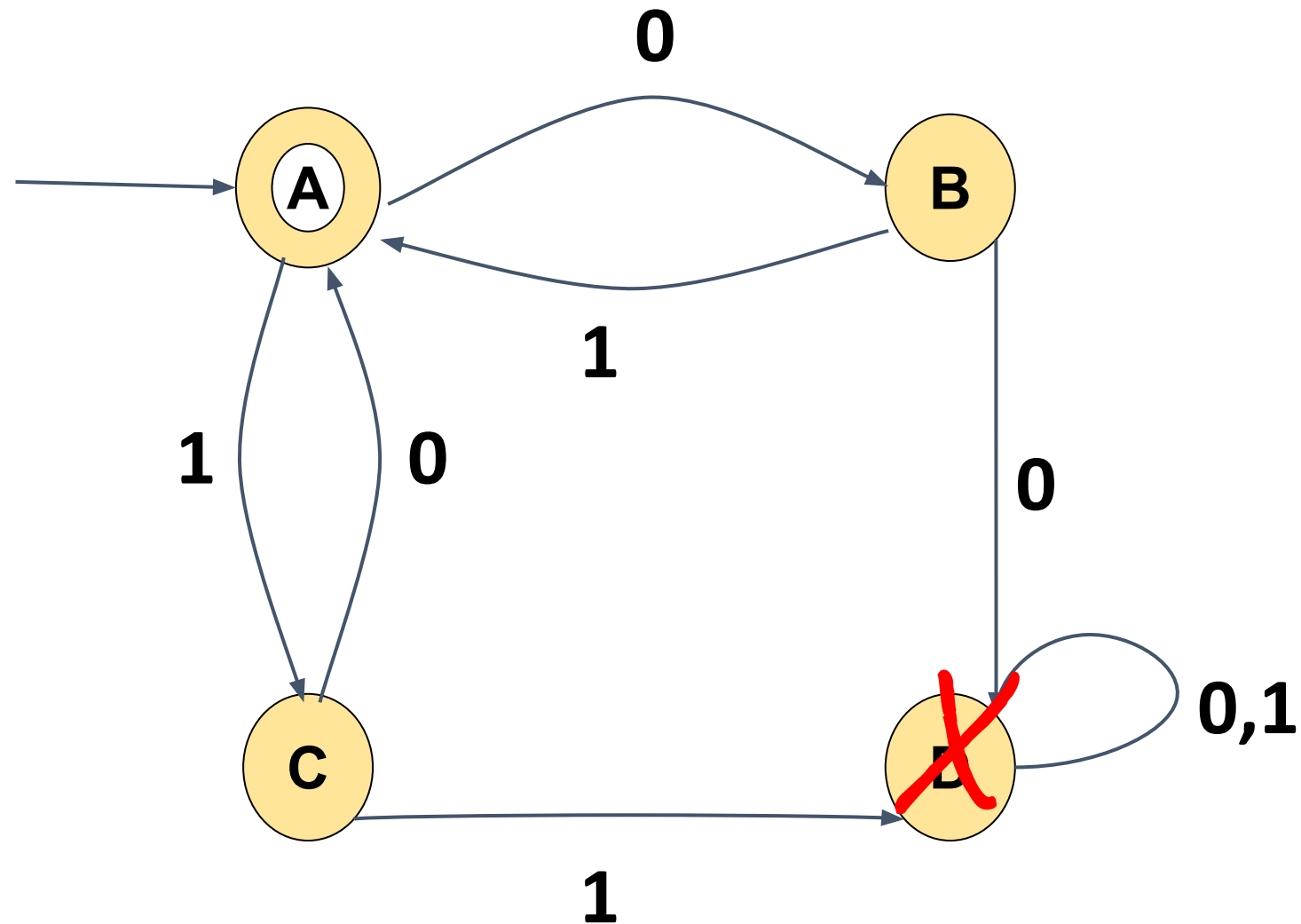
$$= 0^*(1^++\lambda)$$

$$\text{RE} = 0^*1^*$$

Example 6 :



Example 6 :

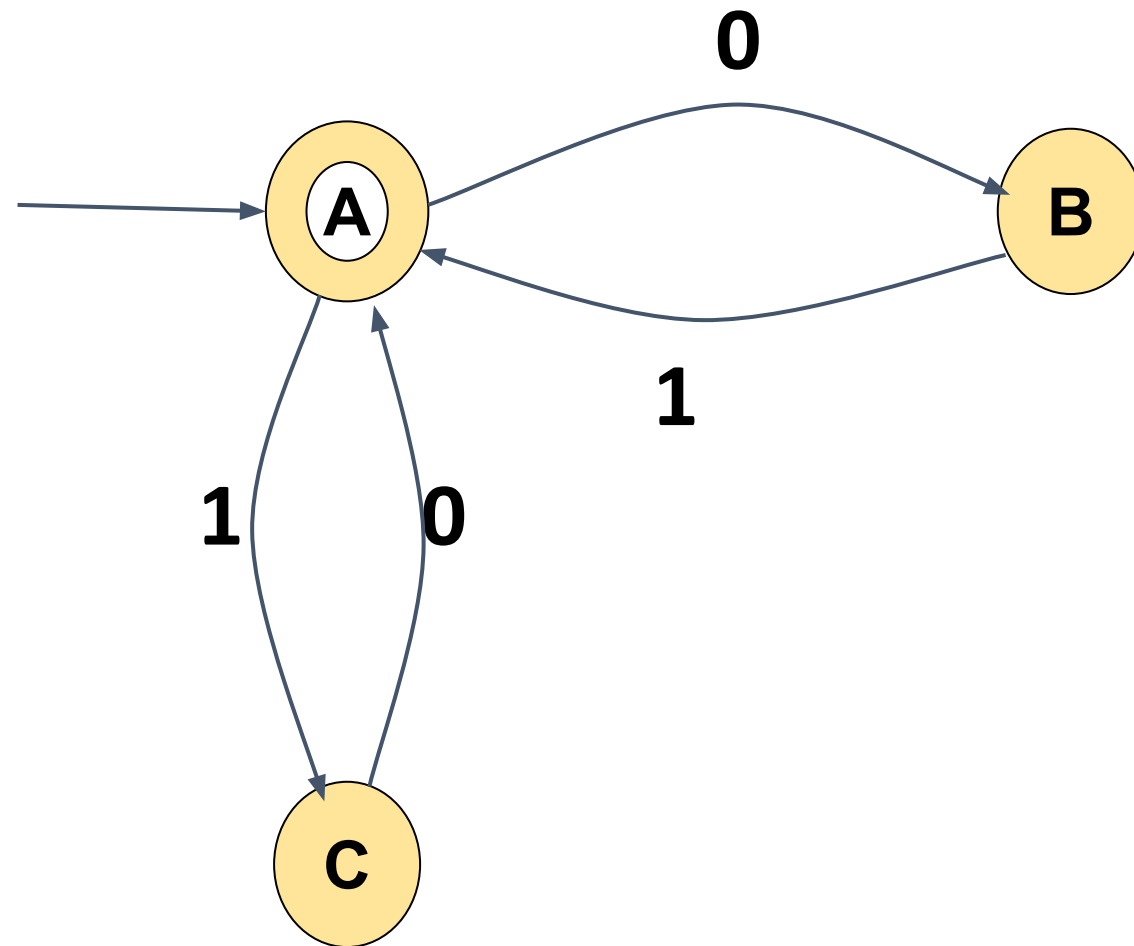


1. Eliminate D
(Dead state)

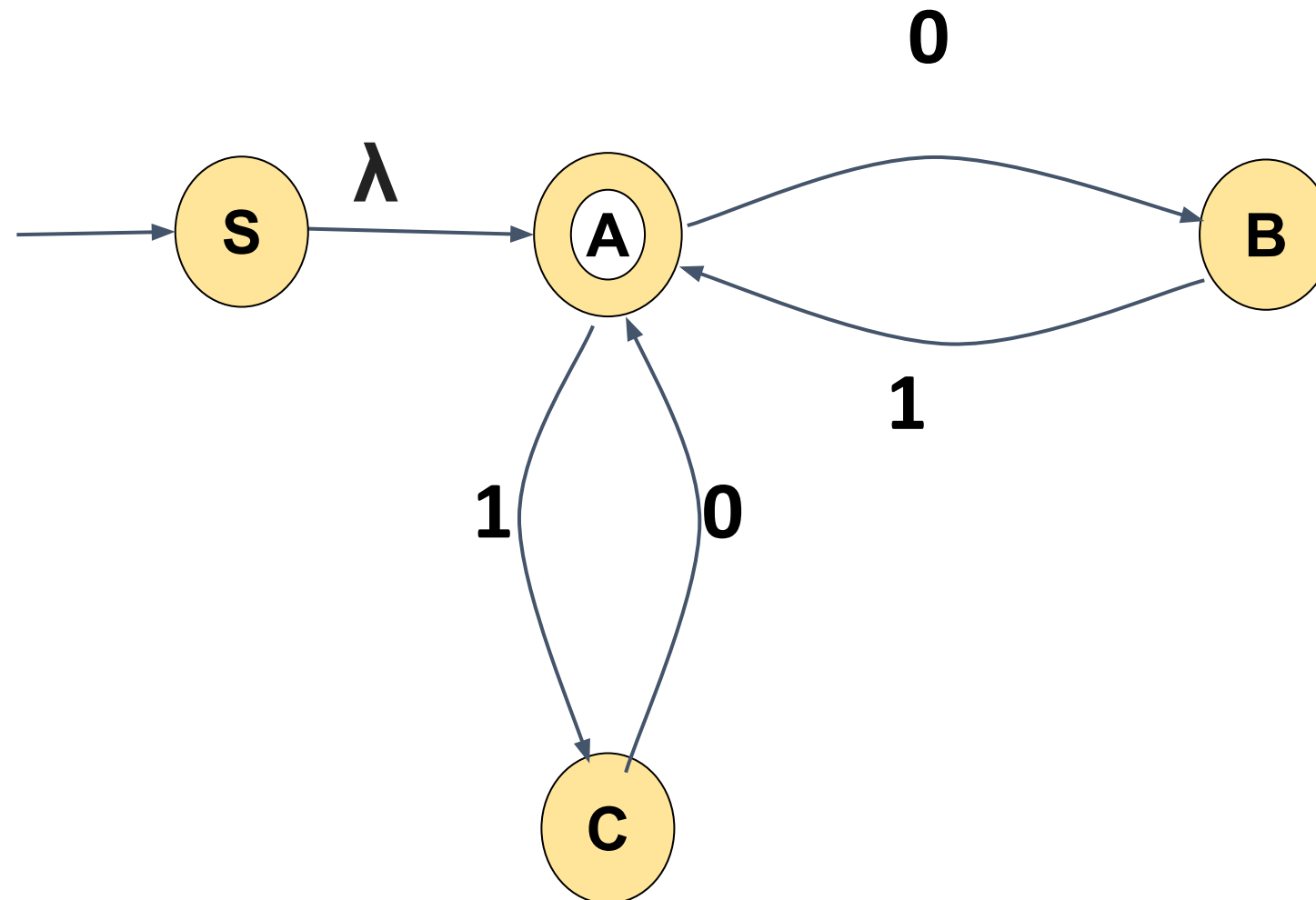
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 6 :

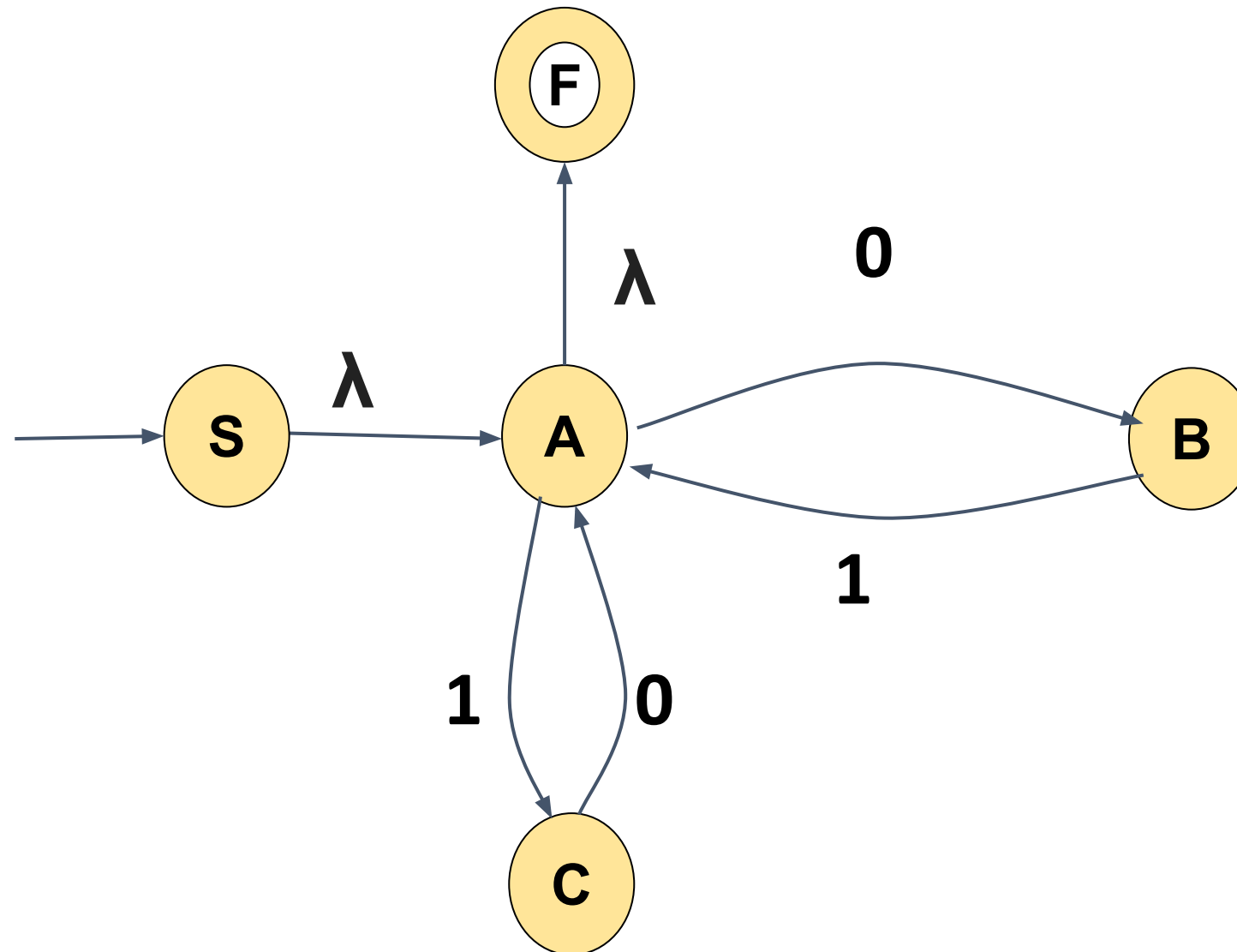


Example 6 :



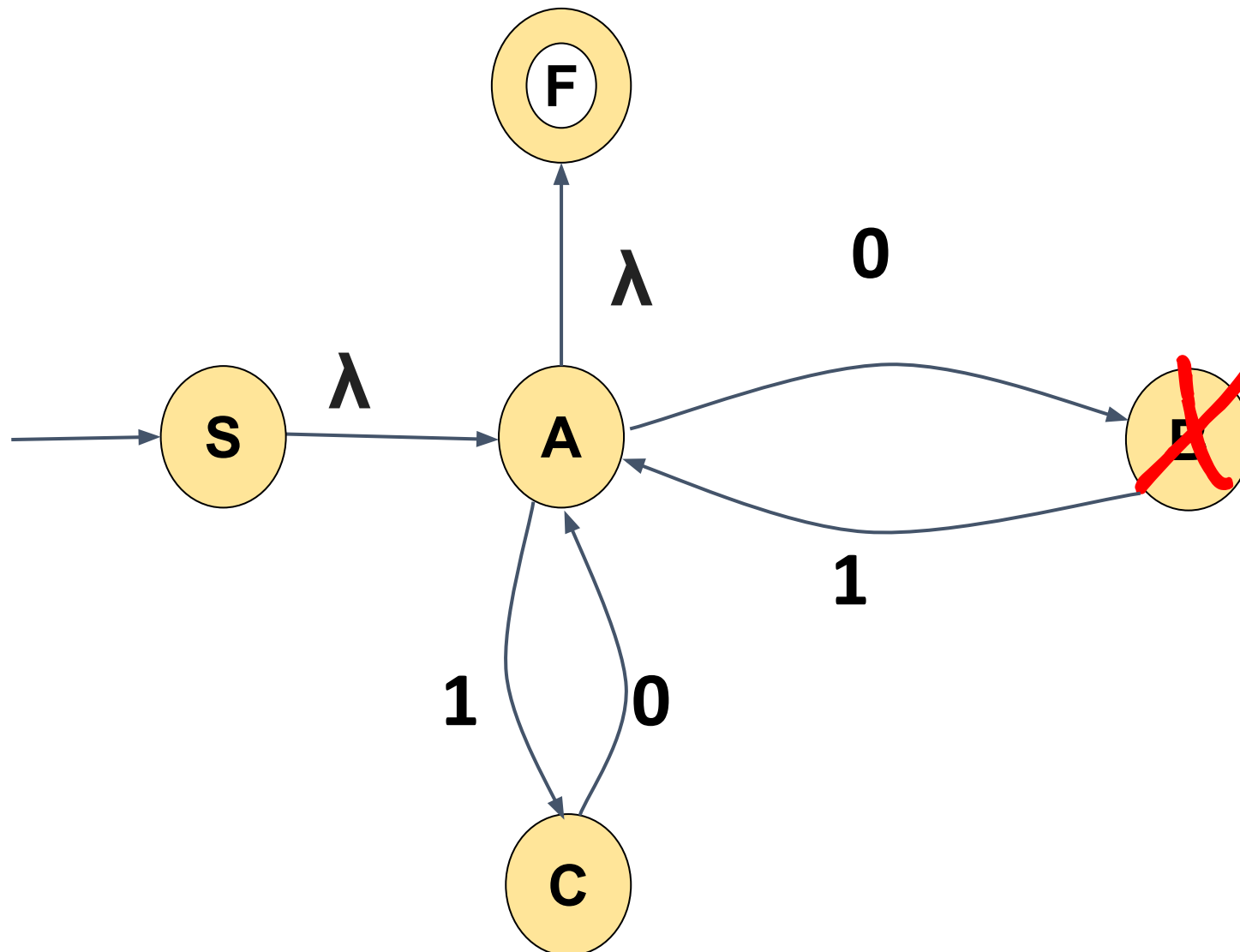
A new start state (S) is introduced as there is an incoming edge to the existing start state

Example 6 :



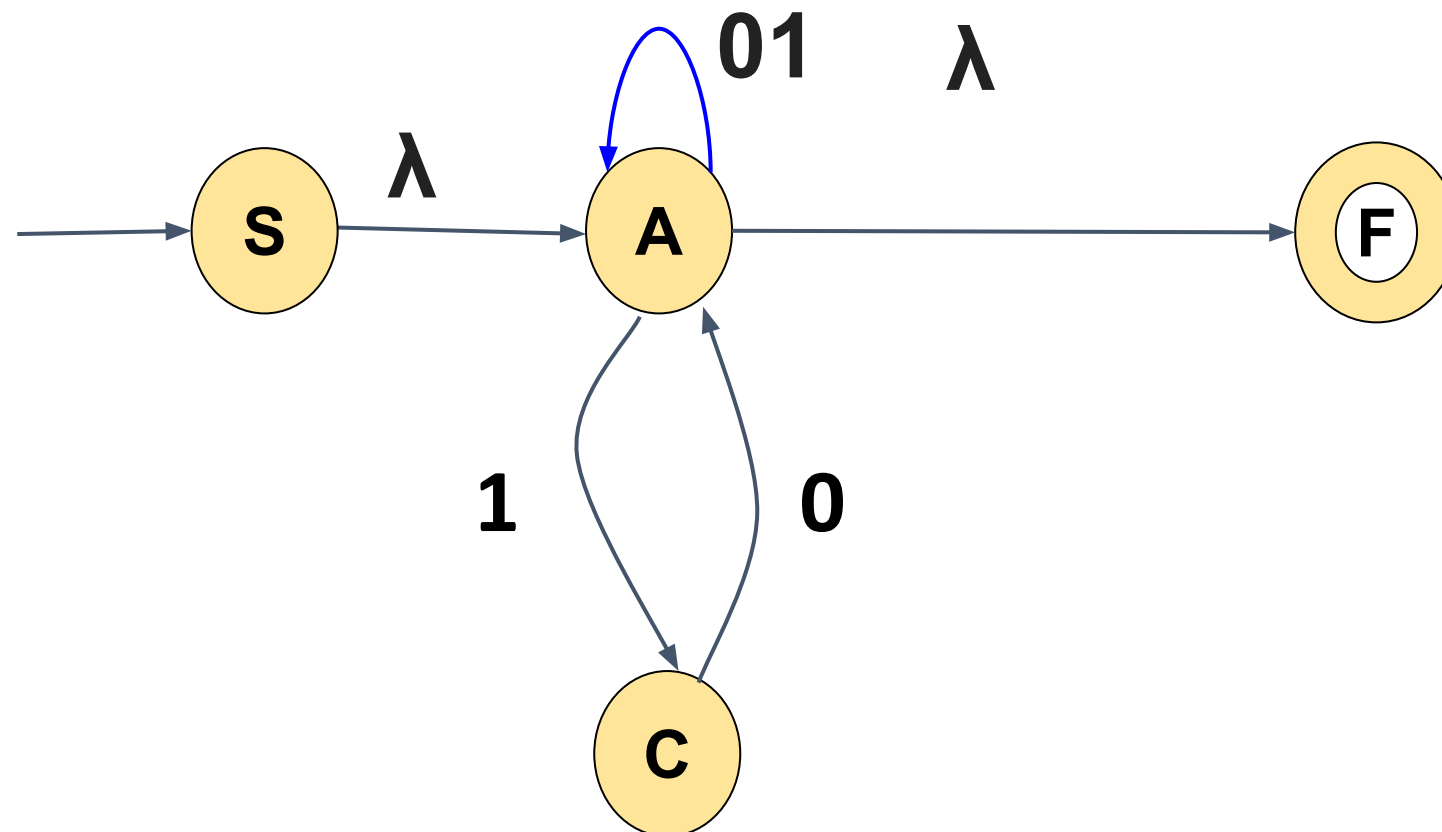
A new final state (F) is introduced as there is an outgoing edge from the existing final state

Example 6 :



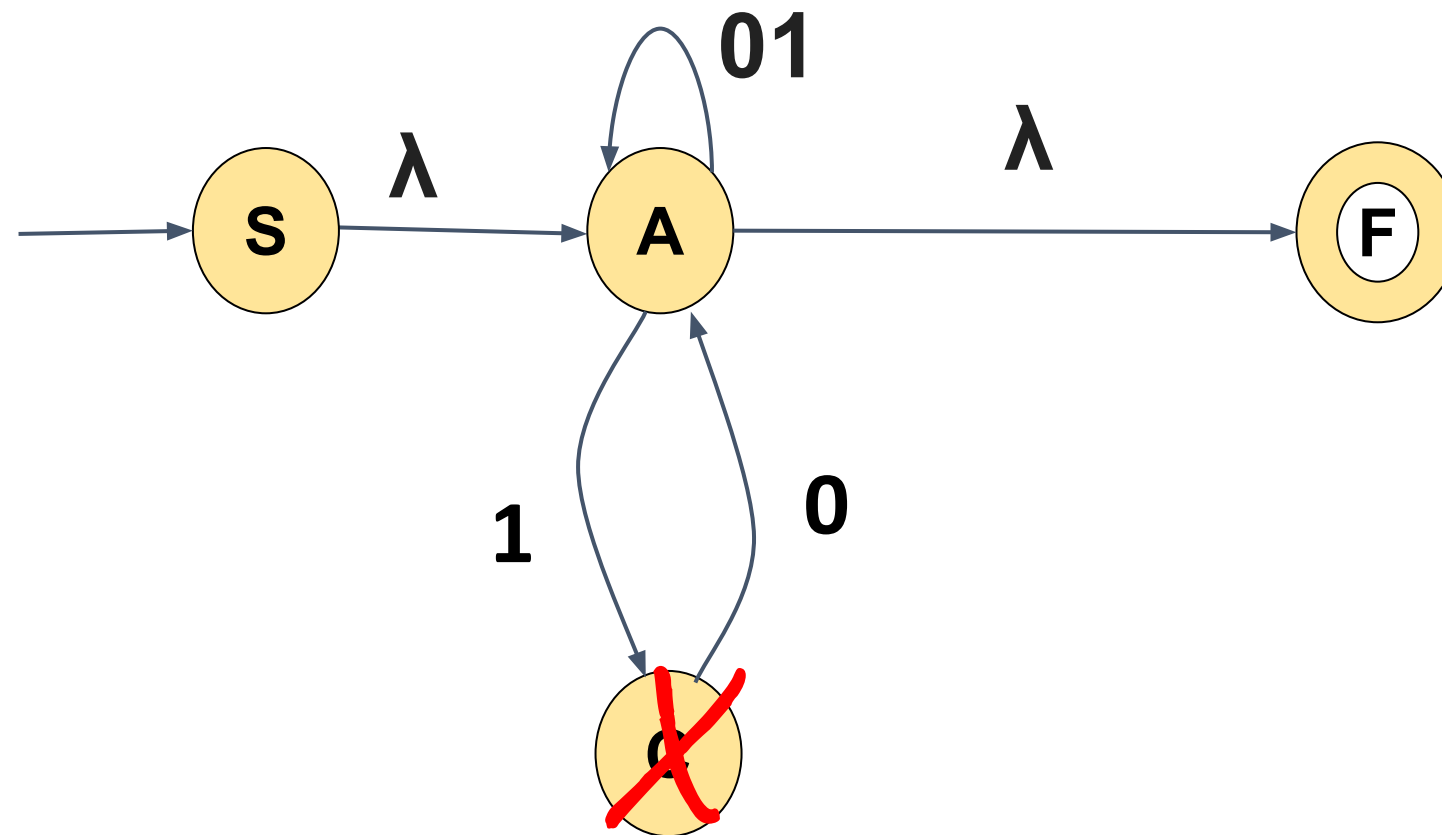
1. Eliminate D
2. Eliminate B

Example 6 :



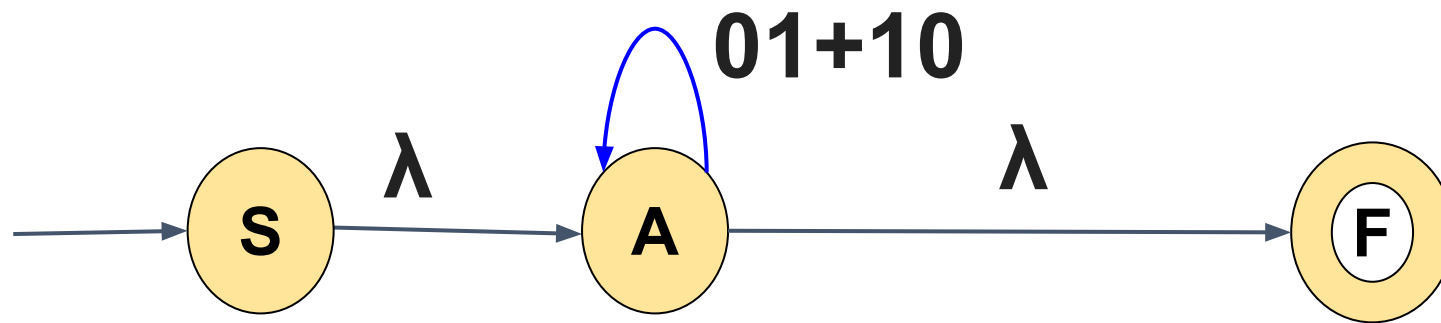
1. Eliminate D
2. Eliminate B

Example 6 :



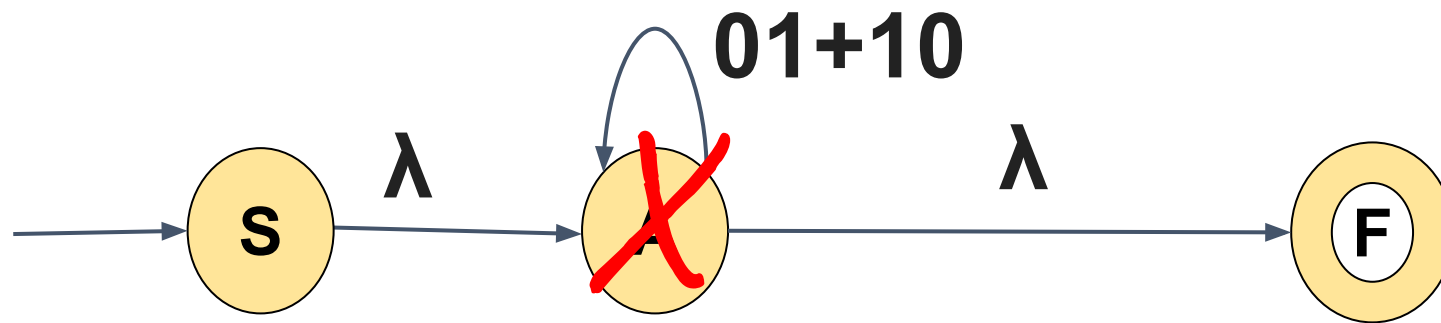
1. Eliminate D
2. Eliminate B
3. Eliminate C

Example 6 :



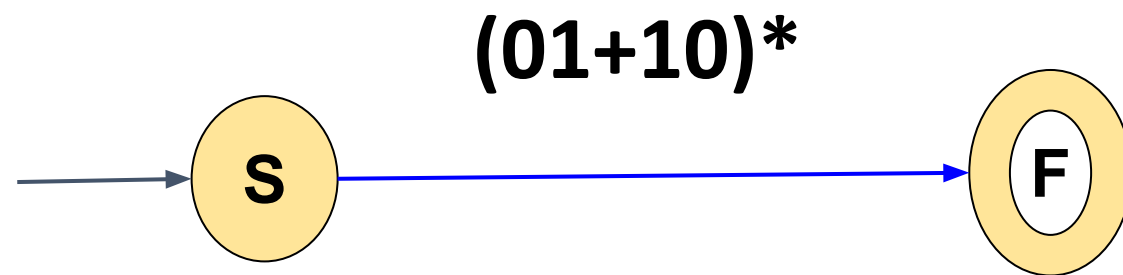
1. Eliminate D
2. Eliminate B
3. Eliminate C

Example 6 :



1. Eliminate D
2. Eliminate B
3. Eliminate C
4. Eliminate A

Example 6 :

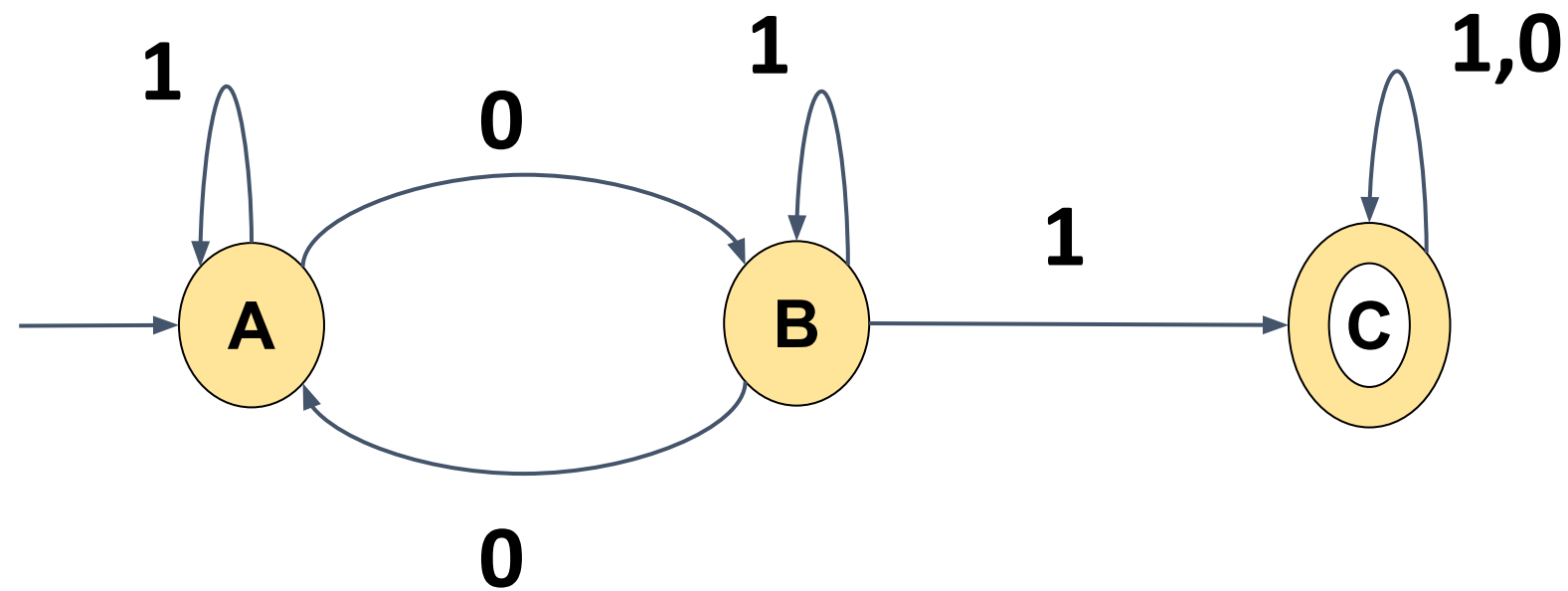


$RE=(01+10)^*$

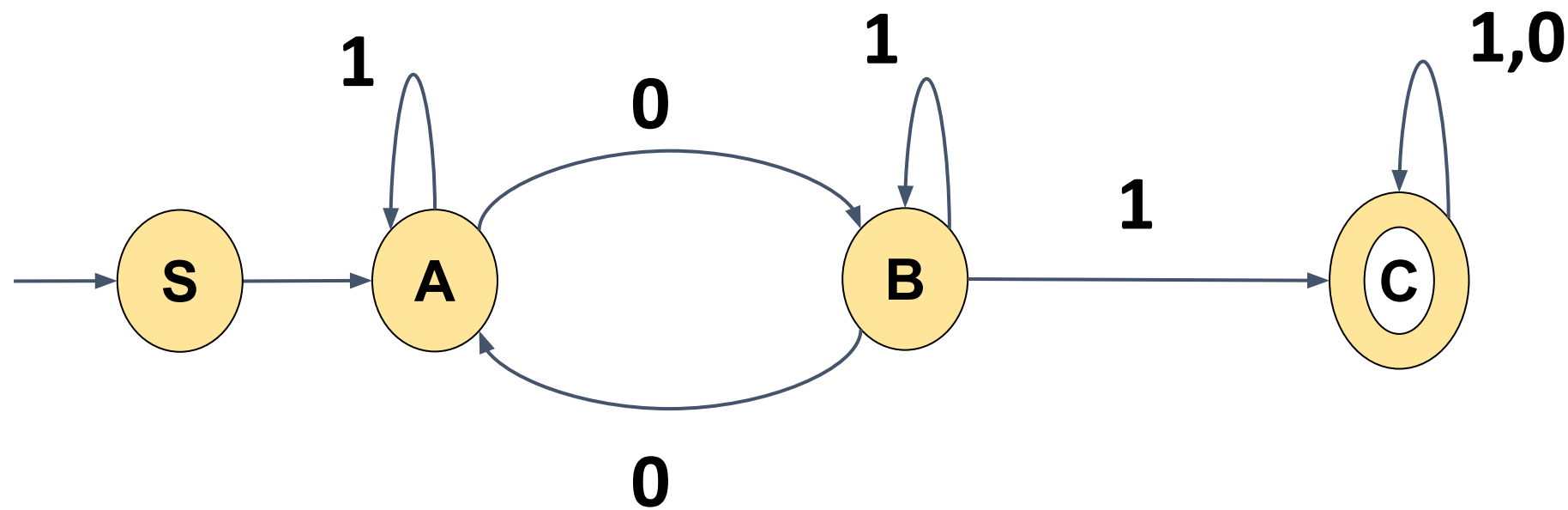
Automata Formal Languages and Logic

Unit 2 - Finite Automata to Regular Expression

Example 7 :

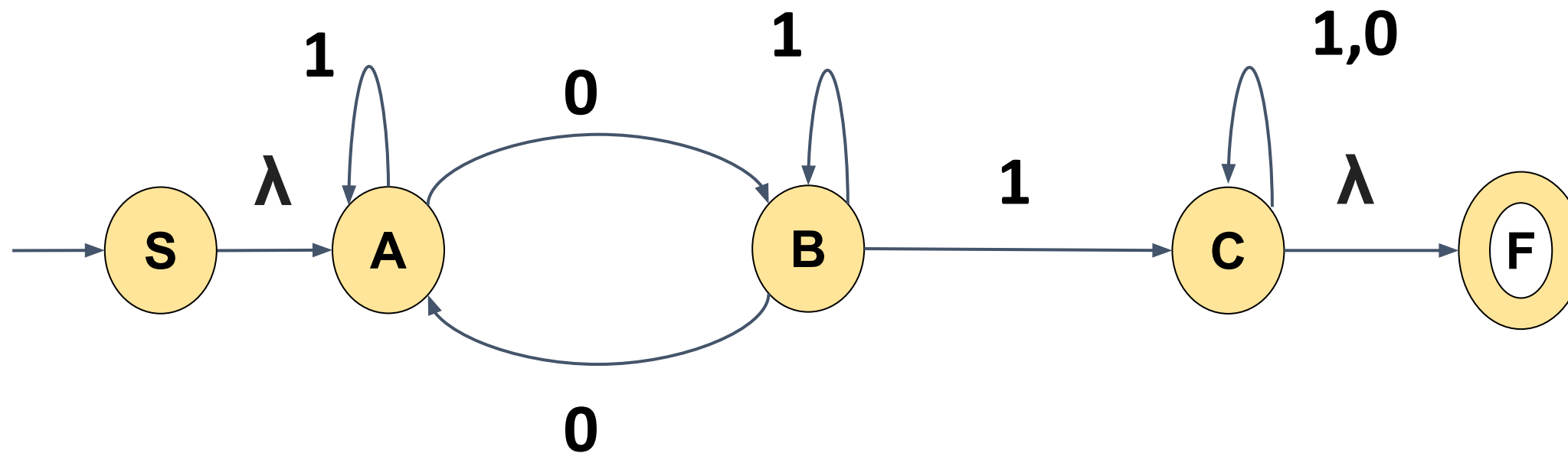


Example 7 :



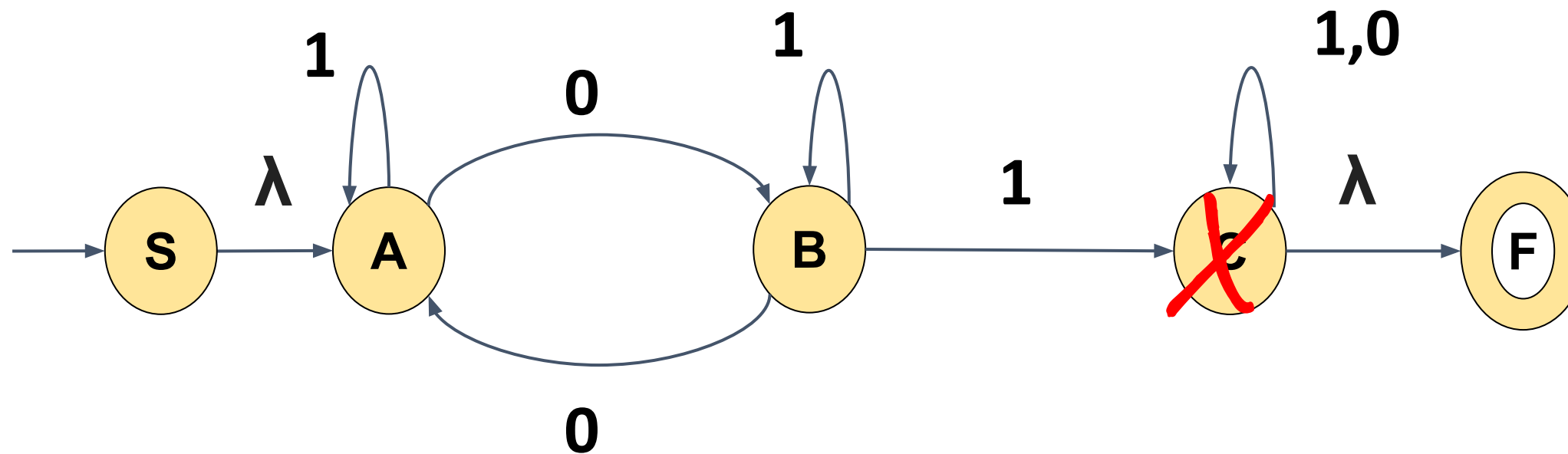
A new start state (S) is introduced as there is an incoming edge to the existing start state

Example 7 :



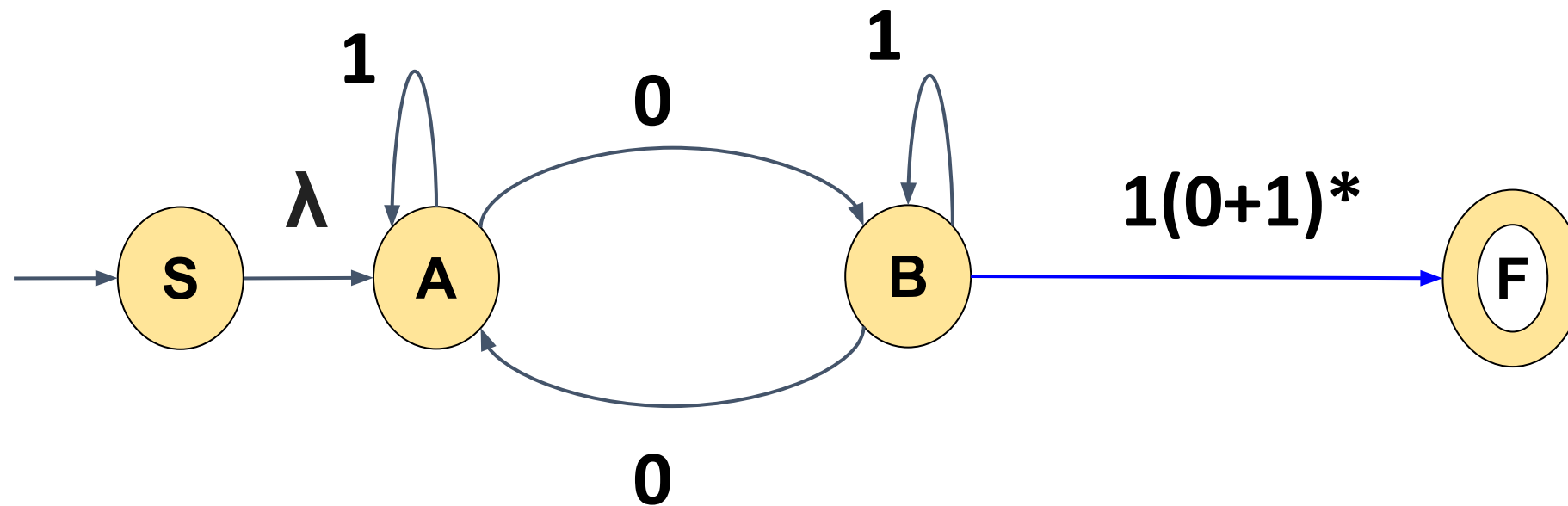
A new final state (F) is introduced as there is an outgoing edge from the existing final state

Example 7 :



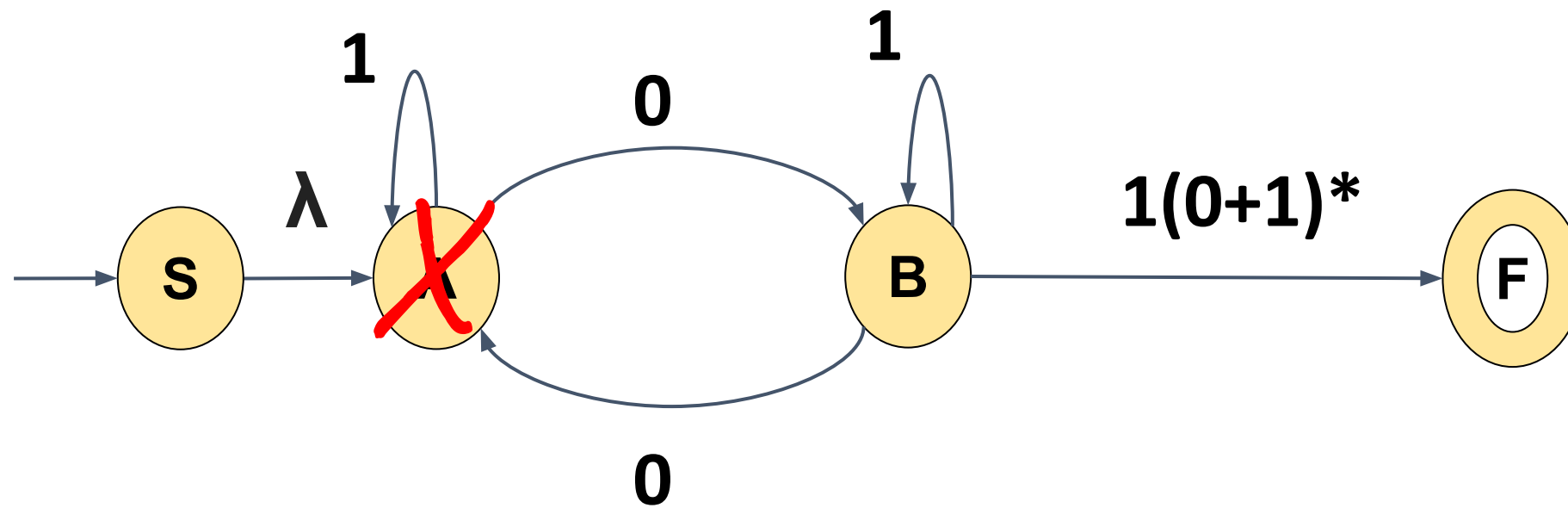
1. Eliminate C

Example 7 :



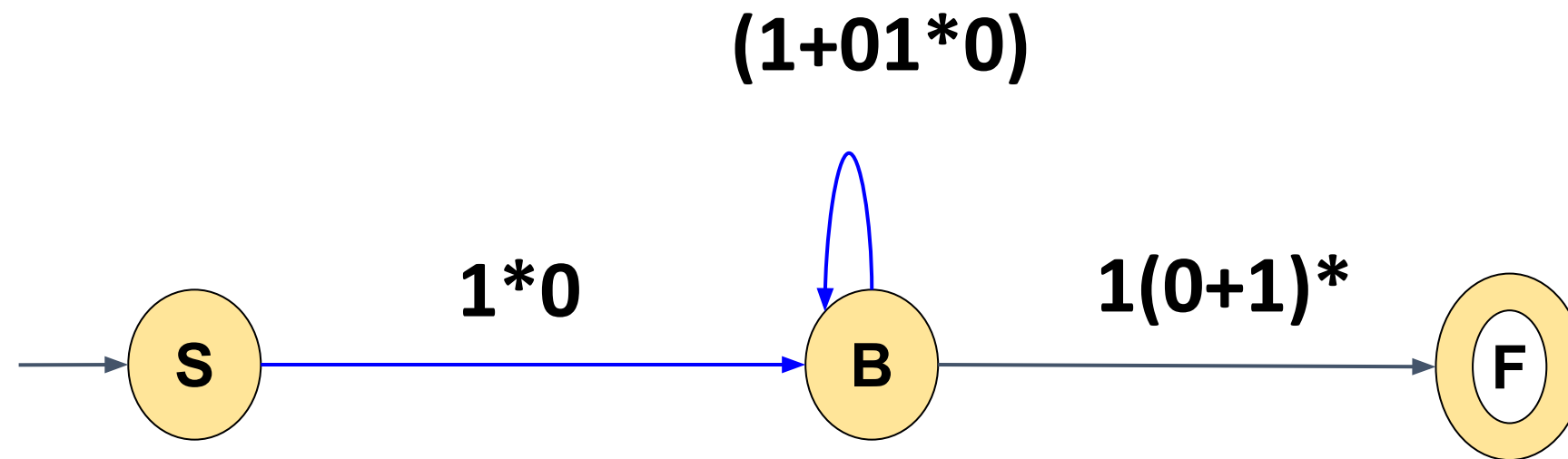
1. Eliminate C

Example 7 :



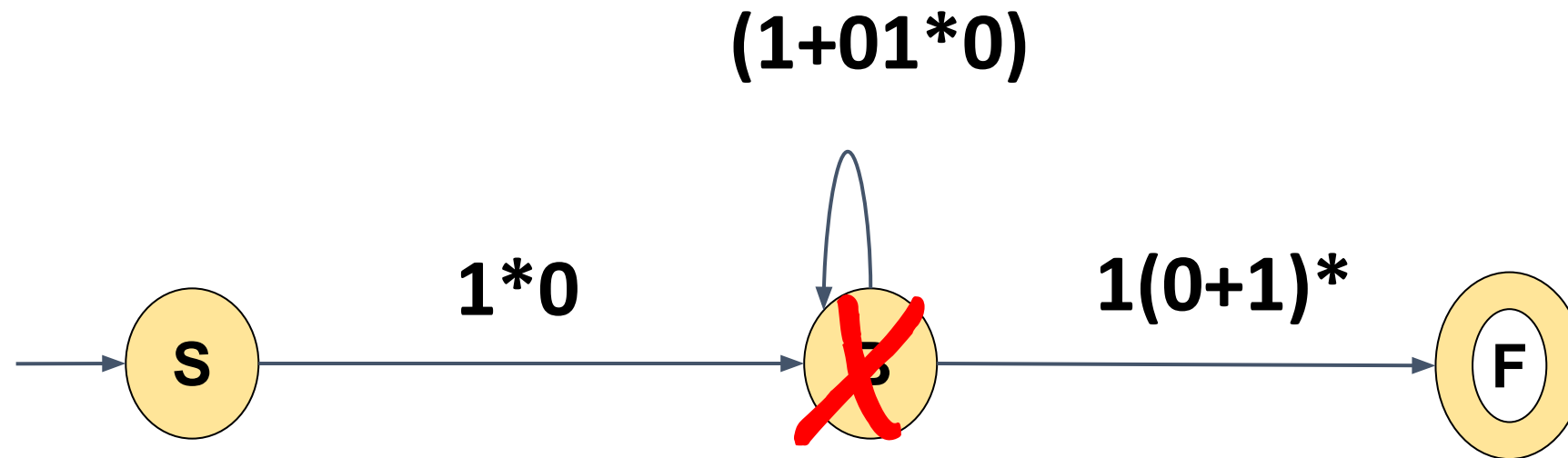
1. Eliminate C
2. Eliminate A

Example 7 :



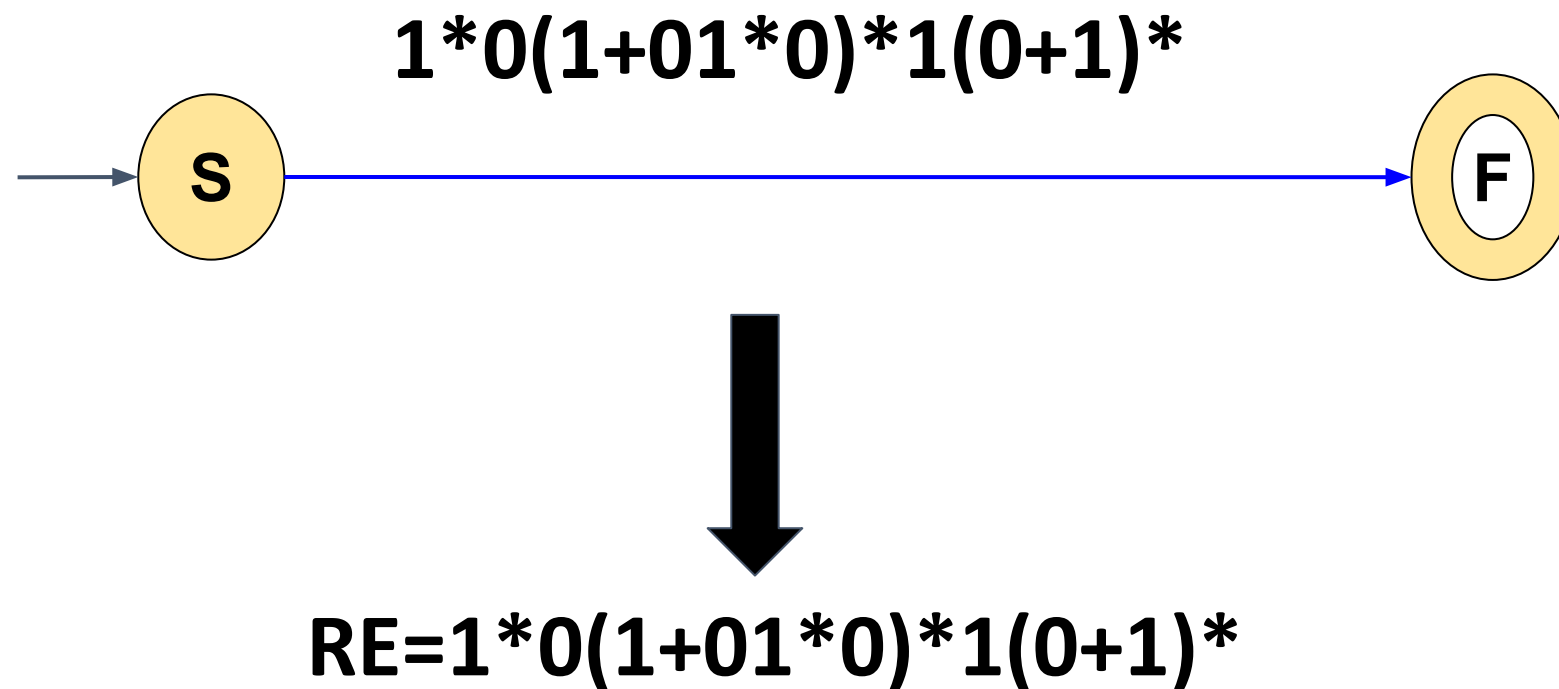
1. Eliminate C
2. Eliminate A

Example 7 :

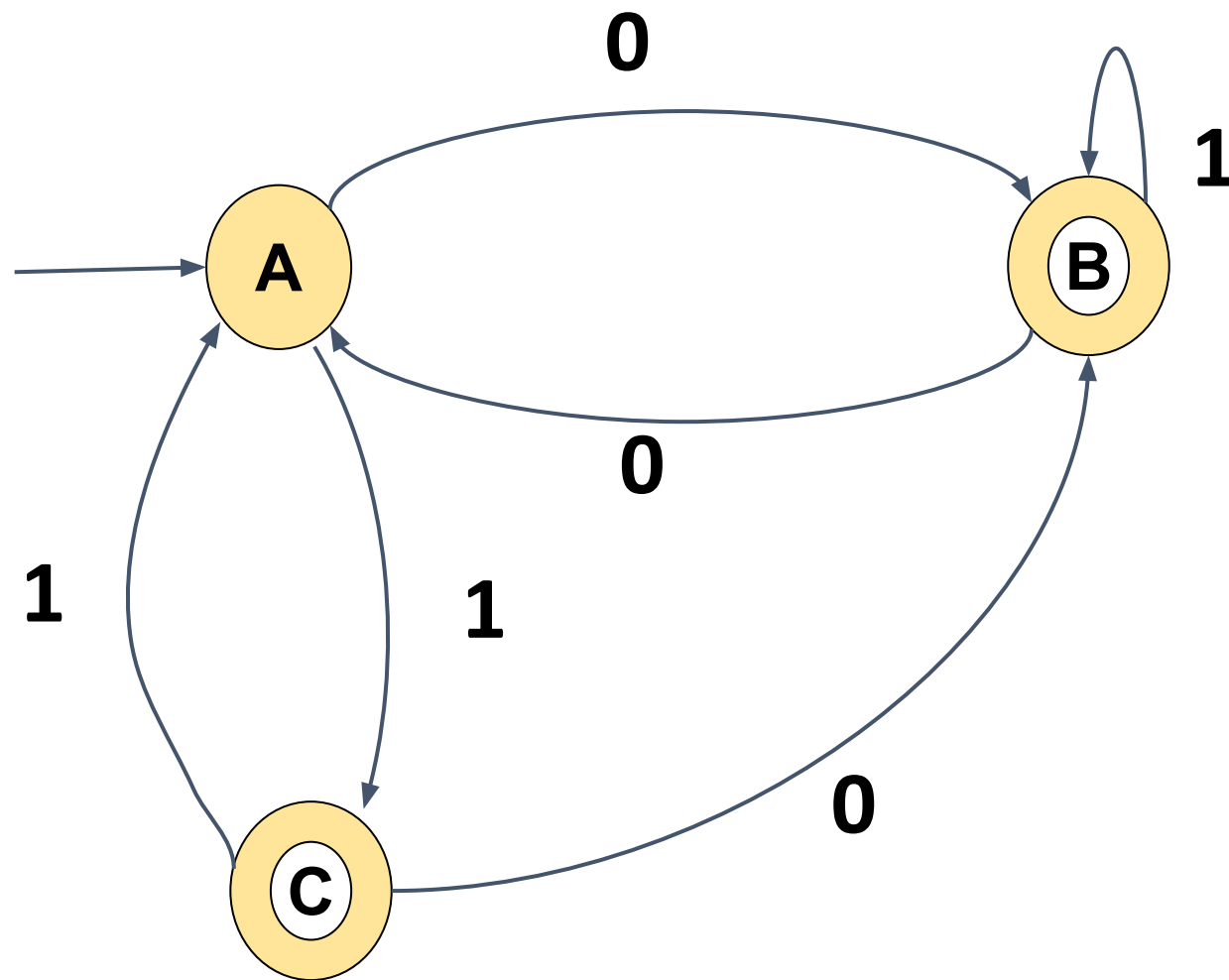


1. Eliminate C
2. Eliminate A
3. Eliminate B

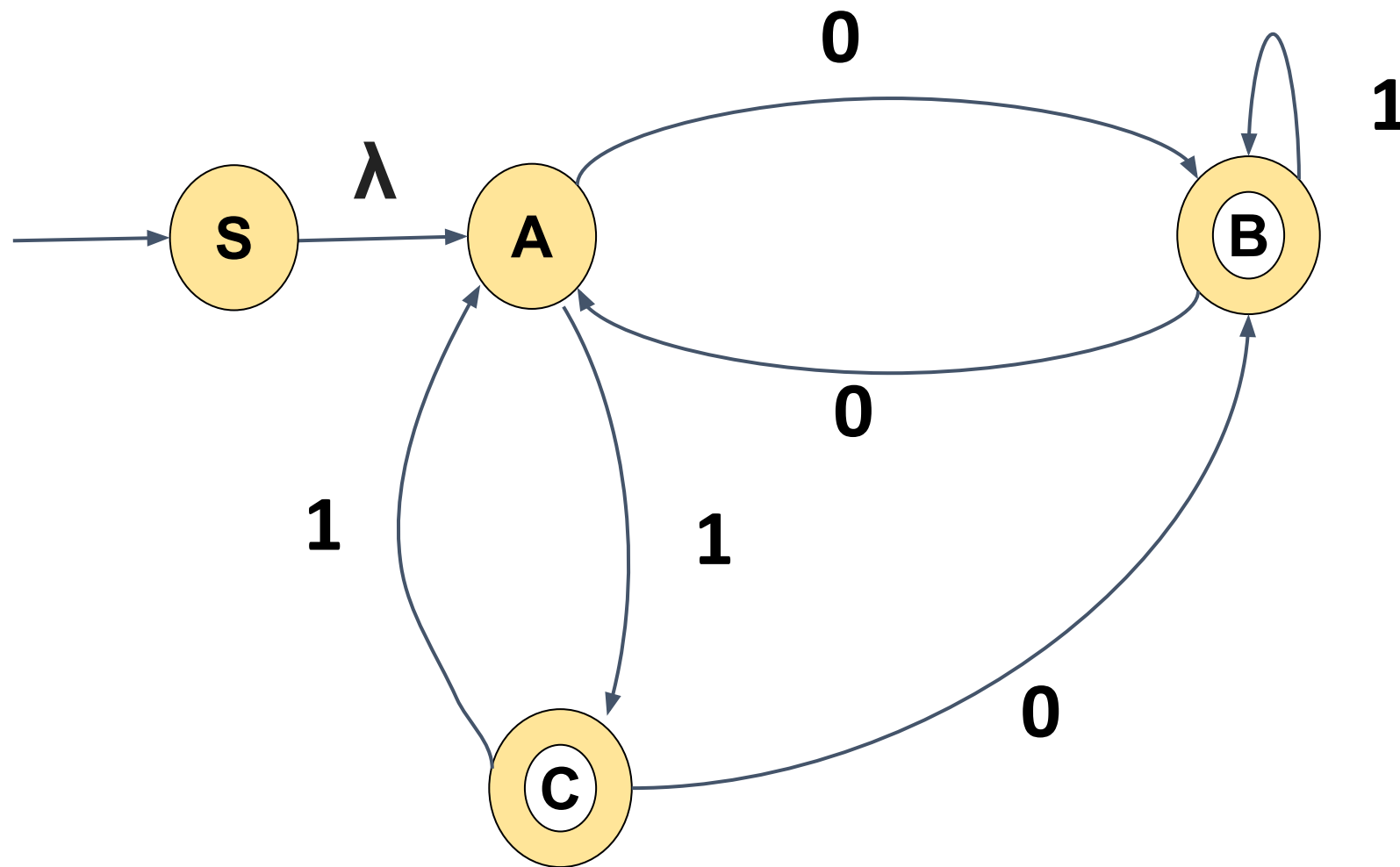
Example 7 :



Example 8 :

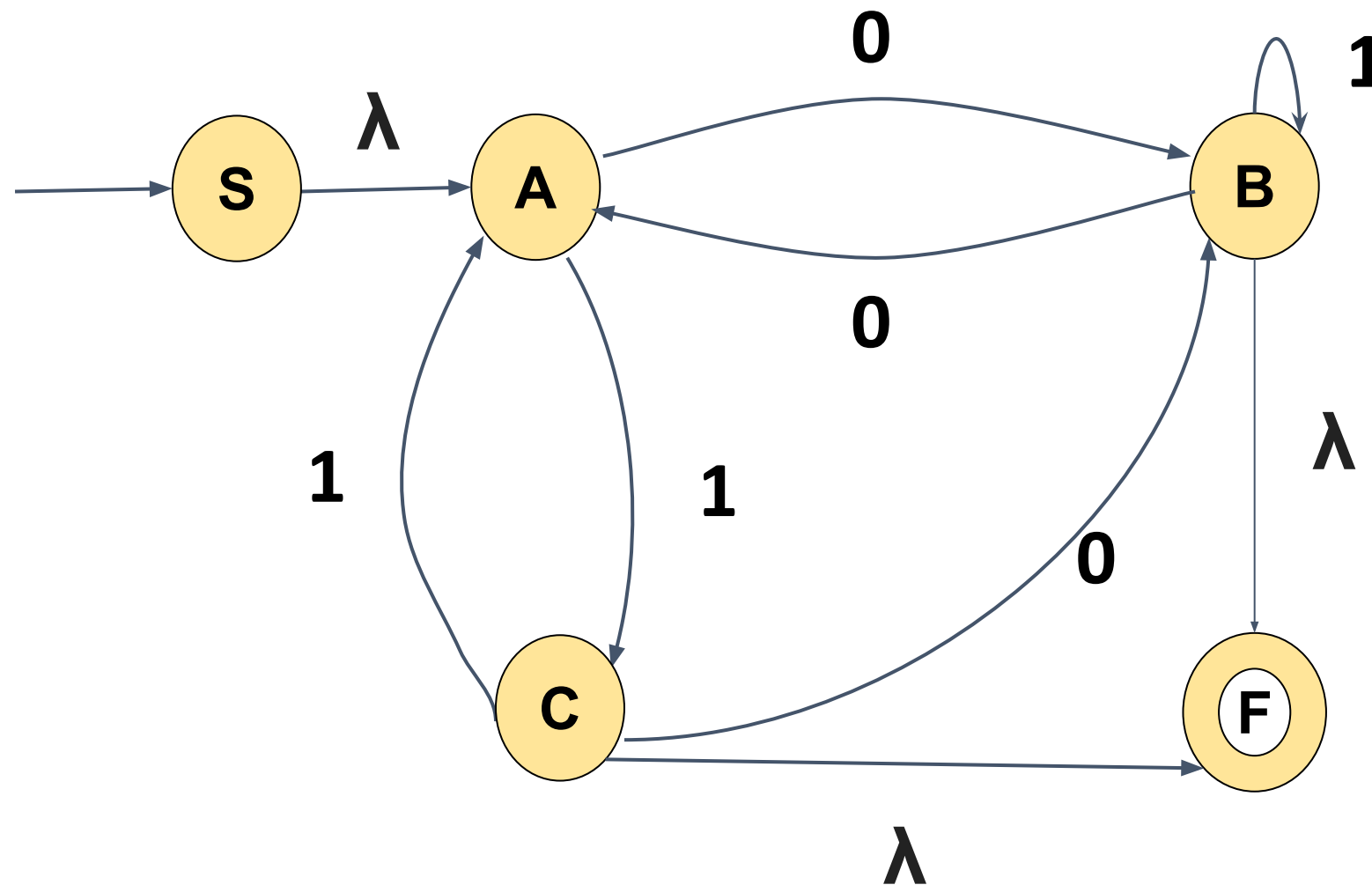


Example 8 :



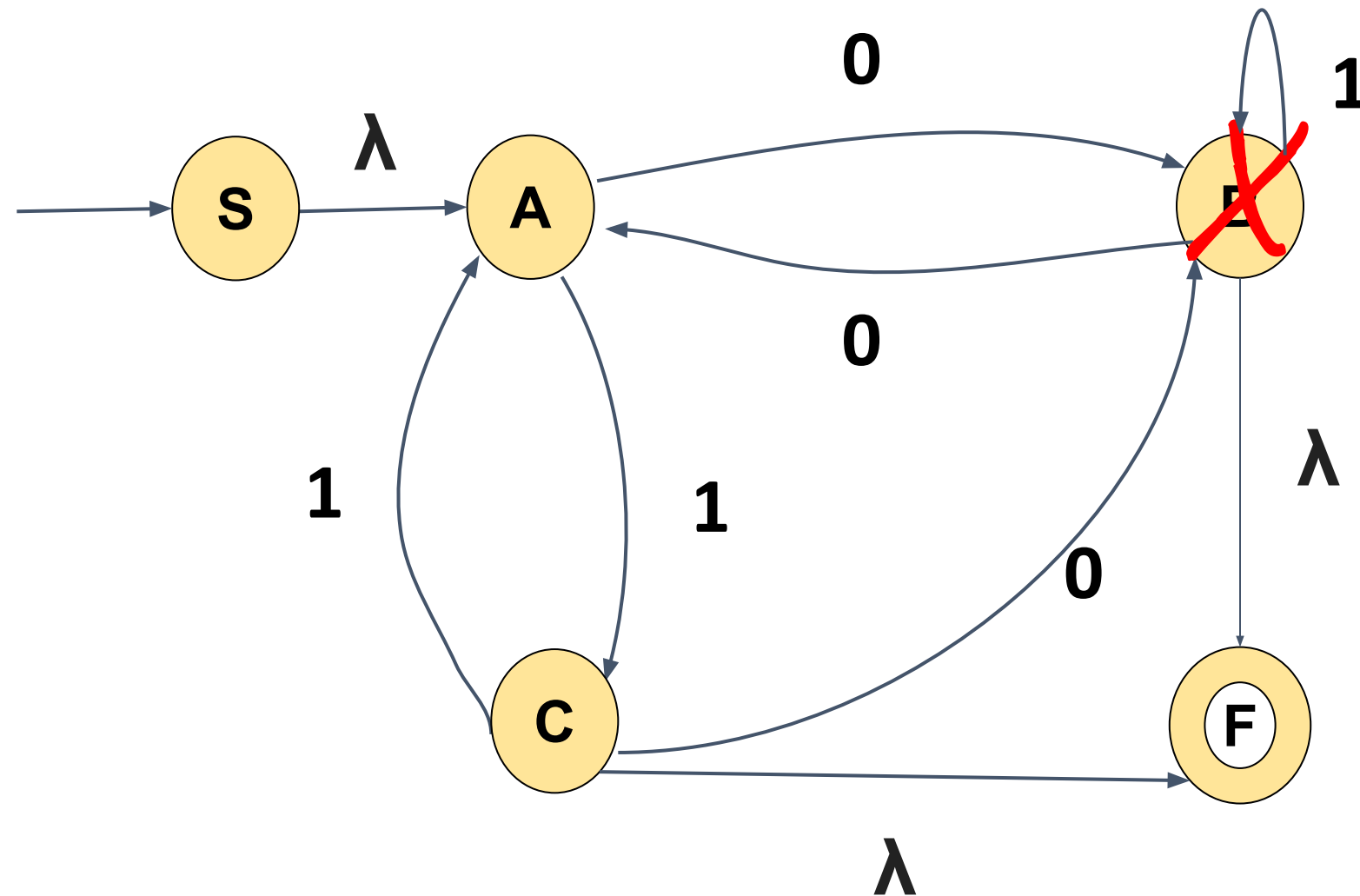
A new start state (S) is introduced as there is an incoming edge to the existing start state

Example 8 :



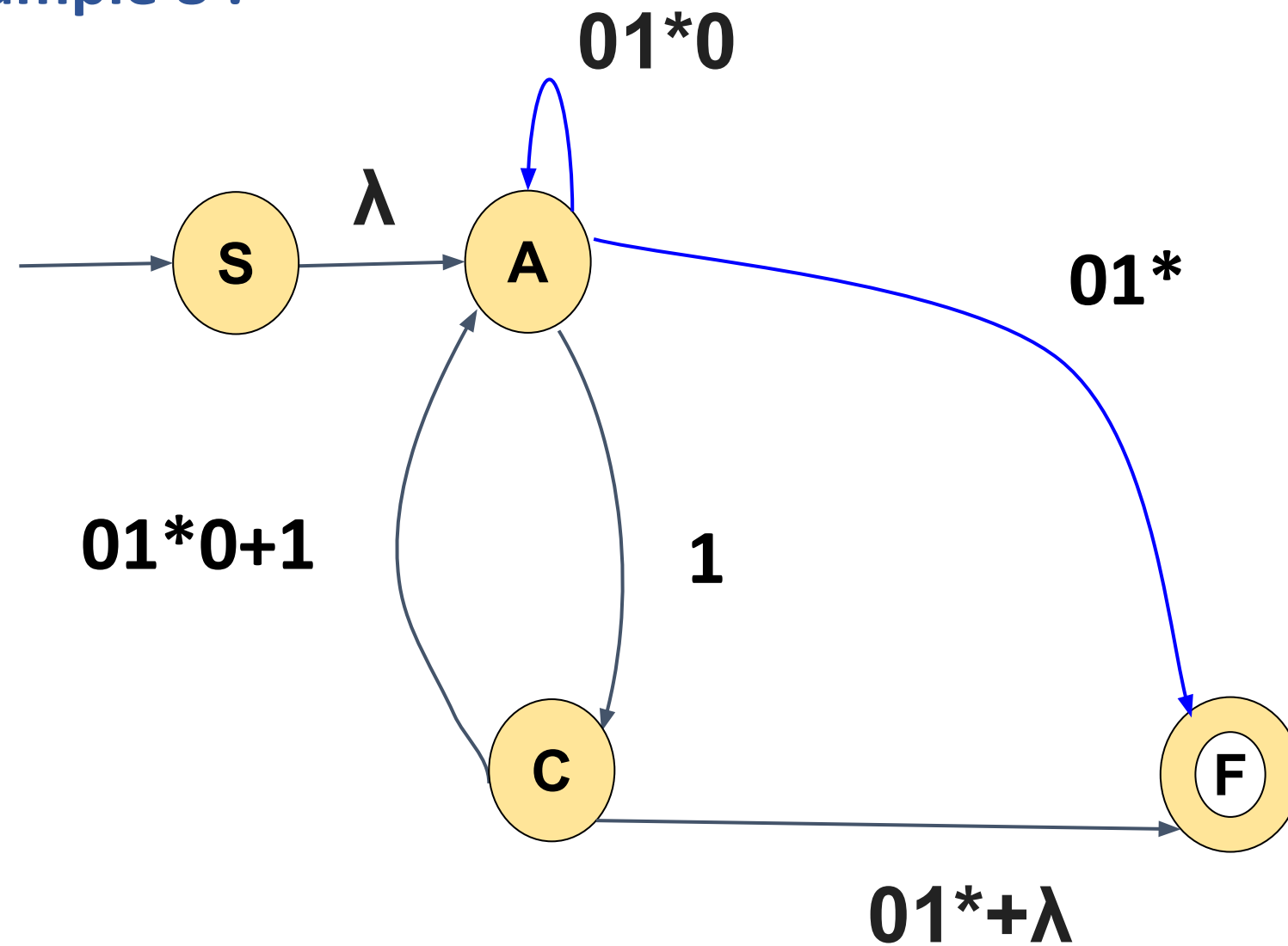
A new final state (F) is introduced as there is an outgoing edge from the existing final state

Example 8 :



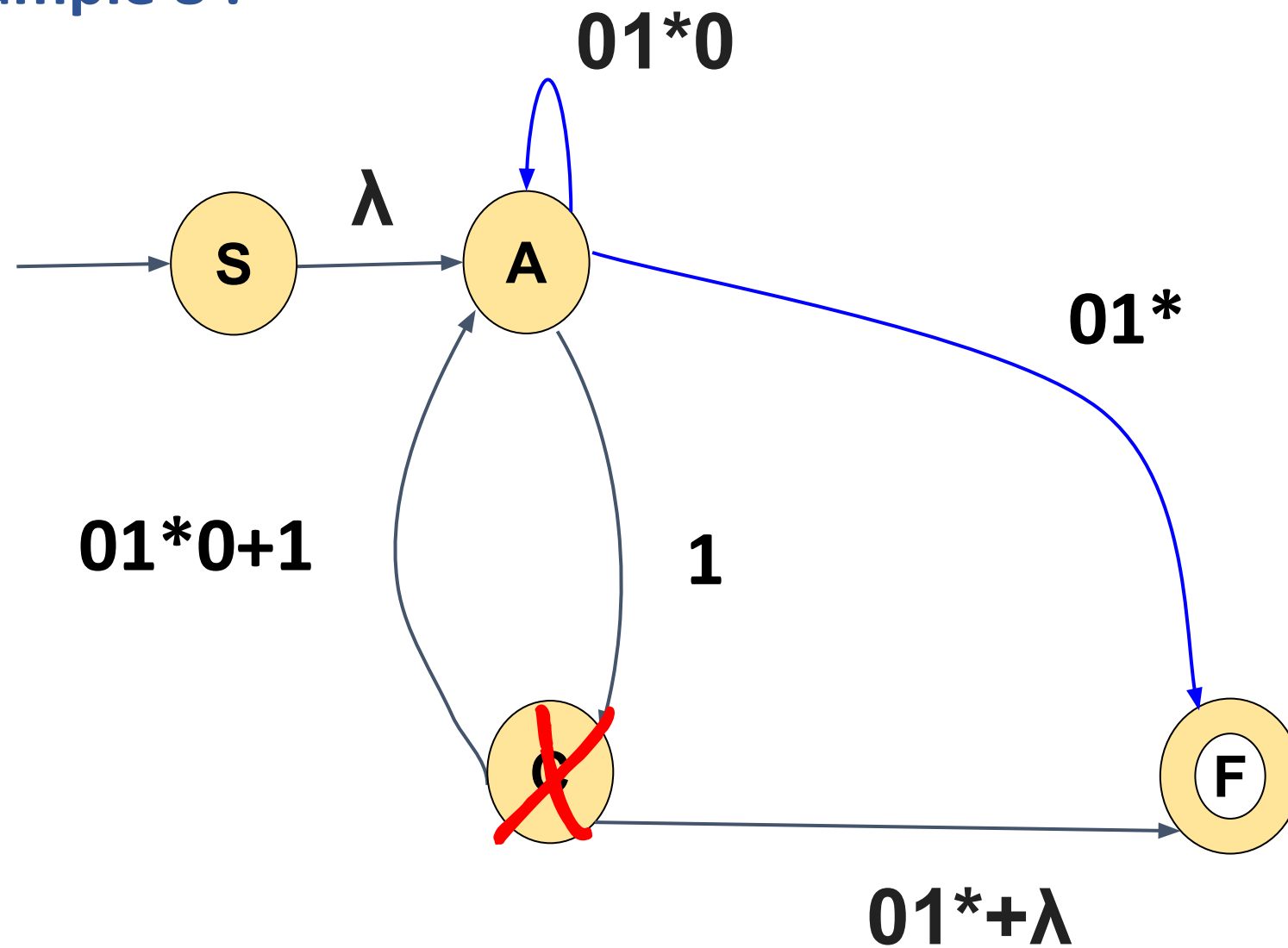
1. Eliminate B

Example 8 :



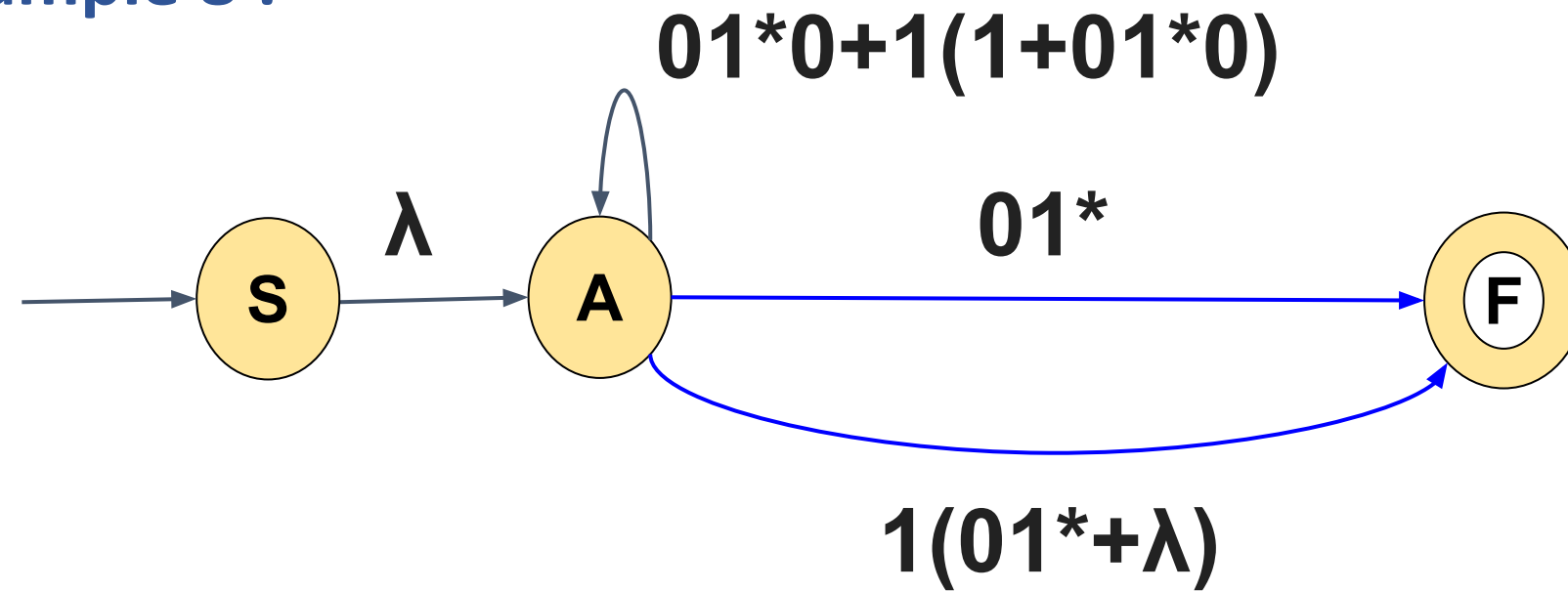
1. Eliminate B

Example 8 :



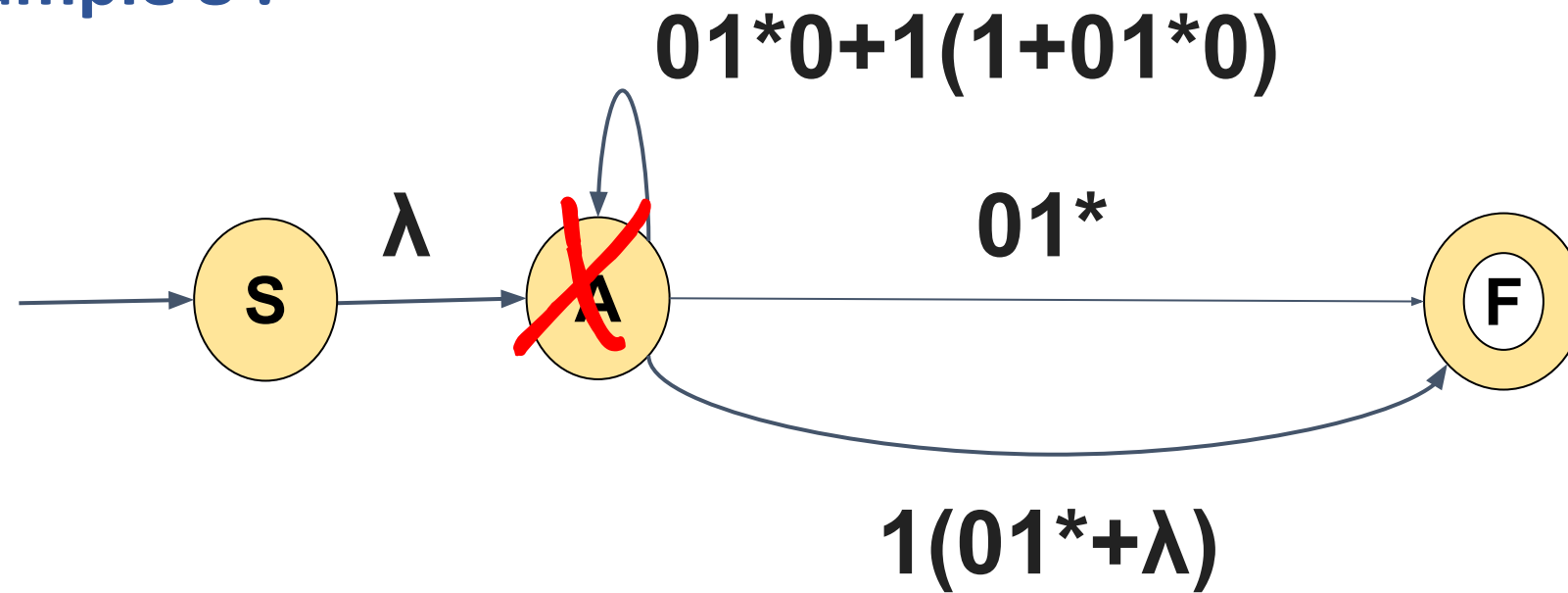
1. Eliminate B
2. Eliminate C

Example 8 :



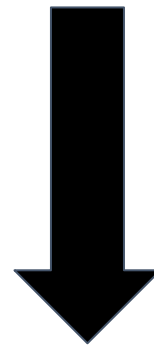
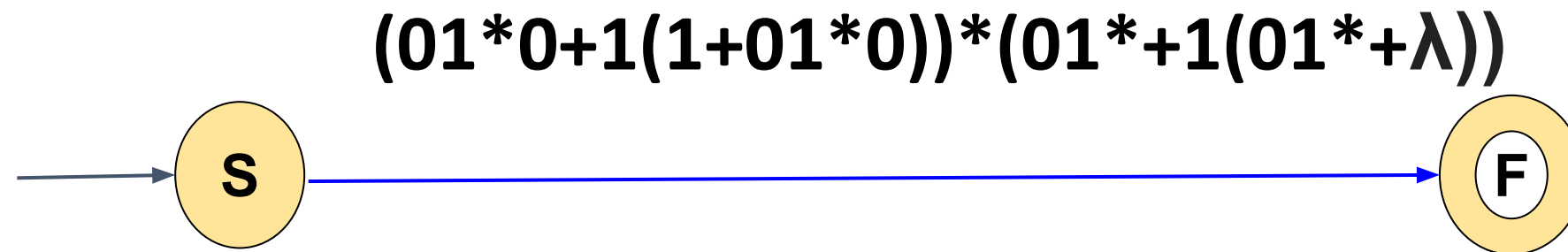
1. Eliminate B
2. Eliminate C

Example 8 :



1. Eliminate B
2. Eliminate C
3. Eliminate A

Example 8 :



$RE = (01^*0 + 1(1 + 01^*0))^*(01^* + 1(01^* + \lambda))$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^*(01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^*(01^* + (101^* + 1))$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

$$= (01^*0(1 + \lambda))^* (11)^* (01^*(\lambda + 1) + 1)$$

Example 8 :

$$RE = (01^*0 + 1(1 + 01^*0))^* (01^* + 1(01^* + \lambda))$$

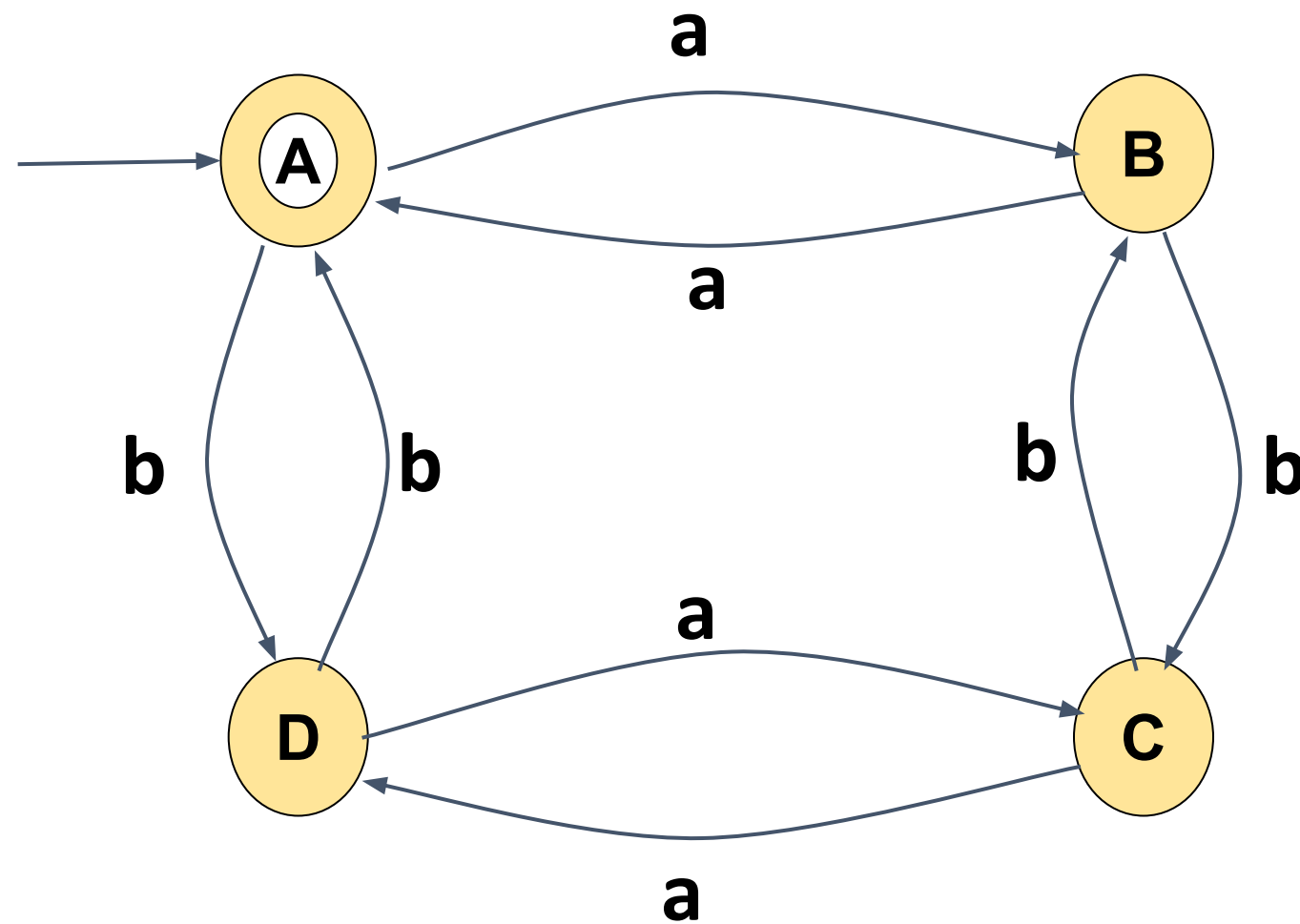
$$= (01^*0 + 11 + 101^*0)^* (01^* + (101^* + 1))$$

$$= (101^*0 + 01^*0 + 11)^* (01^*(\lambda + 1) + 1)$$

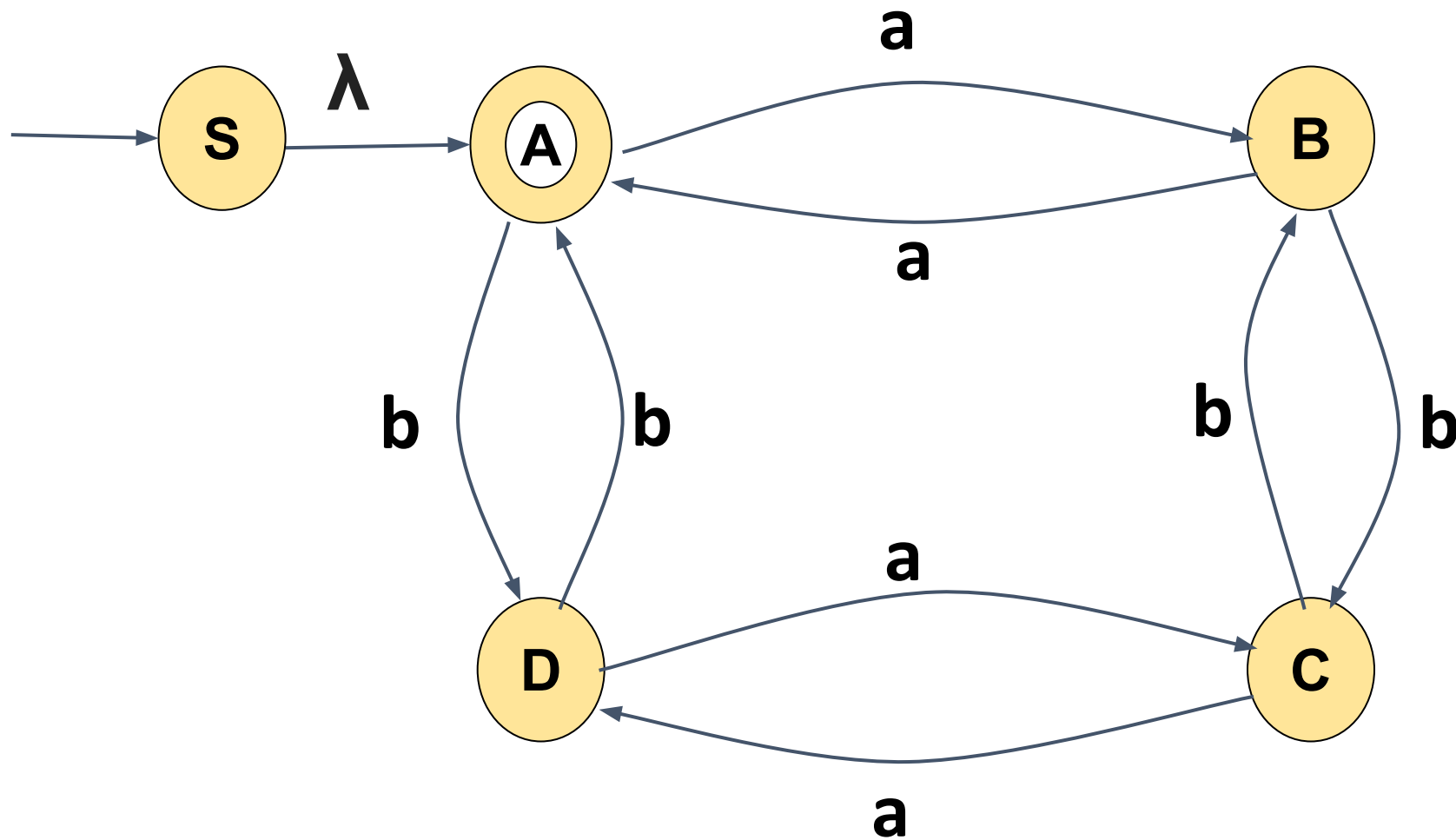
$$= (01^*0(1 + \lambda) + 11)^* (01^*(\lambda + 1) + 1)$$

$$RE = (01^*0(1 + \lambda))^* (11)^* (01^*(\lambda + 1) + 1)$$

Example 9 :order of elimination(B,D,C,A)

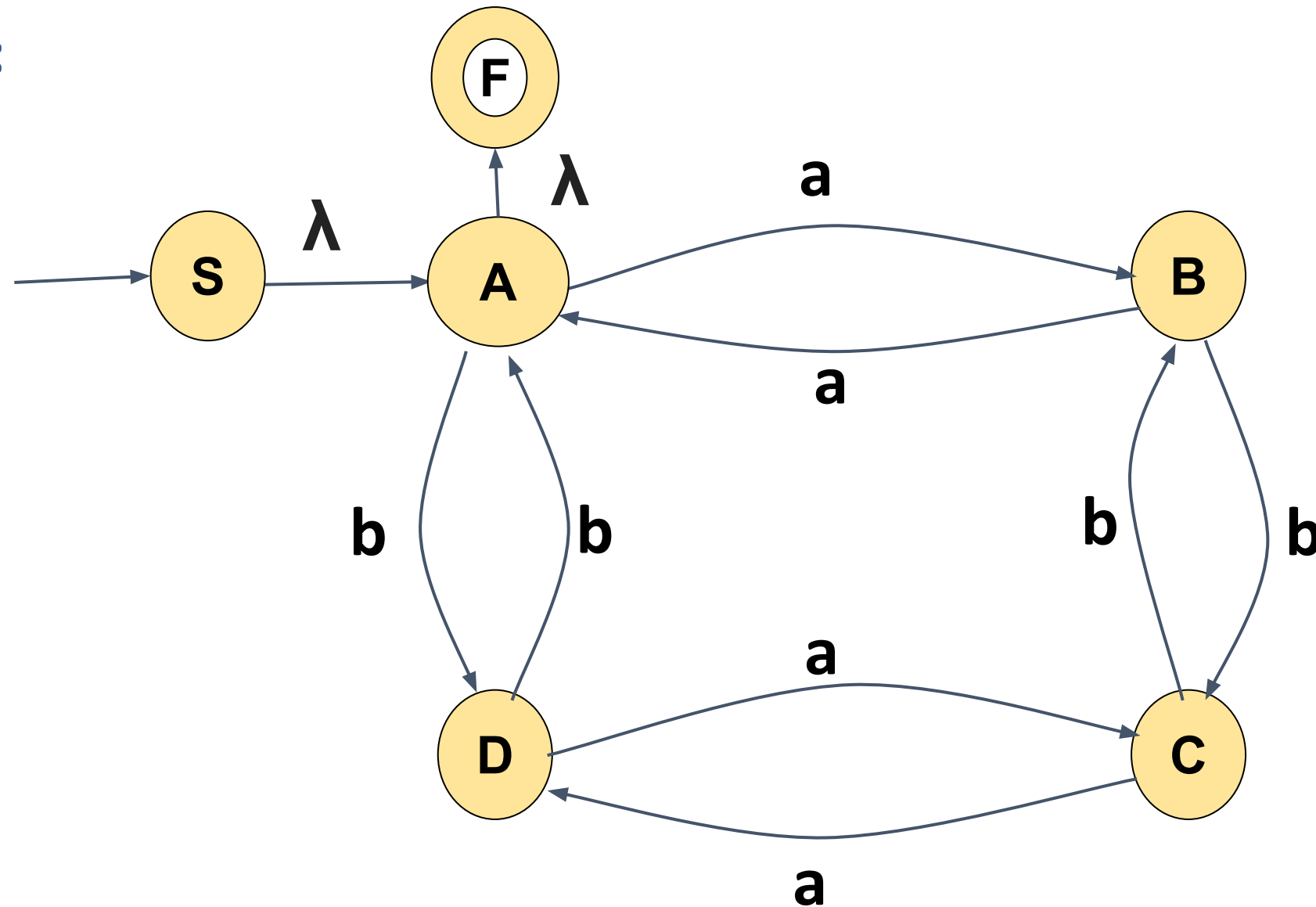


Example 9 :



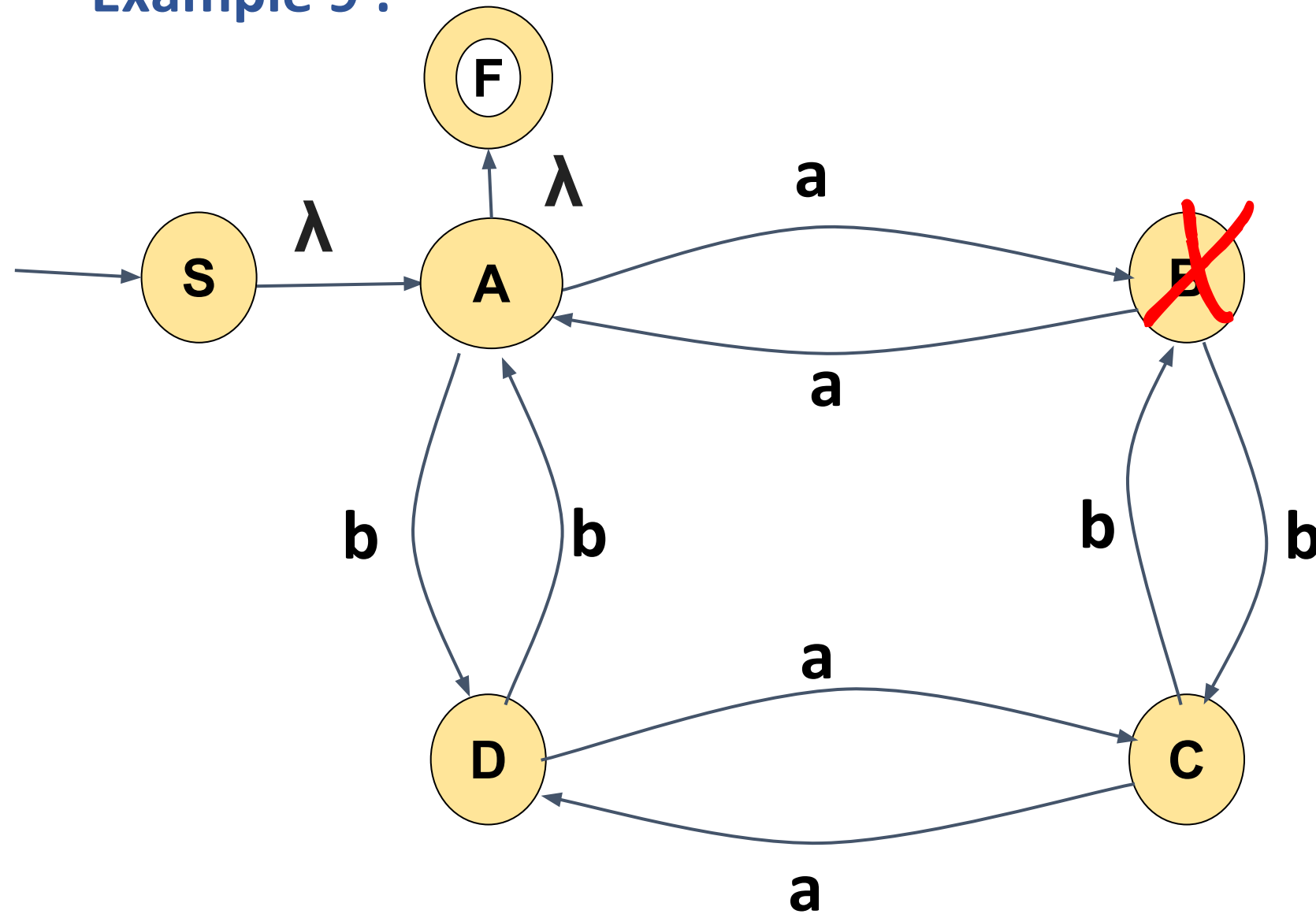
A new start state (S) is introduced as there is an incoming edge to the existing start state

Example 9 :



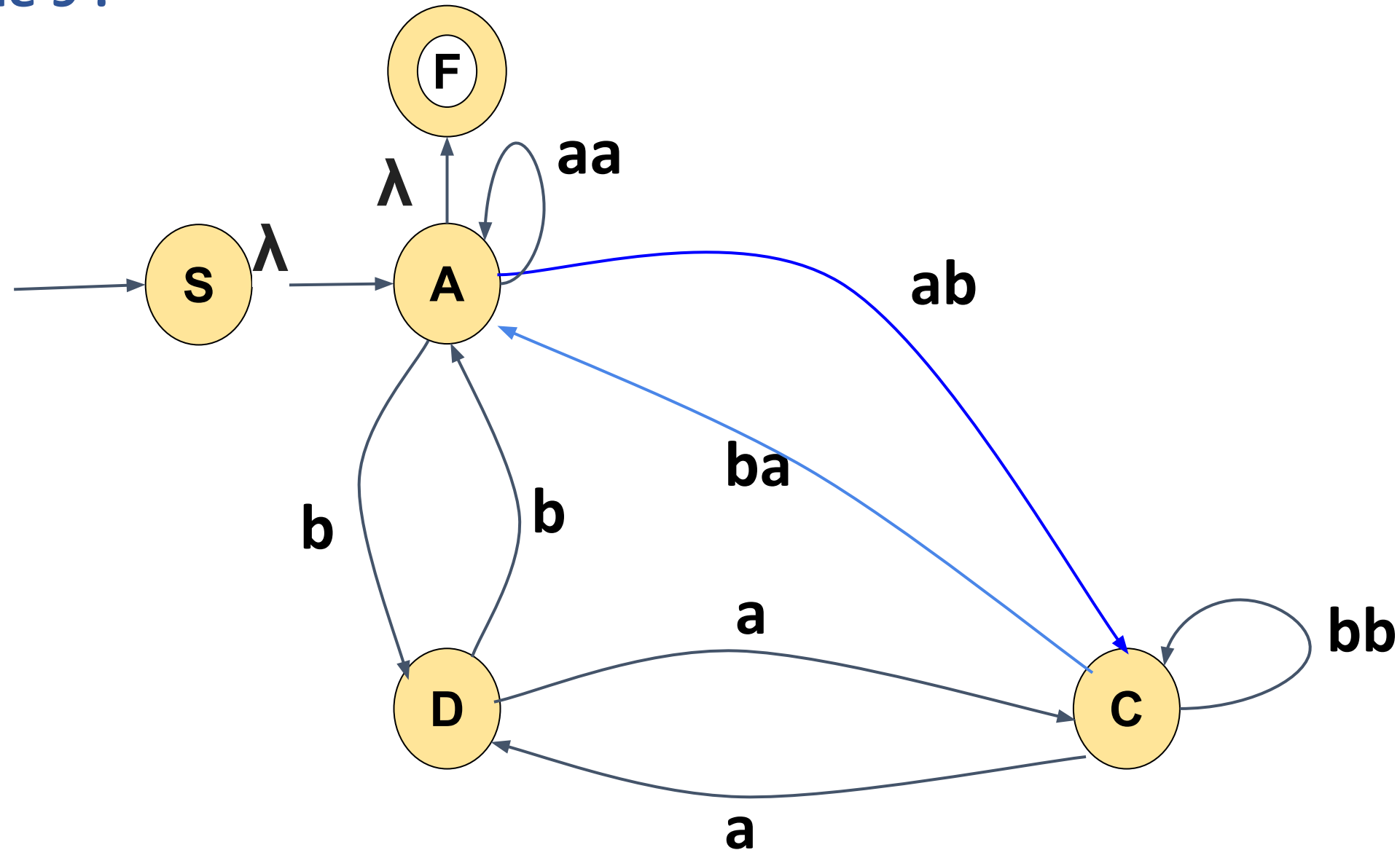
A new final state (F) is introduced as there is an outgoing edge from the existing final state

Example 9 :



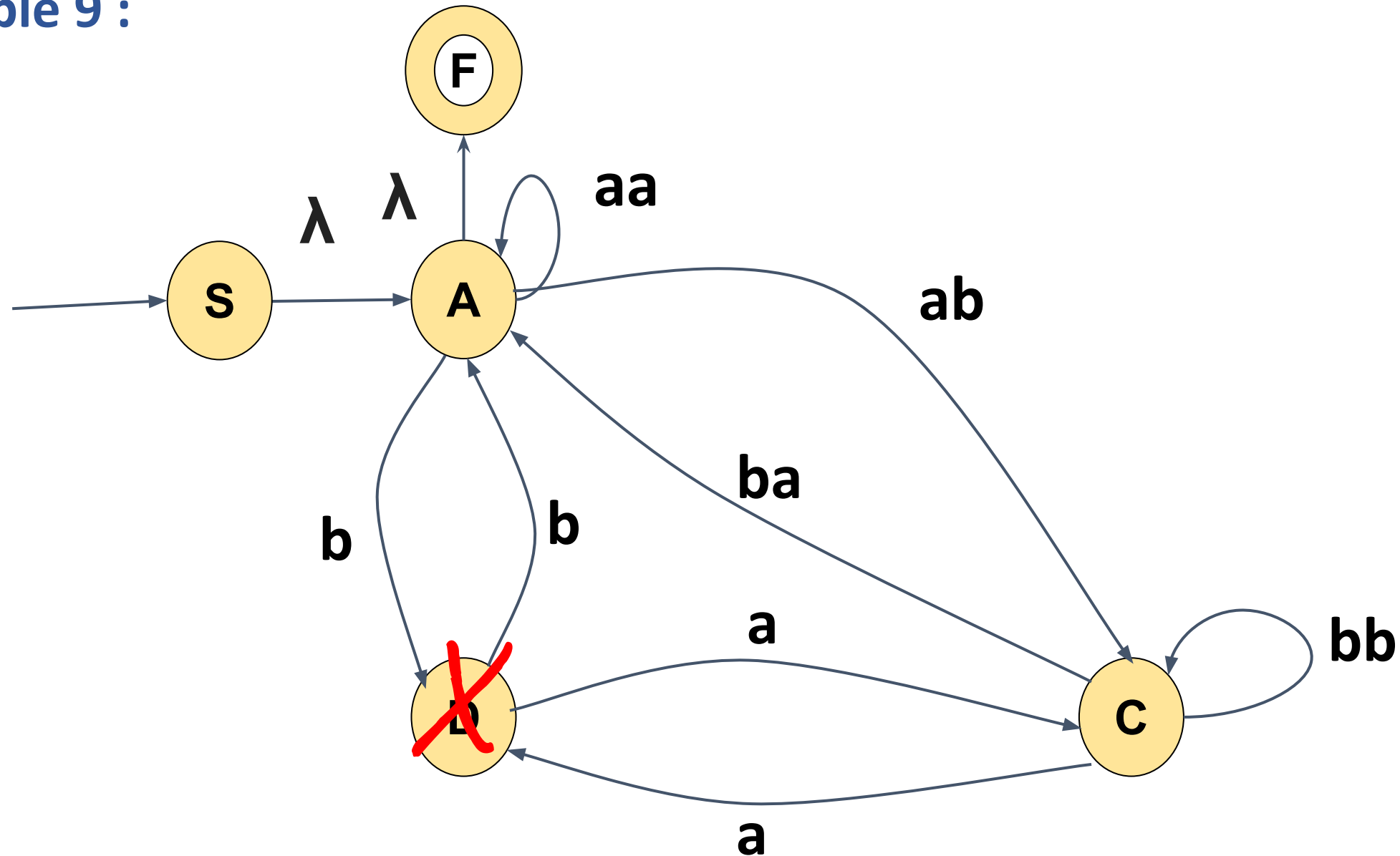
1. Eliminate B

Example 9 :



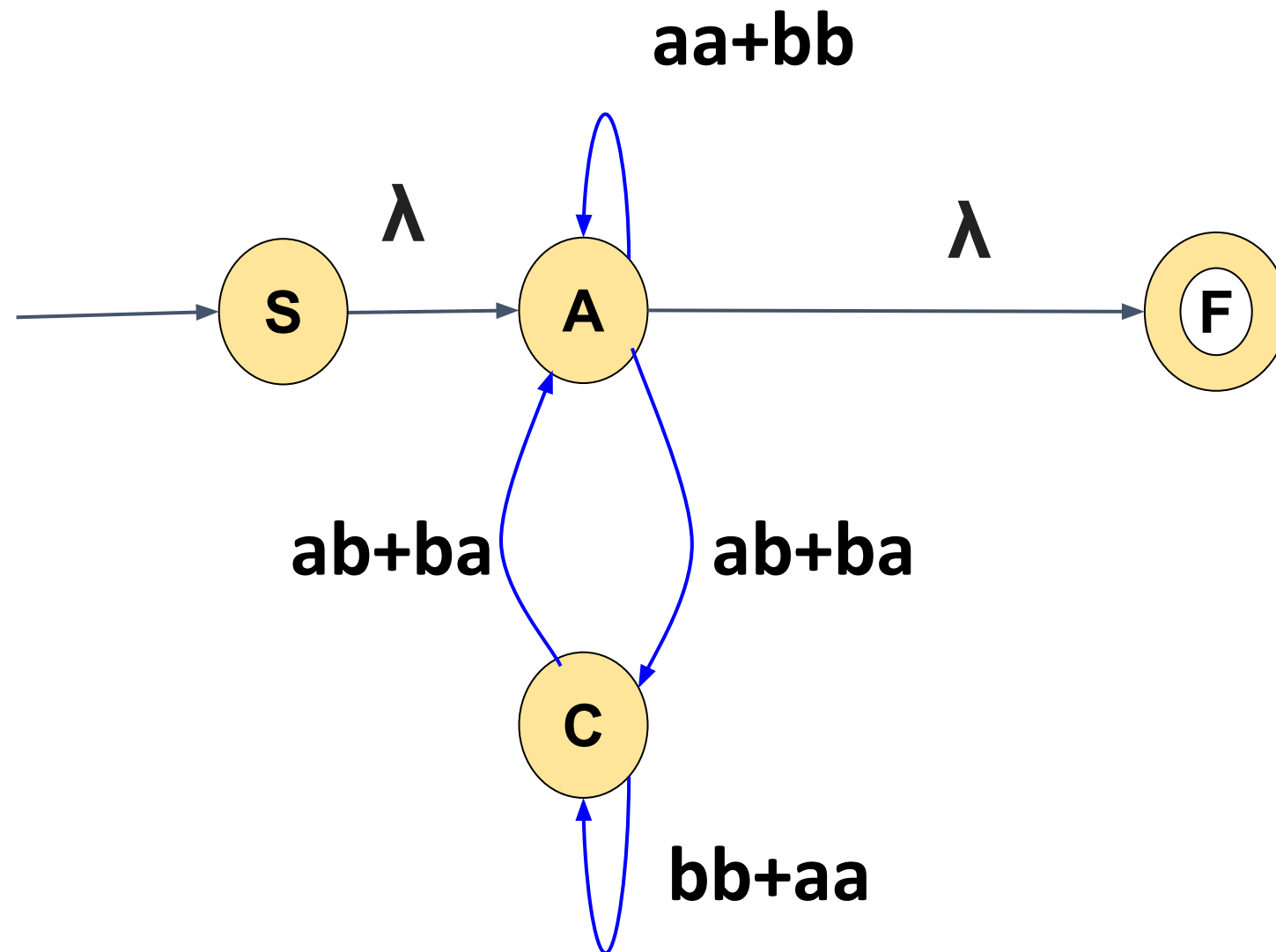
1. Eliminate B

Example 9 :



1. Eliminate B
2. Eliminate D

Example 9 :

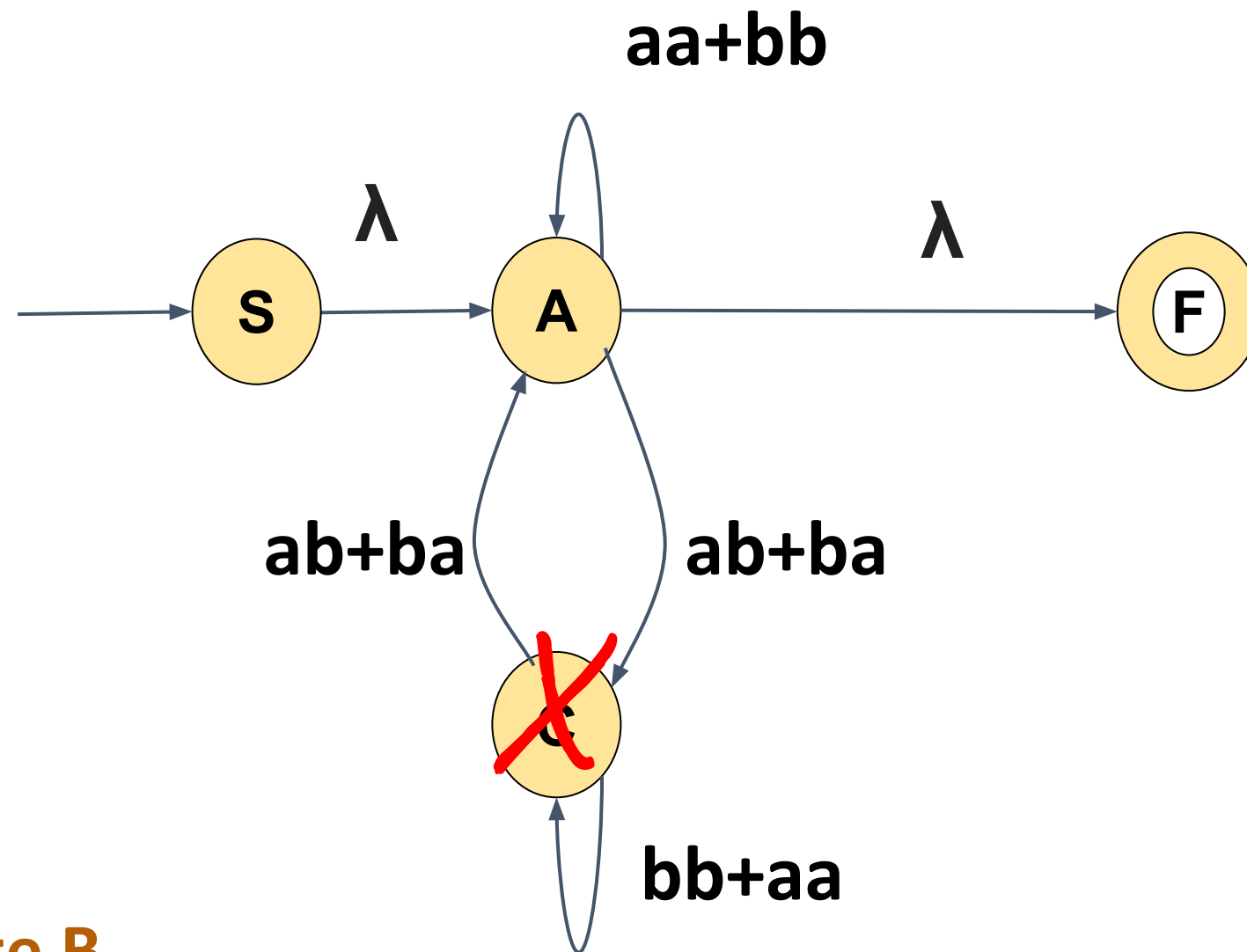


1. Eliminate B
2. Eliminate D

Automata Formal Languages and Logic

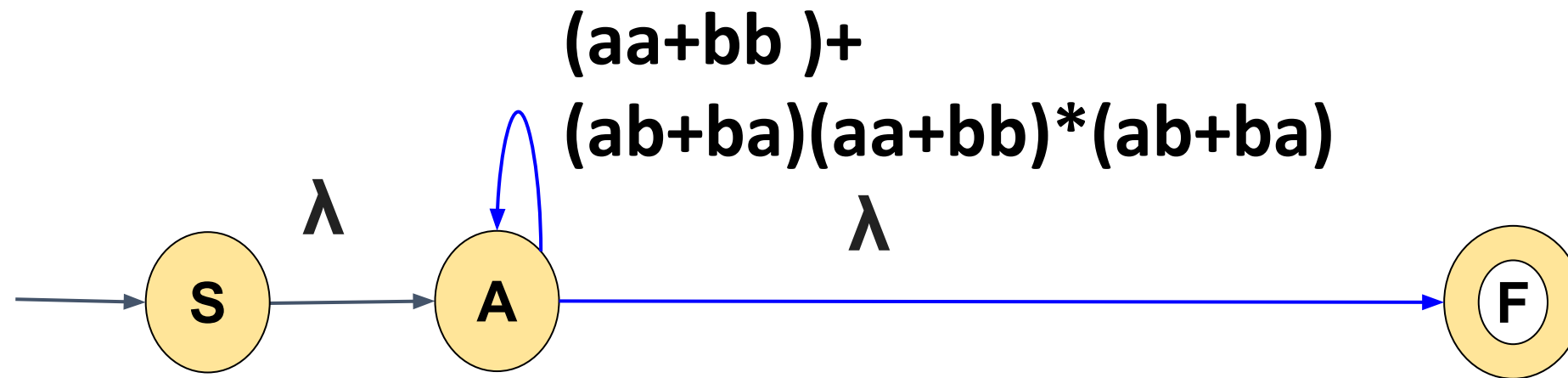
Unit 2 - Finite Automata to Regular Expression

Example 9 :



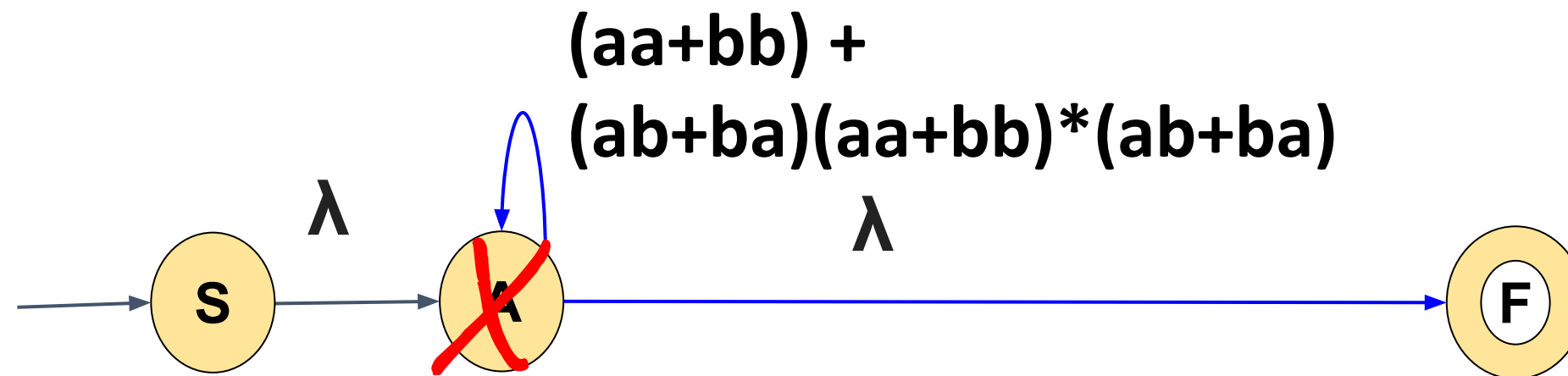
1. Eliminate B
2. Eliminate D
3. Eliminate C

Example 9 :



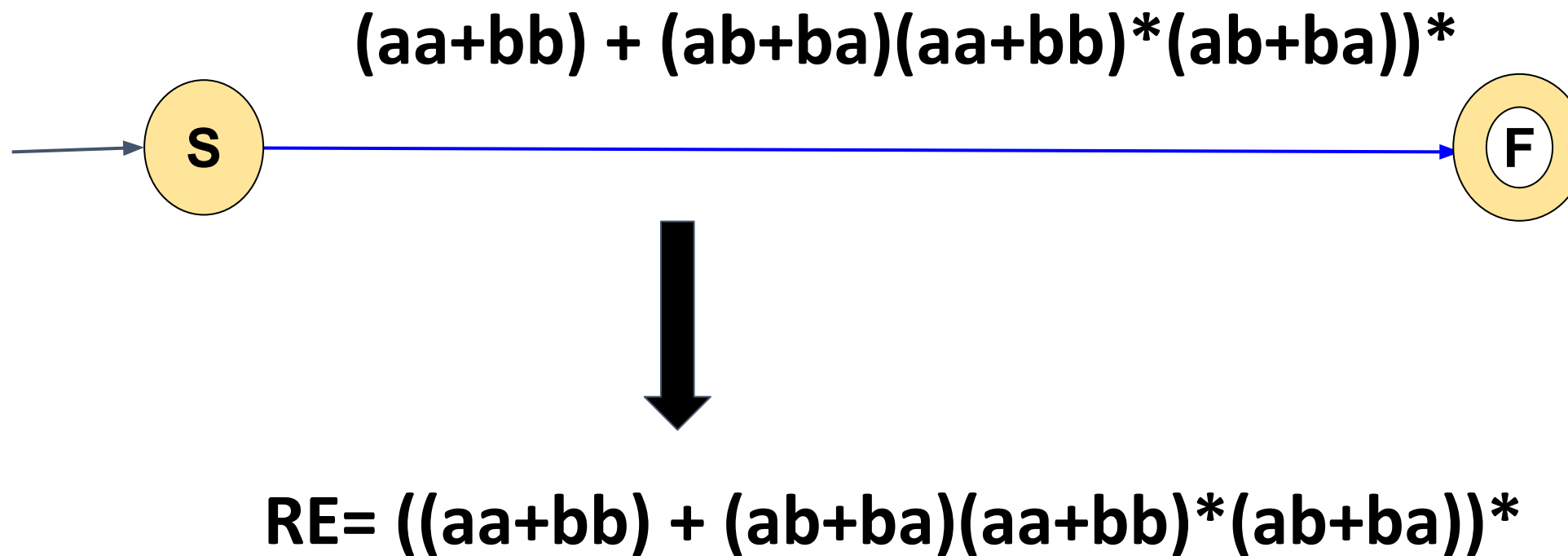
1. Eliminate B
2. Eliminate D
3. Eliminate C

Example 9 :



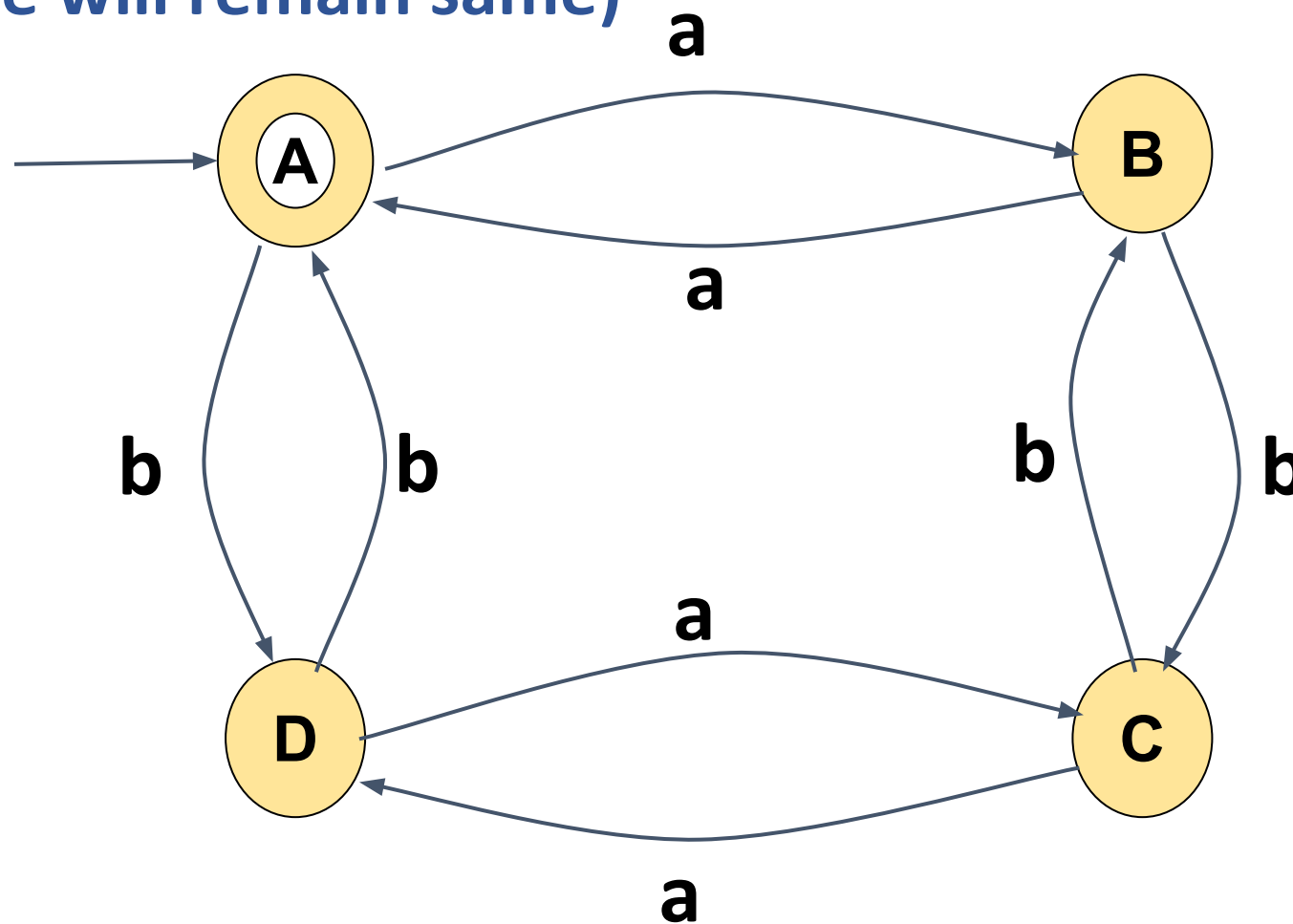
1. Eliminate B
2. Eliminate D
3. Eliminate C
4. Eliminate A

Example 9 :

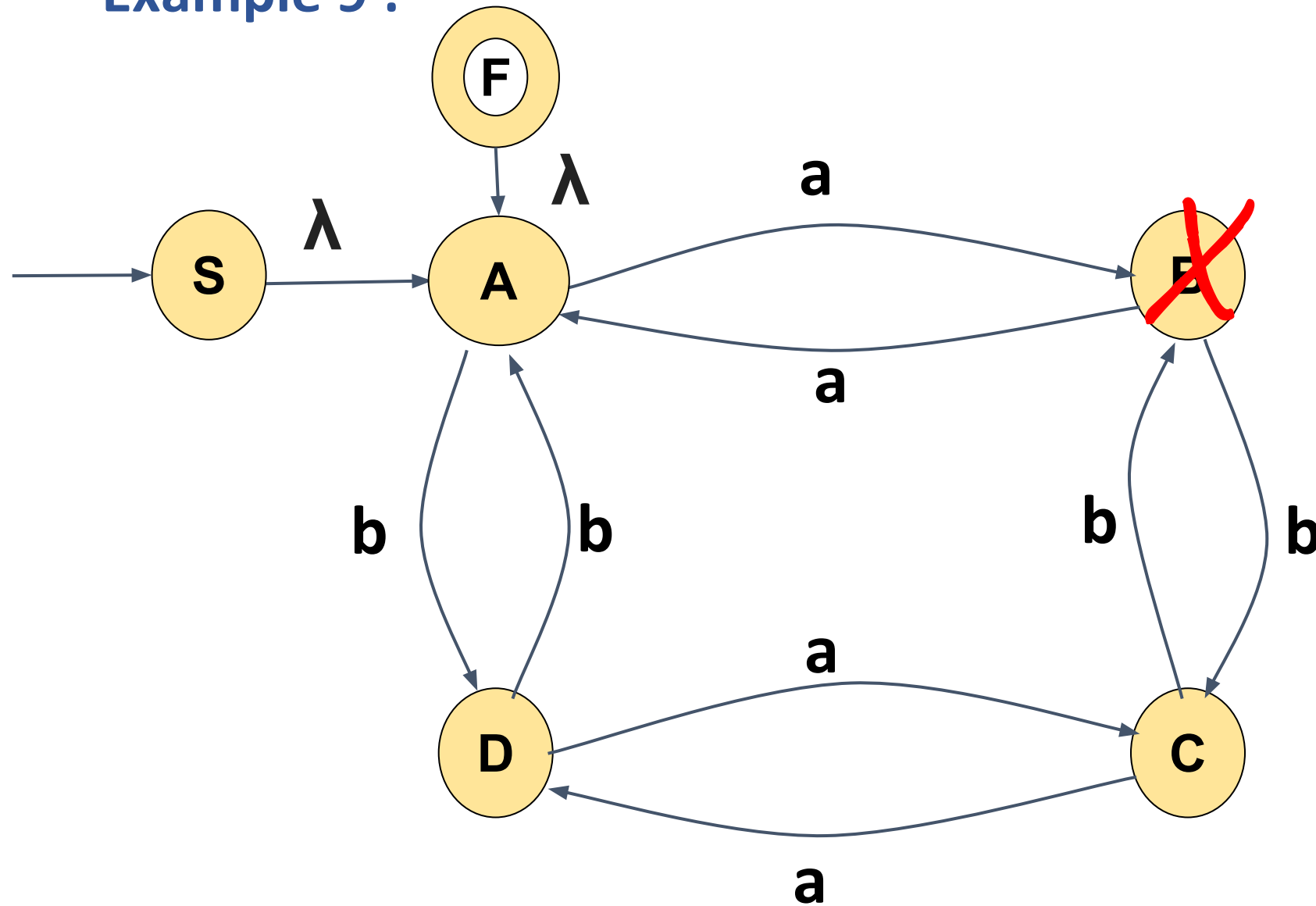


Example 9 :

Consider the same example: Order of elimination: B,C, D ,A (Adding new start state and new final state will remain same)



Example 9 :

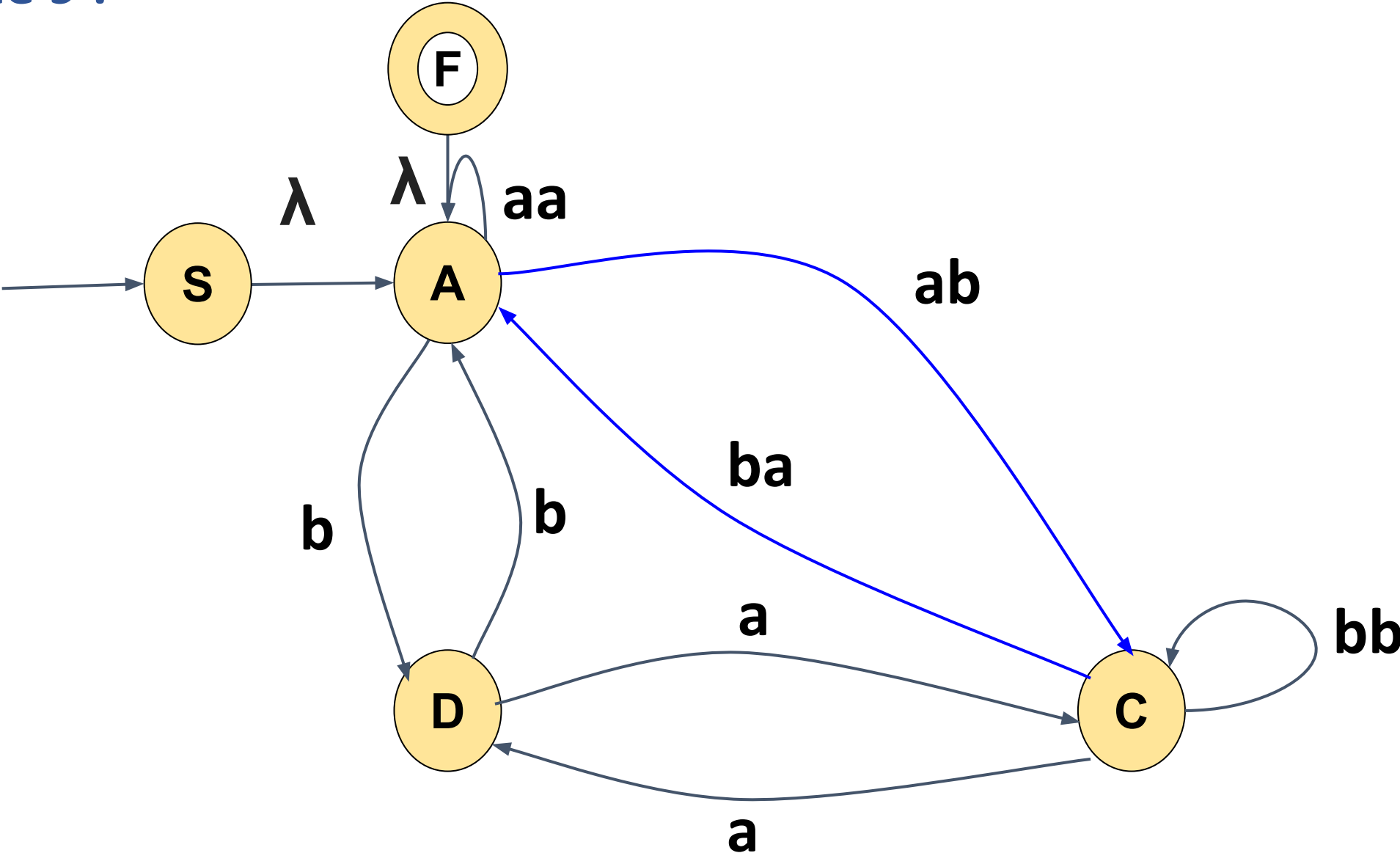


1. Eliminate B

Automata Formal Languages and Logic

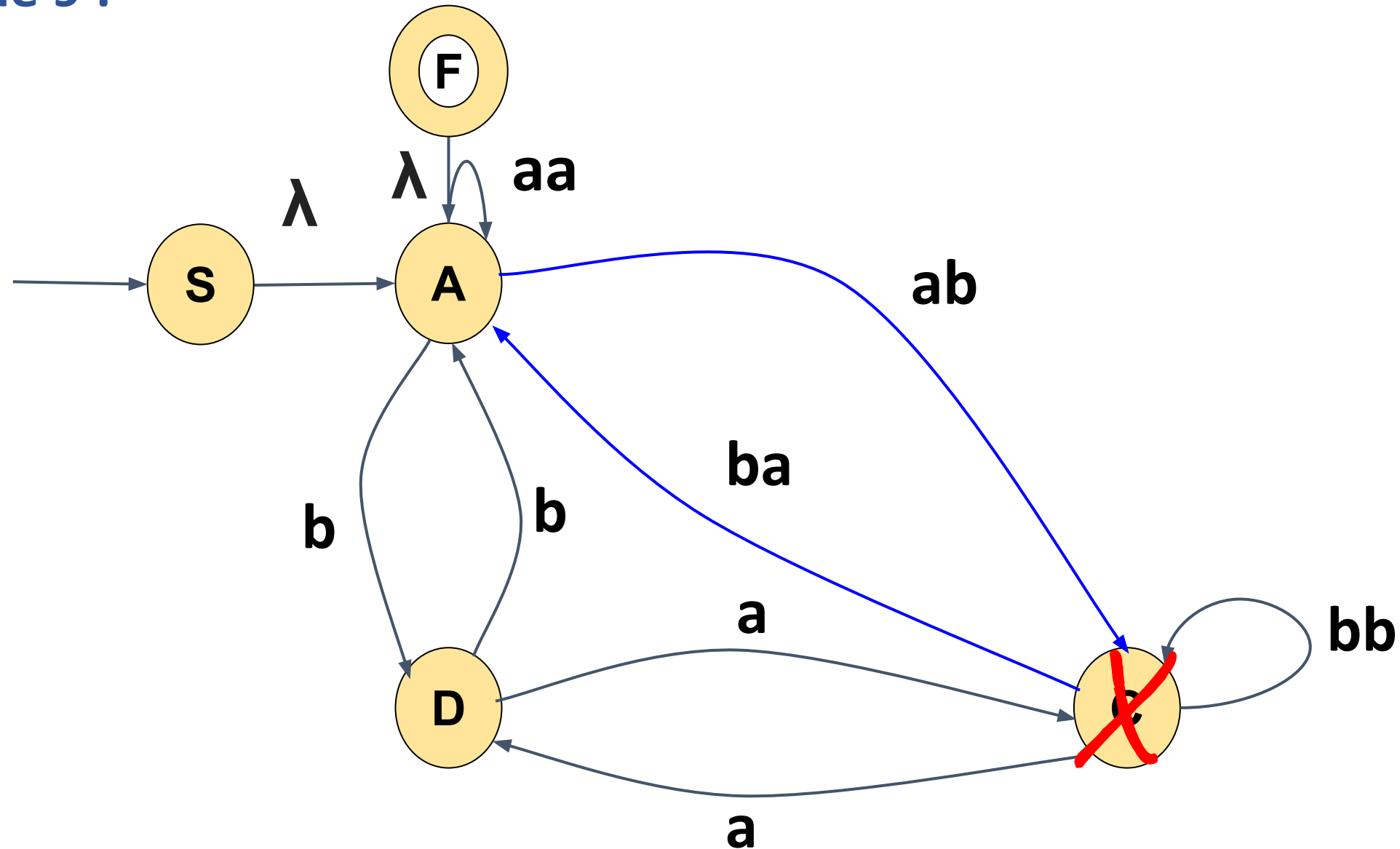
Unit 2 - Finite Automata to Regular Expression

Example 9 :



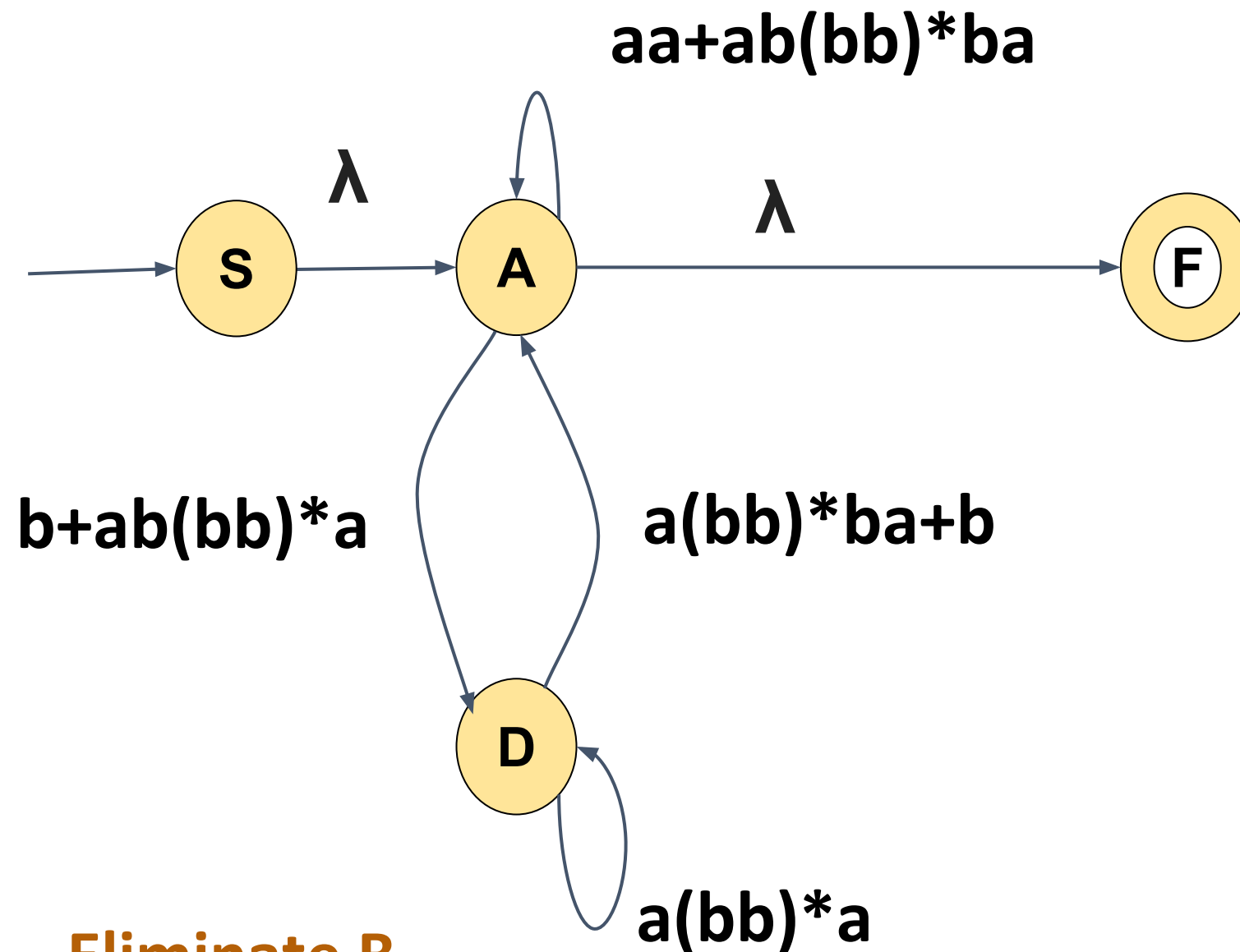
1. Eliminate B

Example 9 :



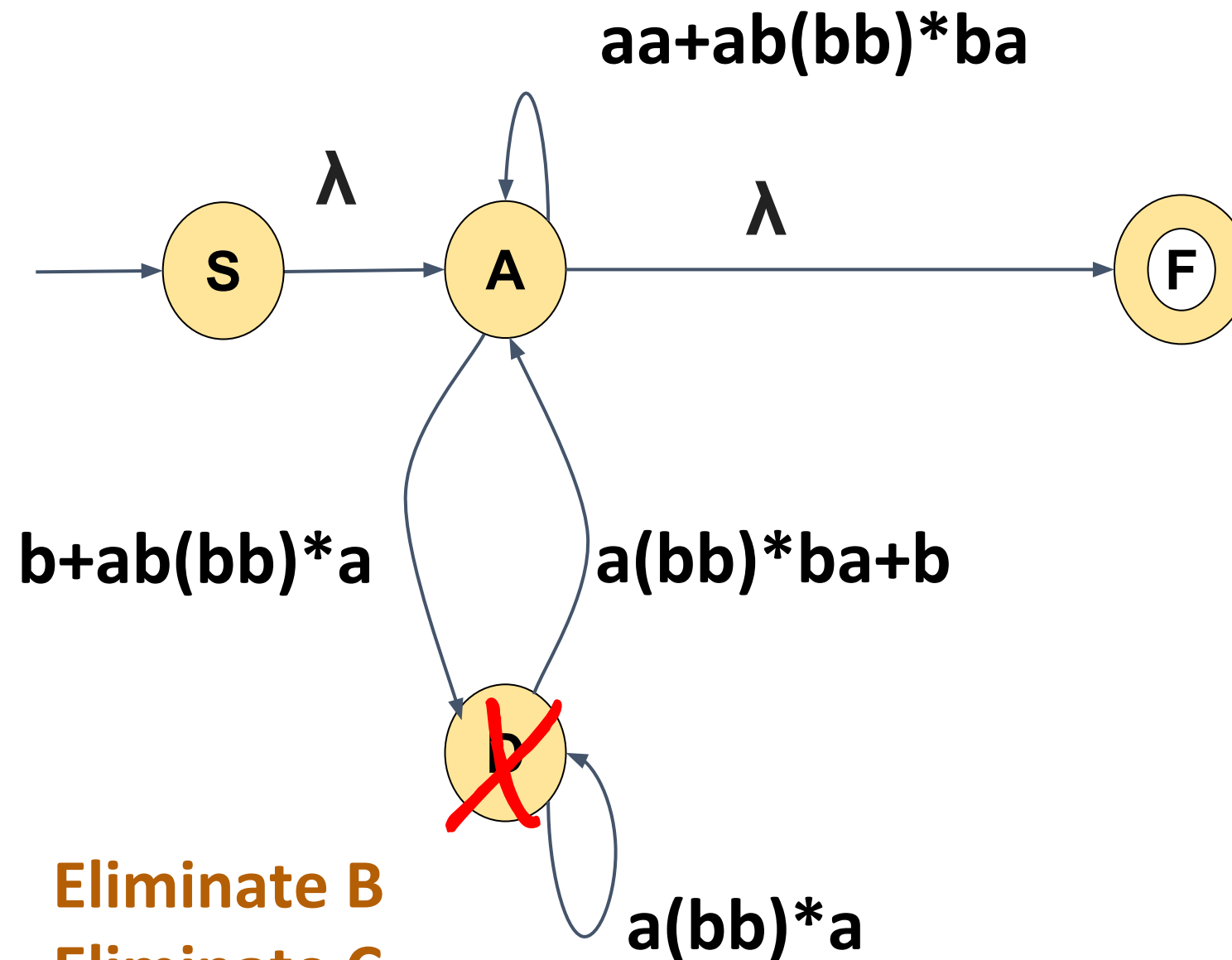
1. Eliminate B
2. Eliminate C

Example 9 :



1. Eliminate B
2. Eliminate C

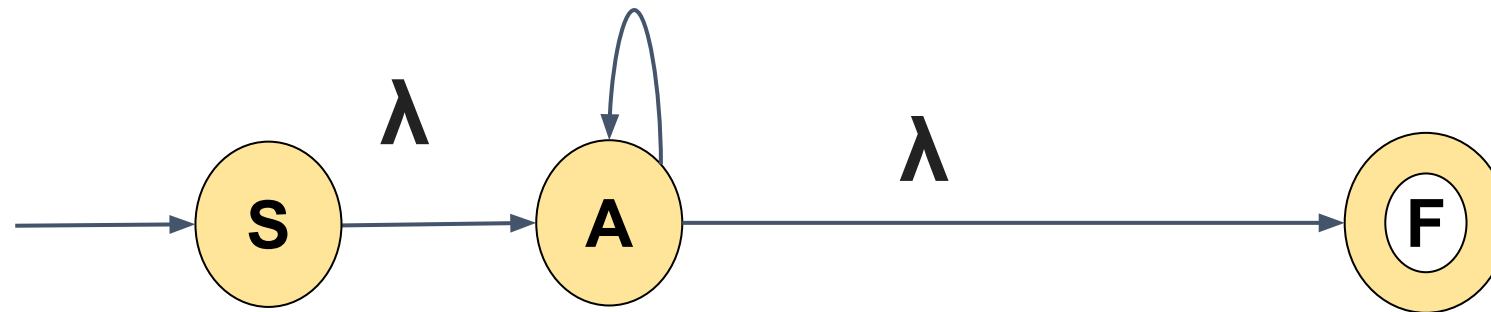
Example 9 :



1. Eliminate B
2. Eliminate C
3. Eliminate D

Example 9 :

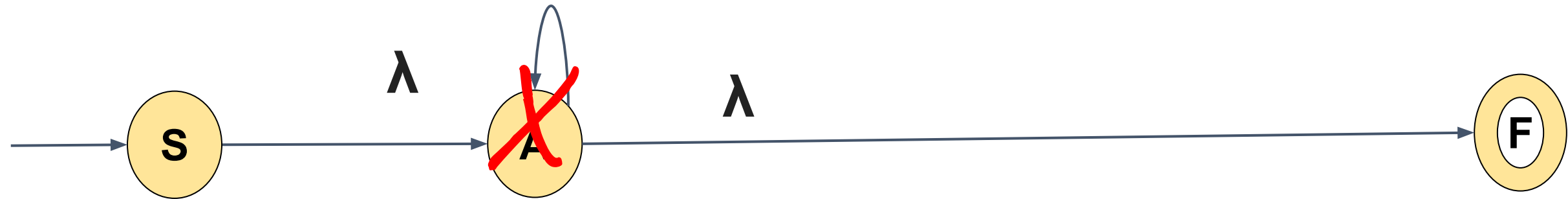
$aa+ab(bb)^*ba + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b$



1. Eliminate B
2. Eliminate C
3. Eliminate D

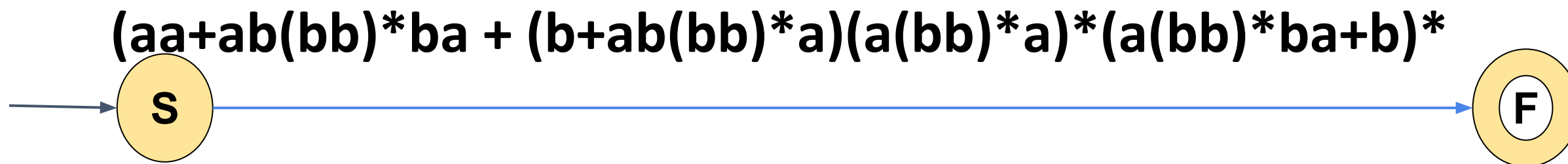
Example 9 :

$(aa+ab(bb)^*ba + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b)$



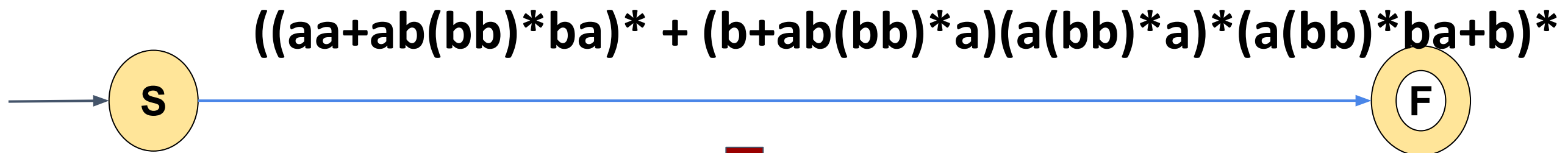
1. Eliminate B
2. Eliminate C
3. Eliminate D
4. Eliminate A

Example 9 :



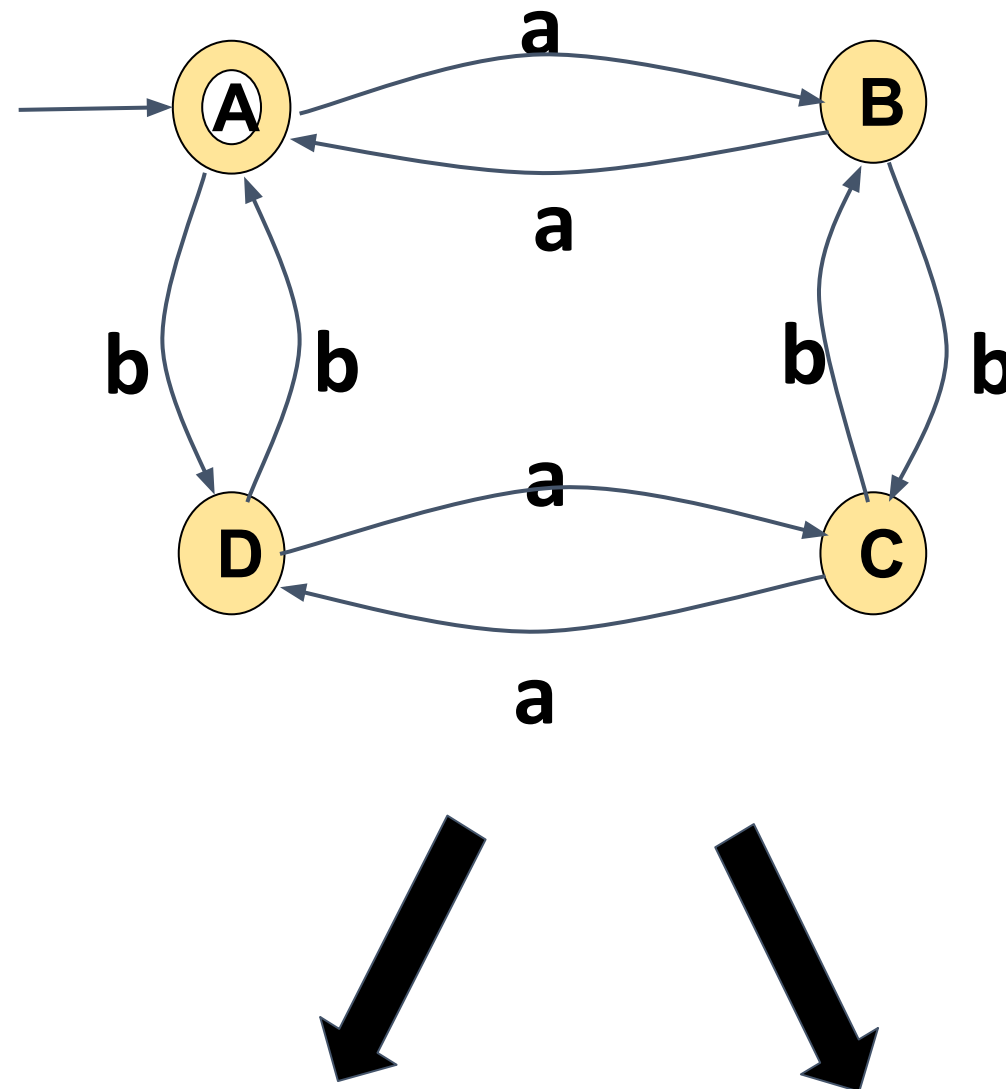
1. Eliminate B
2. Eliminate C
3. Eliminate D
4. Eliminate A

Example 9 :



$$\text{RE} = ((aa+ab(bb)^*ba) + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b))^*$$

Example 9 :



$(aa+bb + (ab+ba+(aa+bb)^*(ab+ba))^*$

Eliminate : B,D,C,A

$((aa+ab(bb)^*ba)^* + (b+ab(bb)^*a)(a(bb)^*a)^*(a(bb)^*ba+b))^*$

Eliminate B,C,D,A



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

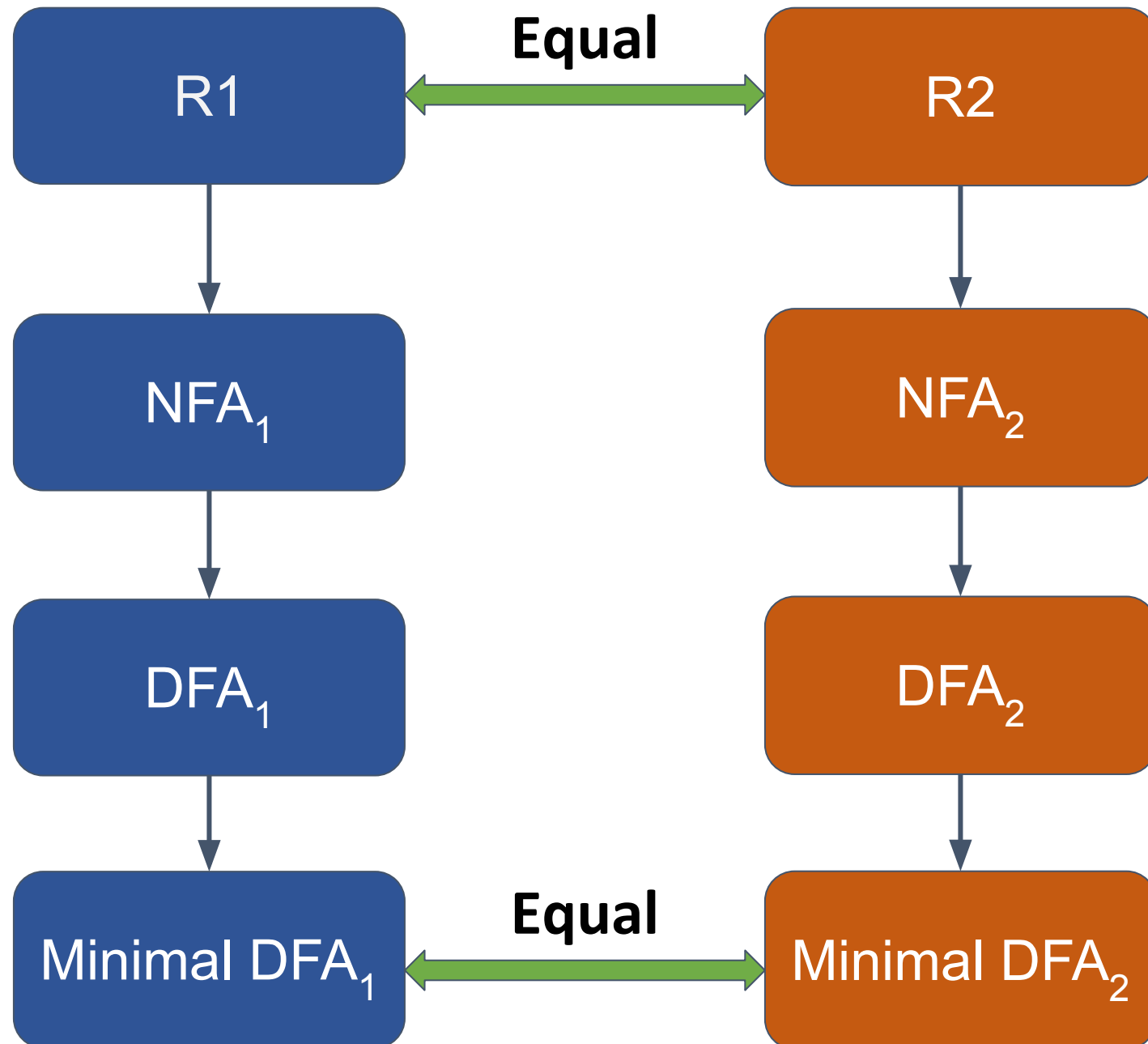
Two regular expressions(R_1 and R_2) are equivalent($R_1 = R_2$) iff:

$$L(R_1) = L(R_2)$$

We can determine the equivalence using:

- 1) A Formal method
- 2) An Informal method:

Formal method to prove Equivalence of two Regex



Informal Method -

We try proving $R_1 \neq R_2$

- Find a string that can be matched with only one of the regex hence proving that the two regex are not equivalent.
- Faster
- But, based on hit and trail.
- Can only be used to prove inequality!!

Let us look at examples and find out whether two regex are equivalent or not??

Example 1:

$$(0+1)^*(0+\lambda)$$
$$(1+\lambda)(1+0)^*$$

Example 1:

$(0+1)^*(0+\lambda)$

**Formal
Method**

$(1+\lambda)(1+0)^*$

Automata Formal Languages and Logic

Unit 2 - Equivalence of two regular expression

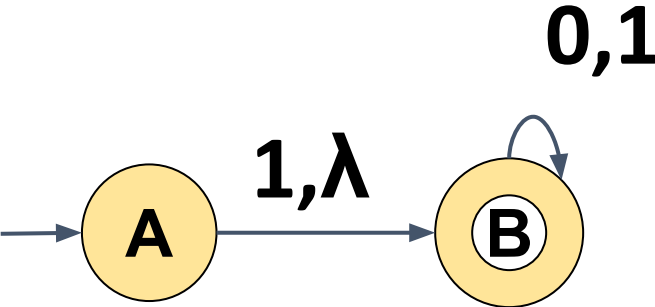
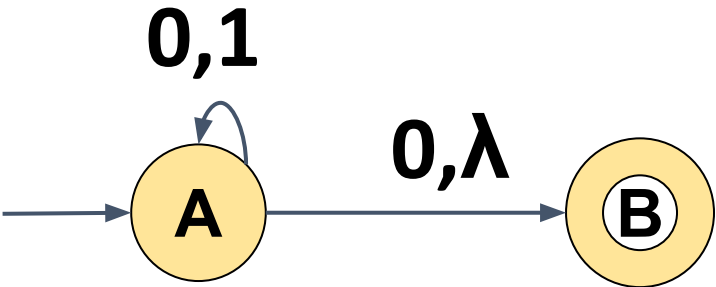
Example 1:

Formal
Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

NFA



Automata Formal Languages and Logic

Unit 2 - Equivalence of two regular expression

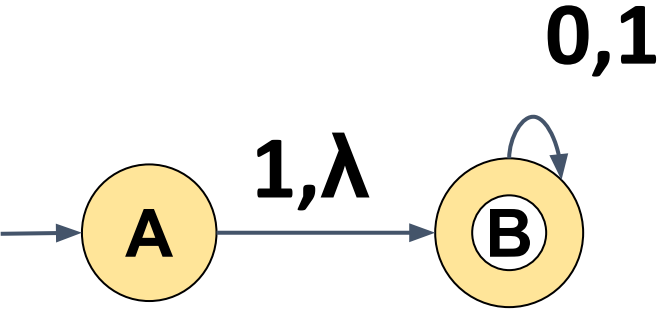
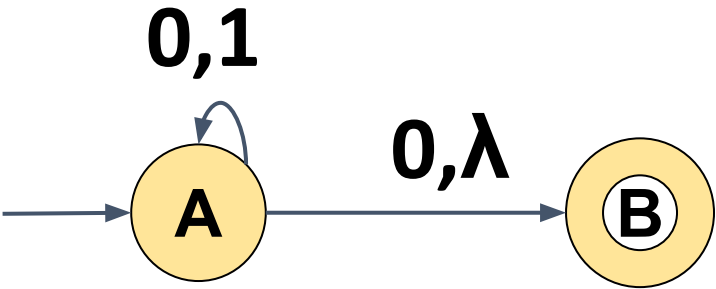
Example 1:

Formal
Method

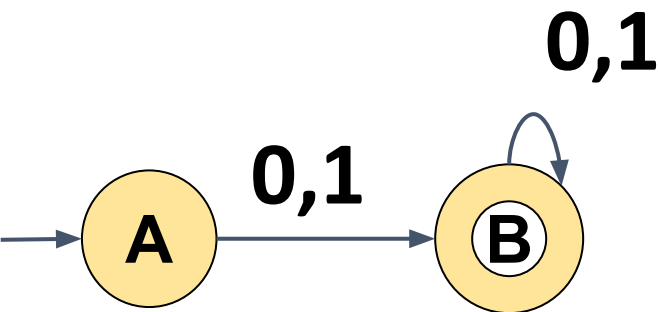
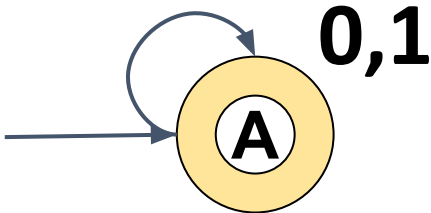
$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

NFA



DFA



Automata Formal Languages and Logic

Unit 2 - Equivalence of two regular expression

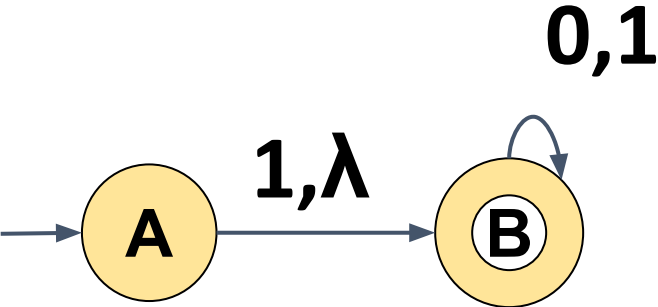
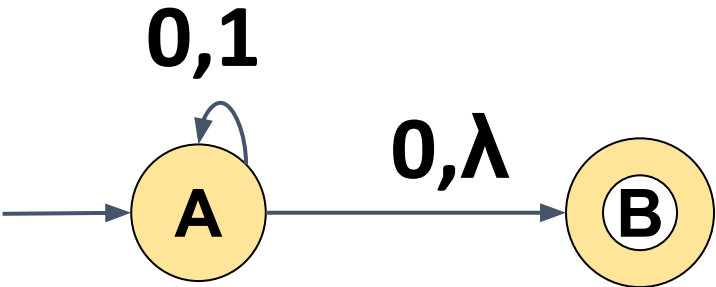
Example 1:

Formal
Method

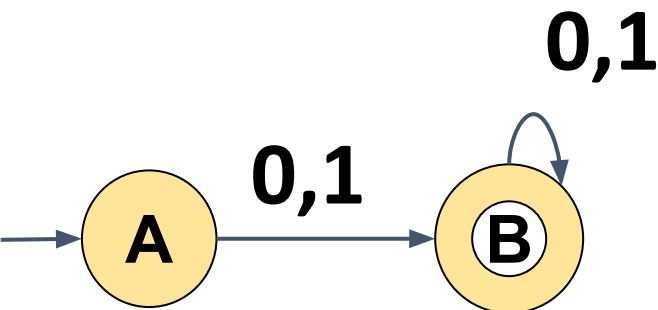
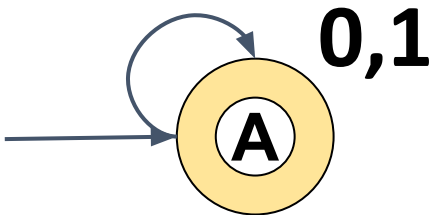
$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

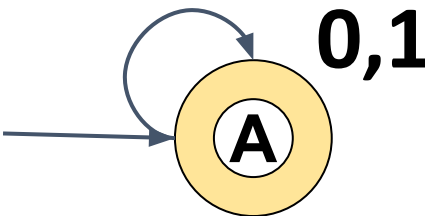
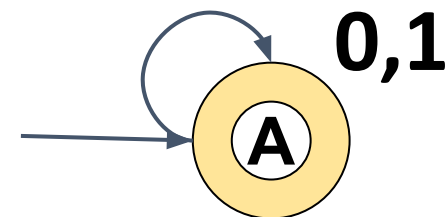
NFA



DFA



Minimal
DFA



Automata Formal Languages and Logic

Unit 2 - Equivalence of two regular expression

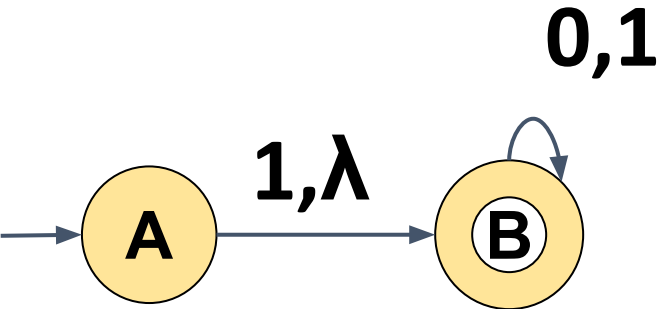
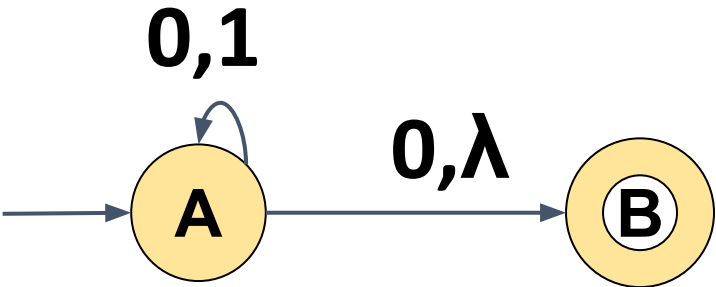
Example 1:

Formal
Method

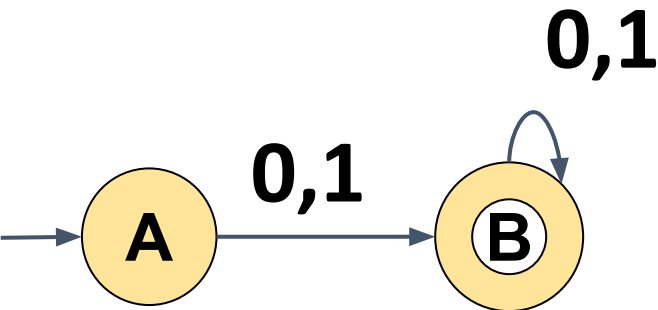
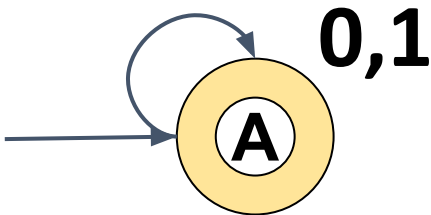
$(0+1)^*(0+\lambda)$

$(1+\lambda)(1+0)^*$

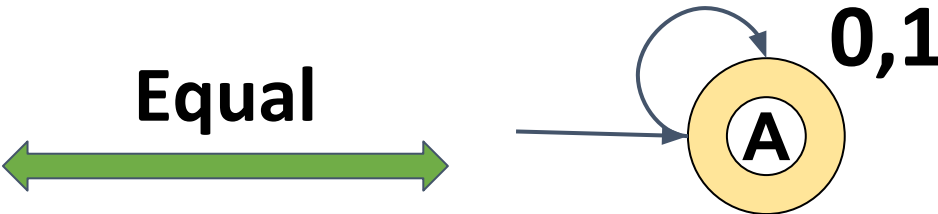
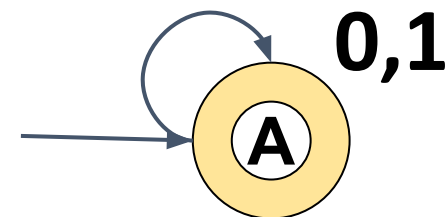
NFA



DFA

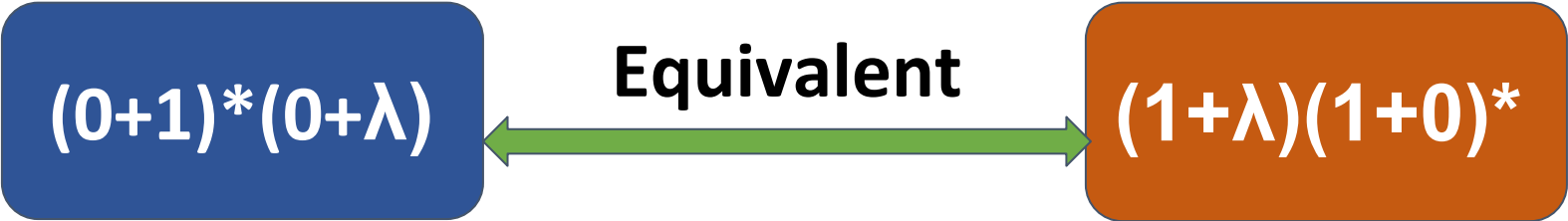


Minimal
DFA



Example 1. Equivalence of two regular expression

Formal
Method



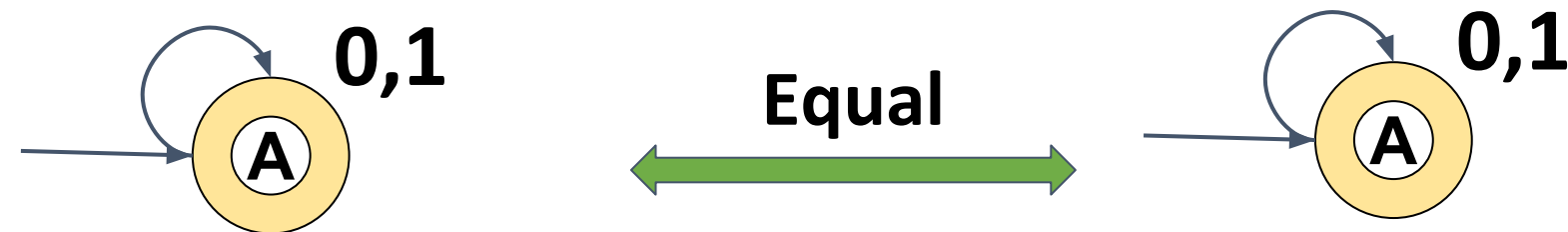
NFA



DFA



Minimal
DFA



Example 2:

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

Example 2:

$(0+\lambda)(11^*0)^*(1+\lambda)$

**Informal
Method**

$(1+\lambda)(011^*)(0+\lambda)$

Example 2:

**Informal
Method**

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

Example 2:

**Informal
Method**

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(1+\lambda)(011^*)(0+\lambda)$

Example 2:

$(0+\lambda)(11^*0)^*(1+\lambda)$

Informal
Method

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(1+\lambda)(011^*)(0+\lambda)$

011

Example 2:

Informal Method

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

011

Example 2:

Informal Method

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

011 cannot be
generated

011

$(0+\lambda)(11^*0)^*(1+\lambda)$

$\neq$

$(1+\lambda)(011^*)(0+\lambda)$

Find a string belongs to only one of the regex

$(0+\lambda)(11^*0)^*(1+\lambda)$

$(1+\lambda)(011^*)(0+\lambda)$

011 cannot be
generated

011

Can you answer whether the two regex are equivalent or not??

1) $(1+\lambda) (00^*1)^* 0^*$ and $(0+\lambda) (11^*0)1^*$

2) $0^*(10^*)^*$ and $(1^*0)^*1^*$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Regular Grammar

- Formal Definition of a Grammar
- Terminology : Sentence , Sentential Form
- Examples on how to construct a Regular Grammar for a given L

Grammar

V - Variables

$\text{expr, var, S, A, B...}, w, x, y$

T - Terminals

$a, b, c \dots, 0, 1, \dots \text{id, num}$

P - Production Rules

$\alpha \text{ or } \beta = (V \cup T)^*$

$\alpha \rightarrow \beta$

S - Start Symbol

$L(G) =$

$\{w \mid S \Rightarrow w, \text{ where } w \in \Sigma^*\}$

Sentence (or a String)

A sentence is made up only of terminals.

Example:

For the Grammar $S \rightarrow aS \mid a \mid \lambda$

Sentence (or string) is { a or λ or aa or aaaa)

Sentential Form

A sentential form is made up over set of terminals and non-terminals (VUT)*

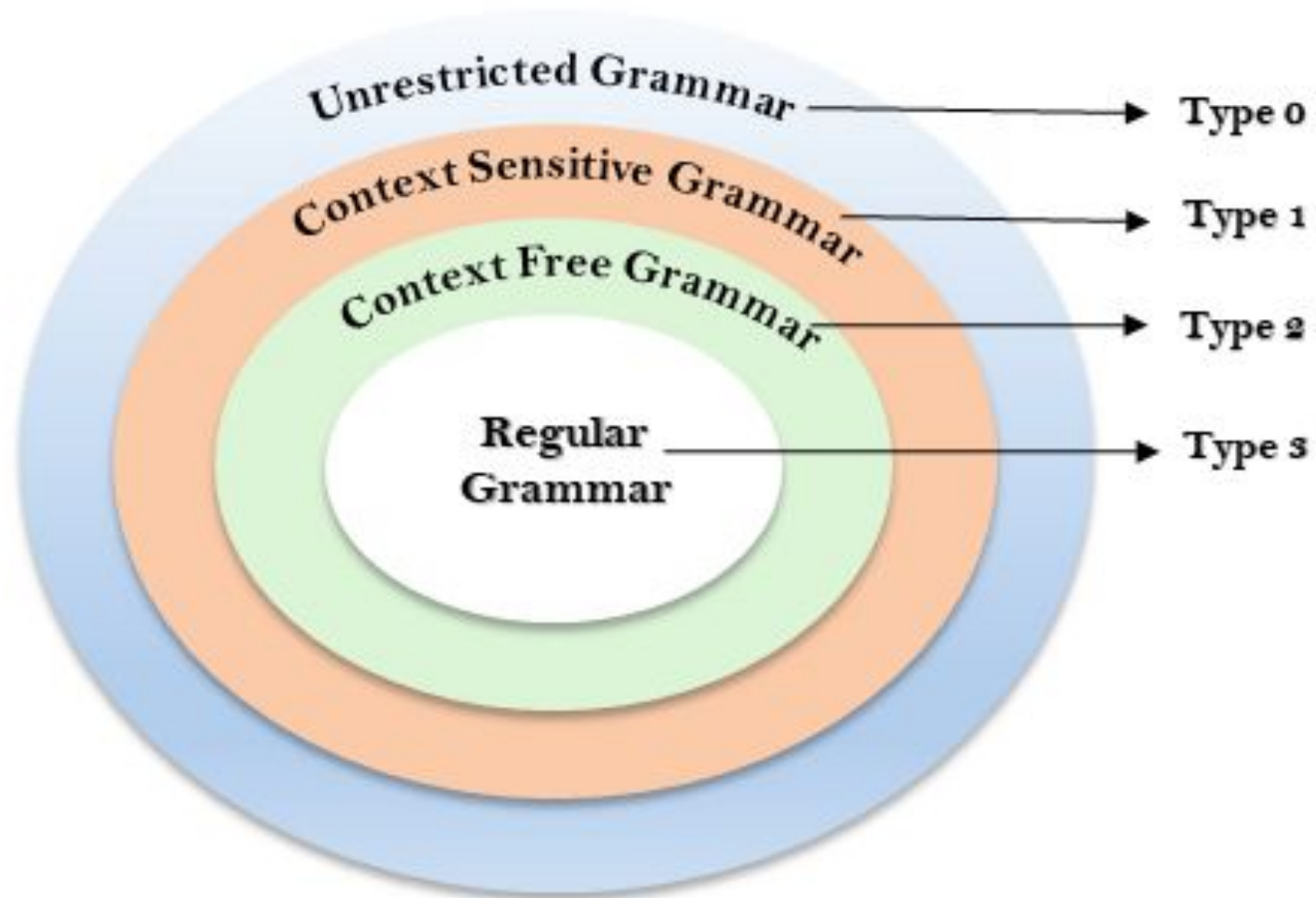
Example:

For the Grammar $S \rightarrow aS \mid a \mid \lambda$

Sentential form is { a or λ or aS or aa or or aaS aaaa)

Automata Formal Languages and Logic

Unit 2 - Regular Grammar

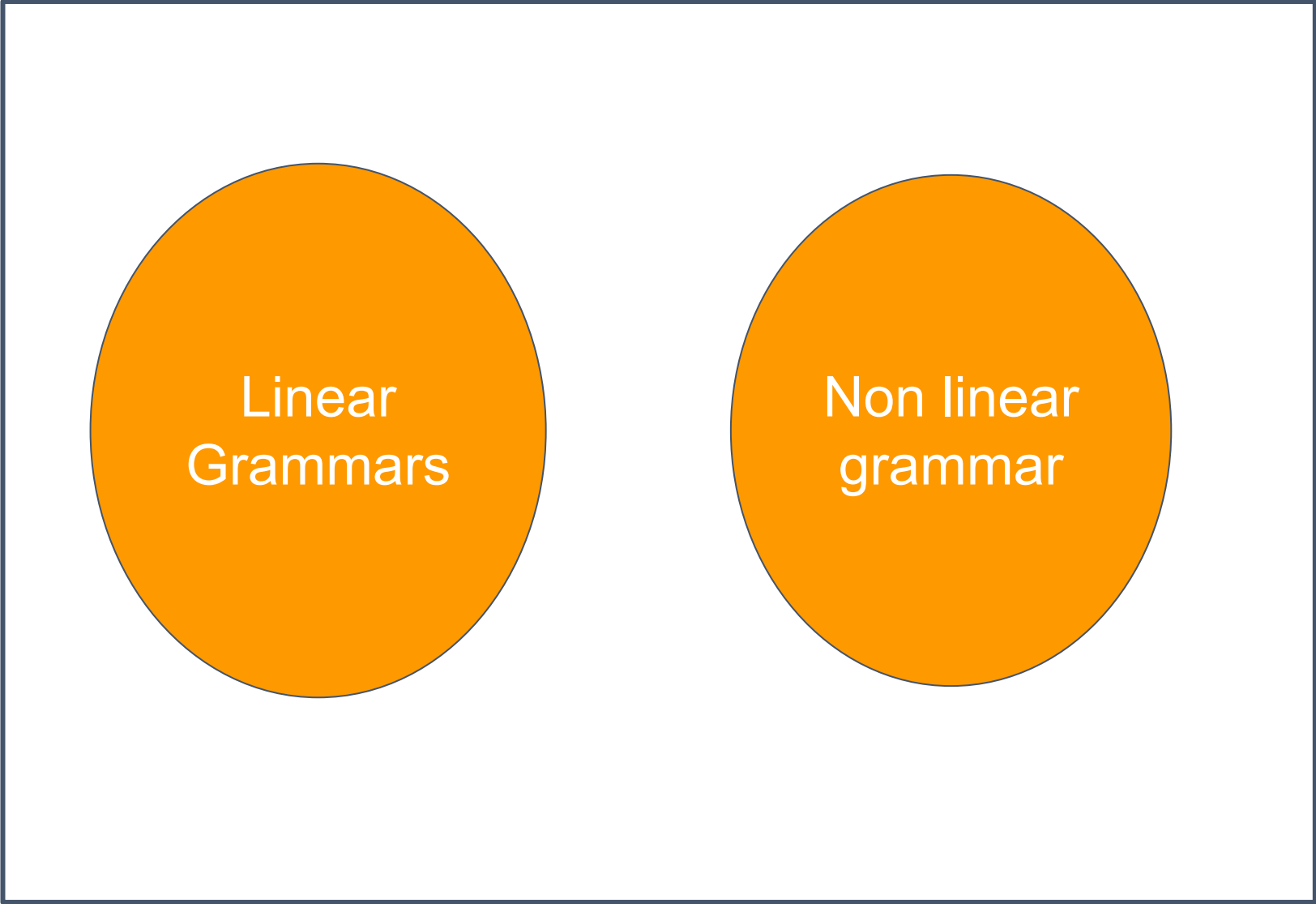


Noam Chomsky

Chomsky Hierarchy

Automata Formal Languages and Logic

Unit 2 - Regular Grammar



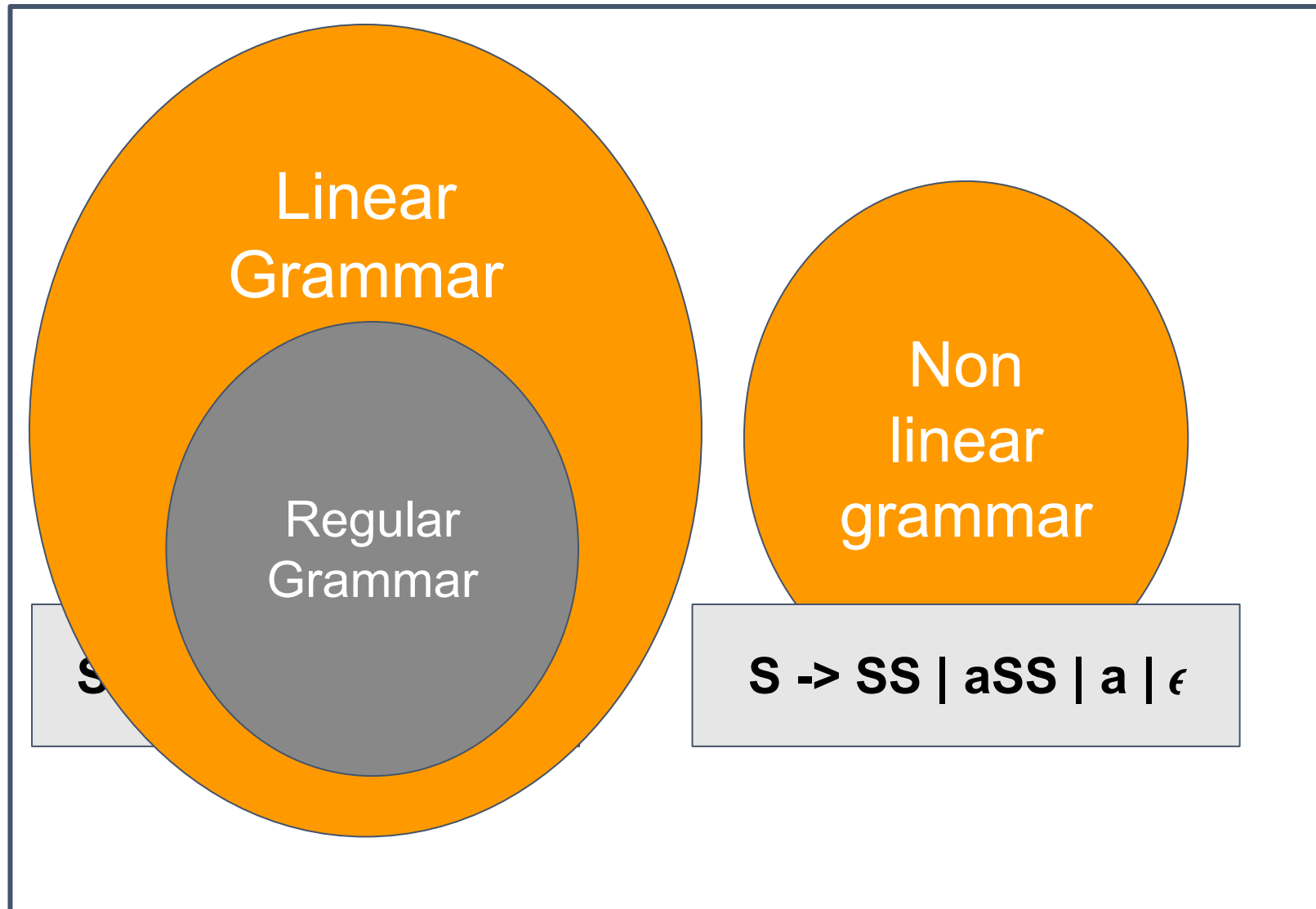
Linear
Grammars

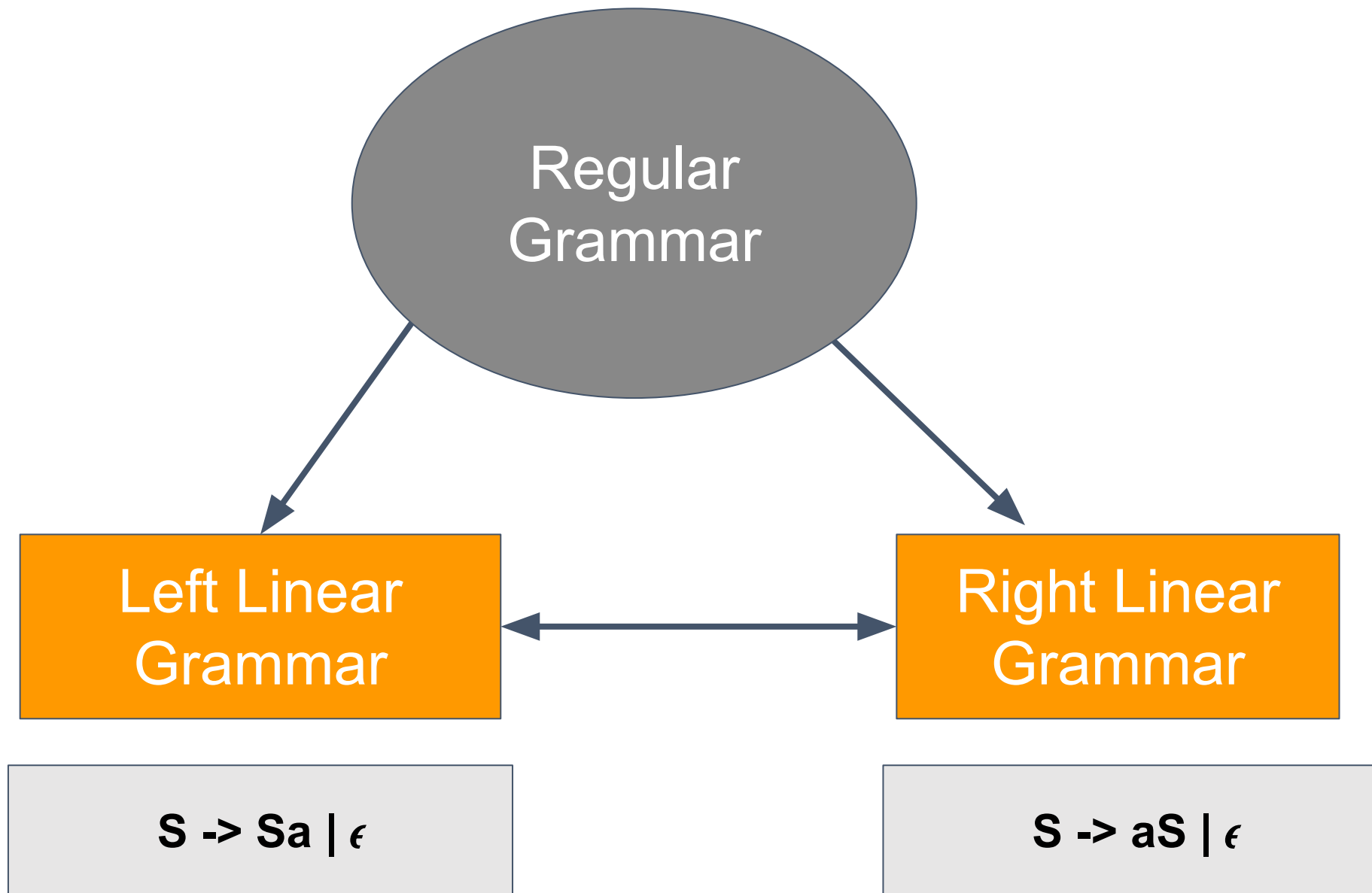
Non linear
grammar

Linear
Grammar
s

$S \rightarrow aSa \mid aS \mid Sa \mid \epsilon$

Non
linear
grammar





Construct a regular grammar for a given language description



Construct a regular grammar for the given language

Language	RE	RG
$L = \{a\}$	a	$S \rightarrow a$



Construct a regular grammar for the given language

Language	RE	RG
$L = \{a, b\}$	$a+b$	$S \rightarrow a \mid b$ or $S \rightarrow a$ $S \rightarrow b$



Construct a regular grammar for the given language

Language	RE	RG
$L = \{ab\}$	ab	$S \rightarrow ab$



Construct a regular grammar for the language $L=\{a^n|n\geq 0\}$

Language	RE	RG
$L = \{\lambda, a, aa, aaa, \dots\}$	a^*	$S \rightarrow aS \mid \lambda$



Construct a regular grammar for the language $L=\{a^n|n\geq 1\}$

Language	RE	RG
$L = \{a,aa,aaa,\dots\}$	a^+	$S \rightarrow aS \mid a$



Construct a regular grammar for the regular expression $(a+b)^*$

Language	RE	RG
$L =$ $\{\lambda, a, b, abb, baaaaab, ..$ $.....\}$ $L = \{\text{any number of 'as and b's'}\}$	$(a+b)^*$	$S \rightarrow aS \mid bS \mid \lambda$



Construct a regular grammar for the regular expression $(a+b)^+$

Language	RE	RG
$L =$ $\{\lambda, a, b, abb, baaaaab, ..$ $.....\}$ $L = \{\text{any number of 'as and b's'}\}$	$(a+b)^+$	$S \rightarrow aS \mid bS \mid a \mid b$



Construct a regular grammar for the regular expression $(ab)^*$

Language	RE	RG
$L =$ $\{\lambda, a, b, ab, ababab, \dots$ $\dots\}$ $L = \{\text{any number of}$ $ab's\}$	$(ab)^*$	$S \rightarrow abS \mid \lambda$



Construct a regular grammar for the language $L=\{a^m b^n \mid n,m \geq 0\}$

Language	RE	RG
$L=\{\lambda,a,b,bb,abb,.....\}$. $L=\{\text{any number of a's followed by any number of b's}\}$	a^*b^*	$S \rightarrow A B$ A:any number of a's B:any number of b's $A \rightarrow aA \lambda$ $B \rightarrow bB \lambda$



Construct a regular grammar for the given language with even number of a's. $L=\{a^{2n} \mid n \geq 0\}$

Language	RE	RG
$L=\{\lambda,$ $aa,$ $aaaa,$ $aaaaaa \dots$	$(aa)^*$	$S \rightarrow aaS \mid \lambda$



Construct a regular grammar for the given language with odd number of a's. $L=\{a^{2n+1} \mid n \geq 0\}$

Language	RE	RG
$L=\{a, aaa, aaaaa, aaaaaaa \dots\}$	$(aa)^*a$	$S \rightarrow aaS \mid a$



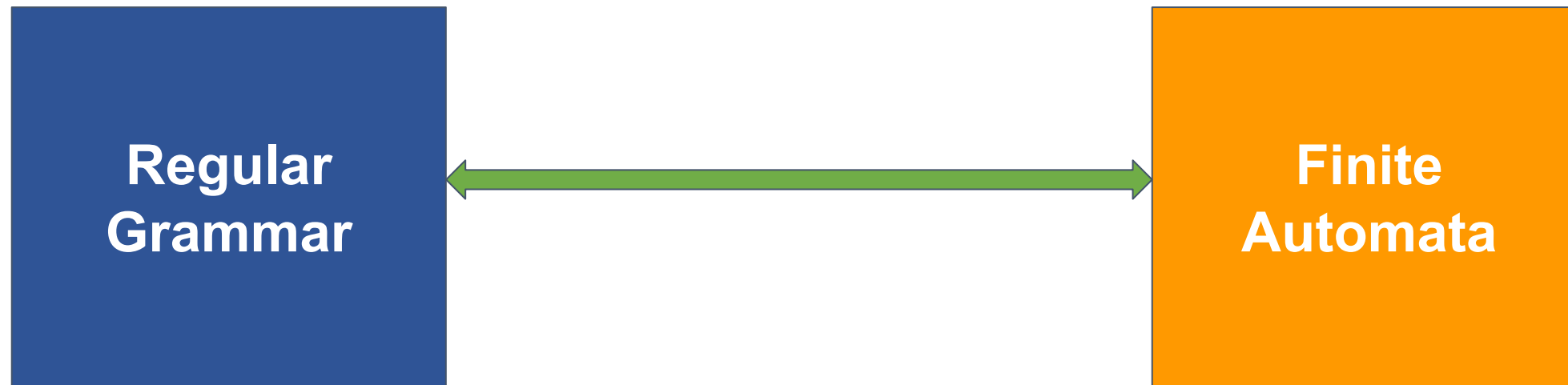
Construct a regular grammar for the given language with number of a’s as multiples of 4. $L=\{a^{4n} \mid n \geq 0\}$

Language	RE	RG
$L=\{\lambda,$ aaaa, aaaaaaaaa, aaaaaaaaaaaaa,	$(aaaa)^*$	$S \rightarrow aaaaS \mid \lambda$



Convert finite automata to regular grammar for the language to accept at least one ‘a’ over the alphabet $\Sigma=\{a,b\}$

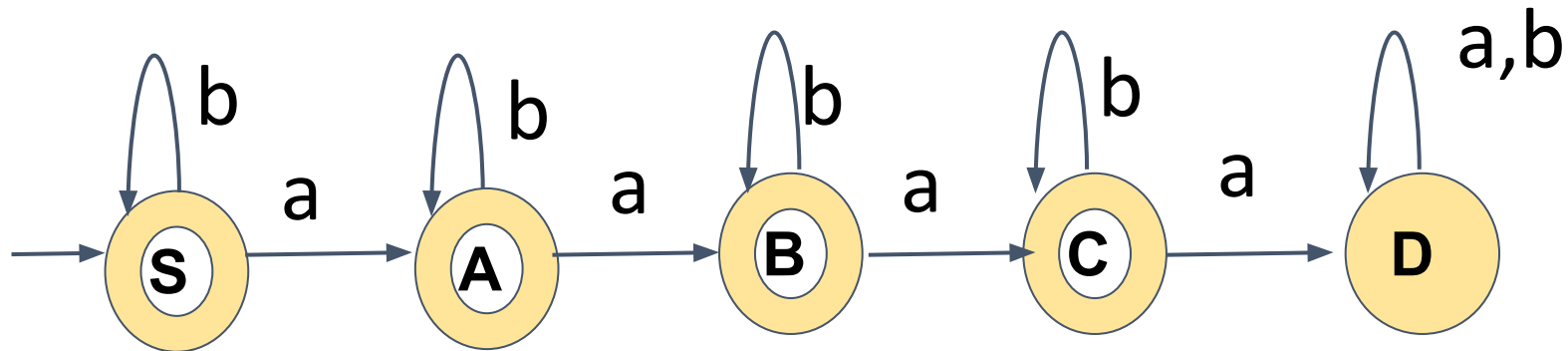
Language	RE	RG
$L=\{a, bb... a, ba \text{ any \# of } a's \text{ \& } b's\}$	$b^* a (a+b)^*$	$S \rightarrow bS \mid aA$ $A \rightarrow aA \mid bA \mid \lambda$



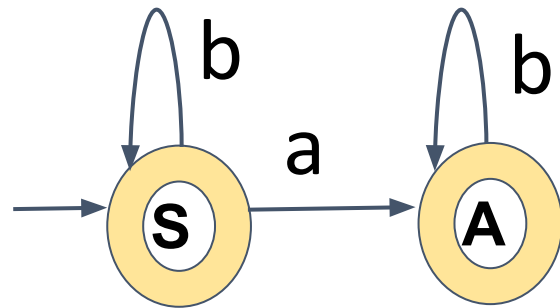
Let's look at examples to convert :

- 1. Finite automata to regular grammar**
- 2. Regular Grammar to Finite Automata**

1. Convert finite automata to regular grammar for the language to accept at most 3 'a's over the alphabet $\Sigma = \{a, b\}$.

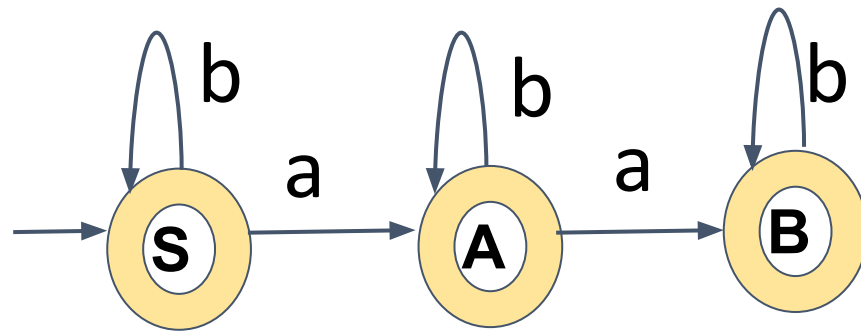


Let's look at each transition and perform the conversion


$$S \rightarrow bS \mid aA \mid \lambda$$
$$A \rightarrow bA \mid \lambda$$

λ on RHS indicates S
and A are final states

Let's look at each transition and perform the conversion



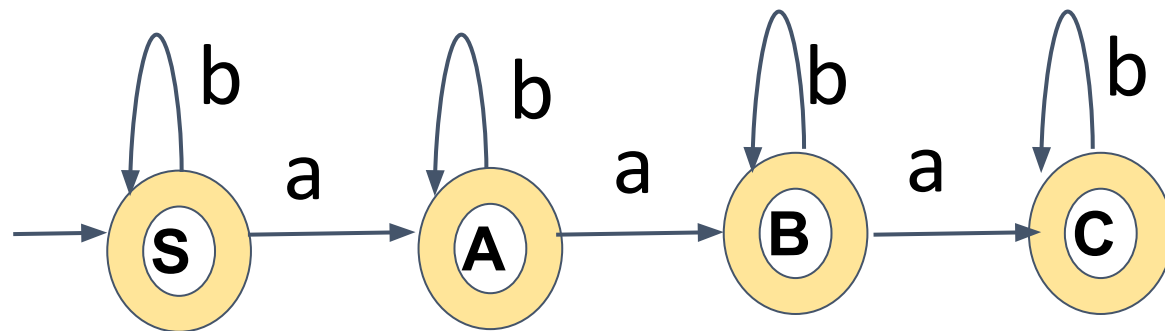
$$S \rightarrow bS \mid aA \mid \lambda$$

$$A \rightarrow bA \mid aB \mid \lambda$$

$$B \rightarrow bB \mid aC \mid \lambda$$

λ on RHS indicates S,A
and B are final states

Let's look at each transition and perform the conversion



$$S \rightarrow bS \mid aA \mid \lambda$$

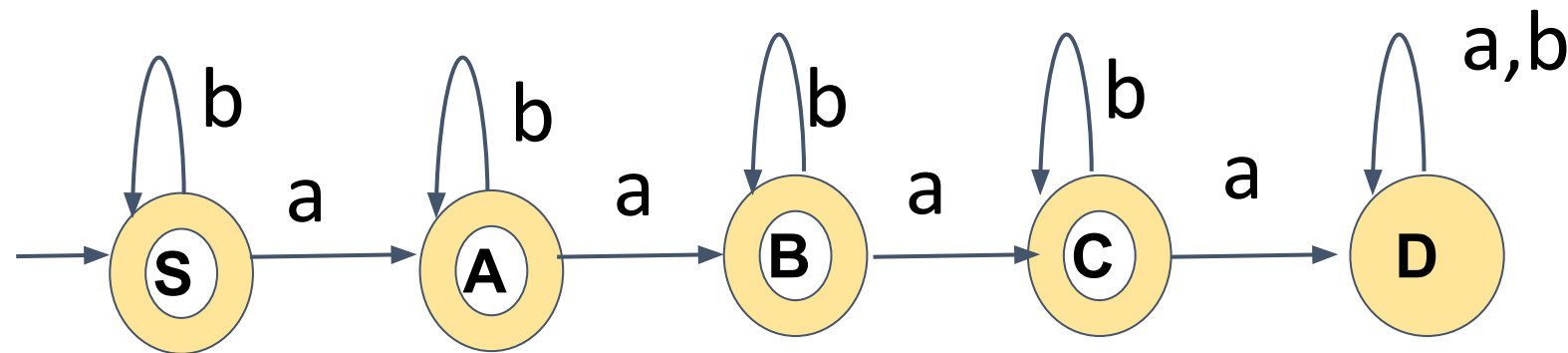
$$A \rightarrow bA \mid aB \mid \lambda$$

$$B \rightarrow bB \mid aC \mid \lambda$$

$$C \rightarrow bC \mid aD \mid \lambda$$

Let's look at each transition and perform the conversion

Since D is a dead state, we will not encode $C \rightarrow aD$ and $D \rightarrow aD \mid bD$ as D is a useless Non-terminal(that means it never terminates)



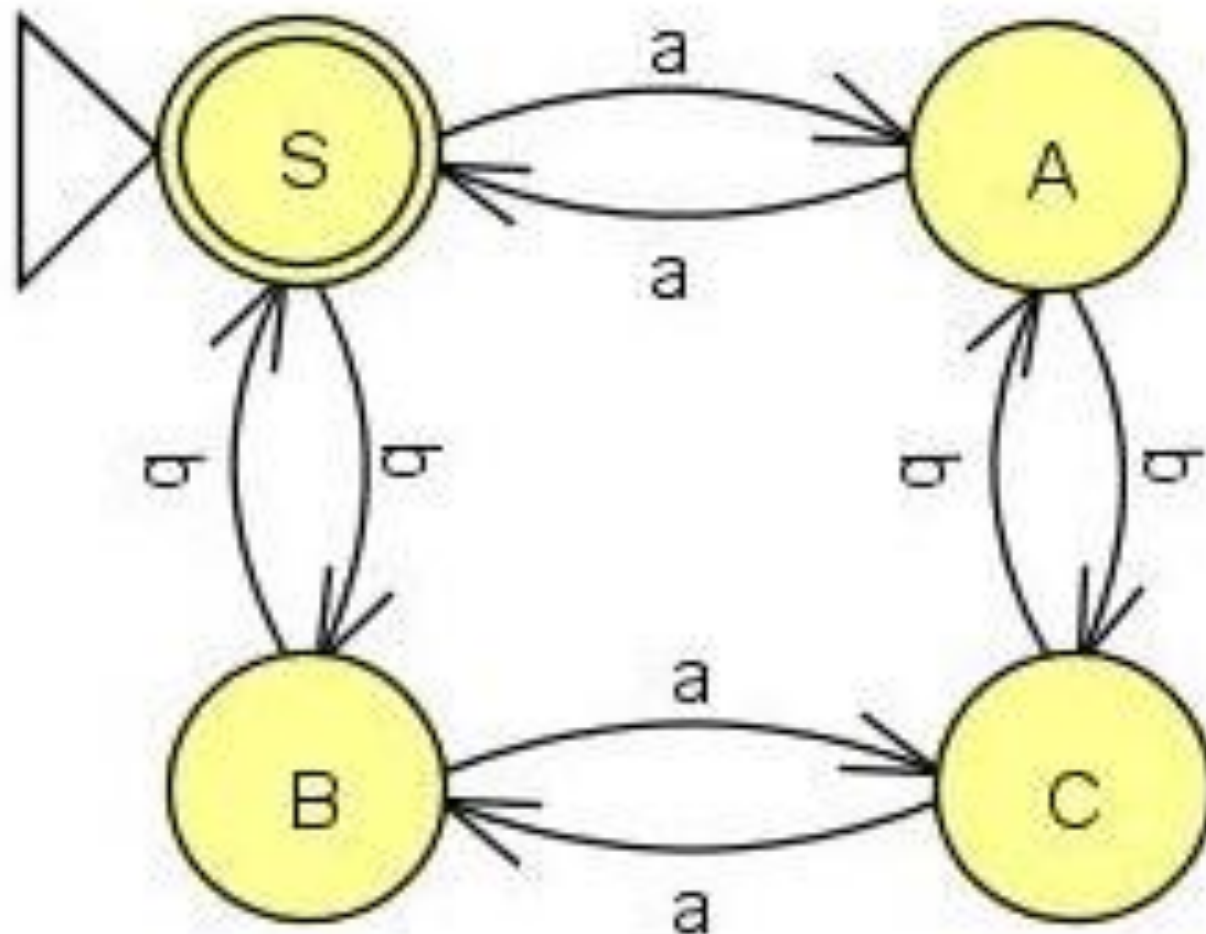
$$S \rightarrow bS \mid aA \mid \lambda$$

$$A \rightarrow bA \mid aB \mid \lambda$$

$$B \rightarrow bB \mid aC \mid \lambda$$

$$C \rightarrow bC \mid \lambda$$

2. Convert a given finite automata accepting $L = \{n_a(w) \bmod 2 = 0 \text{ and } n_b(w) \bmod 2 = 0\}$ to regular grammar.



2. Convert a given finite automata accepting $L = \{n_a(w) \bmod 2 = 0 \text{ and } n_b(w) \bmod 2 = 0\}$ to regular grammar.

Start state of automata will be the start symbol of the grammar.

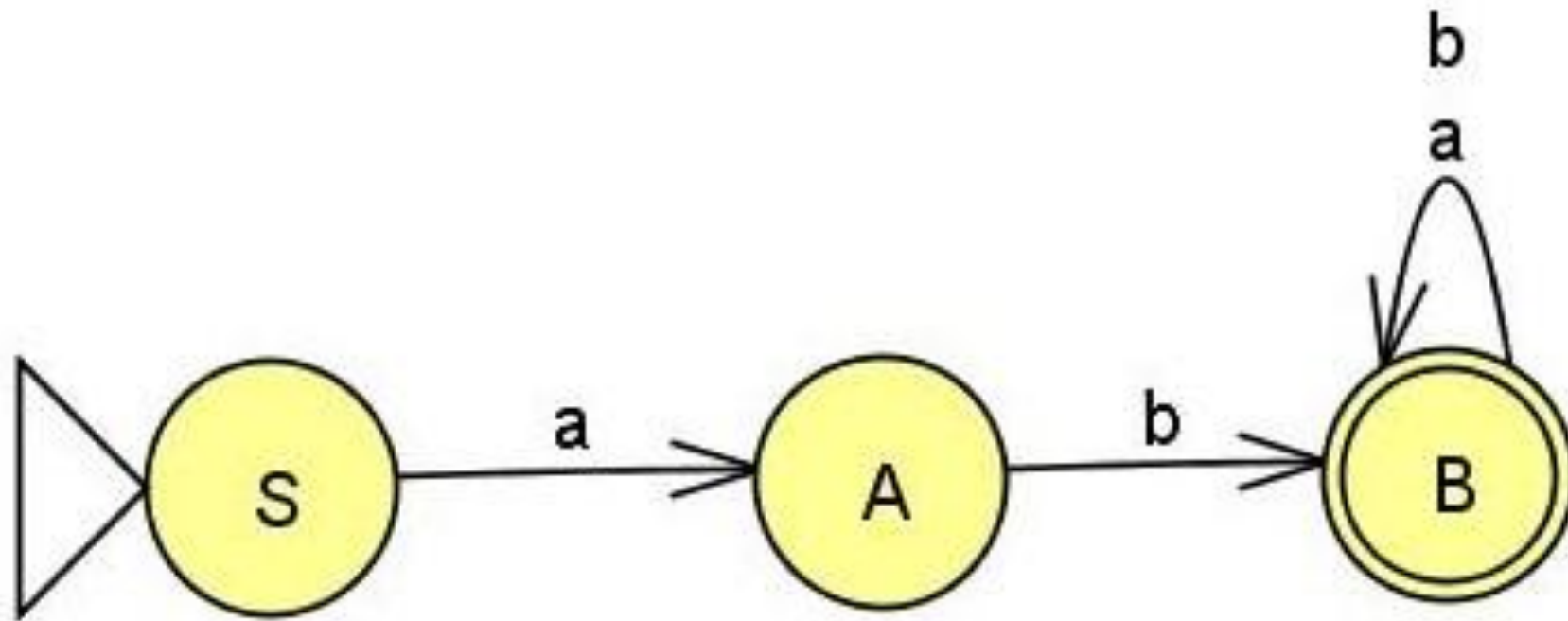
We start with S, S on seeing terminal a it moves to state A ($S \rightarrow aA$) and on seeing terminal b it moves to state B ($S \rightarrow bB$).

Since S is also the final state, we introduce the production $S \rightarrow \lambda$.

Grammar is

$$S \rightarrow aA \mid bB \mid \lambda$$
$$A \rightarrow aS \mid bC$$
$$B \rightarrow aC \mid bS$$
$$C \rightarrow aB \mid bA$$

3. Converting a given finite automata accepting $L=\{abw, w \in \{a,b\}^*\}$ to regular grammar.



3. Converting a given finite automata accepting $L=\{abw, w \in \{a,b\}^*\}$ to regular grammar.

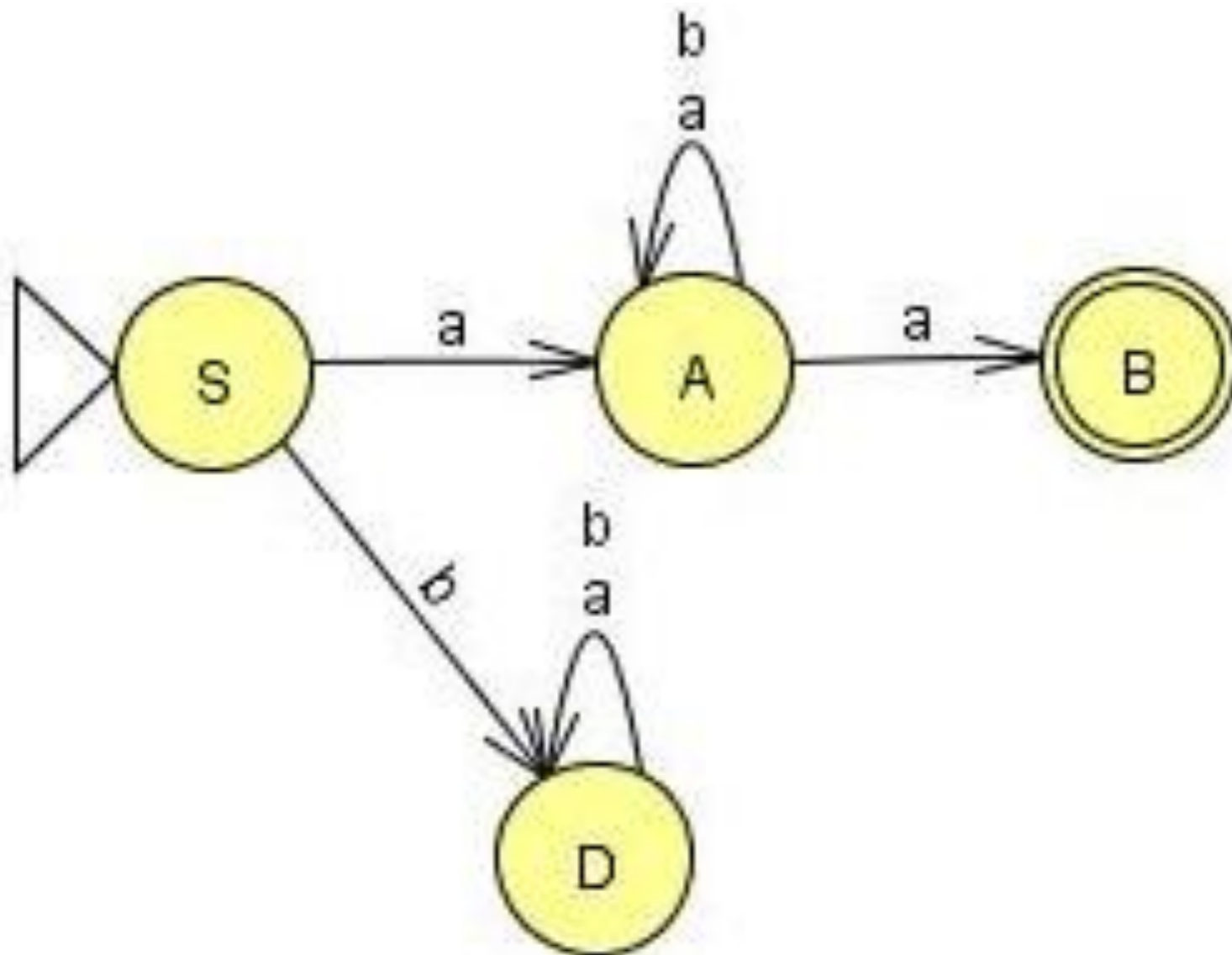
Grammar is,

$S \rightarrow aA$

$A \rightarrow bB$

$B \rightarrow aB \mid bB \mid \lambda$

4. Converting given finite automata accepting $L=\{awa, w \in \{a,b\}^*\}$ to regular grammar.



4. Converting given finite automata accepting $L=\{awa, w \in \{a,b\}^*\}$ to regular grammar.

So the grammar is,

$$S \rightarrow aA$$
$$A \rightarrow aA \mid bA \mid aB$$
$$B \rightarrow \lambda$$

Converting Regular grammar to finite automata

Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

Variables $\rightarrow$ state

terminal symbols $\rightarrow$ symbols of finite
automata

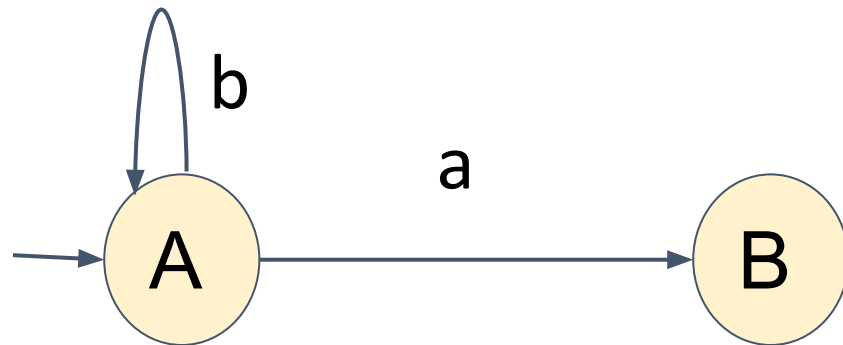
Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

- Initial start symbol will be the start state



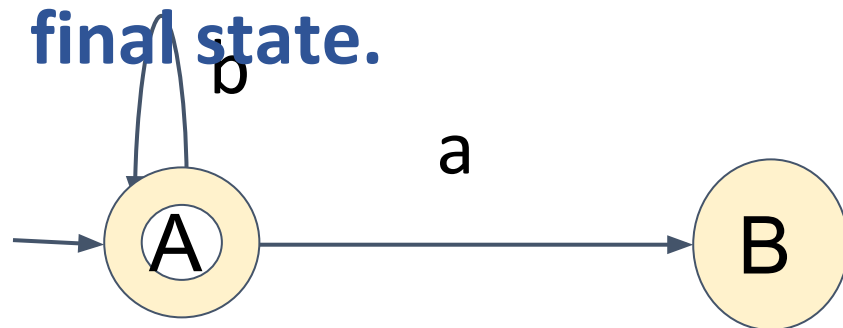
Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

- Transition $A \rightarrow bA$ transforms to $A \rightarrow b$, when $A \rightarrow \lambda$ $A \rightarrow \lambda$ indicates A is a final state.

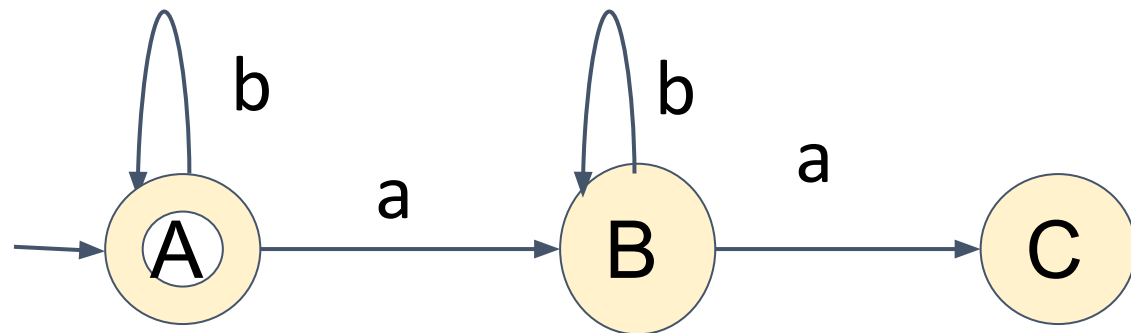


Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$



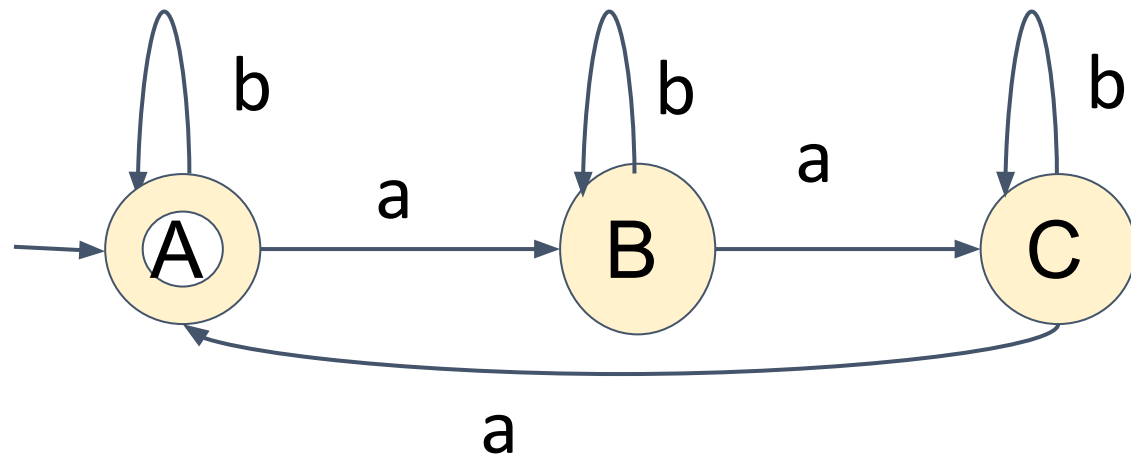
Example 1 :

$A \rightarrow aB \mid bA \mid b$

$B \rightarrow aC \mid bB$

$C \rightarrow aA \mid bC \mid a$

- Transition $C \rightarrow aA$ to $C \rightarrow a$ when $A \rightarrow \lambda$ indicates A is a final state.



Automata Formal Languages and Logic

Unit 2 - Regular Grammar to Finite Automata



Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$

Automata Formal Languages and Logic

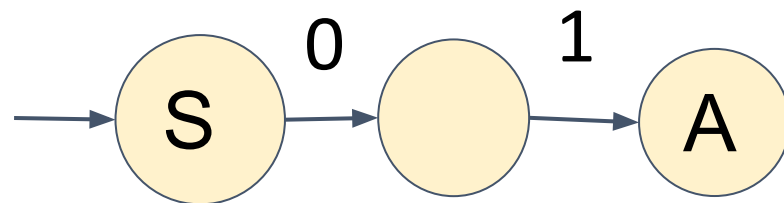
Unit 2 - Regular Grammar to Finite Automata

Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$

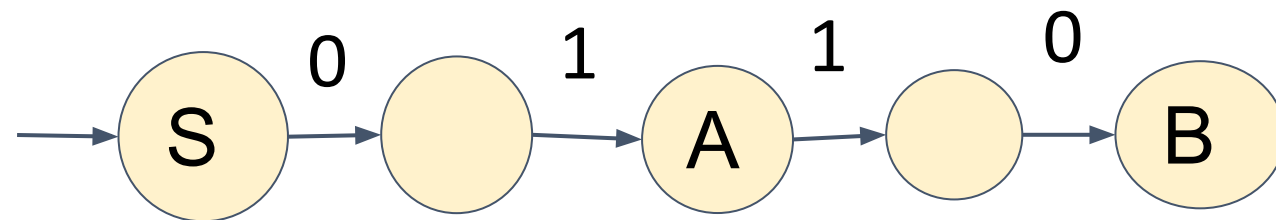


Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$



Automata Formal Languages and Logic

Unit 2 - Regular Grammar to Finite Automata

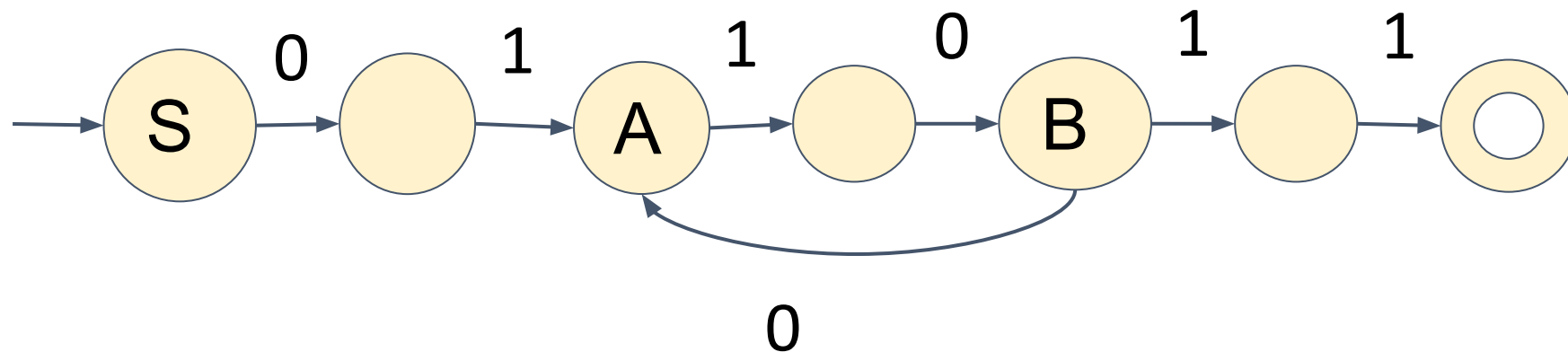
Example 2 :

$S \rightarrow 01A$

$A \rightarrow 10B$

$B \rightarrow 0A \mid 11$

$B \rightarrow 11$ indicates on state B on consuming the input 11 we reach final state



Example 3 :

$S \rightarrow bS \mid aA$

$A \rightarrow aA \mid bA$

$A \rightarrow aB$

$B \rightarrow bbB$

$B \rightarrow \lambda$

Example 3 :

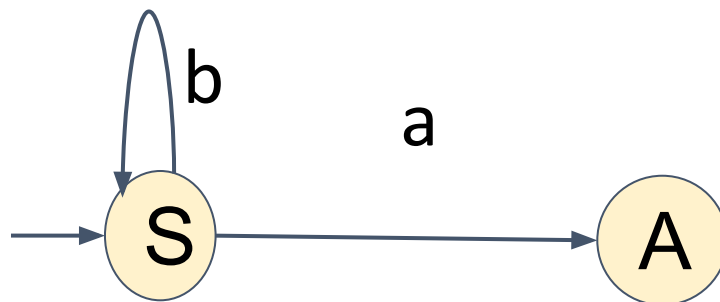
$S \rightarrow bS \mid aA$

$A \rightarrow aA \mid bA$

$A \rightarrow aB$

$B \rightarrow bbB$

$B \rightarrow \lambda$



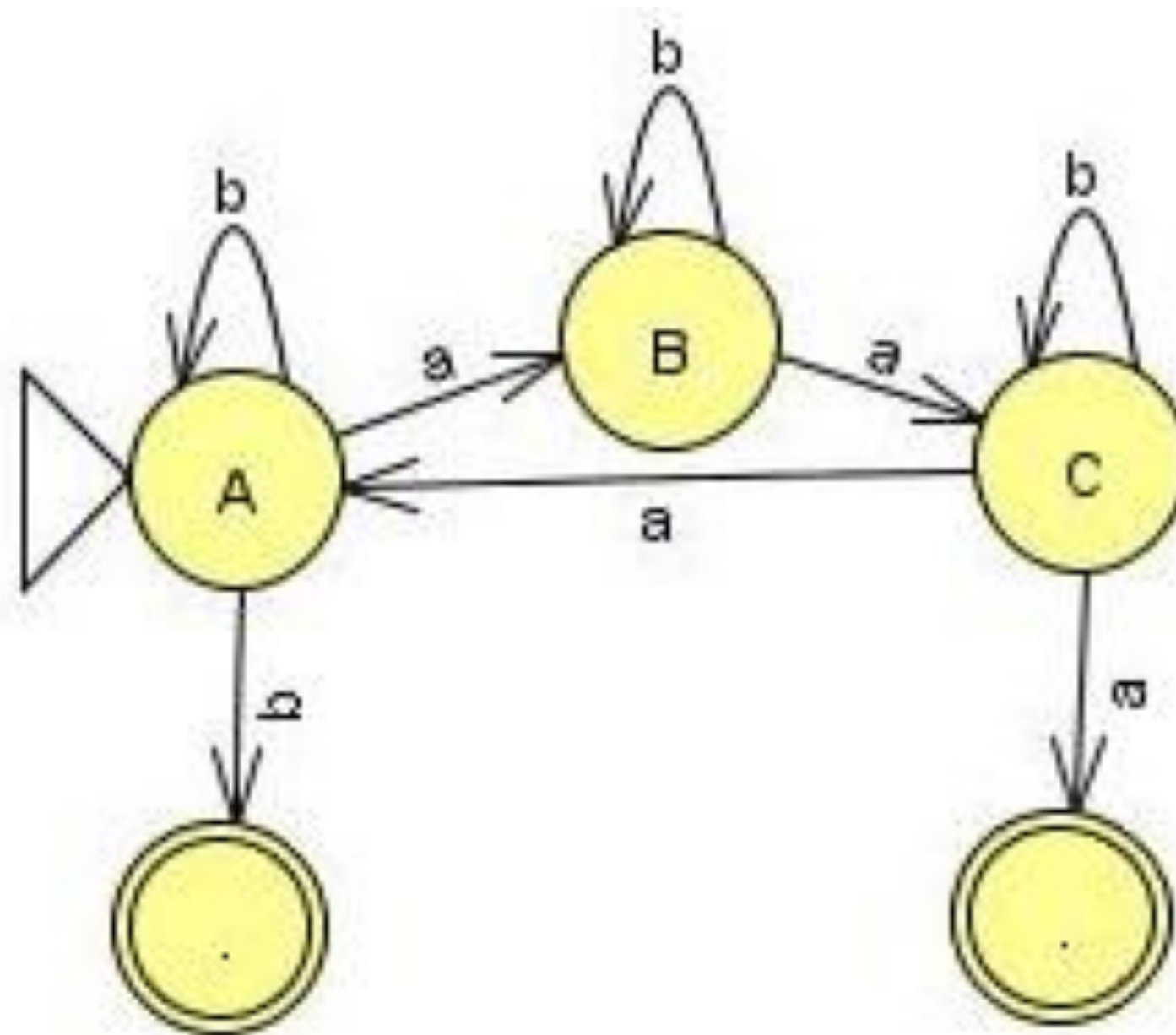
Grammar

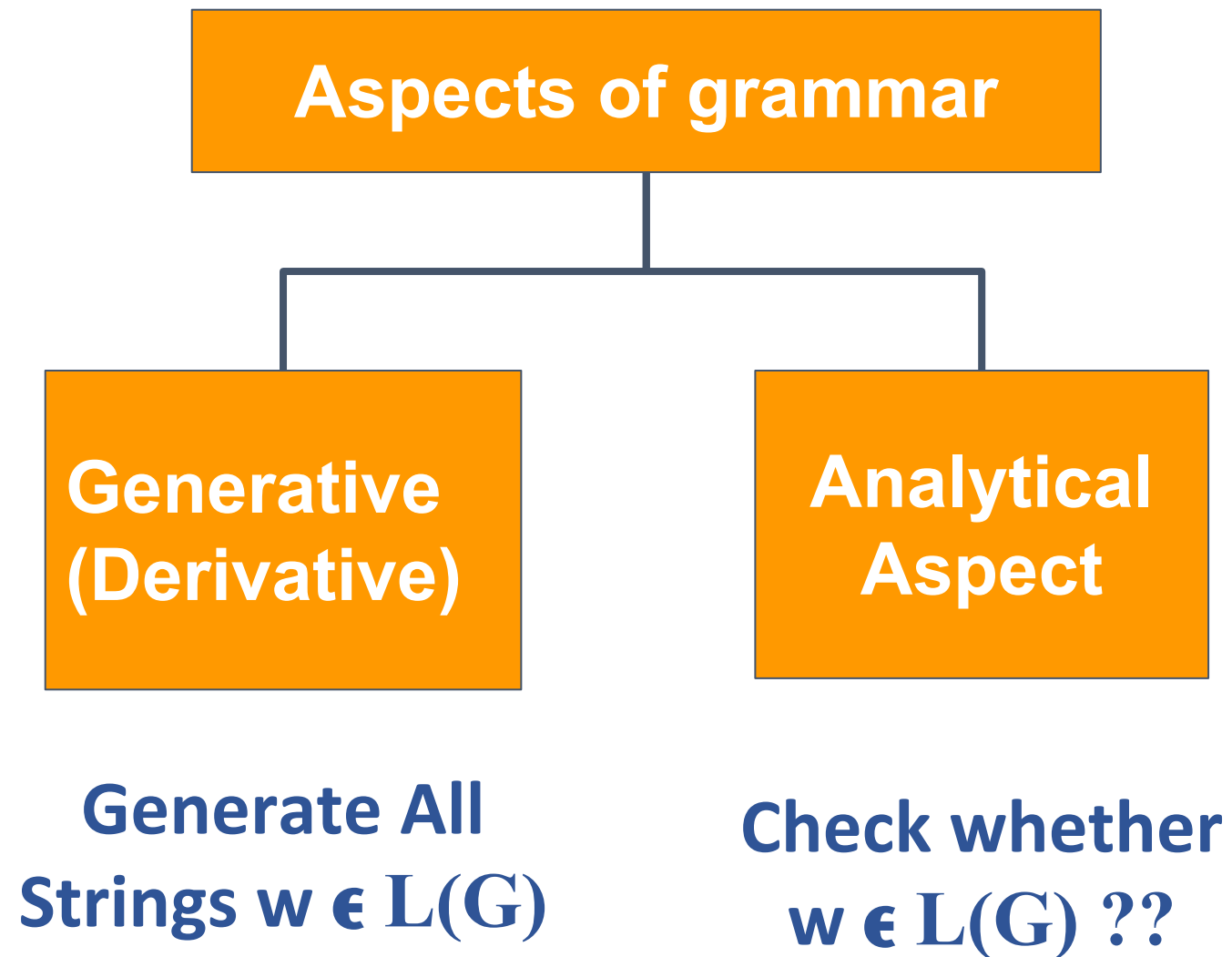
$S \rightarrow bS \mid aA$

$A \rightarrow aA \mid bA \mid aB$

$B \rightarrow bbB \mid \lambda$

Automata





Parse Tree

|

What is a Tree?

Leaves

Branches

Root



Technically we represent a Tree upside down

Root-

Branches

Leaves

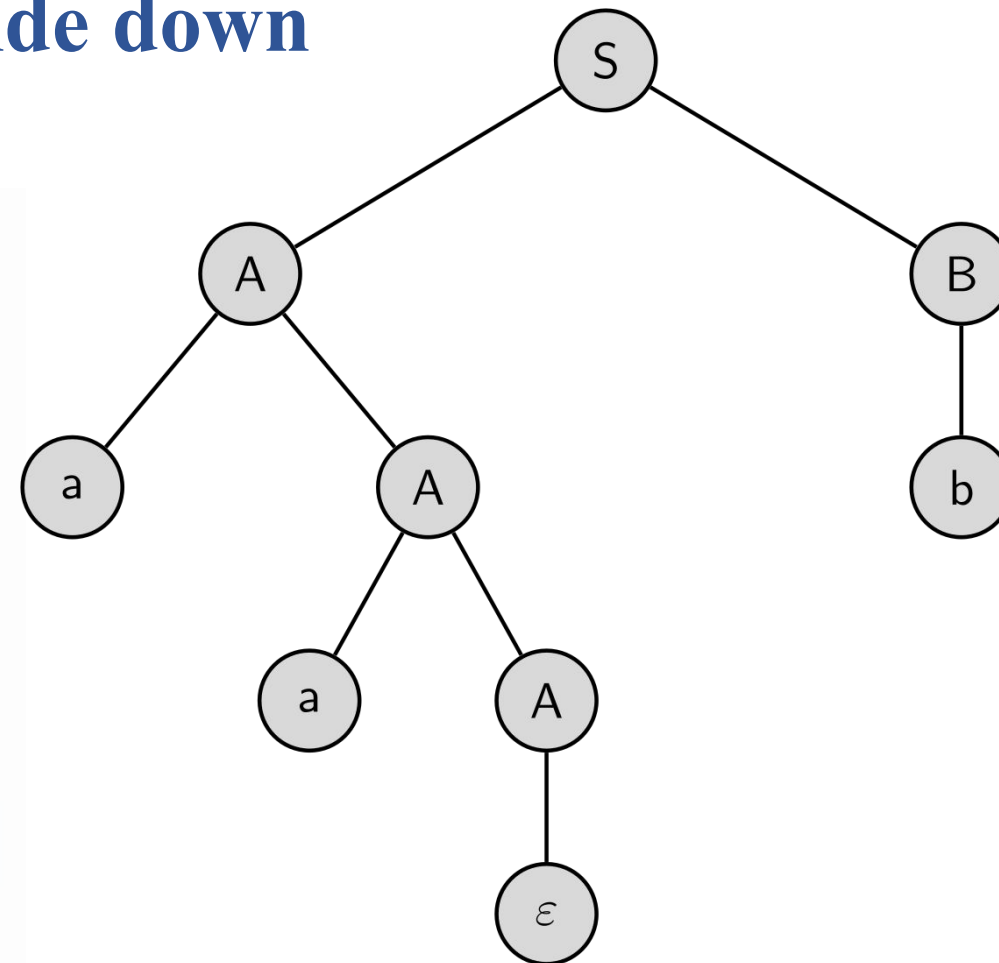


Technically we represent a Tree upside down

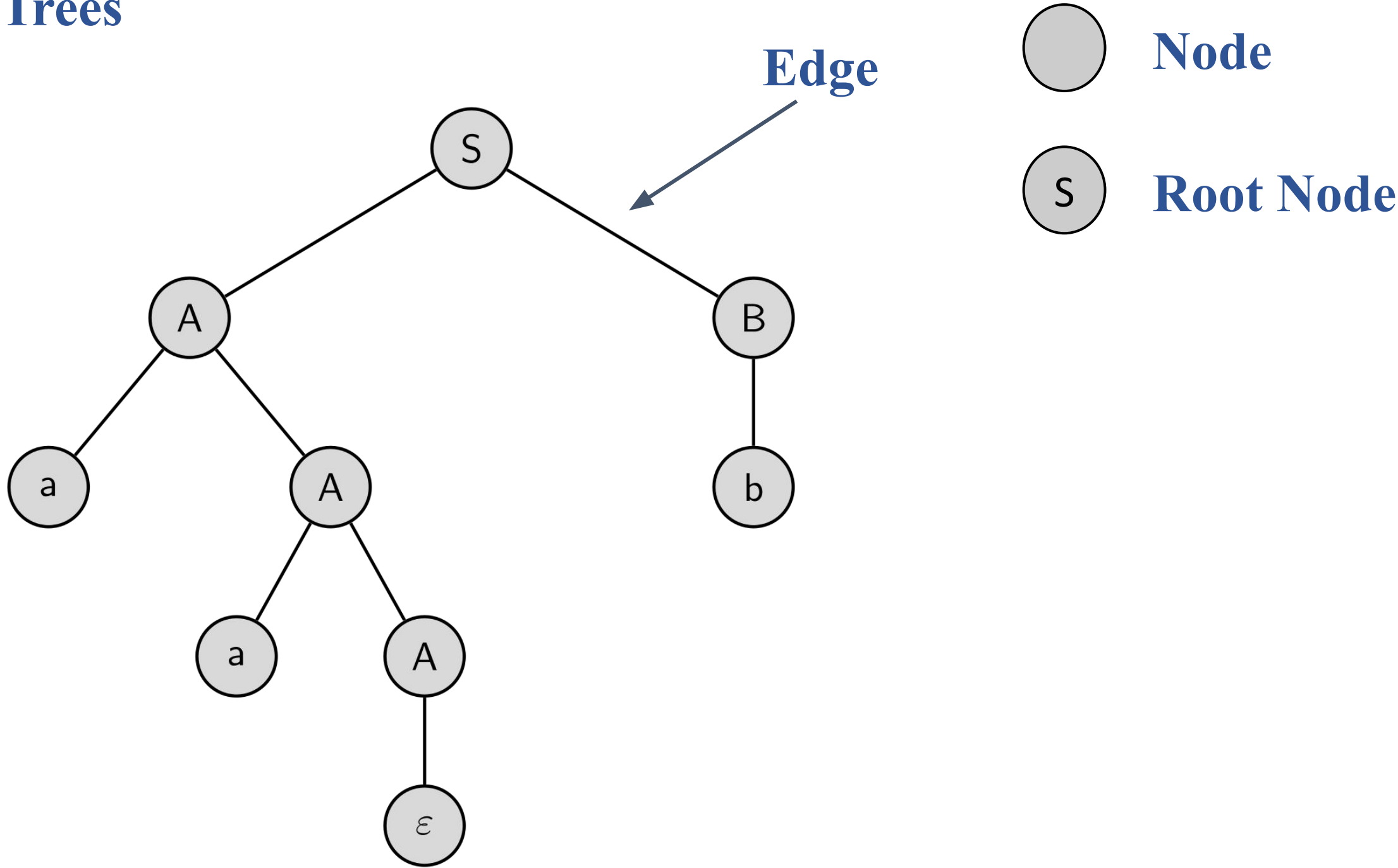
Root

Branches

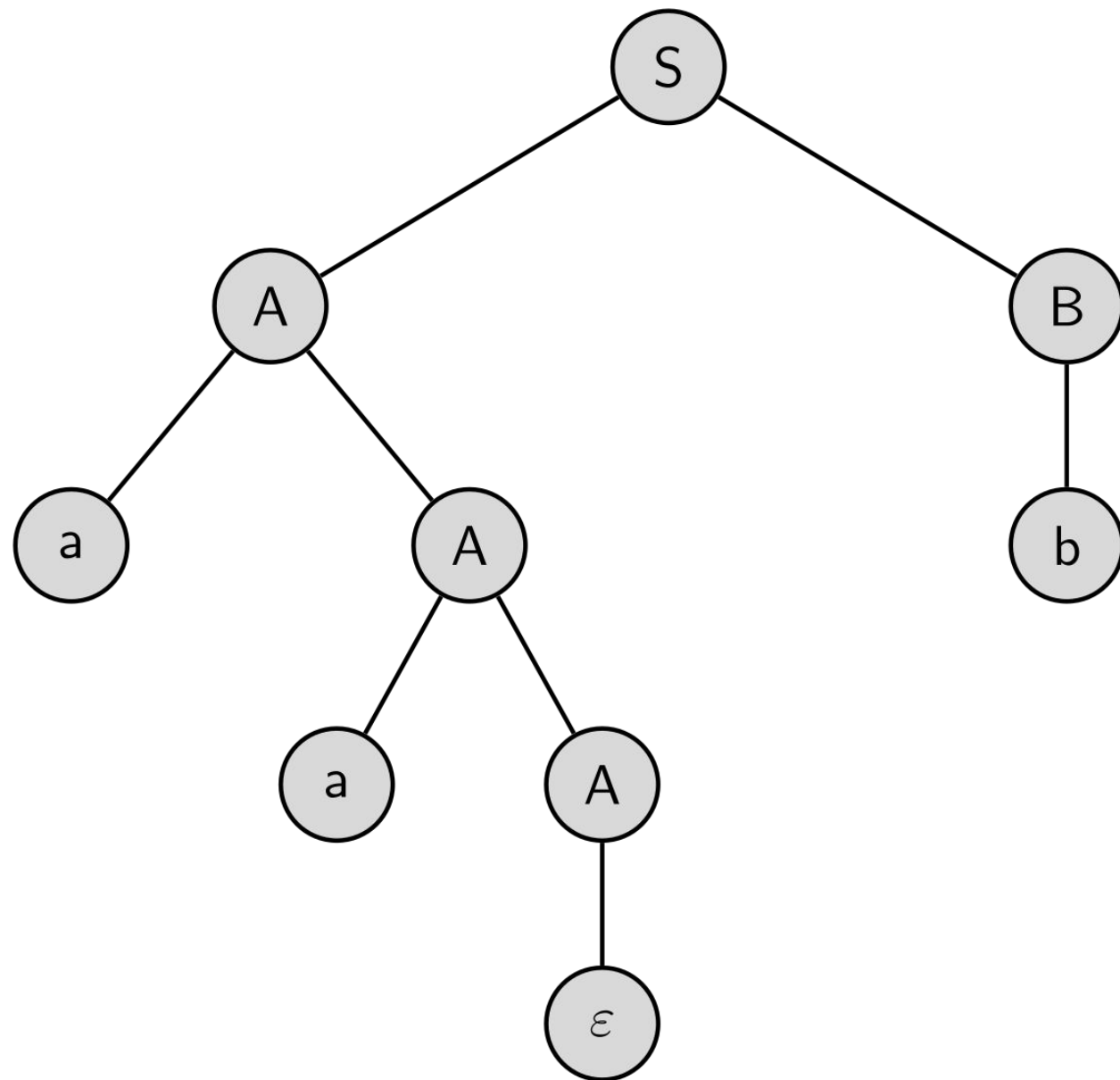
Leaves



Trees



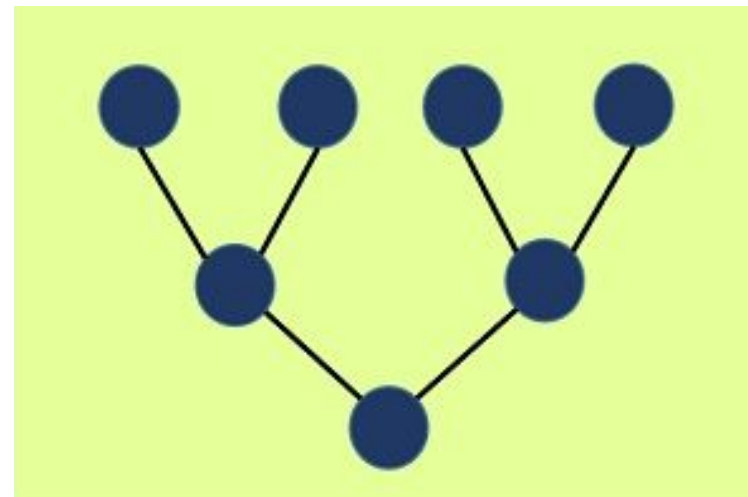
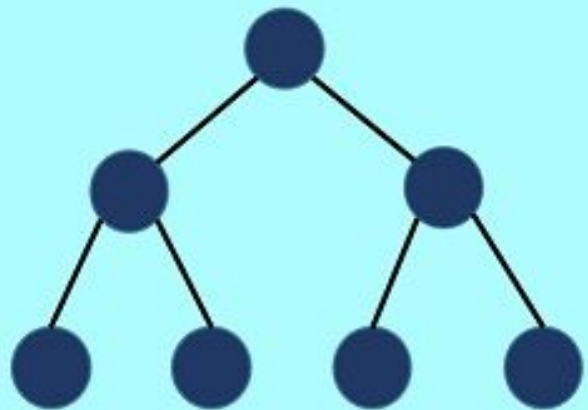
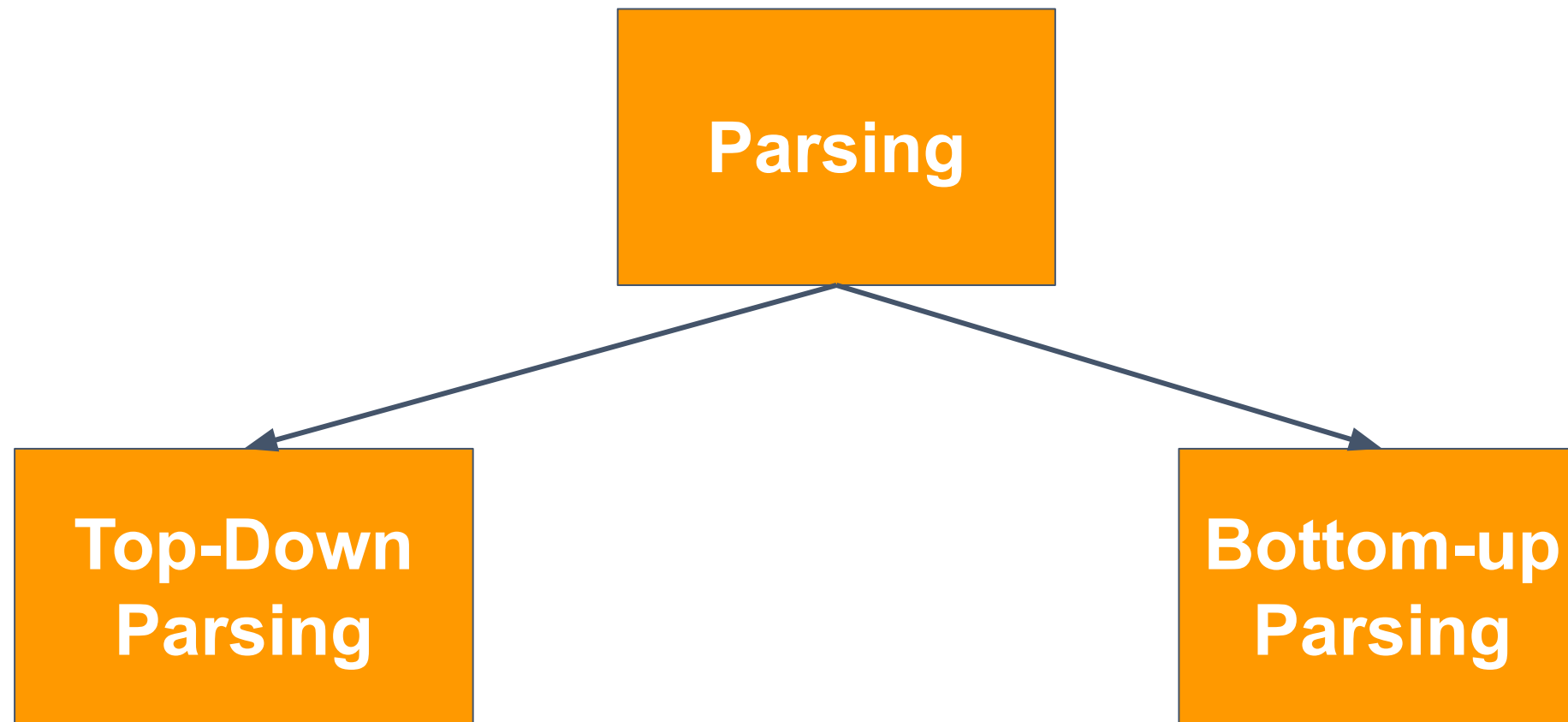
Parse Tree / Derivation Tree

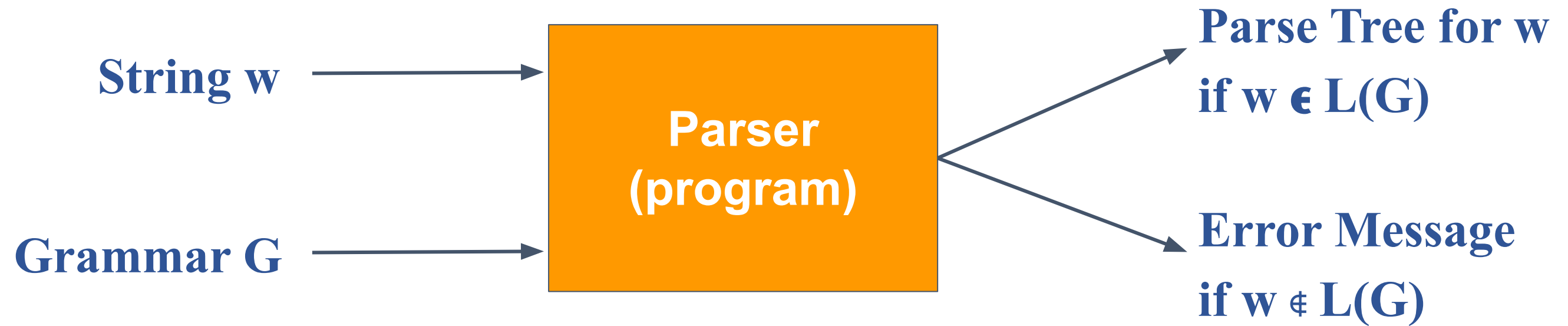


- S** Root Node (Start Symbol)
- V** Non-Terminals - Interior Nodes
- T** Terminal Symbols - Leaf nodes

Yield of Parse Tree - aab

Parsing is the process of determining whether a String
 $w \in L(G)??$





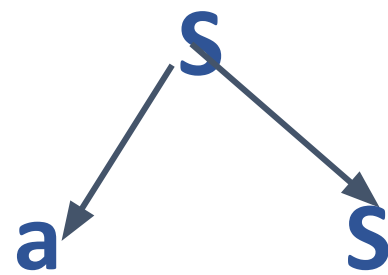
Let us look at few examples and generate parse tree for a given String w and Grammar G .

Parse Tree

S

Root node

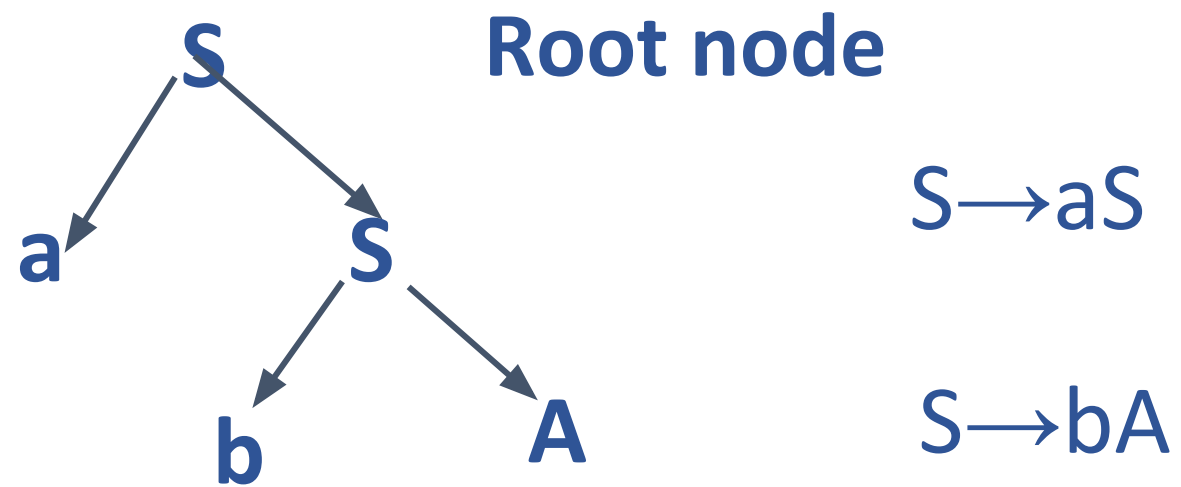
Parse Tree



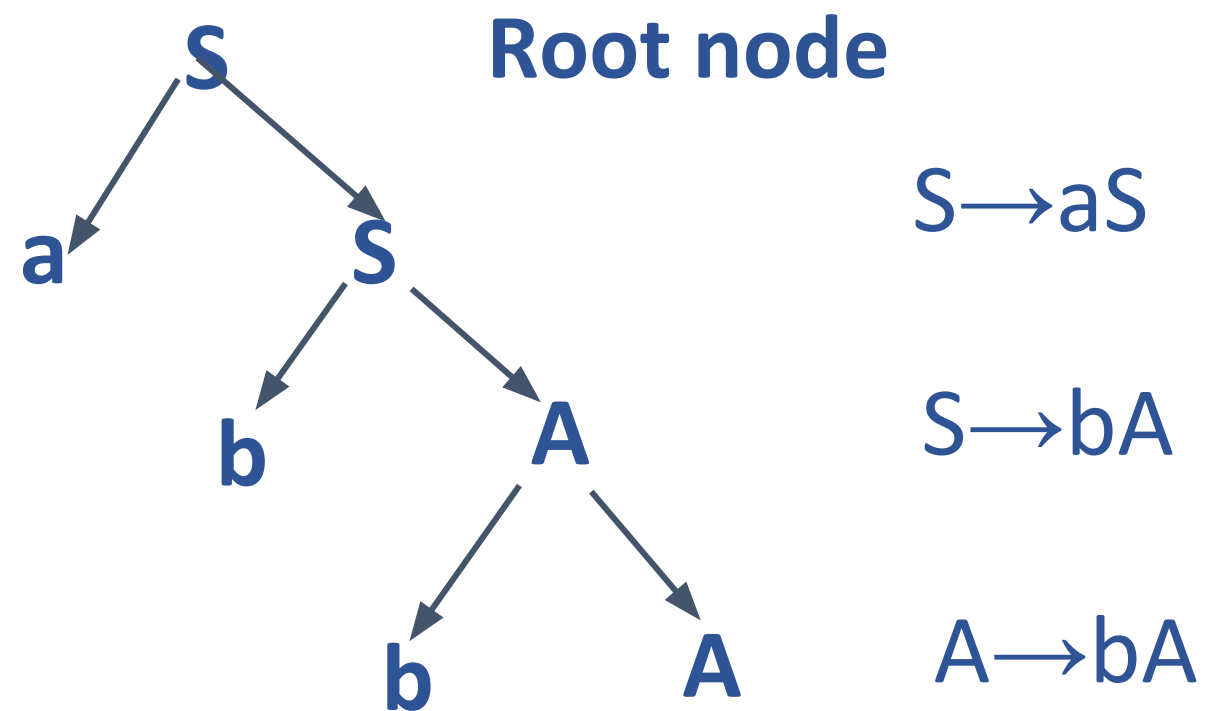
Root node

$S \rightarrow aS$

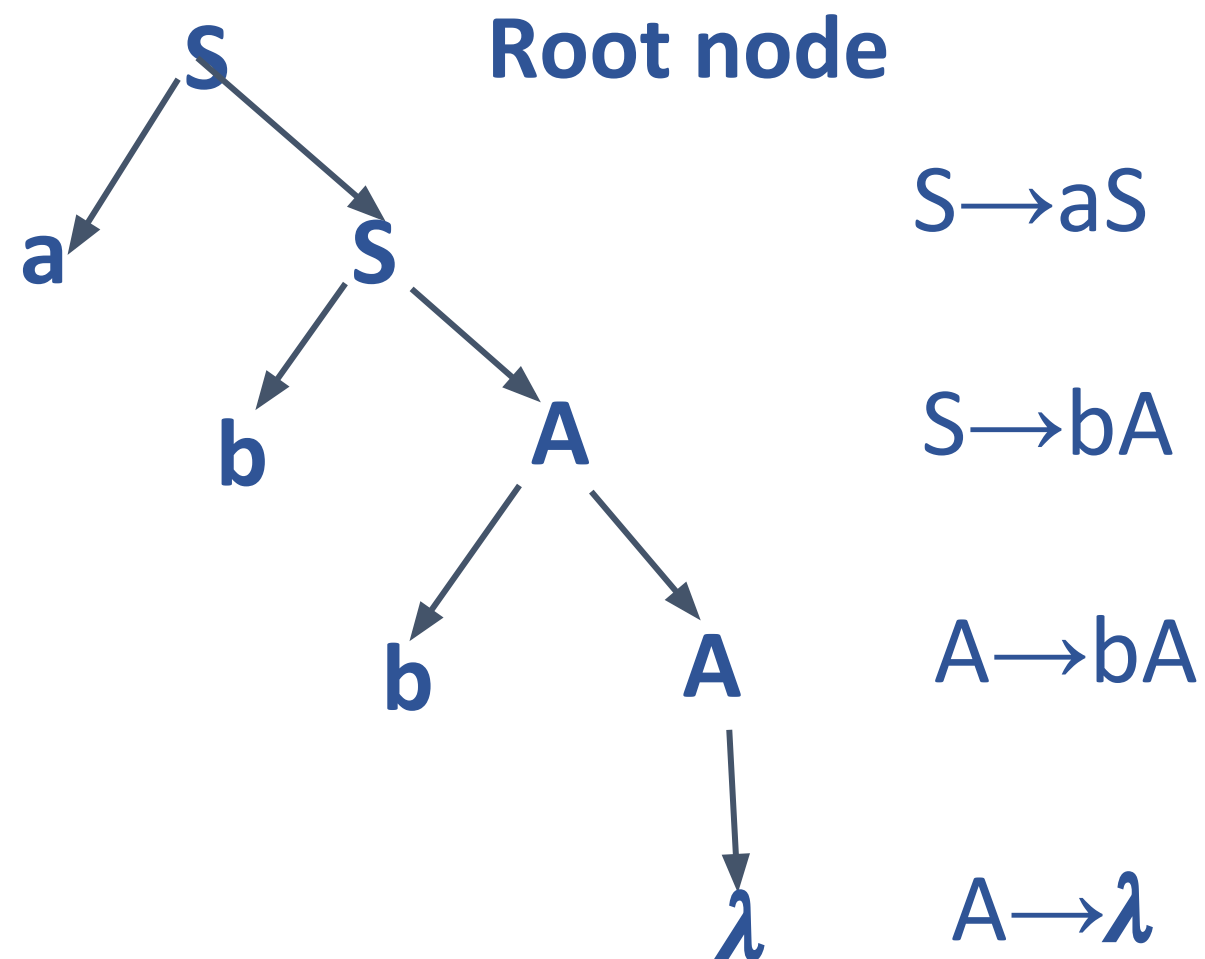
Parse Tree



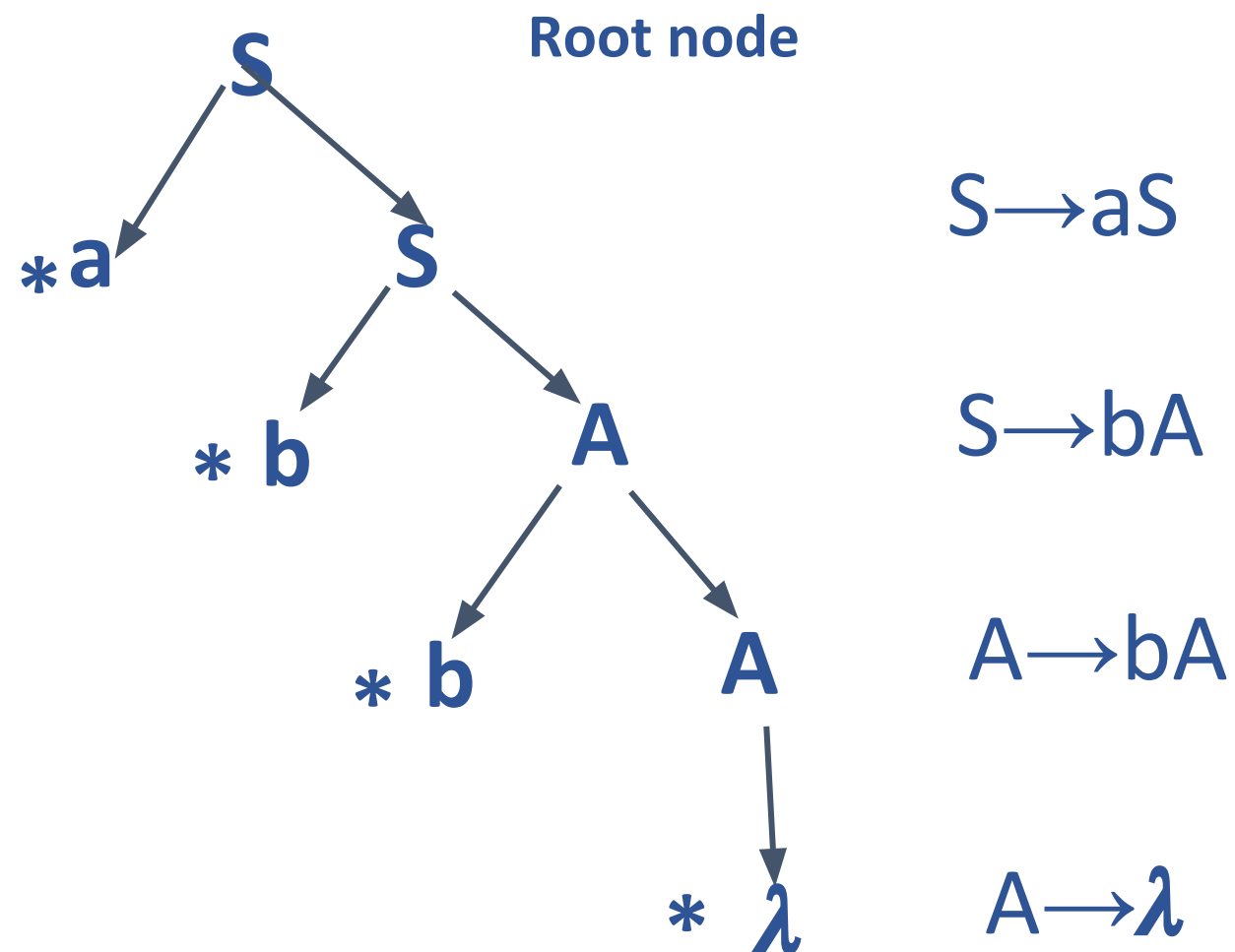
Parse Tree



Parse Tree



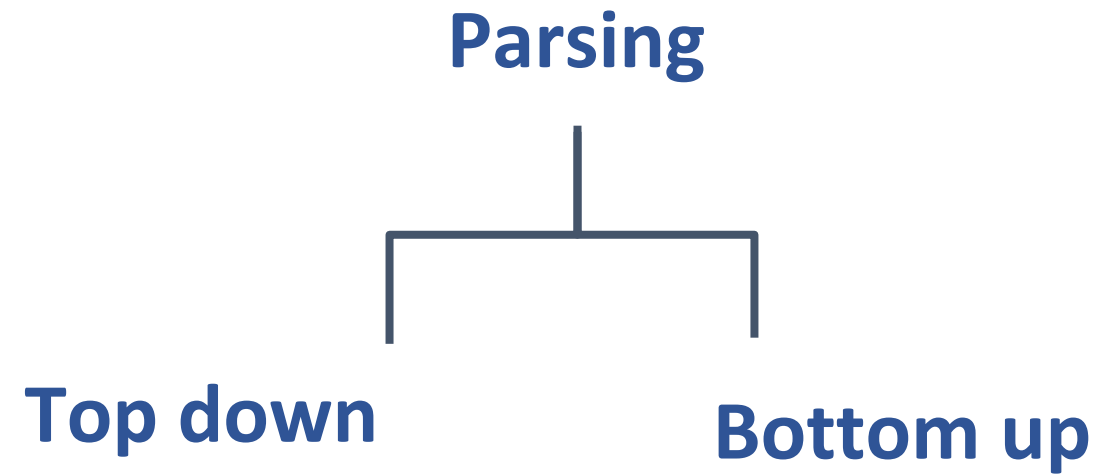
Parse Tree



Leaf Node:*

Yield of the tree:abb

2. Analytical aspect



2. Analytical aspect

Parsing



$S \rightarrow aS$

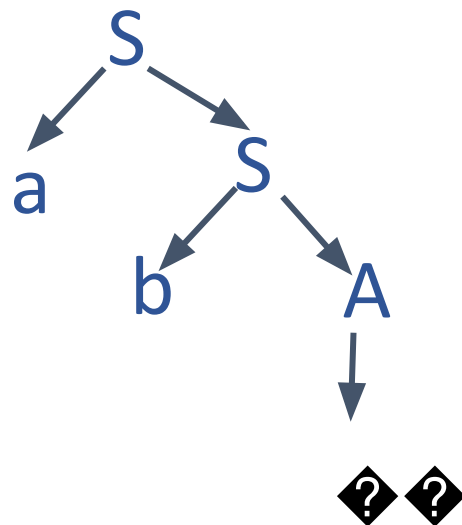
$S \rightarrow bA$

$A \rightarrow bA$

$A \rightarrow \lambda$

Top down

Bottom up



2. Analytical aspect

Parsing



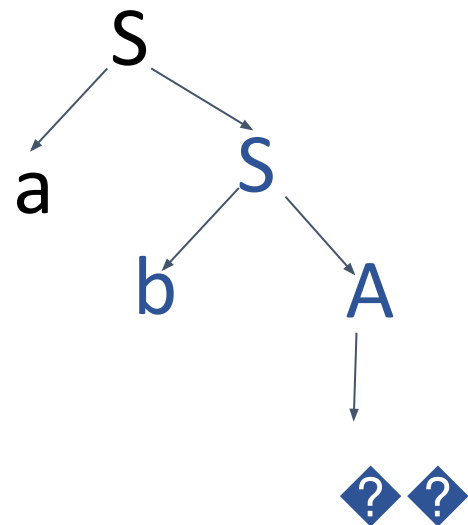
$S \rightarrow aS$

$S \rightarrow bA$

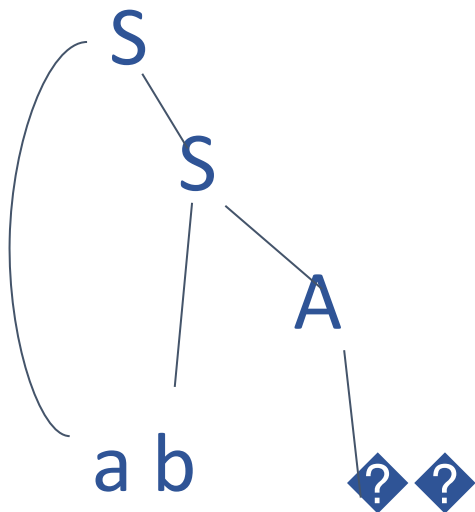
$A \rightarrow bA$

$A \rightarrow \lambda$

Top down



Bottom up



All about Left Linear Grammar

Right Linear

$A \rightarrow aB$

$B \rightarrow aB \mid bB \mid \lambda$

$L = \{\text{starting with 'a'}\}$

Left Linear

??

Right Linear

$A \rightarrow aB$

$B \rightarrow aB \mid bB \mid \lambda$

$L = \{\text{starting with 'a'}\}$

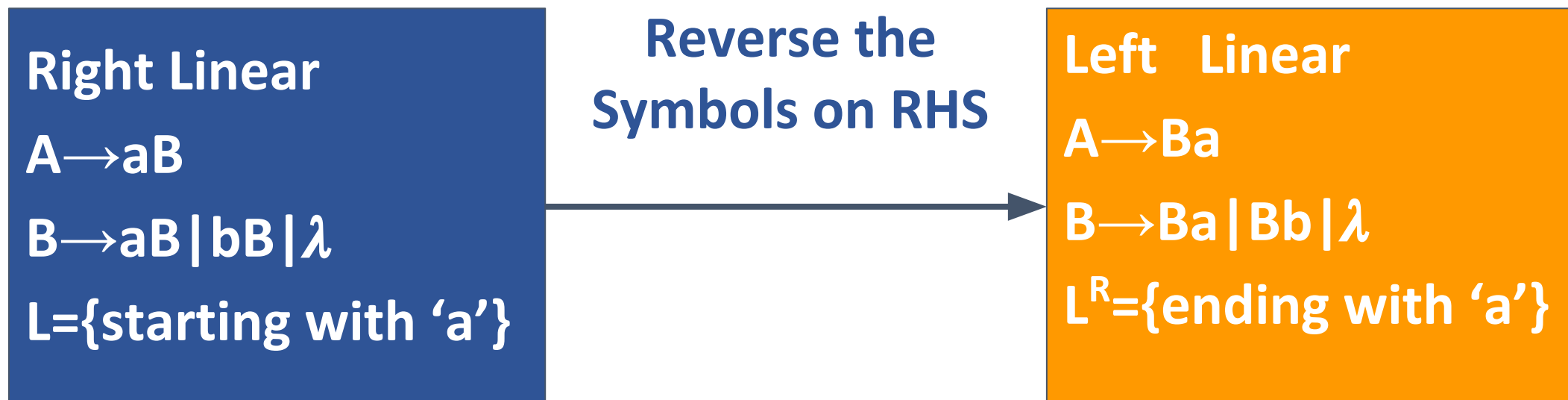
Reverse the
Symbols on RHS

Left Linear

$A \rightarrow Ba$

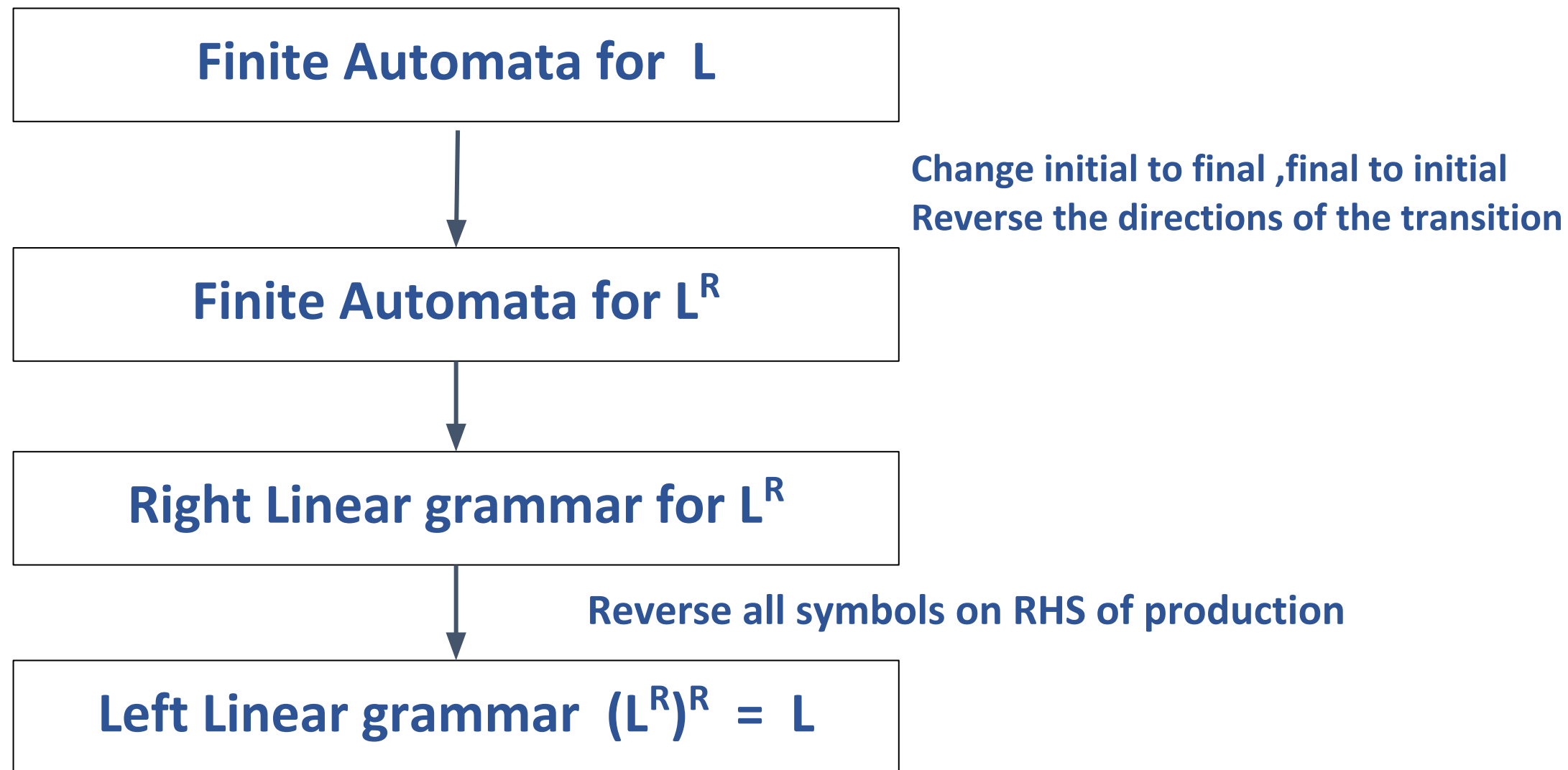
$B \rightarrow Ba \mid Bb \mid \lambda$

$L = ?$



When we reverse the RHS in every production in the right linear grammar for “L” we get a left linear grammar which will represent L^R .

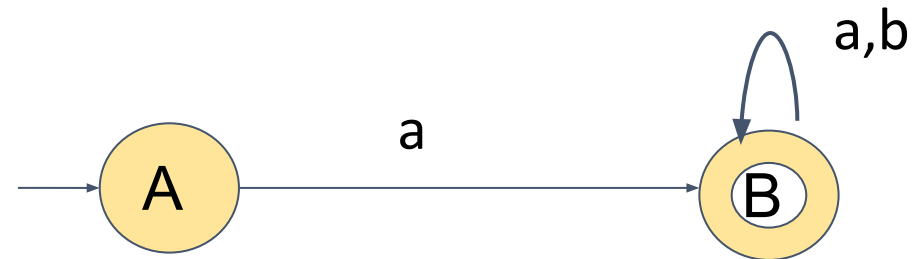
Converting Finite Automata to Left linear grammar



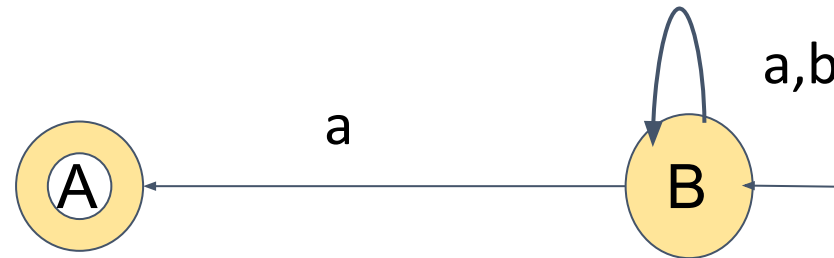
Automata Formal Languages and Logic

Unit 2 - Constructing a Left Linear Grammar

FA for $L = \{aw, w \in \{a,b\}^*\}$



FA for $L^R = \{wa, w \in \{a,b\}^*\}$



RLG for L^R

Right Linear Grammar

$B \rightarrow aB \mid bB \mid aA$

$A \rightarrow \lambda$

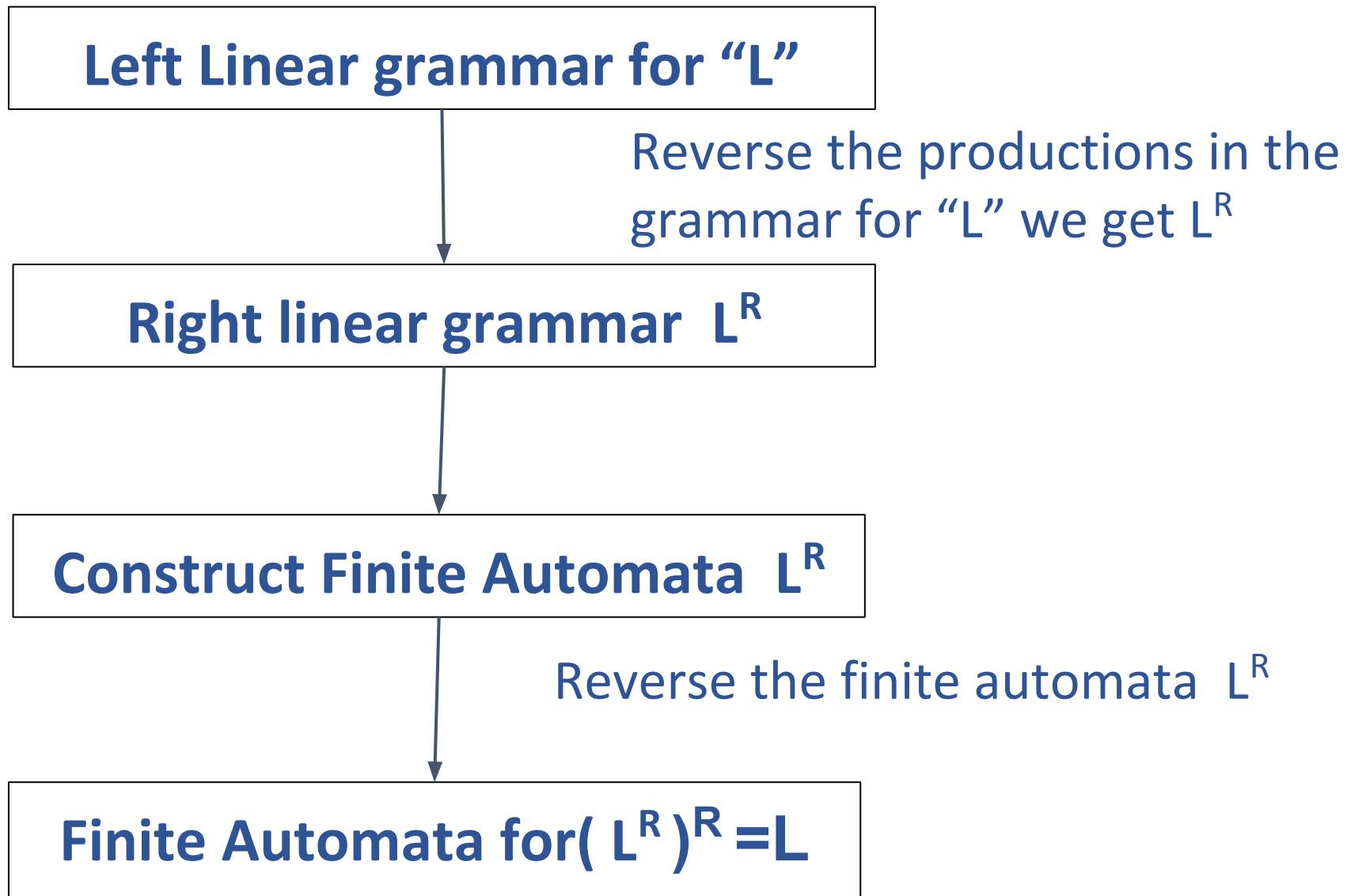
LLG for $(L^R)^R = L$

Left Linear Grammar

$B \rightarrow Ba \mid Bb \mid Aa$

$A \rightarrow \lambda$

Converting Left linear grammar to Finite automata



Automata Formal Languages and Logic

Unit 2 - Constructing a Left Linear Grammar



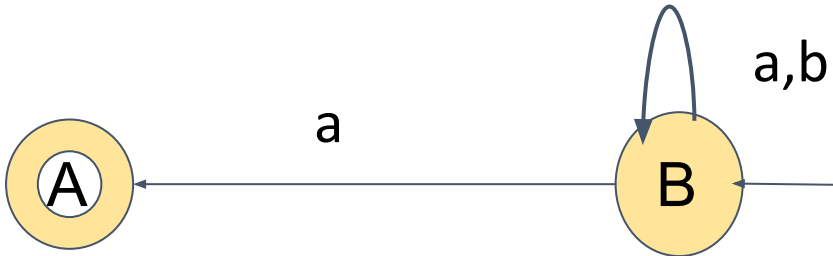
LLG for L

Left Linear Grammar
 $B \rightarrow Ba \mid Bb \mid Aa$
 $A \rightarrow \lambda$

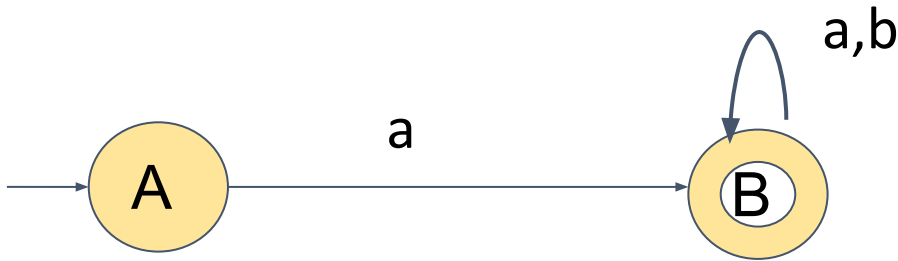
RLG for L^R

Right Linear Grammar
 $B \rightarrow aB \mid bB \mid aA$
 $A \rightarrow \lambda$

FA for $L^R = \{w\mathbf{a}, w \in \{a,b\}^*\}$



FA for $(L^R)^R = L = \{\mathbf{a}w, w \in \{a,b\}^*\}$



Automata Formal Languages and Logic

Unit 2 - LLG or RLG?



Which one is easier LLG or RLG??

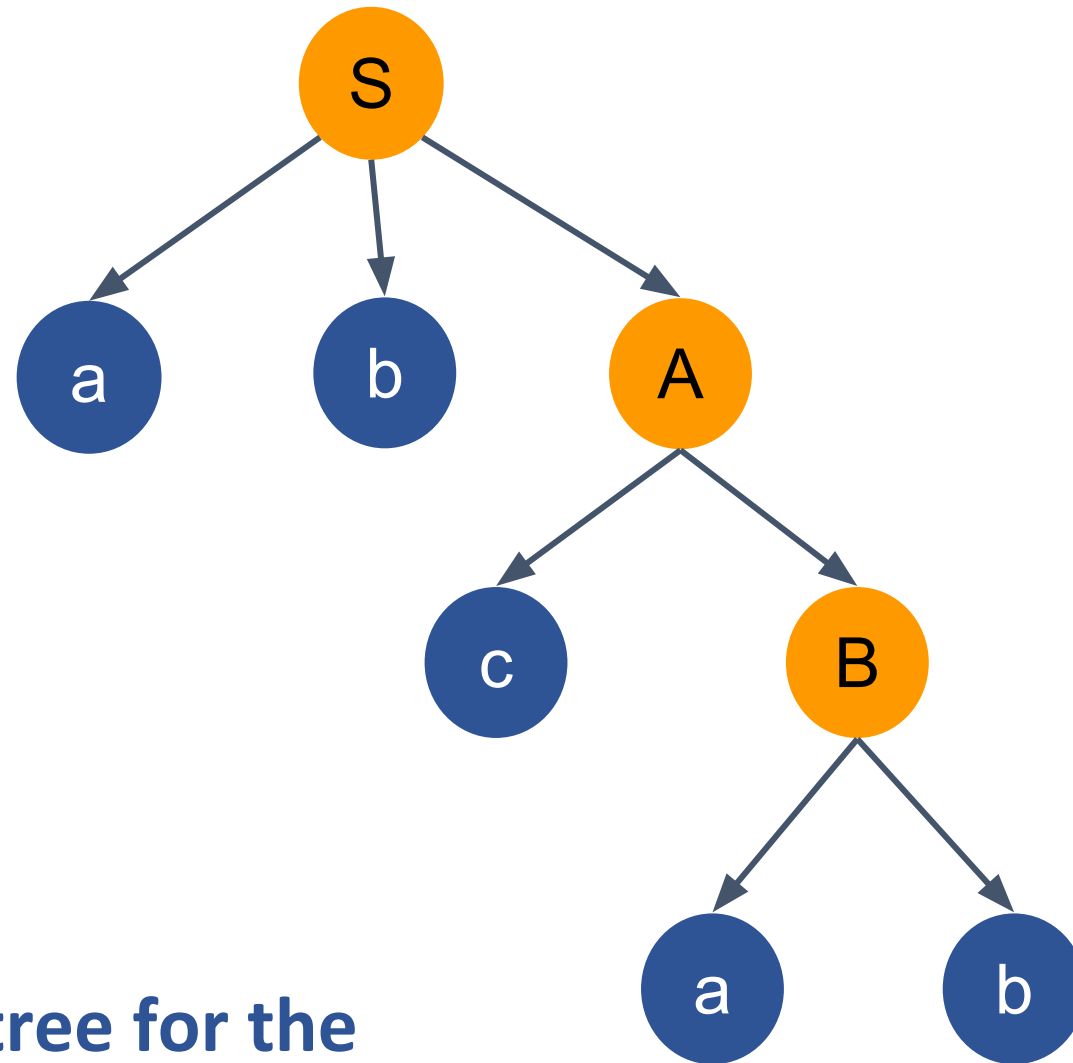
Right Linear

$S \rightarrow abA$

$A \rightarrow cB | aC$

$B \rightarrow ab$

$C \rightarrow b$



Construct Parse tree for the
String "abcab"

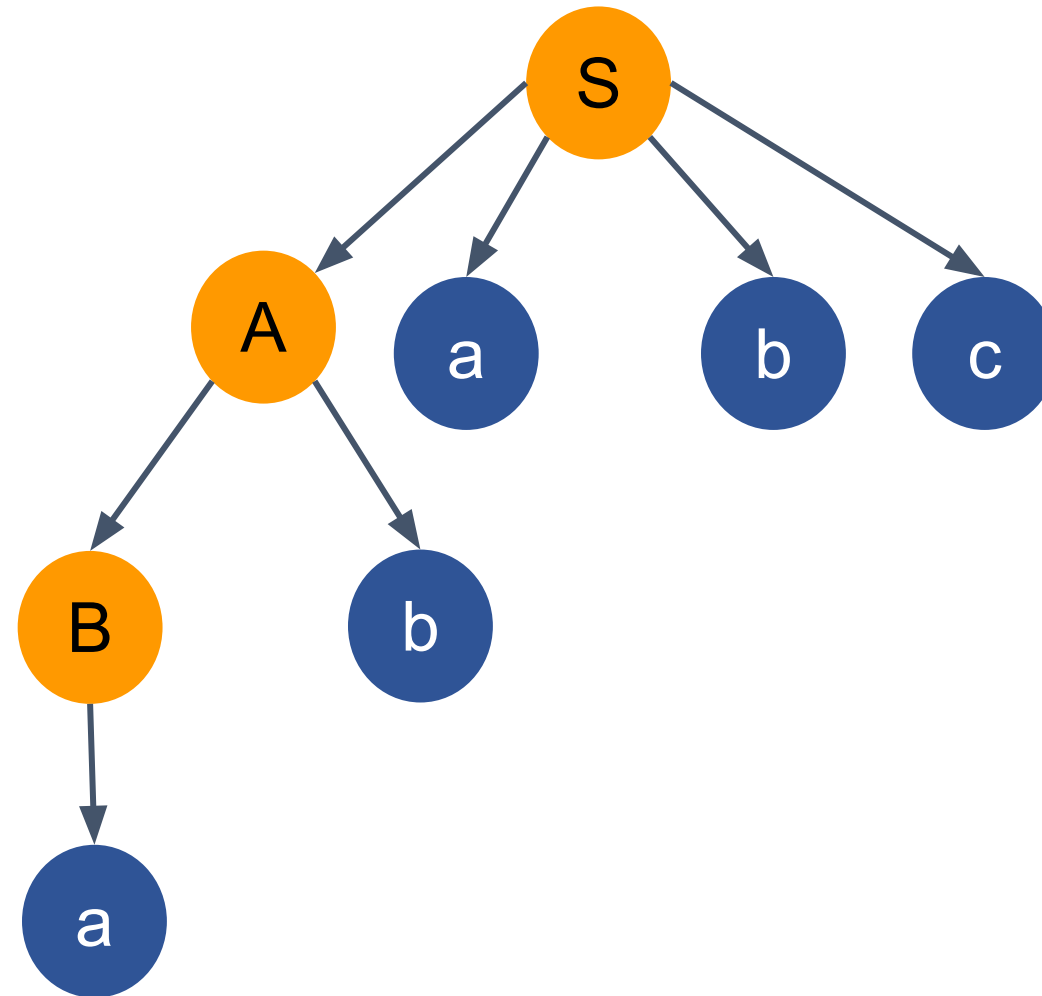
Left Linear

$S \rightarrow Aabc$

$A \rightarrow Bb|C$

$B \rightarrow a$

$C \rightarrow b$



Construct Parse tree for the
String "ababc"



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724